

计算机网络课程设计

任兴田 王勇 杨建红 编著

清华大学出版社

前言

计算机网络课程是一门实践性很强的课程,实践教学环节在教学过程中起着举足轻重的作用。通过课程设计不仅可以使学生加深对网络协议工作原理的理解和掌握,而且可以培养学生的网络运行管理和维护能力,网络协议分析能力,以及网络编程开发能力。

本书重点讲述了实际网络的组建、网络协议的分析和网络编程的基本方法。本书介绍了交换机和路由器等常用网络设备的配置和管理,从众多网络协议中选出数据链路层、网络层、传输层和应用层的 7 个常用网络协议进行了分析,并且精心设计了 11 个网络编程。

本书的主要特色体现在以下四点:

1) 理论实践相结合。每一个课程设计都简要介绍相关协议的工作原理,不需要读者再去查阅其他资料;

2) 覆盖面广。涵盖实际网络的组建、常用网络协议的分析和网络程序的设计与实现等方面;

3) 对网络环境的要求较低。网络组建只需要由 PC、交换机和路由器组成的基本网络环境。协议分析只需要实际连接 Internet 的环境。网络编程是基于套接口的,不需要大量的网络设备和复杂的网络环境,就可以进行各层协议的编程开发;

4) 灵活性高。可以根据课程设计的课时情况,选择其中的某些课程设计作为教学内容,便于组织教学实践。

全书共分 3 章。第 1 章介绍实际网络的组建,包括网线制作、路由器配置、虚拟局域网(VLAN)划分、访问控制列表(ACL)配置以及动态主机配置协议(DHCP)配置等。在配置和管理实际网络过程中,加深理解网络协议的基本工作原理,掌握网络的运行维护和管理。第 2 章介绍网络协议分析,利用 Wireshark 捕获在局域网中传输的数据包,对以太网、ARP、IP、ICMP、UDP、TCP 和 HTTP 等常用网络协议进行分析,能够更好地理解这些协议的数据包结构,掌握这些协议的工作原理。第 3 章介绍网络编程,包括以太网 ARP 请求发送程序、以太网 ARP 响应分析程序、ping 程序、多播客户/服务器程序、UDP 报文发送程序、TCP SYN 报文发送程序、UDP 客户/服务器程序、TCP 客户/服务器程序、服务器接口监测程序、邮件客户端程序和聊天客户/服务器程序等。通过数据链路层、网络层、传输层和应用层的常用协议的设计与实现,加深理解这些协议的工作原理,掌握网络编程的基本流程和基本方法,培养编程开发能力。

本书在编写过程中参考了国内外最新文献资料,可以作为高等院校计算机、物联网、软件工程、电子信息类及相关专业的配套的课程设计教材,也可作为从事计算机网络的工程技术人员的参考书。作者在教学中利用这些内容做了一些尝试,并取得了较好的教学效果。

本书由任兴田、王勇和杨建红共同编写,由任兴田完成全书的统稿。

限于作者的学术水平,书中难免存在不妥之处,敬请读者批评指正。

编者

2015 年 7 月

目录

第一章 网络组建	1
1.1 网线制作	1
1.1.1 课程设计目的	1
1.1.2 课程设计内容	1
1.1.3 相关知识	1
1.1.4 课程设计步骤	2
1.2 路由器配置	3
1.2.1 课程设计目的	3
1.2.2 课程设计内容	3
1.2.3 相关知识	3
1.2.4 课程设计步骤	5
1.3 VLAN 配置	9
1.3.1 课程设计目的	9
1.3.2 课程设计内容	9
1.3.3 相关知识	9
1.3.4 课程设计步骤	11
1.4 ACL 配置	12
1.4.1 课程设计目的	12
1.4.2 课程设计内容	12
1.4.3 相关知识	13
1.4.4 课程设计步骤	15
1.5 DHCP 配置	16
1.5.1 课程设计目的	16
1.5.2 课程设计内容	16
1.5.3 相关知识	17
1.5.4 课程设计步骤	17
第二章 网络协议分析	20
2.1 Wireshark 使用	20
2.1.1 课程设计目的	20
2.1.2 课程设计内容	20
2.1.3 相关知识	20
2.1.4 课程设计步骤	21
2.2 以太网协议分析	26
2.2.1 课程设计目的	26
2.2.2 课程设计内容	26
2.2.3 相关知识	26
2.2.4 课程设计步骤	27
2.3 ARP 协议分析	27
2.3.1 课程设计目的	27
2.3.2 课程设计内容	27

2.3.3 相关知识	28
2.3.4 课程设计步骤	28
2.4 IP 协议分析	30
2.4.1 课程设计目的	30
2.4.2 课程设计内容	30
2.4.3 相关知识	30
2.4.4 课程设计步骤	32
2.5 ICMP 协议分析	34
2.5.1 课程设计目的	34
2.5.2 课程设计内容	34
2.5.3 相关知识	34
2.5.4 课程设计步骤	36
2.6 UDP 协议分析	37
2.6.1 课程设计目的	37
2.6.2 课程设计内容	37
2.6.3 相关知识	37
2.6.4 课程设计步骤	38
2.7 TCP 协议分析	39
2.7.1 课程设计目的	39
2.7.2 课程设计内容	39
2.7.3 相关知识	39
2.7.4 课程设计步骤	42
2.8 HTTP 协议分析	47
2.8.1 课程设计目的	47
2.8.2 课程设计内容	47
2.8.3 相关知识	48
2.8.4 课程设计步骤	49
第三章 网络编程	52
3.1 以太网 ARP 请求发送程序	52
3.1.1 课程设计目的	52
3.1.2 课程设计内容	52
3.1.3 相关知识	52
3.1.4 课程设计分析	52
3.1.5 进一步的扩展	57
3.2 以太网 ARP 响应分析程序	57
3.2.1 课程设计目的	57
3.2.2 课程设计内容	58
3.2.3 相关知识	58
3.2.4 课程设计分析	58
3.2.5 进一步的扩展	60
3.3 ping 程序	61
3.3.1 课程设计目的	61
3.3.2 课程设计内容	61
3.3.3 相关知识	61

3.3.4 课程设计分析	62
3.3.5 进一步的扩展	67
3.4 多播客户/服务器程序	68
3.4.1 课程设计目的	68
3.4.2 课程设计内容	68
3.4.3 相关知识	68
3.4.4 课程设计分析	69
3.4.5 进一步的扩展	72
3.5 UDP 报文发送程序	72
3.5.1 课程设计目的	72
3.5.2 课程设计内容	72
3.5.3 相关知识	72
3.5.4 课程设计分析	72
3.5.5 进一步的扩展	76
3.6 TCP SYN 报文发送程序	76
3.6.1 课程设计目的	76
3.6.2 课程设计内容	76
3.6.3 相关知识	76
3.6.4 课程设计分析	76
3.6.5 进一步的扩展	83
3.7 UDP 客户/服务器程序	83
3.7.1 课程设计目的	83
3.7.2 课程设计内容	83
3.7.3 相关知识	83
3.7.4 课程设计分析	83
3.7.5 进一步的扩展	86
3.8 TCP 客户/服务器程序	86
3.8.1 课程设计目的	86
3.8.2 课程设计内容	86
3.8.3 相关知识	86
3.8.4 课程设计分析	86
3.8.5 进一步的扩展	90
3.9 服务器接口流量监测程序	90
3.9.1 课程设计目的	90
3.9.2 课程设计内容	90
3.9.3 相关知识	90
3.9.4 课程设计分析	95
3.9.5 进一步的扩展	96
3.10 邮件客户端程序	97
3.10.1 课程设计目的	97
3.10.2 课程设计内容	97
3.10.3 相关知识	97
3.10.4 课程设计分析	100
3.10.5 进一步的扩展	107

3.11 聊天客户/服务器程序	108
3.11.1 课程设计目的	108
3.11.2 课程设计内容	108
3.11.3 相关知识	108
3.11.4 课程设计分析	108
3.11.5 进一步的扩展	120
附录	121
附录 A 网络编程的参考答案	121
A.1 以太网 ARP 请求发送程序参考答案	121
A.2 以太网 ARP 响应分析程序参考答案	125
A.3 ping 程序参考答案	127
A.4 多播客户/服务器程序参考答案	133
A.5 UDP 报文发送程序参考答案	136
A.6 TCP SYN 报文发送程序参考答案	142
A.7 UDP 客户/服务器程序参考答案	148
A.8 TCP 客户/服务器程序	150
A.9 服务器接口流量监测程序参考答案	153
A.10 邮件客户端程序参考答案	157
A.11 聊天客户/服务器程序参考答案	183
附录 B Linux 网络编程基础知识	211
B.1 地址结构	211
B.2 相关函数	212
附录 C Java 网络编程基础知识	222
C.1 客户套接口	222
C.2 服务器套接口	225
参考文献	227

第一章 网络组建

本章主要是学习动手组建实际的网络环境。在配置和管理实际网络中，理解网络协议的基本原理，掌握网络的运行维护和管理。

在介绍了双绞线的规格、作用和制作方法之后，重点讲述了路由器和交换机等网络设备的基本功能和基本配置，包括在路由器上配置静态路由、路由信息协议（Routing Information Protocol, RIP）动态路由和开放最短路径优先协议（Open Shortest Path First, OSPF）动态路由，在交换机上划分虚拟局域网（Virtual LAN, VLAN），配置访问控制列表（Access Control List, ACL）以及配置动态主机配置协议（Dynamic Host Control Protocol, DHCP）等内容。

1.1 网线制作

1.1.1 课程设计目的

本课程设计的主要目的是掌握直通线和交叉线的规格、作用和制作方法，学习网线测试工具的使用。

1.1.2 课程设计内容

本课程设计要求制作和测试网线，具体的内容如下：

- 1) 制作直通线和交叉线；
- 2) 使用网线测试仪检测制作网线的连通情况。

1.1.3 相关知识

1. 双绞线

双绞线由两根相互绝缘的铜线组成，当两根线绞合在一起后，不同电线产生的干扰波会相互抵消，可以显著降低电线的辐射。双绞线可分为屏蔽双绞线和非屏蔽双绞线两大类。非屏蔽双绞线由多对双绞线和一个塑料外皮构成。屏蔽双绞线电缆的外层由铝箔包裹，以减小辐射，但并不能完全消除辐射。非屏蔽双绞线分为 3 类、5 类和 6 类等，屏蔽双绞线为 7 类。

2. RJ-45 连接器

RJ-45 连接器也称水晶头。通常它被安装在双绞线两头，以便于网卡、集线器、ADSL 调制解调器、交换机、路由器等网络设备相连。它共有八个金属引脚，当插入 RJ-45 插座时，分别与插座中的八个引脚相连，各引脚作用各不相同，有排列次序关系。当金属引脚面对我们时，引脚从左至右序号分别为 1~8。

3. 压线钳

通常具有剪线、剥线和压线三项功能。

4. 网线测试仪及使用方法

将网线两端的水晶头分别插入主测试仪和远程测试端的 RJ45 端口，将开关拨到“ON”

(S 为慢速档)。

1) 直通连线的测试。测试直通连线时,主测试仪的指示灯应该从 1 到 8 逐个顺序闪亮,而远程测试端的指示灯也应该从 1 到 8 逐个顺序闪亮。如果是这种现象,说明直通线的连通性没问题,否则就得重做。

2) 交错线连线的测试。测试交错连线时,主测试仪的指示灯也应该从 1 到 8 逐个顺序闪亮,而远程测试端的指示灯应该是按着 3、6、1、4、5、2、7、8 的顺序逐个闪亮。如果是这样,说明交错线连通性没问题,否则就得重做。

5. 双绞线接线标准与接线方法

双绞线接线通常有 EIA/TIA568A 和 568B 两种国际标准,其排列顺序如下所示。

568B: 橙白 橙 绿白 蓝 蓝白 绿 棕白 棕

568A: 绿白 绿 橙白 蓝 蓝白 橙 棕白 棕

通常双绞线连接的方法有直通线缆和交叉线缆两种方法。直通线缆两端的水晶头遵循 568A 或 568B 标准(通常以 B 标准为多),双绞线的每组线在两端是一一对应的,颜色相同的在两端水晶头的相应槽中保持一致。交叉线的一端采用 568A 标准,另一端采用 568B 标准。

1.1.4 课程设计步骤

1. 直通线制作

直通线一端的制作步骤:

- 1) 准备好双绞线、水晶头和压线钳;
 - 2) 用压线钳的剥线刀口将双绞线的外保护套划开,刀口距双绞线的端头至少 2 厘米,将划开的外保护套管剥去,露出 5 类线电缆中的 4 对双绞线;
 - 3) 按照 568B 标准(橙白、橙、绿白、蓝、蓝白、绿、棕白、棕)将导线排好;
 - 4) 用压线钳的剪线刀口将剥去保护套的 8 根导线多余部分剪去,留下约 1.2 厘米;
 - 5) 将剪去多余部分后的导线插入 RJ-45 插头,试试长短(要插到底),电缆线的外保护套应能在 RJ-45 插头内的凹陷处被压实。反复进行调整;
 - 6) 将 RJ-45 插头放入压线钳的压头槽内压实。
- 按照上述方法制作双绞线的另一端,就完成直通线的制作了。

2. 交叉线制作

交叉线一端的制作方法和导线的线序与直通线相同,另一端将 8 根导线按绿白、绿、橙白、蓝、蓝白、橙、棕白、棕的顺序平行排列,其它步骤与直通线的相同。

3. 网线测试

将双绞线两端的水晶头安装完成后,还需要用网线测试仪进行测试。

1) 直通线的测试。测试直通连线时,主测试仪的指示灯应该从 1 到 8 逐个顺序闪亮,而远程测试端的指示灯也应该从 1 到 8 逐个顺序闪亮。如果是这种现象,说明直通线的连通性没问题,否则就得重做。

2) 交叉线的测试。测试交叉线时,主测试仪的指示灯也应该从 1 到 8 逐个顺序闪亮,而远程测试端的指示灯应该是按着 3、6、1、4、5、2、7、8 的顺序逐个闪亮。如果是这样,说明交叉线连通性没问题,否则就得重做。

1.2 路由器配置

1.2.1 课程设计目的

本课程设计的主要目的是理解路由器工作原理,掌握静态路由协议和动态路由协议的配置,以及基本的测试命令。

1.2.2 课程设计内容

本课程设计要求配置路由器,具体的内容如下:

- 1) 配置路由器的接口;
- 2) 配置路由器的静态路由,使得各网络能够互相连通;
- 3) 配置路由器的 RIP 动态路由,使得各网络能够互相连通;
- 4) 配置路由器的 OSPF 动态路由,使得各网络能够互相连通;
- 5) 配置 PC 的地址和默认路由,使得各 PC 机能够互相连通;
- 6) 测试网络连通性。

1.2.3 相关知识

1. 路由

路由是指通过相互连接的网络把数据包从源地点传送到目标地点的活动。在路由过程中,数据包至少会经过一个或多个中间结点。而路由器是执行这种行为动作的网络通信设备,它工作在网络层,具有连接不同类型网络的能力,并能够选择数据传送路径的网络设备。

2. 路由表

为了给经过结点(主机、路由器等)的每个数据包寻找一条最佳传输路径,在结点中要保存各种传输路径的相关信息,供路由选择时使用,结点中保存的这种传输路径的相关信息称为路由表。路由表中保存着目的地址、前缀长度、接口和下一跳地址等内容。路由表可以由系统管理员根据网络情况预先设定,不能根据当前测量或估计的网络流量和拓扑结构的变化更新路由表,这种路由表称为静态路由表。其特点是简单,开销少,但灵活性差。也可以根据网络流量和拓扑结构的变化更新路由表,这种路由表称为动态路由表。其特点是健壮性和灵活性好,但开销大。

3. 路由器组成

CPU 负责执行路由器操作系统的指令以及通过控制台或者通过远程登录连接而键入的用户口令。CPU 的处理功能直接与路由器的处理能力相关。

RAM/DRAM 用于存储临时的运算结果,例如,存储路由选择表、快速交换缓存、数据包缓冲器以及数据包控制队列。RAM 中的数据在路由器断电后会丢失。

NVRAM 为非易失性 RAM,用于存储路由器的备份配置文件。路由器断电后,NVRAM 中的内容仍然保存。

Flash 为可擦除、可编程的 ROM,用于存放路由器的 IOS,FLASH 的可擦除特性允许更新、升级 IOS,而不用更换路由器内部的芯片。路由器断电后,FLASH 的内容不会丢失。当 FLASH 容量较大时,可以存放多个 IOS 版本。

ROM 为只读存储器，包含开机诊断程序、引导程序和特殊版本的 IOS 软件。

数据包进入和离开路由器的网络连接必须通过接口才能实现。路由器的接口类型非常多，它们各自用于不同的网络连接。路由器接口类型主要分为与局域网设备之间的连接和与广域网设备之间的连接两种类型接口。

4. 路由协议分类

静态路由协议。一种特殊的路由，根据互联网的拓扑结构和连接方式，由网络管理员采用手工方法在路由器中配置而成。这种方法适合规模较小、路由表相对简单的网络中使用。静态路由的优点是简单，开销少。

动态路由协议分为距离矢量协议和链路状态协议。

距离矢量协议是一种动态路由协议，采用距离矢量算法。距离矢量算法就是相邻的路由器之间互相交换整个路由表，并进行矢量的叠加，最终达到知道整个网络路由。它适合于小型到中型、多路径、动态的互联网环境。如：RIP 协议。

链路状态协议使用链路状态算法，最适宜较大型到特大型、多路径、动态的互联网环境。如：OSPF 协议。

5. 路由器的配置模式

路由器常见的配置模式如图 1-1 所示。

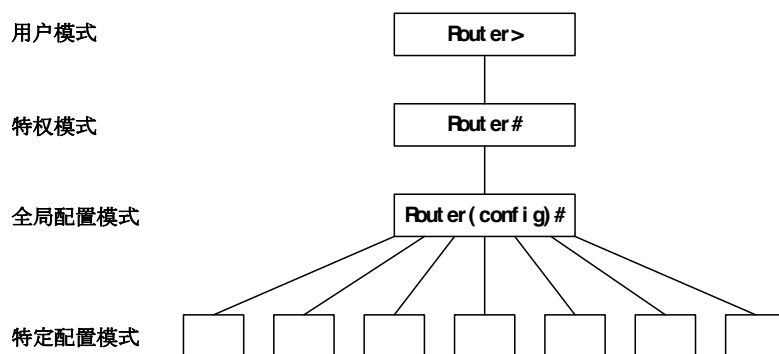


图 1-1 路由器常见的配置模式

用户模式 (User EXEC mode)。在该命令模式下，用户只能运行少数的命令，只能查看一些路由器当前的使用状态。在用户模式下无法做任何的配置，要配置路由器，必须转入特权模式。

```
Router> // 用户模式
Router> show interface // 查看接口信息
Router> show ip router // 查看路由表信息
```

特权模式 (Privileged EXEC mode)。这种模式支持调试和测试命令，详细检查路由器，配置文件操作和访问配置模式。

```
Rouer>enable
Router# // 特权模式
```

全局配置模式 (Global configuration mode)。这种模式实现强大的执行简单配置任务的单行命令。

```
Rouer#configure terminal
Router(config)# // 全局配置模式
```

特定配置模式 (Specific configuration modes)。有接口配置模式、路由协议配置模式等，这些模式提供更详细的配置。

```
Router(config)#interface fastEthernet 0/1
Router(config-if)# // 接口配置子模式
```

```

Router(config-if)# exit
Router(config)#router rip
Router(config-rouer)#          // 路由配置子模式
6. 路由器配置常用命令
1) 帮助
无论何种状态和位置，都可以键入“？”得到系统的帮助。
2) 改变状态命令
进入特权命令：enable
退出特权命令状态：disable
进入设置对话状态：setup
进入全局设置状态：config
进入接口设置状态：interface type slot/number①
退出局部设置状态：exit
进入子接口设置状态：interface type number.subinterface[point-to-point |
multipoint]
进入线路设置状态：line type slot/number
进入路由设置状态：router protocol
退出局部设置状态：exit
3) 显示命令
查看版本及引导信息：show version
查看运行设置：show running-config
查看开机设置：show startup-config
显示路由信息：show ip router
显示接口信息：show interface type slot/number
4) 基本设置命令
全局设置：config terminal
设置特权密码：enable secret password
设置访问用户及密码：username username password password
设置路由器名：hostname name
设置登录密码：password password
设置静态路由：ip route destination subnet-mask next-hop
启动 IP 路由：ip routing
设置 IP 地址：ip address address subnet-mask
激活接口：no shutdown
启动登陆进程：login[local | tacacs server]

```

1.2.4 课程设计步骤

1. 配置静态路由

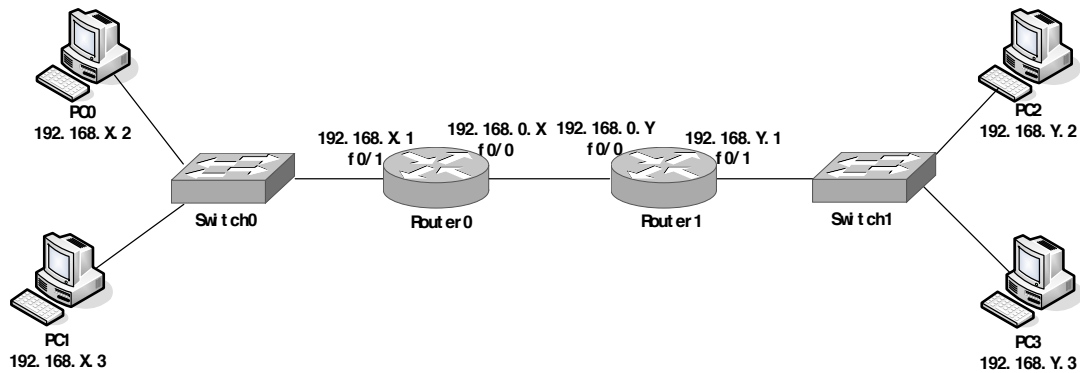


图 1-2 路由器配置拓扑图（注：图中 X、Y 为组号）

配置静态路由的拓扑图如图 1-2 所示。假设 X 为 1 组，Y 为 2 组，根据图 1-2 连接设备，配置路由器的静态路由，使 192.168.0.0/24 网络、192.168.1.0/24 网络和 192.168.2.0/24 网络能相互通信。

1) 根据图 1-2，连接设备。

2) 配置路由器接口的 IP 地址，配置静态路由，并查看路由配置。

(1) 在 Router0 上配置静态路由。

```
Router>
Router>enable
Router#configure terminal
Router(config)#hostname Router0
// 配置路由器 f0/0 接口
Router0(config)#interface f0/0
// 配置 f0/0 接口的 IP 为 192.168.0.1，子网掩码为 255.255.255.0
Router0(config-if)#ip address 192.168.0.1 255.255.255.0
// 激活接口
Router0(config-if)#no shutdown
Router0(config-if)#exit
// 配置路由器 f0/1 接口
Router0(config)#interface f0/1
Router0(config-if)#ip address 192.168.1.1 255.255.255.0
Router0(config-if)#no shutdown
Router0(config-if)#exit
// 配置静态路由。目的网络为 192.168.2.0/24 和下一跳地址为 192.168.0.2
Router0(config)#ip route 192.168.2.0 255.255.255.0 192.168.0.2
Router0(config)#end
// 显示路由表
Router0#show ip route
```

(2) 在 Router1 上配置静态路由。

```
// 配置 f0/0 接口
Router1(config)#interface f0/0
Router1(config-if)#ip address 192.168.0.2 255.255.255.0
Router1(config-if)#no shutdown
Router1(config-if)#exit
```

```
// 配置 f0/1 接口
Router1(config)#interface f0/1
Router1(config-if)#ip address 192.168.2.1 255.255.255.0
Router1(config-if)#no shutdown
Router1(config-if)#exit
// 配置静态路由
Router1(config)#ip route 192.168.1.0 255.255.255.0 192.168.0.1
Router1(config)#end
// 显示路由表
Router1#show ip route
```

3) 配置 PC 的 IP 地址和网关。PC0 的 IP 地址和网关配置如图 1-3 所示，其它 PC 的配置和 PC0 类似。

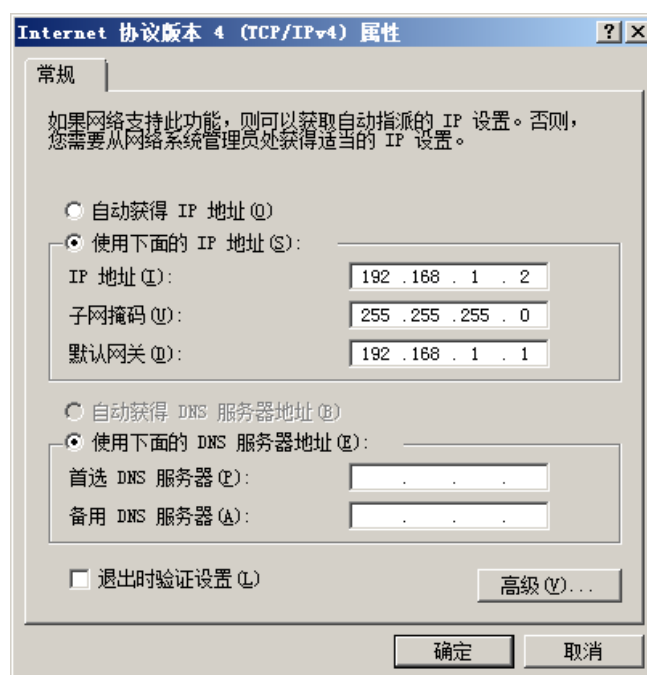


图 1-3 PC0 的 IP 地址和网关配置

4) 通过 ping 命令检查 4 台 PC 机能够互相连通性。

2. 配置 RIP 动态路由

配置 RIP 动态路由的拓扑图如图 1-2 所示。假设 X 为 1 组，Y 为 2 组，根据图 1-2 连接设备，配置路由器的 RIP 动态路由，使 192.168.0.0/24 网络、192.168.1.0/24 网络和 192.168.2.0/24 网络能相互通信。

1) 根据图 1-2，连接设备。

2) 配置路由器接口的 IP 地址，配置 RIP 动态路由，并查看路由配置。

(1) 在 Router0 上配置 RIP 动态路由。

```
// 配置路由器 f0/0 接口
Router0(config)#interface f0/0
// 配置 f0/0 接口的 IP 为 192.168.0.1，子网掩码为 255.255.255.0
Router0(config-if)#ip address 192.168.0.1 255.255.255.0
Router0(config-if)#no shutdown
Router0(config-if)#exit
```

```
// 配置路由器 f0/1 接口
Router0(config)#interface f0/1
Router0(config-if)#ip address 192.168.1.1 255.255.255.0
Router0(config-if)#no shutdown
Router0(config-if)#exit
// 配置 RIP 动态路由
Router0(config)#router rip
Router0(config-router)#network 192.168.0.0
Router0(config-router)#network 192.168.1.0
Router0(config-router)#end
// 查看 RIP 路由
Router0#show ip protocols
```

(2) 在 Router1 上配置 RIP 动态路由。

```
// 配置 f0/0 接口
Router1(config)#interface f0/0
Router1(config-if)#ip address 192.168.0.2 255.255.255.0
Router1(config-if)#no shutdown
Router1(config-if)#exit
// 配置 f0/1 接口
Router1(config)#interface f0/1
Router1(config-if)#ip address 192.168.2.1 255.255.255.0
Router1(config-if)#no shutdown
Router1(config-if)#exit
// 配置 RIP 路由
Router1(config)#router rip
Router1(config-router)#network 192.168.0.0
Router1(config-router)#network 192.168.2.0
Router1(config-router)#end
// 查看 RIP 动态路由
Router1#show ip protocols
```

3) 配置好各 PC 机的 IP 地址和网关。PC 的配置参见配置静态路由。

4) 通过 ping 命令检查 4 台 PC 机能够互相连通性。

3. 配置 OSPF 动态路由

配置 OSPF 动态路由的拓扑图如图 1-2 所示。假设 X 为 1 组，Y 为 2 组，根据图 1-2 连接设备，配置路由器的 OSPF 动态路由，使 192.168.0.0/24 网络、192.168.1.0/24 网络和 192.168.2.0/24 网络能相互通信。

1) 根据图 1-2，连接设备。

2) 配置路由器接口的 IP 地址，配置 OSPF 动态路由，并查看路由配置。

(1) 在 Router0 上配置 OSPF 动态路由。

```
// 配置路由器 f0/0 接口，参见 RIP 动态路由配置
// 配置路由器 f0/1 接口，参见 RIP 动态路由配置
// 配置 OSPF 动态路由
Router0(config)#router OSPF 100
Router0(config-router)#network 192.168.0.0 255.255.255.0 area 0
```

```

Router0(config-router)#network 192.168.1.0 255.255.255.0 area 0
Router0(config-router)#end
// 查看 OSPF 动态路由。
Router0#show ip protocols
    (2) 在 Router1 上配置 OSPF 动态路由。
// 配置路由器 f0/0 接口, 参见 RIP 动态路由配置
// 配置路由器 f0/1 接口, 参见 RIP 动态路由配置
// 配置 OSPF 动态路由
Router1(config)#router OSPF 200
Router1(config-router)#network 192.168.0.0 255.255.255.0 area 0
Router1(config-router)#network 192.168.2.0 255.255.255.0 area 0
Router1(config-router)#end
// 查看 OSPF 动态路由
Router1#show ip protocols
3) 配置好各 PC 机的 IP 地址和网关。PC 的配置参见配置静态路由。
4) 检查 4 台 PC 机能够互相连通性。

```

1.3 VLAN 配置

1.3.1 课程设计目的

本课程设计的主要目的是理解交换机工作原理和虚拟局域网（VLAN）的作用，掌握配置 VLAN 的方法。

1.3.2 课程设计内容

本课程设计要求配置 VLAN，具体的内容如下：

- 1) 按照 VLAN 划分的拓扑图，连接设备，配置 PC 的 IP 地址；
- 2) 使用超级终端程序，通过控制台端口对交换机进行配置，在一台交换机上创建 VLAN；
- 3) 测试端口的配置状态、VLAN 的配置状态，记录测试的结果。

1.3.3 相关知识

1. 交换机的工作原理

交换机工作在第二层，根据帧中的 MAC 地址进行工作。

交换机工作在混杂(promiscuous)方式，接收所有和它相连的 LAN 上传送的帧；交换机刚启动时，所有的内容可寻址存储器（CAM）为空，采用扩散算法(flooding)转发帧；在转发过程中采用逆向学习算法(backward learning)更新 CAM 的表项。对于一个进入的帧，它在交换机中的路由过程取决于它从哪个端口来(源端口)，以及它要往哪个目的地址去(目的端口)。若去往目的地址的端口与源端口相同，则丢弃该帧；若去往目的地址的端口与源端口不同，则转发该帧到目的端口；若目的端口未知，则使用扩散法。

2. VLAN

虚拟局域网（VLAN）是交换技术的重要组成部分，也是交换机的重要进步之一。把物理上直接相连的网络从逻辑上划分为多个子网。每一个 VLAN 对应着一个广播域，处于不同 VLAN 上的主机不能直接进行通信，需要引入三层路由技术才能通信。VLAN 组网技术具有高速、灵活、易于扩展和管理的特点，被广泛用来能够与局域网建设。VLAN 的实现原理是通过交换机的控制，某一 VLAN 成员发出的数据包只发给同一 VLAN 的其它成员，而不会发给该 VLAN 以外的成员。

3. VLAN 划分方法

基于端口的 VLAN 划分。将交换机中的若干个端口定义为一个 VLAN，同一个 VLAN 中的计算机具有相同的网络地址，不同 VLAN 之间进行通信需要通过三层路由协议。采用这种方式的 VLAN 在工作过程中，把一个网络结点迁移时，如果新旧端口不在一个 VLAN 内，则用户必须对该端口重新设置。

基于 MAC 地址的 VLAN 划分。VLAN 一旦划分完成，无论结点在网络上怎样移动，由于 MAC 地址保持不变，不需要重新配置。但是如果新增加结点，需要对交换机进行复杂的配置，以确定该结点属于哪一个 VLAN。

基于 IP 地址的 VLAN。新增加结点时，无须进行太多配置，交换机根据 IP 地址会自动将其划分到不同的 VLAN。这种 VLAN 智能化最高，实现最复杂。一旦离开该 VLAN，原 IP 地址将不可用，从而防止了非法用户通过修改 IP 地址来越权使用资源。

4. VLAN 划分步骤

VLAN 划分步骤：创建以太网 VLAN；将交换机端口加入到 VLAN 里；配置 Trunk 链路（可选）。

1) 创建以太网 VLAN

// 输入 VLAN ID，进入 VLAN 配置模式

```
Switch(config)#vlan {vlan-id}
```

// 为 VLAN 设置名字（可选）

```
Switch(config-vlan)#name {vlan-name}
```

2) 将交换机端口加入到指定的 VLAN 里。假如将端口加到了不存在的 VLAN 里，那么新的 VLAN 将自动被创建。

// 进入端口配置状态

```
Switch(config)#interface {interface}
```

// 将其设置为接入模式。用于接入终端设备

```
Switch(config-if)#switchport mode access
```

// 将端口加入到指定的 VLAN 里

```
Switch(config-if)#switchport access vlan {vlan-id}
```

3) 将交换机一组端口加入到指定的 VLAN 里

// 进入组配置状态

```
Switch(config)#interface range {port-range}
```

// 将其设置为接入模式。用于接入终端设备

```
Switch(config-if-range)#switchport mode access
```

// 将交换机端口加入到指定的 VLAN 里

```
Switch(config-if-range)#switchport access vlan {vlan-id}
```

当使用 interface range 命令时有如下的规则：

有效的组范围：vlan 从 1 到 4094；fastethernet 槽位/{first port} - {last port}，槽位为 0；gigabitethernet 槽位/{first port} - {last port}，槽位为 0；port-channel

port-channel-number - port-channel-number, port-channel-number 从 1 到 64。

端口号之间需要加入空格, 如: interface range fastethernet 0/1 - 5 是有效的, 而 interface range fastethernet 0/1-5 是无效的。

interface range 命令只能配置已经存在的 interface vlan。

所有在同一组的端口必须是相同类别的。

4) 配置 Trunk 端口 (可选)

// 进入接口配置模式

```
Switch(config)#interface {interface}
```

// 定义中继模式

```
Switch(config-if)#switchport trunk encapsulation {isl|dot1q|negotiate}
```

// 定义端口为 2 层的中继端口

```
Switch(config-if)#switchport mode trunk
```

// 定义允许中继的 VLAN 列表, VLAN 顺序由低到高, 以连字符相连。这些 VLAN 是在原有的基础上进行增加而不是覆盖, 默认为所有 VLAN

```
Switch(config-if)#switchport trunk allowed vlan {{add|all|except|remove}  
{vlan-list} | none}}
```

1.3.4 课程设计步骤

在交换机上 VLAN 划分的拓扑图如图 1-4 所示。

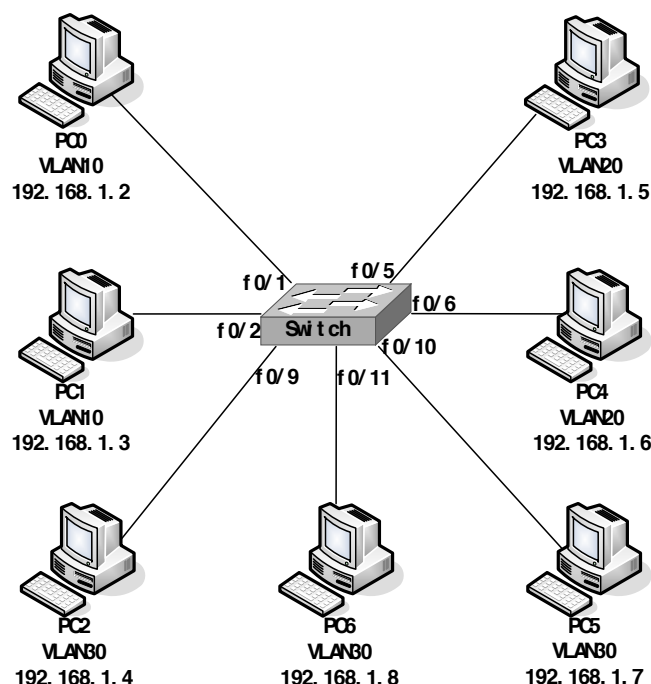


图 1-4 VLAN 划分的拓扑图

1. 将专用配置线的一端连接到交换机的 Console 口上, 另一端连接在 PC 机的串口转换器上。

2. 将 PC0、PC1 分别连接在交换机 1、2 端口上, 将 PC3、PC4 分别连接在交换机 5、6 端口上, 将 PC2、PC5、PC6 分别连接在交换机 9、10、11 端口上, 并按拓扑图配置 IP 地址。将交换机的 1-4 端口划分到 VLAN10, 交换机的 5-8 端口划分到 VLAN20, 交换机的 9-12 端口划分到 VLAN30。交换机的配置如下:

```

// 进入特权模式
Switch>enable
// 配置 VLAN10，端口范围 1-4
Switch#config t
Switch(config)#vlan 10
Switch(config-vlan)#exit
Switch(config)#interface range FastEthernet 0/1 - 4
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 10
Switch(config-if-range)#end
// 配置 VLAN20，端口范围 5-8
Switch#config t
Switch(config)#vlan 20
Switch(config-vlan)#exit
Switch(config)#interface range FastEthernet 0/5 - 8
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 20
Switch(config-if-range)#end
// 配置 VLAN30，端口范围 9-12
Switch#config t
Switch(config)#vlan 30
Switch(config-vlan)#exit
Switch(config)#interface range FastEthernet 0/9 - 12
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 30
Switch(config-if-range)#end
// 显示 VLAN
Switch#show vlan

```

3. 进行验证。配置前 PC 机之间都能够 ping 通。配置后，同一 VLAN 内 PC 机间能相互 ping 通，而不同 VLAN 之间的 PC 机不能 ping 通，表明 VLAN 划分成功。

1.4 ACL 配置

1.4.1 课程设计目的

本课程设计的主要目的是理解访问控制列表（ACL）原理，掌握配置扩展访问列表的方法。

1.4.2 课程设计内容

本课程设计要求在路由器上配置 ACL，具体的内容如下：

- 1) 配置路由器和主机，使网络之间相互连通；

- 2) 在此基础上配置 ACL;
- 3) 测试 ACL 的访问控制功能。

1.4.3 相关知识

1. 网络防火墙

网络防火墙指的是一个有软件和硬件设备组合而成、在内部网和外部网之间或专用网与公网之间的界面上构造的保护屏障。防火墙是一种保护计算机网络安全的技术性措施,它通过在网络边界上建立相应的网络通信监控系统来隔离内部和外部网络,以阻挡来自外部的网络入侵。

防火墙分为包过滤防火墙和应用层报文过滤。包过滤防火墙对 IP 包进行过滤,先获取 IP 包头信息,如:IP 层所承载的上层协议的协议号、源地址、目的地址、源端口和目的端口等,然后和设定的规则进行比较,根据比较的结果决定数据包是否被允许通过。应用层报文过滤维护每一个连接的状态,并且检查应用层协议的数据,以此决定数据包是否被通过。

2. 访问控制列表

访问控制列表 (ACL) 是实现包过滤规则库的一般方法,它由一系列 permit 或 deny 的规则组成。这些规则根据 IP 包头中源地址、目的地址和端口号等来描述。ACL 通过这些规则对数据包进行分类,并将这些规则应用到路由器的接口中。路由器根据这些规则来判断哪些数据可以接收,哪些数据包需要拒绝。

访问列表分为标准访问控制列表和扩展访问控制列表。标准访问列表利用 IP 包头中源 IP 地址来做出过滤决定,其特点是配置简单,但应用场合有限,不能够进行复杂条件的过滤。标准访问控制列表的表号范围为 1-99。扩展访问列表可以利用多个条件来做过滤,如:IP 包头中的源 IP 地址、目的 IP 地址、协议和端口号等。扩展访问列表的表号范围为 100-199。

根据在设备中定义列表的方式,还可以将列表分为编号列表和命名列表两类

3. 定义访问控制列表

1) 定义编号法标准访问列表

定义编号法标准访问列表的格式为

```
access-list listnumber {permit | deny} address [wildcard-mask]
```

它表示允许或拒绝来自指定网络的数据包,该网络由 IP 地址 (address) 和反掩码指定。其中

listnumber 为规则序号,其范围为 1-99。

permit 和 deny 表示允许或禁止满足该规则的数据包通过。

Address 和 wildcard-mask 分别为 IP 地址和反掩码,指定某个网络。如果 IP 地址指定为 any,则表示所有 IP 地址,而且不需配置指定相应的掩码。

注意: wildcard mask 由 32 位长的 2 进制数字组成,4 个八位位组。其中 0 代表必须精确匹配,1 代表任意匹配(即不关心)。

列表的形成完全依赖于写入的先后顺序。列表对进入的数据进行匹配查询时也是按照先后顺序,一旦找到对应的列表项即退出列表不再往下继续查询。如果定义列表时没有将小范围源地址放在列表的前几项,而是把整个大范围的网路采取的动作写入了列表的前面,就会使设备形成错误的判断。

例:在神州数码 DCR-2626 路由器上配置标准访问列表。禁止 192.168.0.0/24 网络中所有数据包通过路由器,而允许从其它主机来的数据包通过。

```
Router#config
```

```
Router_config#ip access-list standard 1 // 定义标准的访问控制列表
```

```
Router_config_std_nacl#deny 192.168.0.0 0.0.0.255
```

```
Router_config_std_nacl#permit any
```

2) 定义命名法标准访问列表

由于数值表示的访问控制列表在列表进行改动时无法将其中的某条删除,当需要对访问列表的某条进行更改时,只能先将整个列表一同删除,再一条一条将修改后的写入。而且数值表示的列表很不直观,一段时间之后,网管员也容易忘记当时配置访问列表的目的了,因此,在设备实现中可以采用另外一种方式配置,这就是命名的访问控制列表。

命名的列表配置分两步:1) 创建一个列表并进入到列表配置模式;2) 在独立的模式中配置具体列表项。整个过程如下所示。

```
// 定义了一个标准的访问控制列表,其名字为 for_test,并进入到这个列表配置模式
```

```
Router_config#ip access-list standard for_test
```

```
// 定义这个列表的第一个列表项允许了 1.1.1.0 为源地址的所有数据包通过
```

```
Router_config_std_nacl#permit 1.1.1.1 0.0.0.255
```

命名的列表与编号的列表本质上是一致的,只是在具体的操作环节有一点差异。命名的方法使得对列表的修改显得稍微灵活一些,可以独立删除某一个列表项,不必删除全部的。

但无论是编号的还是命名的访问控制列表,都不能对列表项进行修改覆盖,而只能添加,并且它们也只能将新的列表项添加在已有列表的最后(默认隐含的 deny 所有的前面),不能插入,所以当删除了命名列表的某项之后,新添加的列表项是出于整个列表的最末位置的。

3) 定义扩展访问控制列表

定义扩展访问列表的格式为

```
Router(config)#access-list listnumber { permit | deny } protocol source  
source-wildcard-mask destination destination-wildcard-mask [operator operand]
```

它表示允许或拒绝满足如下条件的数据包通过:

带有指定的协议,如 TCP、UDP、ICMP 等。

数据包来自由 *source* 和 *source-wildcard-mask* 指定的网络。

数据包去往 *destination* 和 *destination-wildcard-mask* 指定的网络。

该数据包的目的端口由 *operator operand* 规定的端口范围之内。

listnumber 为规则序号,扩展访问控制列表的规则序号范围为 100-199。

permit 和 *deny* 表示允许或禁止满足该规则的数据包通过。

protocol 可以指定为 0~255 之间的任一协议号(如 1 表示 ICMP 协议),对于常见协议(如 IP、TCP 和 UDP),可以直观地指定协议名,若指定为 IP,则该规则对所有 IP 包均起作用。

source 和 *source-wildcard-mask* 以及 *destination* 和 *destination-wildcard-mask* 之间的关系参见标准访问列表中相关内容。如果 IP 地址指定为 any,则表示所有 IP 地址,而且不需配置指定相应的反掩码。反掩码缺省为 255.255.255.255。

operand 用于指定端口范围,缺省为全部端口号 0-65535,只有 TCP 和 UDP 协议需要指定端口范围。支持的操作符及其语法如下:

eq portnumber 表示等于端口号 *portnumber*;

gt portnumber 表示大于 *portnumber*;

lt portnumber 表示小于 *portnumber*;

neq portnumber 表示不等于 *portnumber*;

range portnumber1 portnumber2 表示介于端口号 *portnumber1* 和 *portnumber2* 之间在指定 *portnumber* 时,对于用相应的助记符来代替其实际数字。例:

```
// 允许 129.8.0.0/16 网段的主机访问 202.39.160.0/24 网段上主机的 WWW 服务
```

```
access-list 100 permit tcp 129.8.0.0 0.0.255.255 202.39.160.0 0.0.0.255 eq www
// 禁止所有主机访问 202.38.160.1 的主机的 telnet 服务
access-list 101 deny tcp any 202.39.160.1 0.0.0.0 eq telnet
// 禁止从 129.9.8.0/24 网段内的主机建立与 202.38.160.0/24 网段内的主机的端口大于 128 的 UDP（用户数据报协议）的连接
access-list 102 deny udp 129.8.0.0 0.0.0.255 202.38.160.0 0.0.0.255 gt 128
```

用户通过添加适当的访问规则，就可以利用包过滤来对通过路由器的 IP 包进行检查，从而过滤掉用户不希望通过路由器的包，起到网络安全的作用。

4. 删除访问控制列表

删除访问控制列表的格式为

```
Router(config)#no access-list listnumber
```

5. 显示访问控制列表的信息

显示访问控制列表的信息的格式为

```
Router#show access-list listnumber
```

6. 将访问列表应用于接口

将一个访问控制列表应用于接口的步骤：1) 定义一个访问控制列表；2) 指定应用的接口；3) 用 ip access-group 命令定义数据流的方向。例：

```
// 定义一个访问控制列表，参见定义访问控制列表部分
```

```
// 指定应用的接口
```

```
Router(config)#interface f0/1
```

```
// 将访问控制列表 listnumber 应用于 f0/1 接口的入口方向
```

```
Router(config-if)#ip access-group listnumber in
```

```
// 在 f0/1 接口的入口方向取消访问控制列表 listnumber 的应用
```

```
Router(config-if)#no ip access-group listnumber in
```

```
// 将访问控制列表 listnumber 应用于 f0/1 接口的出口方向
```

```
Router(config-if)#ip access-group listnumber out
```

```
// 在 f0/1 接口的出口方向取消访问控制列表 listnumber 的应用
```

```
Router(config-if)#no ip access-group listnumber out
```

若对进入该接口的数据包进行检查选择 in，若对该接口发出的数据包进行检查选择 out。

1.4.4 课程设计步骤

配置 ACL 的拓扑图如图 1-5 所示。假设 X 为 1 组，Y 为 2 组。根据图 1-5 连接设备，配置路由器和主机，互相连通后，配置 ACL。

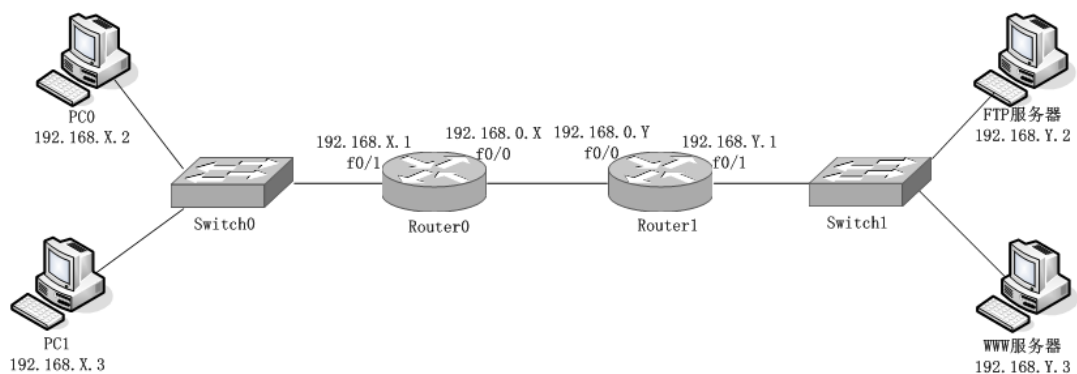


图 1-5 配置 ACL 的拓扑图（注：X、Y 为组号）

1. 根据图 1-5 的拓扑图，连接设备。
2. 配置路由器和主机，使网络之间相互连接。
配置路由器和主机知识，请参考 1.2 路由器配置。
3. 分别开启 FTP 服务和 WWW 服务。
4. 在路由器上进行过滤规则的配置，绑定在相应的接口，实现用访问列表控制 IP 通信。如：拒绝或允许 ping，拒绝或允许计算机访问 FTP、WWW 服务。

1) 配置 ACL101 禁止 ping

// 禁止 192.168.1.0/24 中主机向 192.168.2.0/24 中主机发送 ICMP 报文

```
Router1(config)#access-list 101 deny icmp 192.168.1.0 0.0.0.255 192.168.2.0
0.0.0.255
```

// 允许任何网段的主机向任何网段的主机发送 IP 包

```
Router1(config)#access-list 101 permit ip any any
```

```
Router1(config)#interface f0/1
```

// 绑定 ACL101 在接口 f0/1 的出口方向

```
Router1(config-if)#ip access-group 101 out
```

```
Router1(config-if)#
```

2) 配置 ACL100 禁止访问 WWW

// 禁止 192.168.1.0 网段的主机访问 192.168.2.0 网段的主机的 WWW 服务

```
Router1(config)#access-list 100 deny tcp 192.168.1.0 0.0.0.255 192.168.2.0
0.0.0.255 eq www
```

```
Router1(config)#access-list 100 permit ip any any
```

```
Router1(config)#interface f0/1
```

// 绑定 ACL100 在接口 f0/1 的出口方向

```
Router1(config-if)#ip access-group 100 out
```

```
Router1(config-if)#
```

注意：

- 1) 在定义访问控制列表时，要特别注意语句输入的先后顺序，因为路由器在执行该列表时的顺序是自上而下的。
- 2) 路由器不对自身产生的 IP 包进行过滤，在实验时应由其他的设备发包进行测试。

1.5 DHCP 配置

1.5.1 课程设计目的

本课程设计的主要目的是理解动态主机配置协议（DHCP）的工作过程，掌握配置 DHCP 的方法。

1.5.2 课程设计内容

本课程设计要求配置 DHCP，具体的内容如下：

- 1) 规划 IP 地址池，配置 DHCP 服务器和客户；

- 2) 对 DHCP 进行测试。

1.5.3 相关知识

动态主机配置协议 (DHCP) 是一种动态的向 Internet 终端提供配置参数的协议。在终端提出申请之后, DHCP 可以向终端提供 IP 地址、网关和 DNS 服务器地址等参数。它是 Bootstrap 协议的一种扩展, 基于 UDP 协议, 客户的端口号是 68, 服务器的端口号是 67。采用 DHCP 的好处是减少错误, 减少网络管理。

DHCP 工作过程:

- 1) 客户在本网段内广播 DHCP DISCOVER 报文已发现网络中的 DHCP 服务器, DHCP Relay 可将此报文广播到其他的网段;

- 2) 服务器向客户回应请求, 并给出一个可用的 IP 地址。此地址并非真的被分配。但在给出此地之前, 应当用 ICMP ECHO REQUEST 报文进行检查;

- 3) 如果收到多个 DHCP OFFER 报文, DHCP 客户会根据报文的内容从其中选择一个给与响应。如果客户之前曾经获得过一个 IP 地址, 她会将此地址写在 DHCP REQUEST 报文的 OPTIONS 域的 “REQUESTD IP ADDRESS” 中发给服务器;

- 3) 当收到 DHCP REQUEST 报文后, 服务器将客户的网络的(网络地址, 硬件地址)同分配的 IP 地址绑定, 在将 IP 地址发送给客户;

- 4) 当收到 DHCP REQUEST 报文后, 如果发现其申请的地址无法被分配, 则用 DHCP NAK 报文回应;

- 5) 客户收到 DHCP ACK 报文后, 再对所有的参数进行一次最后的检查, 如果发现由地址冲突存在, 则使用 DHCP DECLINE 报文回复服务器;

- 6) 如果客户放弃现在使用的 IP 地址, 则她使用 DHCP RELEASE 报文通知服务器, 服务器将此地址回收以备下次使用;

- 7) 当客户的地址到达 50%租用期 (T1) 时, 客户进入 RENEW 状态, 使用 DHCP REQUEST 报文续约;

- 8) 当客户的地址到达 87.5%租用期(T2)时, 客户进入 REBINDING 状态, 使用 DHCPREQUEST 报文续约。

交换机配置常用命令, 请参考 1.3 VLAN 配置。

1.5.4 课程设计步骤

Cisco3750 交换机配置 DHCP 拓扑图如图 1-6 所示。在交换机上将端口 f0/1 - 4 划分给 vlan10, IP 地址段为 192.168.10.0, 子网掩码 255.255.255.0, 默认网关为 192.168.10.1, DNS 服务器为 192.168.10.2。DHCP 保留 192.168.10.1-192.168.10.5 不分配。

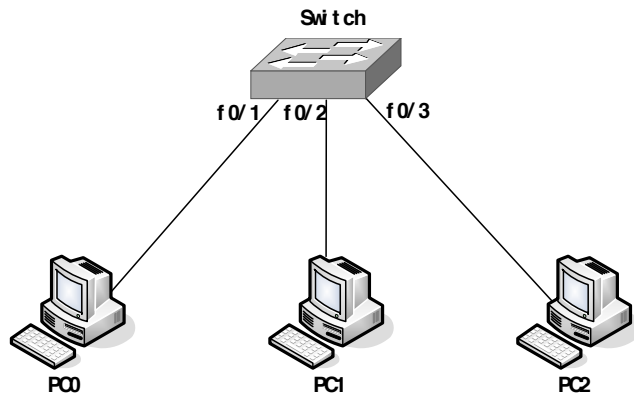


图 1-6 DHCP 拓扑图

1. 建立 VLAN

// 进入特权模式

```
Switch>enable
```

// 配置 VLAN10, 端口范围 1-4

```
Switch#config t
```

```
Switch(config)#vlan 10
```

```
Switch(config-vlan)#exit
```

```
Switch(config)#interface range f0/1 - 4
```

```
Switch(config-if-range)#switchport mode access
```

```
Switch(config-if-range)#switchport access vlan 10
```

```
Switch(config-if-range)#end
```

2. 设置 VLAN IP 地址

```
Switch#config t
```

```
Switch(config)#interface vlan 10
```

```
Switch(config-if)#ip add 192.168.10.1 255.255.255.0
```

```
Switch(config-if)#end
```

3. 配置交换机将其作为 DHCP 服务器

```
Switch#config t
```

// DHCP 地址池中需要排除的地址, 如: 默认网关地址和 DNS 等

```
Switch(config)#ip dhcp excluded-address 192.168.10.1 192.168.10.5
```

// 建立名为 abc 的地址池

```
Switch(config)#ip dhcp pool abc
```

// 分配地址池可用的 IP 范围

```
Switch(dhcp-config)#network 192.168.10.0 255.255.255.0
```

// 分配给 PC 的默认网关

```
Switch(dhcp-config)#default-router 192.168.10.1
```

// 分配给 PC 的 DNS

```
Switch(dhcp-config)#dns-server 192.168.10.2
```

// 设置租期为无限时间 (路由器上默认是 24h)

```
Switch(dhcp-config)#lease infinite
```

4. 检查 PC 是否获取到地址, 能否 ping 默认网关。

在 PC 上通过 ipconfig 观察其是否获得 IP 地址、默认网关等;

在交换机上通过 Switch#show ip dhcp binding 命令显示地址分配情况，观察 PC 是否获得 IP 地址；

通过 ping 命令观察是否 ping 通默认网关。

第二章 网络协议分析

在实际网络环境中，使用网络数据包分析工具捕获网络中的数据包，并显示数据包的尽可能详细的情况，从而了解协议的运行机制。本章通过 Wireshark 软件的安装使用，捕获在局域网中传输的数据包，并结合在计算机网络课程中学习到的理论知识，对以太网、ARP、IP、ICMP、UDP、TCP 和 HTTP 等常用网络协议进行分析，能够更好地理解它们的数据包结构，掌握它们的工作过程，加深网络协议的理解和掌握。

2.1 Wireshark 使用

2.1.1 课程设计目的

本课程设计的主要目的是学会安装和使用 Wireshark，了解使用 Wireshark 分析 TCP/IP 的主要网络协议的方法，加深数据包结构、协议层次和协议交互过程的理解。

2.1.2 课程设计内容

利用 Wireshark 捕获和分析网络中的数据包，课程设计内容如下：

- 1) Wireshark 安装；
- 2) 启动数据包的捕获，跟踪 PC 之间的报文，并存入文件以备重新查；
- 3) 设置过滤器过滤网络报文以检测特定数据流；
- 4) 对常用协议的数据包的格式进行分析，利用协议分析软件的统计工具显示网络报文的各种统计信息。

2.1.3 相关知识

Wireshark 是目前最好的、开源的、获得广泛应用的网络数据包分析工具之一。它主要应用于网络问题解决、安全隐患检测、协议执行情况测试和网络协议学习等。它具有的特性是支持 UNIX 和 Windows 平台、在接口实时捕获包、能详细显示数据包的详细协议信息、可以打开/保存捕获的包、可以导入导出其他捕获程序支持的包数据格式、可以通过多种方式过滤包、多种方式查找包、通过过滤以多种色彩显示包以及创建多种统计分析等。

Wireshark 的主窗口如图 2-1 所示。这个打开捕获包以后的界面。Wireshark 主窗口由如下部分组成：

- 1) 菜单 (menu) 用于开始操作；
- 2) 主工具栏 (main toolbar) 提供快速访问菜单中经常用到的项目的功能；
- 3) 过滤工具栏 (filter toolbar) 提供处理当前显示过滤得方法；
- 4) 包列表窗格 (packet list pane) 显示打开文件的每个包的摘要。点击窗格中的单独条目，包的其他情况将会显示在包细节窗格和包字节窗格中；
- 5) 包细节窗格 (packet detail pane) 显示在包列表窗格中选择的包的更多详情；

- 6) 包字节窗格 (packet bytes pane) 显示在包列表窗格所选择的包的数据, 以及在包细节窗格高亮显示的字段;
- 7) 状态栏 (statusbar) 显示当前程序状态以及捕获数据的更多详情。

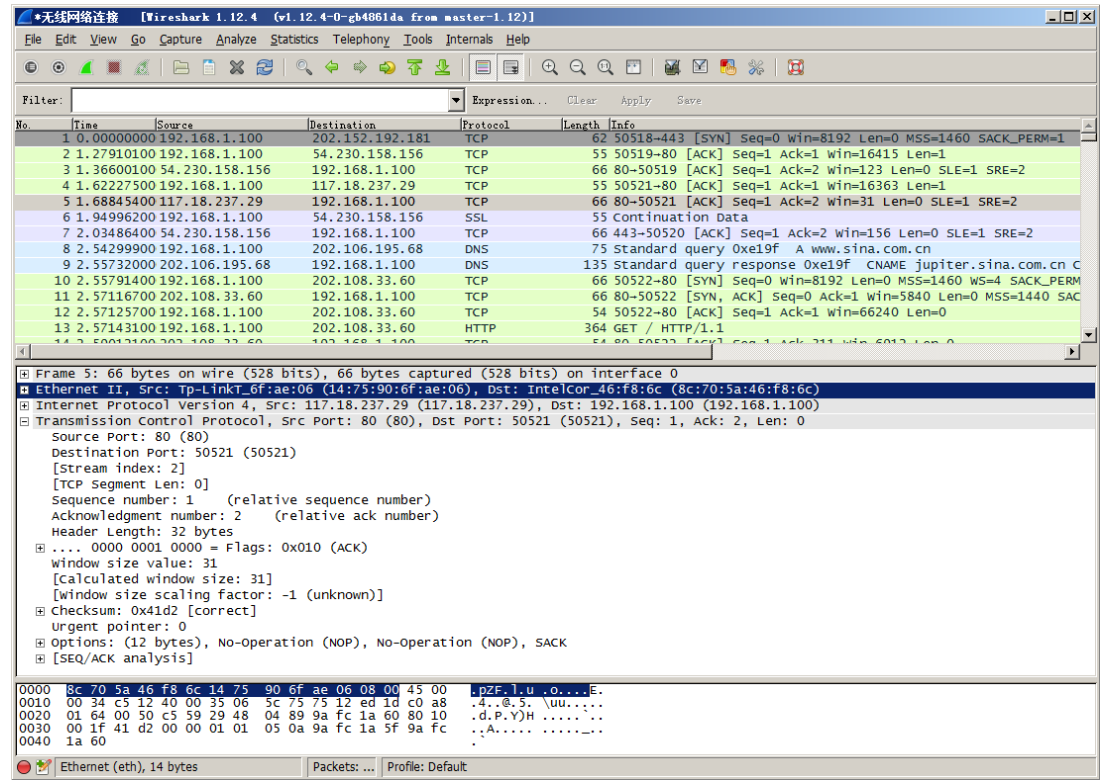


图 2-1 Wireshark 的主窗口

课程设计需要的 Wireshark 详细知识, 请参考 Wireshark User's Guide For Wireshark 1.99。

2.1.4 课程设计步骤

1. 在 Windows 下安装 Wireshark
在 <http://www.wireshark.org/download.html> 中下载后, 安装 Wireshark。Wireshark 安装包包含 WinPcap, 不需要单独下载它, 但是安装过程中需要选择安装 WinPcap。
提示: 如果不了解设置的作用的话, 尽量保持默认设置。
2. 捕获选项设置
选择 Capture 菜单中 Options 按钮, 进入 Capture Options 对话框如图 2-2 所示。
Interface 指定想用于进行捕获的接口。选择想要使用的接口。
Capture packets in promiscuous mode 指定 Wireshark 捕获包时, 设置接口为混杂模式。如果未指定该选项, Wireshark 将只能捕获进出你电脑的数据包 (不能捕获整个局域网段的包)。注意: 如果其他应用程序将网卡设置为混杂模式, 即使不选中该选项, 也会工作于混杂模式。
Capture Filter 指定捕获过滤规则。默认情况下是空的。可以点击 Capture Filter 按钮, 通过弹出的捕获过滤对话框创建或选择一个过滤规则。
File 指定存储捕获数据的文件名。该字段默认是空白。如果保持空白, 捕获数据将会存储在临时文件夹。也可以点击右侧的按钮打开浏览窗口设置文件存储位置。
如果选中 Use multiple files 选项, 指定条件达到临界值, Wireshark 将会自动生成

一个新文件，这个选项不是适用单独文件。

Update list of packets in real time 为在包列表窗格实时更新捕获数据。如果未选定该选项，在 Wireshark 捕获结束之前将不能显示数据。如果选中该选项，Wireshark 将生成两个独立的进程，通过捕获进程传输数据给显示进程。

Automatic scrolling in live capture 指定 Wireshark 在有数据进入时实时滚动包列表窗格，这样将一直能看到最近的包。反之，则最新数据包会被放置在行末，但不会自动滚动窗格。如果未设置“update list of packets in real time”，该选项将是灰色不可选的。

如果选中 Hide capture info dialog 选项，将会隐藏捕获信息对话框。

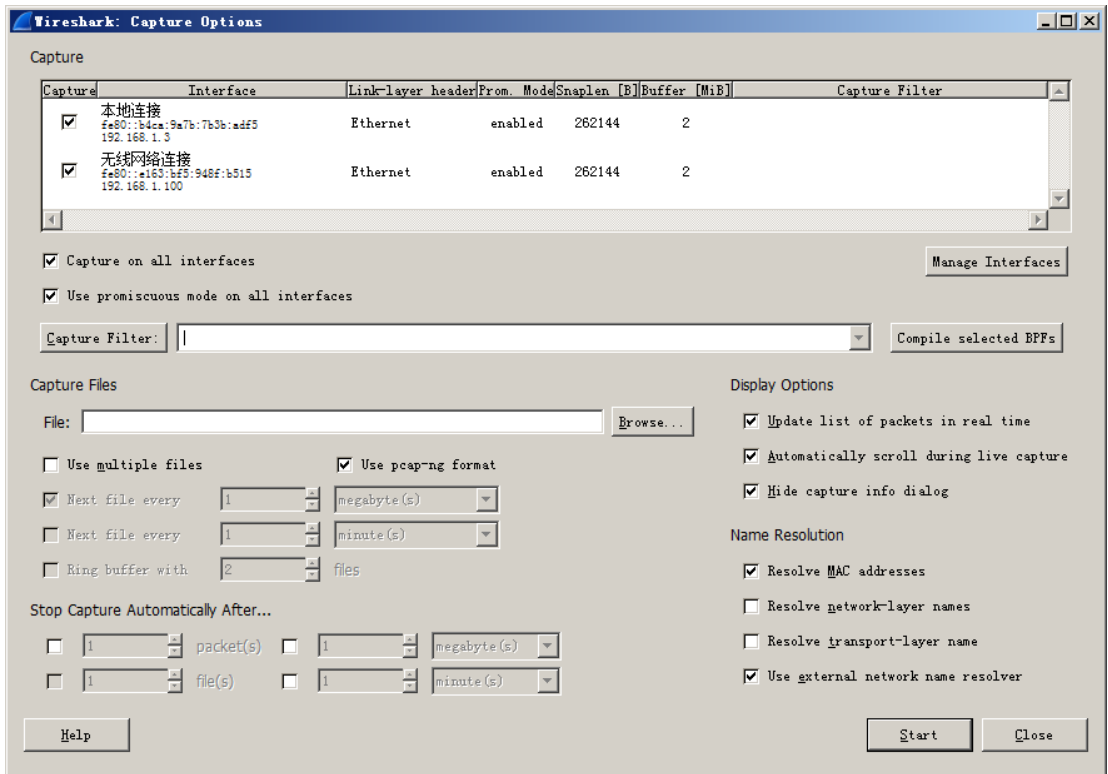


图 2-2 Capture Options 对话框

3. 捕获时过滤

Wireshark 开始捕获数据包前，设置相应的过滤规则，就可以捕获感兴趣的数据包。要设置过滤规则，选择 Capture 菜单 Capture Filter 按钮，进入 Capture Filter 对话框如图 2-3 所示。

在 Capture Filter 对话框选择所需的过滤规则后，单击 OK 按钮，再单击 Capture Options 对话框中的 start 按钮，就开始捕获数据包了。

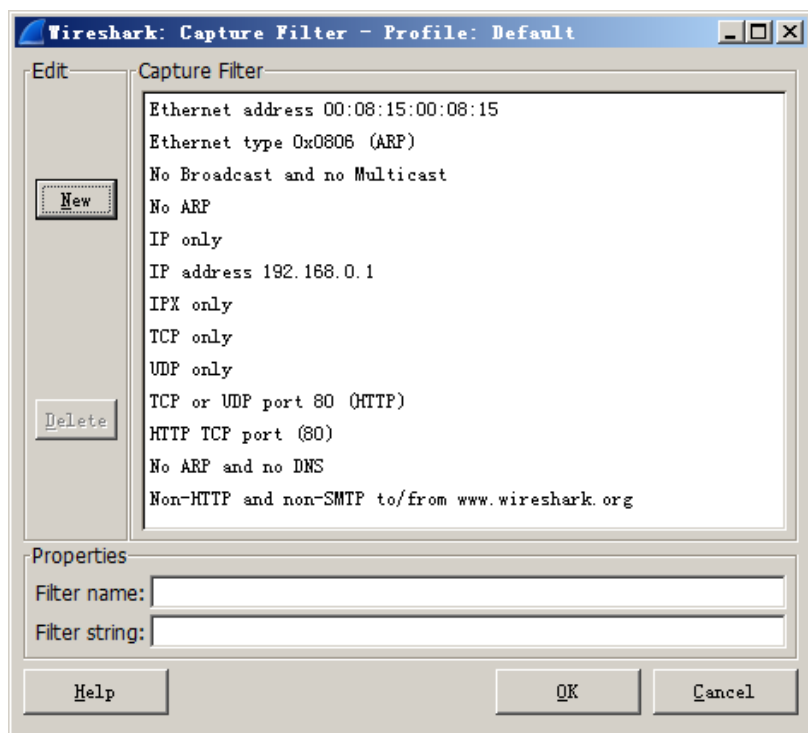


图 2-3 Capture Filter 对话框

设置过滤规则时可以选择已有的过滤规则，也可以添加新过滤规则。在 Wireshark 中添加新过滤规则时需要为其指定名称和规则，点击 New 按钮就可以添加过滤规则。例：添加一条过滤规则，用于捕获来自或指向 192.168.1.5 的 telnet 协议的通信，可以点击 New 按钮，然后在 Filter string 文本框中输入 tcp port 23 and host 192.168.1.5，在 Filter name 文本框中输入 telnet5。

Wireshark 使用 libpcap 过滤语句进行捕获过滤，类似于 tcpdump 捕获过滤语言。捕获过滤的形式为 [not] *primitive* [and|or [not] *primitive* ...]。

其中 primitive 通常是下面中的一个：

[src|dst] *host* <*host*>：允许过滤主机 ip 地址或名称。可以优先指定 src|dst 关键词来指定关注的是源地址还是目标地址。如果未指定，则指定的地址出现在源地址或目标地址中的包会被抓取。

ether [src|dst] *host* <*host*>：允许你过滤主机以太网地址。可以优先指定关键词 src|dst 在关键词 ether 和 host 之间，来确定关注的是源地址还是目标地址。如果未指定，同上。

gateway *host* <*host*>：过滤通过指定 host 作为网关的包。这就是指那些以太网源地址或目标地址是 host，但源 ip 地址和目标 ip 地址都不是 host 的包。

[src|dst] *net* <*net*> [{*mask* <*mask*>}|{*len* <*len*>}]：通过网络号进行过滤。可以选择优先指定 src|dst 来确定感兴趣的是源网络还是目标网络。如果两个都没指定。指定网络出现在源还是目标网络的都会被选择。另外，可以选择子网掩码或者 CIDR(无类别域形式)。

[tcp|udp] [src|dst] *port* <*port*>：过滤 tcp 或 udp 及端口号。可以使用 src|dst 和 tcp|udp 关键词来确定来自源还是目标，tcp 协议还是 udp 协议。tcp|udp 必须出现在 src|dst 之前。

less|greater <*length*>：选择长度符合要求的包。（大于等于或小于等于）。

ip|ether *proto* <*protocol*>：选择有指定的协议在以太网层或是 ip 层的包

ether|ip broadcast|multicast：选择以太网/ip 层的广播或多播

<expr> relop <expr>: 创建一个复杂过滤表达式，来选择包的字节或字节范围符合要求的包。

捕获时过滤的详细知识，请参考 Wireshark User's Guide For Wireshark 1.99 中 4.13 捕获时过滤部分。

4. 显示时过滤

Wireshark 捕获数据包后设置相应的过滤规则，就可以显示感兴趣的数据包。选择主窗口 Filter 按钮，进入 Display Filter 对话框如图 2-4 所示，单击 Apply 按钮，就会显示过滤后的数据包了。

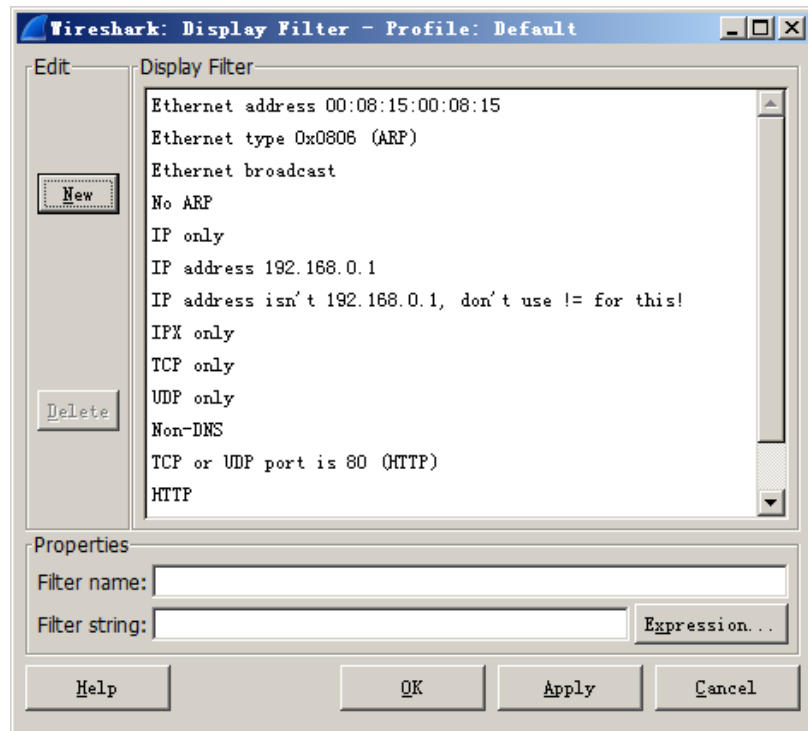


图 2-4 Display Filter 对话框

显示时过滤的详细知识，请参考 Wireshark User's Guide For Wireshark 1.99 中 6.3 显示时过滤部分。

5. 数据包分析

已经捕获完成之后，或者打开先前保存的包文件时，通过点击包列表窗格中的包，可以在包详情窗格看到关于这个包树状结构以及字节窗格。

通过点击左侧“+”标记，可以展开树状视图的任意部分。可以在窗格点击任意字段来选择它。

Wireshark 选择了一个 TCP 包后的界面如图 2-5 所示。图中显示的就是选中 TCP 头部。同样可以选择 TCP 头的头部长度(20 字节)，同时以十六进制形式显示数据包的具体内容会出现在下方的字节浏览窗格中。

图 2-5 中包细节窗格显示的是协议树。通过这棵树可以看到被捕获的数据包更多信息。如：本主机的 MAC 地址为 8C:70:5A:46:F8:6C，IP 地址为 192.168.1.100，TCP 的端口号为 50052。

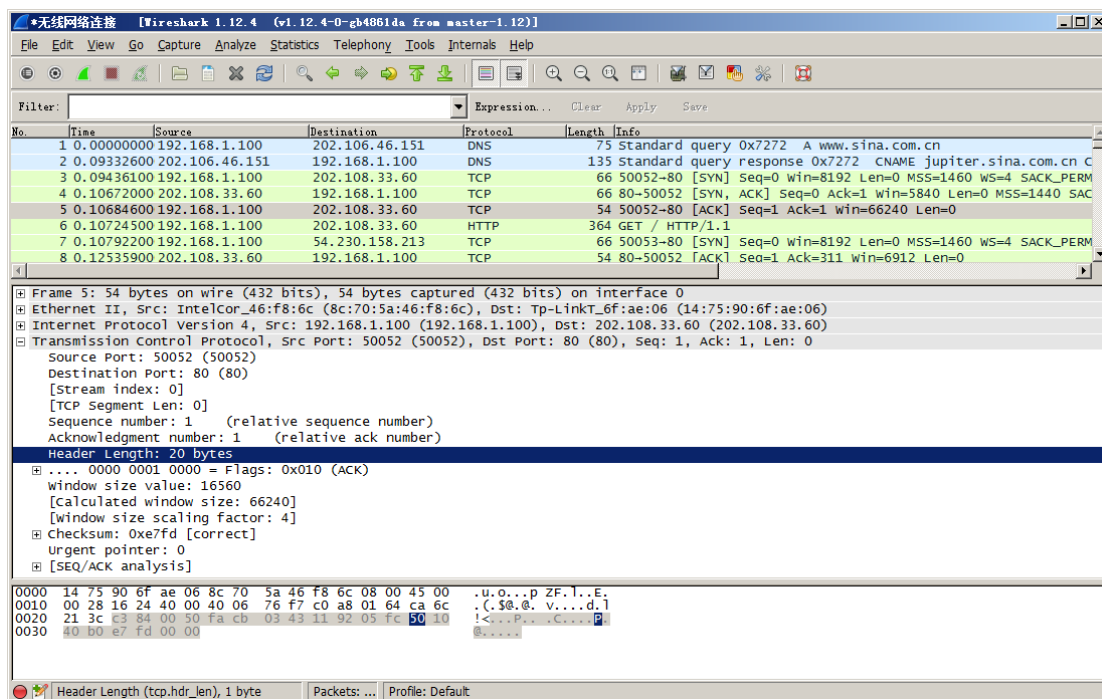


图 2-5 “Wireshark 选择了一个 TCP 包后

6. TCP 流跟踪

在包列表窗格中选择一个 TCP 数据包，然后选择 Analyze 菜单中 Follow TCP Stream 按钮，进入 Follow TCP Stream 对话框，按顺序显示这个 TCP 流的所有数据，如图 2-6 所示。

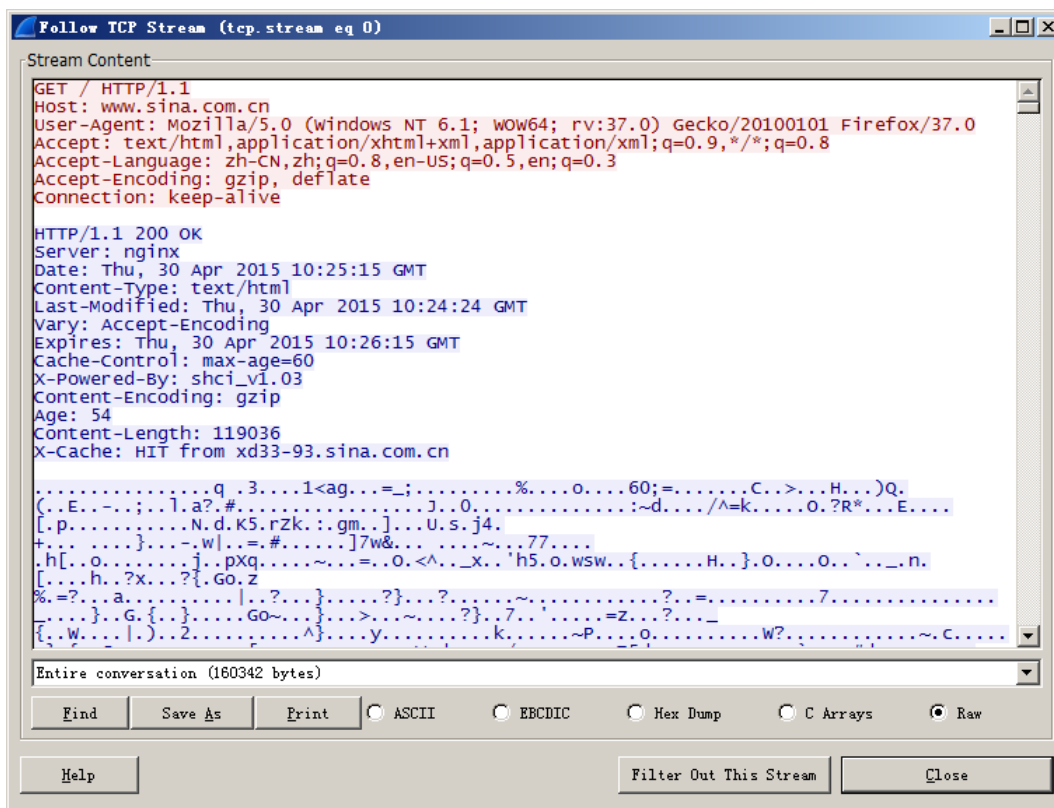


图 2-6 Follow TCP Stream 对话框

7. 捕获包保存

选择 File 菜单中 Save As... 按钮可以将捕获的数据包保存为文件。

8. 统计

可以通过 Statics 菜单实现统计。包括载入文件的基本信息(比如包的数量)，对指定协议的统计(例如，统计包文件内 HTTP 请求和响应数)等。

9. 思考题

- 1) 捕获和分析网络中的数据包，请写出至少 6 种不同的应用层协议。
- 2) 根据图 2-5，请写出服务器 www.sina.com.cn 的 IP 地址是什么？
- 3) 根据图 2-5，请写出本主机的 IP 地址和 MAC 地址分别是什么？
- 4) 根据图 2-5，请写出默认网关的 MAC 地址是多少？
- 5) 根据图 2-5，请写出本主机的 DNS 服务器的 IP 地址是什么？
- 6) 根据图 2-5，请写出从发出 TCP SYN 报文到收到 TCP ACK 报文的时间？

2.2 以太网协议分析

2.2.1 课程设计目的

本课程设计的主要目的是理解以太网帧结构，并了解 IEEE802.3 帧结构和 TCP/IP 的主要协议和协议的层次结构。

2.2.2 课程设计内容

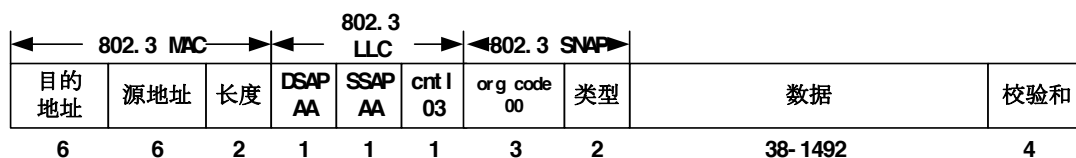
利用 Wireshark 捕获和分析以太网帧，课程设计内容如下：

- 1) 启动数据包的捕获，观察和分析所捕获的以太网帧格式；
- 2) 观察所捕获的 IEEE802.3 帧格式和以太网帧中数据部分有哪些协议。

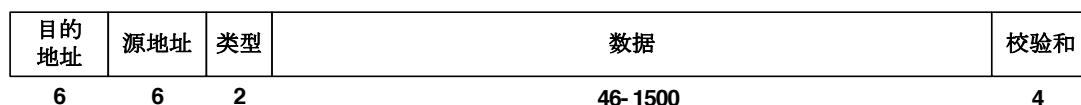
2.2.3 相关知识

70 年代 Xerox 公司的 Bob Metcalfe 和 David Boggs 设计和实现了一个 2.94Mbps 的采用 CSMA/CD 协议的以太网。1978 年 DEC、Intel、Xerox 拟定了 10Mbps 的以太网标准，称为 DIX 标准。1983 年经过小的修改，DIX 标准变成 IEEE 802.3 标准。

IEEE802.3 帧格式和 DIX 以太网帧格式如图 2-7 所示。



(a) IEEE802.3 帧格式



(b) DIX 以太网帧格式

图 2-7 IEEE802.3 和以太网帧格式

目的地址和源地址。这两种帧格式的目的地址和源地址都是 MAC 地址，占 6 字节。

接下来的 2 个字节在两种帧格式中互不相同。IEEE 802.3 的长度是指它后续数据的字节长度，但不包括 CRC 检验码。IEEE 802.3 的类型则由后面的子网接入协议（SNAP）的首部给出。以太网类型定义了后续数据的类型。

以太网的类型后就是数据。IEEE 802.3 的长度后是 3 字节的 IEEE 802.3 LLC 和 5 字节的 IEEE 802.3 SNAP。IEEE 802.3 LLC 的目的服务访问点（DSAP）和源服务访问点（SSAP）的值都为 0xaa，cntl 的值为 3。IEEE 802.3 SNAP 的 3 个字节 org code 的值为 0，2 个字节类型和以太网帧格式一样。

CRC 用于帧内后续字节差错的循环冗余码检验。

IEEE 802.3 标准定义的帧和以太网的帧都有最小长度要求。IEEE 802.3 规定数据部分必须至少为 38 字节，而对于以太网，则要求最少要有 46 字节。为了保证这一点，必须在不足的空间插入填充字节。

主机需求 RFC 对每台主机要求：

- 1) 必须能发送和接收采用 RFC 894（以太网）封装格式的数据包；
- 2) 应该能接收与 RFC 894 混合的 RFC 1042（IEEE 802.3）封装格式的数据包；
- 3) 也许能够发送采用 RFC 1042 格式封装的数据包。如果主机能同时发送两种类型的数据包，那么发送的数据包必须是可以设置的，而且默认条件下必须是 RFC 894 数据包。

2.2.4 课程设计步骤

在主机上启动 Wireshark，准备捕获网络数据帧，然后访问一些 Web 网站。

1. 查看捕获的以太网帧，找出这些以太网帧中长度最小的帧。并查看这个帧有哪些字段以及各个字段的值。
2. 找出捕获的以太网帧中长度最长的帧。并查看这个帧的长度是多大？为什么？
3. 写出捕获的以太网帧中类型的值。观察这些协议的层次结构。
4. 捕获 IEEE802.3 帧，并观察 IEEE802.3 与以太网之间帧格式的区别。

2.3 ARP 协议分析

2.3.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 ARP 报文，理解 ARP 报文格式，掌握 ARP 工作原理。

2.3.2 课程设计要求

利用 Wireshark 捕获和分析 ARP 报文，课程设计要求如下：

- 1) 捕获数据包，观察 ARP 工作过程；
- 2) 观察和分析所捕获的 ARP 报文格式。

2.3.3 相关知识

当将要发送来自网络层的数据时，需要知道目的主机的硬件地址，这就需要通过 IP 地址找到相应的硬件地址，网络接口输出函数会调用 ARP 协议进行 ARP 解析。ARP 解析函数广播 ARP 请求，目标主机收到 ARP 请求后，以单播形式发送 ARP 响应，当发送方主机收到 ARP 响应后就可发送数据。

为了避免频繁发送 ARP 请求和响应，实现时都会有一个 ARP 高速缓存，用来存放已经解析成功的 ARP 信息，所以通常数据发送前先查找 ARP 高速缓存，找不到时才会发送 ARP 请求。

在以太网上解析 IP 地址时，ARP 请求和响应的格式如图 2-8 所示。

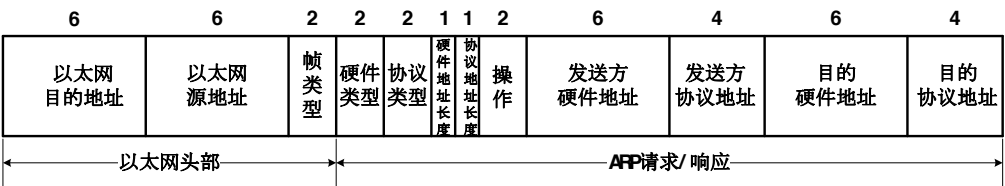


图 2-8 以太网 ARP 请求/响应的格式

以太网目的地址和以太网源地址。目的 MAC 地址为全 1 的特殊地址是广播地址。以太网上的所有以太网接口都要接收这个广播的数据帧。

类型表示后面数据的类型。对于 ARP 请求或响应，该字段的值为 0x0806。

硬件类型表示硬件地址的类型。它的值为 1 表示以太网地址。

协议类型表示要映射的协议地址类型。它的值为 0x0800 表示 IP 地址。

硬件地址长度和协议地址长度分别表示硬件地址和协议地址的长度，以字节为单位。对于以太网 ARP 请求或响应来，它们的值分别为 6 和 4。

操作字段指出四种操作类型，分别是 ARP 请求（值为 1）、ARP 响应（值为 2）、RARP 请求（值为 3）和 RARP 响应（值为 4）。

发送方硬件地址在本例中是 MAC 地址，发送方协议地址在本例中是 IP 地址，目的硬件地址在本例中是 MAC 地址，和目的协议地址在本例中是 IP 地址。当主机收到目的 IP 地址为本机的 ARP 请求后，它就把硬件地址填进去，然后用目的硬件地址和目的协议地址分别替换发送方硬件地址和发送方协议地址，并把操作字段置为 2，最后把它发送回去。

ARP 协议工作过程：每台主机都会建立一个 ARP 高速缓存，以表示 IP 地址和 MAC 地址的对应关系。当发送方主机需要将一个数据包发送到目的主机时，会检查自己 ARP 高速缓存中是否存在这个 IP 地址对应的 MAC 地址，如果有，就直接将数据包发送到这个 MAC 地址；如果没有，就向本以太网中发起一个 ARP 请求的广播，查询此目的主机对应的 MAC 地址。这个 ARP 请求里包括发送方主机的 IP 地址、MAC 地址、以及目的主机的 IP 地址。网络中所有的主机收到这个 ARP 请求后，会检查数据包中的目的 IP 地址是否和自己的 IP 地址一致。如果不相同，就忽略此数据包；如果相同，该主机首先将发送方的 MAC 地址和 IP 地址添加到自己的 ARP 高速缓存中，如果 ARP 高速缓存中已经存在该 IP 的信息，则将其覆盖，然后给发送方主机发送一个 ARP 响应，告诉对方自己是它需要查找的 MAC 地址；发送方主机收到这个 ARP 响应后，将得到的目的主机的 IP 地址和 MAC 地址添加到自己的 ARP 高速缓存中，并利用此信息开始数据的传输。如果源主机一直没有收到 ARP 响应，表示 ARP 查询失败。

2.3.4 课程设计步骤

主机 PC1 和主机 PC2 通过交换机连接在同一网络上，它们运行的操作系统都是

Windows7，主机上的防火墙已关闭。ARP 协议分析网络拓扑图如图 2-9 所示。

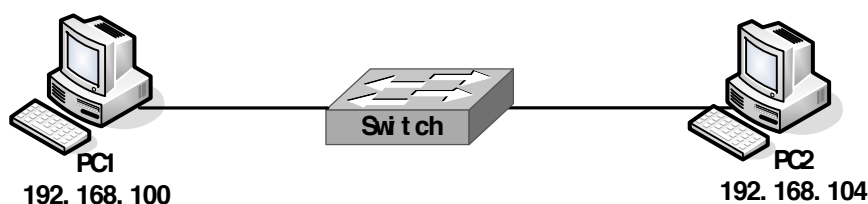


图 2-9 ARP 协议分析网络拓扑图

1. ARP 工作过程

使用 `arp -d` 命令删除主机 PC1 上的所有 ARP 表项，并执行 `ping 192.168.1.104` 命令，使用 Wireshark 捕获 ARP 报文，观察 ARP 工作过程。分别在这两台主机上执行 `arp -a` 命令，观察 ARP 高速缓存中是否存在对方的 IP 地址和 MAC 地址。根据看到的现象解释 ARP 工作过程。

观察到的 ARP 工作过程如图 2-10 所示。

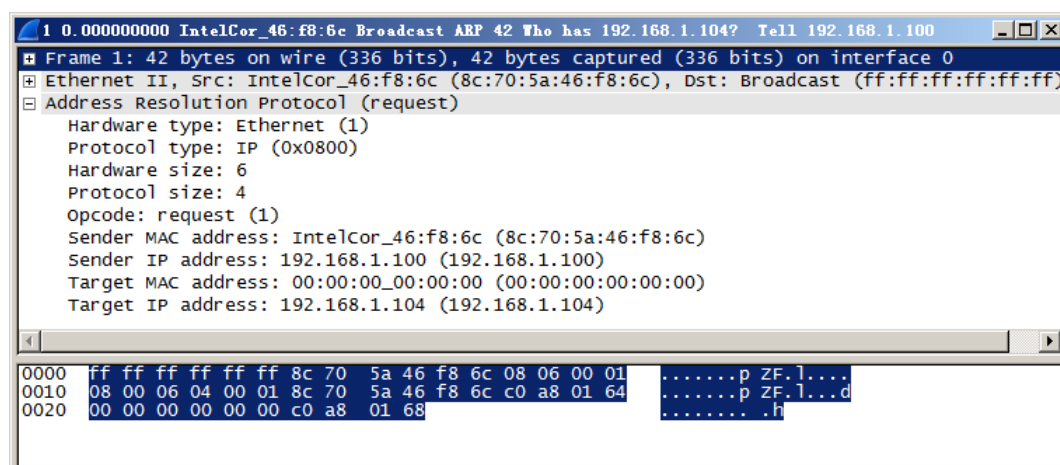
No.	Time	Source	Destination	Protocol	Length	Info
1	0.00000000	IntelCor_46:f8:6c	Broadcast	ARP	42	who has 192.168.1.104? Tell 192.168.1.100
2	0.19314800	HonHaiPr_28:e5:cf	IntelCor_46:f8:6c	ARP	42	192.168.1.104 is at b8:76:3f:28:e5:cf
3	0.19318800	192.168.1.100	192.168.1.104	ICMP	74	Echo (ping) request id=0x0001, seq=48/12288, ttl=64 (reply in 4)
4	0.19797600	192.168.1.104	192.168.1.100	ICMP	74	Echo (ping) reply id=0x0001, seq=48/12288, ttl=64 (request in 3)
5	1.01173700	192.168.1.100	192.168.1.104	ICMP	74	Echo (ping) request id=0x0001, seq=49/12544, ttl=64 (reply in 6)
6	1.01672100	192.168.1.104	192.168.1.100	ICMP	74	Echo (ping) reply id=0x0001, seq=49/12544, ttl=64 (request in 5)
7	2.01008900	192.168.1.100	192.168.1.104	ICMP	74	Echo (ping) request id=0x0001, seq=50/12800, ttl=64 (reply in 8)
8	2.01469100	192.168.1.104	192.168.1.100	ICMP	74	Echo (ping) reply id=0x0001, seq=50/12800, ttl=64 (request in 7)
9	3.00841400	192.168.1.100	192.168.1.104	ICMP	74	Echo (ping) request id=0x0001, seq=51/13056, ttl=64 (reply in 10)
10	3.01149700	192.168.1.104	192.168.1.100	ICMP	74	Echo (ping) reply id=0x0001, seq=51/13056, ttl=64 (request in 9)

图 2-10 ARP 工作过程

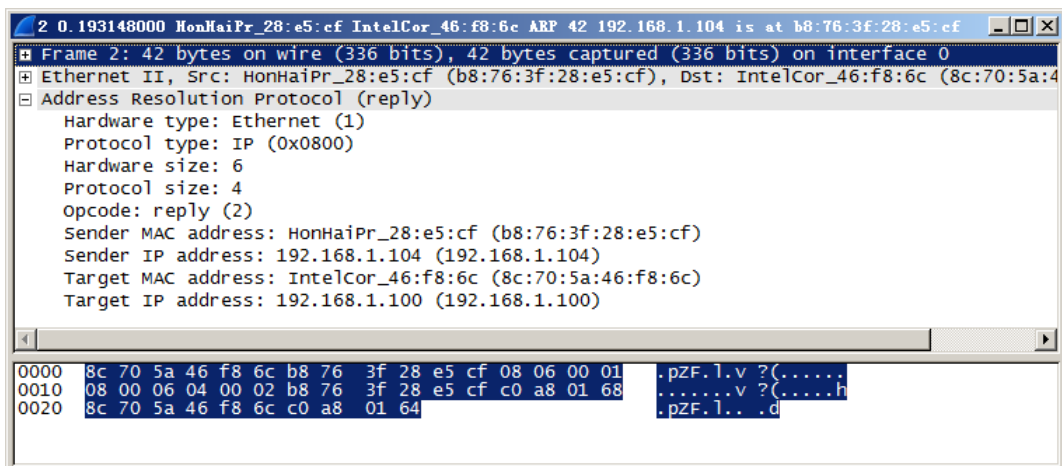
ARP 工作过程：主机 PC1 执行 `ping 192.168.1.104` 命令时，查找 ARP 高速缓存，没有找到 192.168.1.104 对应的表项，于是向网络中广播 ARP 请求报文（图 2-10 中第一个报文），PC2 收到 ARP 请求报文后返回 ARP 响应报文（图 2-10 中第二个报文），告诉 PC1 自己的 MAC 地址。以后 PC1 发送 ICMP 回射请求报文（图 2-10 中第三个报文），PC2 返回 ICMP 回射响应报文（图 2-10 中第四个报文），由于 ARP 高速缓存中已有 192.168.1.104 对应的表项，再发送 ICMP 回射请求报文就不用发送 ARP 请求了。

2. ARP 报文格式

通过 Wireshark 捕获的 ARP 请求报文和 ARP 响应报文，并观察这两种报文的格式。图 2-11 显示了 ARP 请求报文和 ARP 响应报文。



(a) ARP 请求报文



(b) ARP 响应报文

图 2-11 ARP 请求和响应报文

ARP 请求报文和 ARP 响应报文的格式相同地方：硬件类型都是 1，协议类型都是 0x0800，硬件地址长度都是 6，协议地址长度都是 4。

它们不同地方：ARP 请求报文的操作码是 1，发送方 MAC 地址为 8c:70:5a:46:f8:6c，发送方 IP 地址 192.168.1.100，目的 MAC 地址是 00:00:00:00:00:00，目的 IP 地址是 192.168.1.104；ARP 响应报文的操作码是 2，发送方 MAC 地址为 b8:76:3f:28:e5:cf，发送方 IP 地址 192.168.1.104，目的 MAC 地址是 8c:70:5a:46:f8:6c，目的 IP 地址是 192.168.1.100。

3. 思考题

- 1) 解释 ARP 工作过程。
- 2) 比较 ARP 请求报文和 ARP 响应报文的内容有何不同？

2.4 IP 协议分析

2.4.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 IP 包，理解 IP 包格式，掌握 IP 原理。

2.4.2 课程设计内容

利用 Wireshark 捕获和分析 IP 包，课程设计内容如下：

- 1) 启动数据包的捕获，观察 IP 包格式；
- 2) 观察和分析所捕获 IP 包的分段。

2.4.3 相关知识

IP 协议是 TCP/IP 协议族中最为核心的协议，在网络中的 TCP、UDP、ICMP 及 IGMP 协议的数据包都是基于 IP 数据包来进行传输的。IP 协议通过封装传输层的数据包，并把封装好

的 IP 数据包传递给数据链路层。IP 协议规定了 IP 数据包的统一格式，通过这种方式，便可屏蔽了网络差异，从而为网络通信双方建立了一条透明的数据通道。

IP 头部的格式如图 2-12 所示。

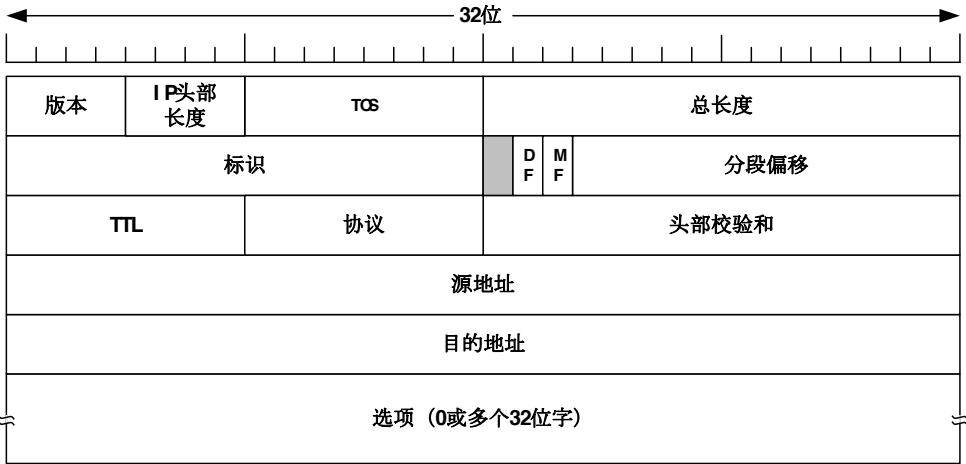


图 2-12 IP 头部的格式

版本的值为 4 表示 IPv4 协议，它的值为 6 表示 IPv6 协议，占 4 位。

头部长度指示 IP 头部的长度，单位为 4 字节，取值范围 5-15(缺省值为 5，即标准头标长 20 字节)，占 4 位。

总长度描述 IP 包的总长（包括 IP 头部和数据），单位为字节，最大包长度为 65535 字节，占 16 位。

标识用于唯一标识该包，占 16 位。

标志占 3 位。第 1 位未定义。第 2 位 DF 表示不分段，DF 为 0 表示该包可分段，否则表示不可分段。第 3 位 MF 表示更多的分段，MF 为 0 表示这是最后分段，否则表示还有后续分段。

分段偏移标明当前分段在原包中的位置。单位为 8 字节，占 13 位。

生存时间(TTL)域表示包的生存时间。实际操作时，包每经过一个路由器，TTL 值减一，当 TTL 值为 0 时，该包被丢弃，占 8 位。

协议(Protocol)表示高层协议类型。如：

- 0 Reserved
- 1 Internet Control Message Protocol (ICMP)
- 2 Internet Group Management Protocol (IGMP)
- 3 Gateway-to-Gateway Protocol (GGP)
- 4 IP (IP encapsulation)
- 5 Stream
- 6 Transmission Control (TCP)
- 8 Exterior Gateway Protocol (EGP)

有关协议号，请参考 <http://www.iana.org/assignments/protocol-numbers>

头部校验和用来检验包头，占 16 位

源地址表示发送方的 IP 地址，占 32 位。

目的地址表示接收方的 IP 地址，占 32 位。

填充(padding)。包头长度必须为 4 字节的整数倍，如果选项的长度不是 4 字节的整数倍，那么就要进行填充。

2.4.4 课程设计步骤

主机 PC1、主机 PC2 和主机 PC3 通过交换机连接在同一网络上，通过路由器连接到互联网上。PC1 和 PC2 运行的操作系统是 Windows7，PC3 运行的操作系统是 CentOS6.5。IP 协议分析网络拓扑图如图 2-13 所示。

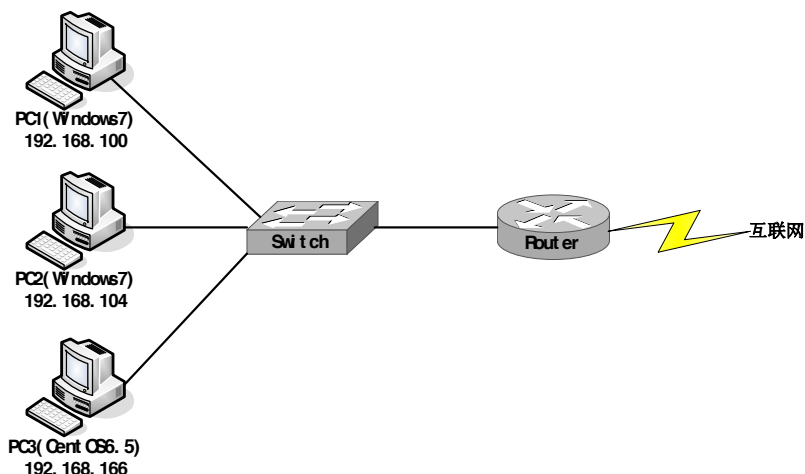


图 2-13 IP 协议分析网络拓扑图

1. IP 包格式

在 PC1 上访问互联网的 Web 网站，ping PC2、PC3、路由器和互联网上的站点，ping -f PC2、PC3、路由器和互联网上的站点，tracert 互联网上的站点。使用 Wireshark 捕获 IP 包，观察 IP 包各字段的值。

观察版本号的值。经观察发现版本号的值为 4，表示 IPv4 协议。

观察头部长度的值一般是多少？头部长度的单位是什么？找出头部长度的值最大是多少？经观察发现长度的值一般是 20 字节，头部长度的单位是 4 字节。

观察 tracert 命令发出一系列数据包中 TTL 字段的值。经观察发现 TTL 从 1 开始，每发送三个数据包 TTL 增加 1。

观察 ping PC2、PC3 和路由器的响应数据包中 TTL 的值。经观察发现 Windows7 和 CentOS6.5 的默认 TTL 的值都是 64。

观察 ping -f 命令发出的数据包中 DF 值为多少？经观察发现 DF=1

观察总长度的值最小是多少？计算总长度时包括 IP 包的哪些部分？

观察不同数据包中标识的值是否相同？经观察发现一般不同。

观察请求数据包和响应数据包中源地址和目的地址有何不同？经观察发现请求数据包中源地址和目的地址分别响应数据包中目的地址和源地址。

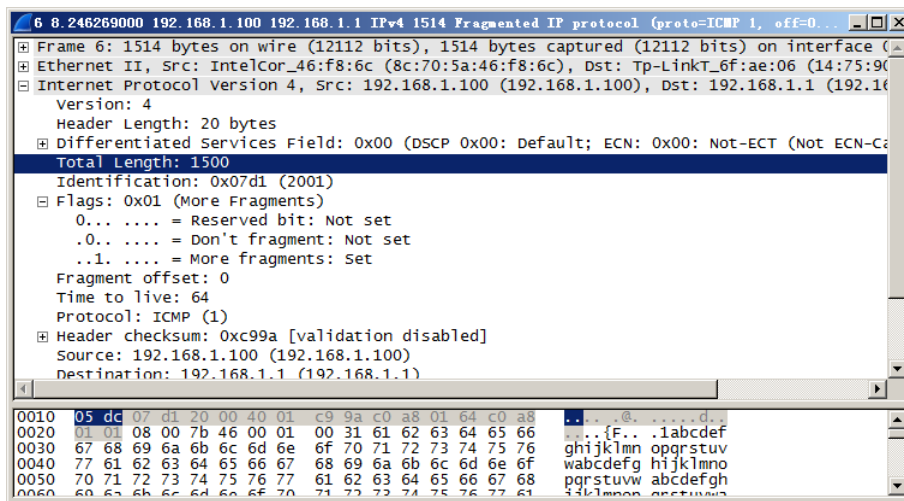
2. IP 分段和重组

1) 运行 ping -f -l 1472 192.168.1.1 和 ping -f -l 1473 192.168.1.1 命令，观察运行结果有何不同。局域网的 MTU 是多少？

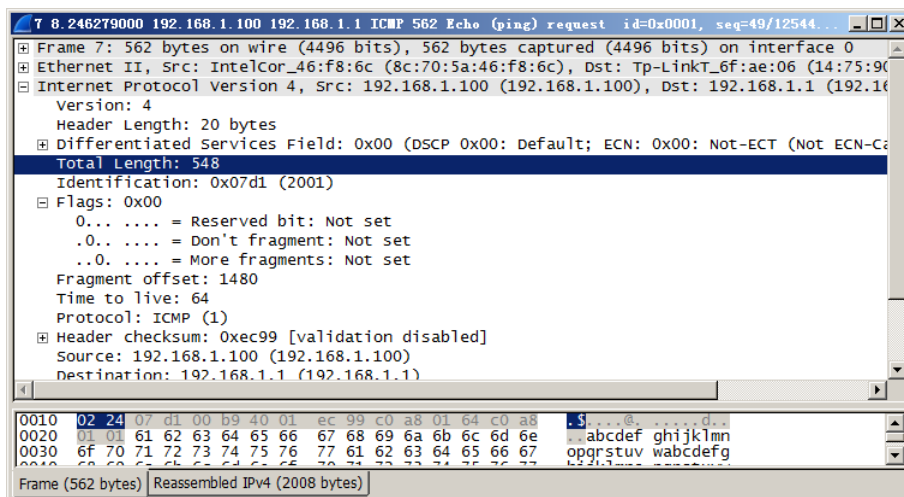
经观察发现第一个命令运行时收到 192.168.1.1 的响应，而第二个命令运行时没有收到响应，并提示需要拆分数据包但是设置 DF。因此，局域网的 MTU 是 $1472+28=1500$ 字节

2) 运行 `ping -l 2000 192.168.1.1` 命令，观察捕获的数据包。

在 Wireshark 中可以看到每个 ICMP 请求都被分成 2 个 IP 分段，如图 2-14 所示。



(a)



(b)

图 2-14 (a) 为 IP 包的第 1 个分段 (b) 为 IP 包的第 2 个分段

图 2-14 中 2 个分段的标识的值是多少？都为 0x07d1

观察第 1 个分段的 DF 和 MF 分别是多少？第 2 个分段的 DF 和 MF 分别是多少？表示什么意思？经观察发现第 1 个分段的 DF 和 MF 分别是 0 和 1，MF=1 表示还有分段；第 2 个分段的 DF 和 MF 都是 0，MF=0 表示第 2 个分段是最后一个分段。

观察第 1 个分段和第 2 个分段的分段偏移分别是多少？分别为 0 和 1480。

2 个分段的总长度为 $1500+548-2*20=2008$ ，比 ping 命令中的参数 2000 多了 8。为什么？因前者还包含了 8 字节的 ICMP 头。

2.5 ICMP 协议分析

2.5.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 ICMP 报文，理解 ICMP 报文格式，掌握 ICMP 原理。

2.5.2 课程设计内容

利用 Wireshark 捕获和分析 ICMP 报文，课程设计内容如下：捕获常用的 ICMP 报文，观察常用 ICMP 报文格式。

2.5.3 相关知识

ICMP 协议通常被认为是 IP 层的一个组成部分。它传递差错报文以及其他需要注意的信息。它通常被 IP 层或更高层协议（TCP 或 UDP）使用。ICMP 的正式规范参见 RFC 792。

ICMP 报文的格式如图 2-15 所示。所有报文的前 4 个字节都是一样的，但是剩下的其他字节则互不相同。



图 2-15 ICMP 报文

类型占 8 位，可以有 15 个不同的值，以描述特定类型的 ICMP 报文。
代码占 8 位，某些 ICMP 报文还使用代码的值来进一步描述不同的条件。
校验和占 16 位，计算范围包括整个 ICMP 报文，计算方法与 IP 首部检验和算法相同。
各种类型的 ICMP 报文如表 2-1 所示，不同类型报文由报文中的类型字段和代码字段来共同决定。表中的最后两列表明 ICMP 报文是一份查询报文还是一份差错报文。因为对 ICMP 差错报文有时需要作特殊处理，需要对它们进行区分。

表 2-1 ICMP 报文类型

类型	代码	名称	查询	差错
0	0	回射响应 (ping 响应)	*	
3		目的地不可达:		
	0	网络不可达		*

	1	主机不可达		*
	2	协议不可达		*
	3	端口不可达		*
	4	需要进行分段但设置了不分段位		*
	5	源站选路失败		*
	6	目的网络不认识		*
	7	目的主机不认识		*
	8	源主机被隔离(作废不用)		*
	9	目的网络被强制禁止		*
	10	目的主机被强制禁止		*
	11	由于服务类型 TOS, 网络不可达		*
	12	由于服务类型 TOS, 主机不可达		*
	13	由于过滤, 通信被强制禁止		*
	14	主机越权		*
	15	优先权中止生效		*
4	0	源端被关闭		*
5		重定向:		
	0	对网络重定向		*
	1	对主机重定向		*
	2	对服务类型和网络重定向		*
	3	对服务类型和主机重定向		*
8	0	回射请求(ping 请求)	*	
9	0	路由器通告	*	
10	0	路由器请求	*	
11		超时:		
	0	传输期间生存时间为 0		*
	1	在数据报组装期间生存时间为 0		*
12		参数问题:		
	0	坏的 IP 首部(包括各种差错)		*
	1	缺少必需的选项		*
13	0	时间戳请求	*	
14	0	时间戳响应	*	
15	0	信息请求(作废不用)	*	
16	0	信息响应(作废不用)	*	
17	0	地址掩码请求	*	
18	0	地址掩码响应	*	

ICMP 报文封装成一个 IP 数据包的格式, 如图 2-16 所示。

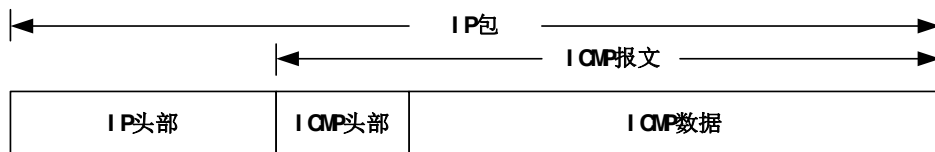


图 2-16 ICMP 封装

2.5.4 课程设计步骤

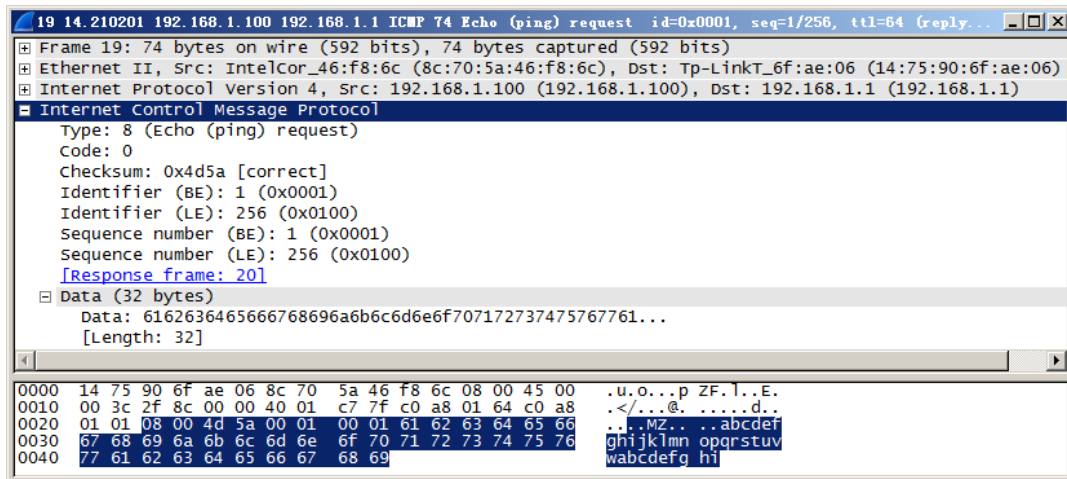
下面将观察几种常见的 ICMP 报文，如：回射请求、回射响应和超时报文等。

1. 观察 ICMP 回射请求和响应报文

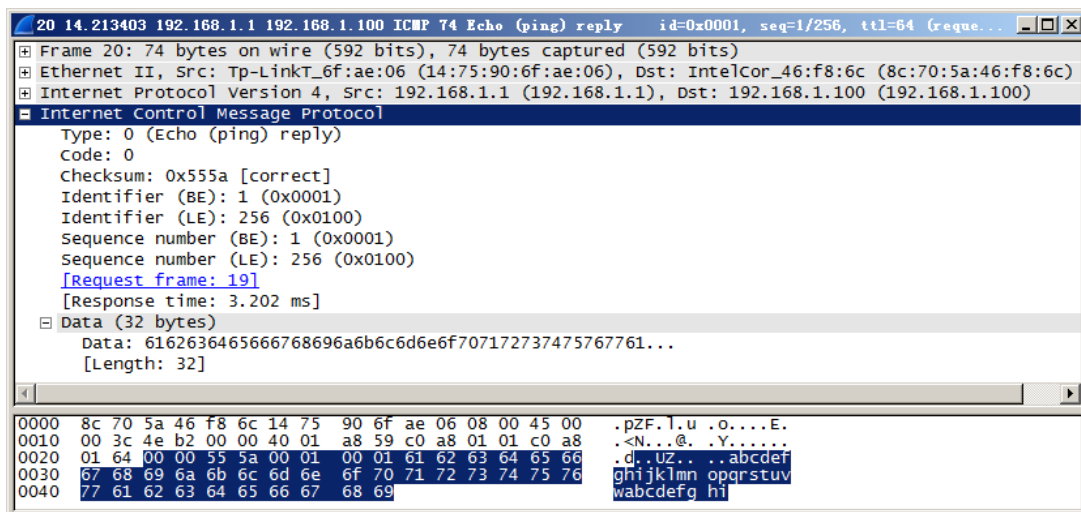
执行 ping 192.168.1.1 命令。使用 Wireshark 捕获网络 ICMP 报文，ICMP 回射请求和响应报文如图 2-17 所示。

观察 ICMP 回射请求和响应报文各字段的值。

观察一对 ICMP 请求和响应报文的标识、序号和数据是否一样？



(a)



(b)

图 2-17 (a) ICMP 回射请求报文 (b) ICMP 回射响应报文

2. 观察 ICMP 超时报文

执行 ping -i 5 www.sina.com.cn 命令。使用 Wireshark 捕获网络 ICMP 报文，ICMP 超时报文如图 2-18 所示。

观察 ICMP 超时报文各字段的值。

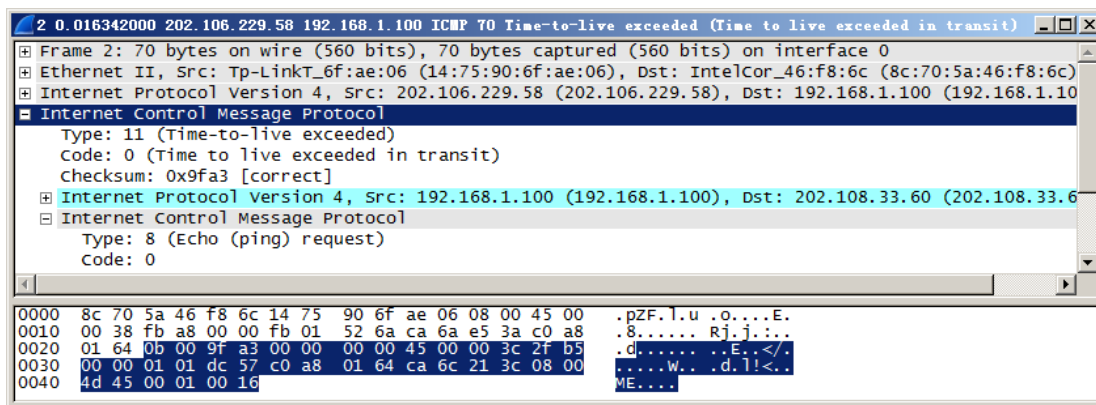


图 2-18 ICMP 超时报文

2.6 UDP 协议分析

2.6.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 UDP 报文,理解 UDP 报文格式,掌握 UDP 原理。

2.6.2 课程设计内容

利用 Wireshark 捕获和分析 UDP 报文,课程设计内容如下:捕获 UDP 报文,观察 UDP 报文格式。

2.6.3 相关知识

UDP 协议是 Internet 协议族的无连接的、不可靠的、端到端的传输协议。UDP 为应用程序提供了一种无需建立连接就可以发送 IP 数据包的方法。RFC 768 描述了 UDP。

UDP 头部的格式如图 2-19 所示。

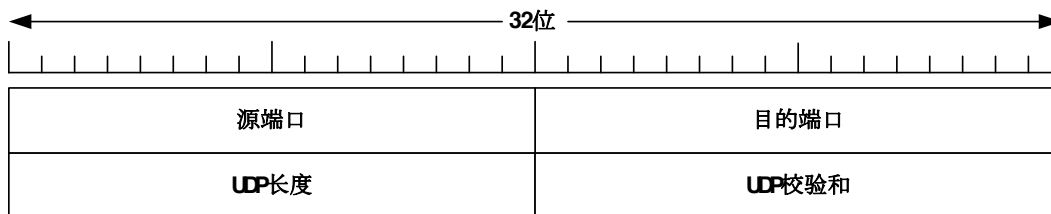


图 2-19 UDP 头部的格式

源端口和目的端口分别适发送方和接收方的端口,各占 16 位。

UDP 长度为 UDP 头和 UDP 数据两部分的总长度。最小长度 8 字节,刚好覆盖 UDP 头,最大长度 65515 字节,占 16 位。

校验和为增加可靠性,对 UDP 头、UDP 数据和一个概念性的 IP 伪头计算校验和,占 16 位。

UDP 数据报中的 IP 伪头格式如图 2-20。

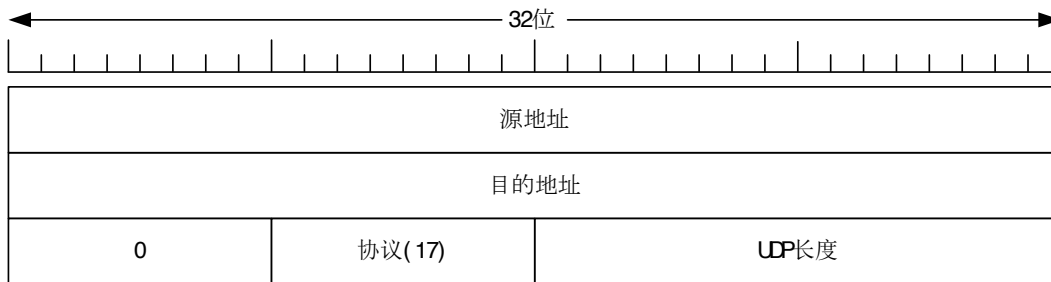


图 2-20 UDP 数据报中的 IP 伪头格式

发送方的 UDP 检验和计算：伪首部填充到 UDP 用户数据报上，把检验和字段填入全零。每 16 位为一段，若字节总数不是偶数，则增加一个字节的填充(全 0)。所有的 16 位段使用反码算术运算相加，把得到的结果取反码后插入到检验和字段。伪首部和任何增加的填充丢掉，把 UDP 用户数据报交付给 IP 软件进行封装。

接收方的检验和计算：伪首部加到 UDP 用户数据报，若需要，就增加填充。每 16 位为一段，把所有的 16 位段使用反码算术运算相加，得到的结果取反码，若得到的结果是全 0，则丢弃伪首部和任何增加的填充并接受这个 UDP 用户数据报，若得到的结果非 0，说明接收方检测到校验和有差错，就丢弃这个 UDP 用户数据报。

UDP 报文封装成一个 IP 数据包的格式，如图 2-21 所示。

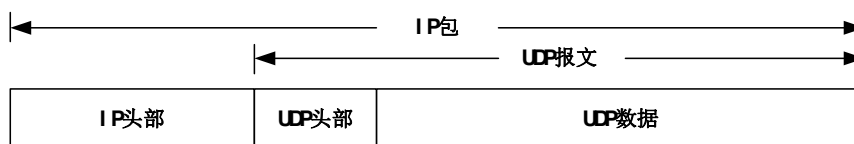


图 2-21 UDP 封装

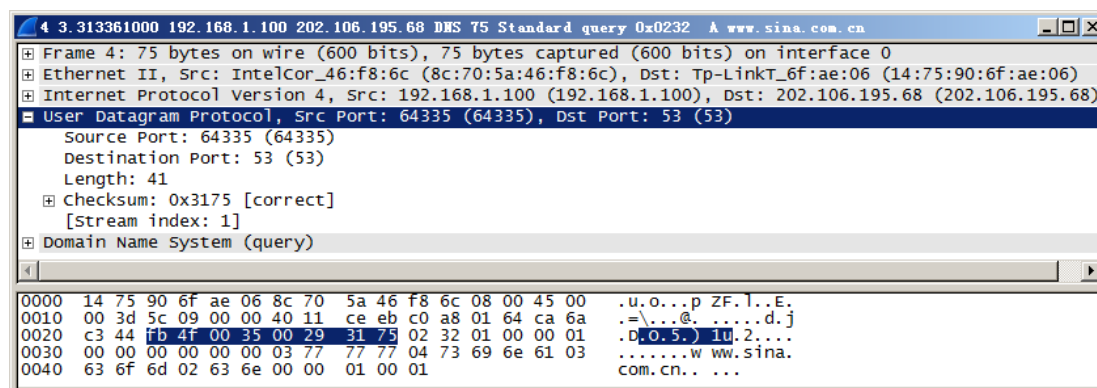
2.6.4 课程设计步骤

1. 观察 DNS 请求过程中的 UDP 报文

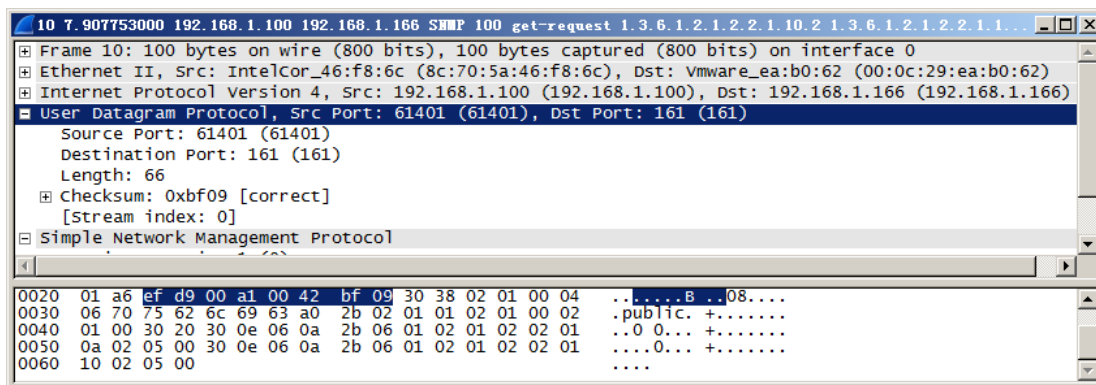
重启网络连接后，访问某个 Web 网站，如：www.sina.com.cn。使用 Wireshark 捕获网络 UDP 报文，DNS 请求过程中的 UDP 报文如图 2-22 (a) 所示。观察 UDP 报文各字段的值。

2. 观察 SNMP 请求过程中的 UDP 报文

在一台主机 (CentOS 6.5) 执行 snmpd，在另一台主机利用 MIB 浏览器向 SNMP 代理发送 get 请求。使用 Wireshark 捕获 UDP 报文，SNMP 请求过程中的 UDP 报文如图 2-22 (b) 所示。观察 UDP 报文各字段的值。



(a) DNS 请求过程中的 UDP 报文



(b) SNMP 请求过程中的 UDP 报文

图 2-22 DNS 和 SNMP 请求过程中的 UDP 报文

2.7 TCP 协议分析

2.7.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 TCP 报文,理解 TCP 报文格式,掌握 TCP 原理。

2.7.2 课程设计内容

利用 Wireshark 捕获和分析 TCP 报文,课程设计内容如下:捕获 TCP 报文,观察 TCP 报文格式,TCP 建立、数据传输和释放过程。

2.7.3 相关知识

1. TCP 协议

TCP 协议是为了在不可靠的互联网上提供一个可靠的、端到端的字节流而专门设计的一个传输协议。

TCP 连接是基于字节流的,而非报文流。端到端之间不保留报文的边界。

每个套接口有一个套接口描述字(地址),该描述字由主机的 IP 地址和一个 16 位端口组成的。每条 TCP 连接用(套接口 1,套接口 2)来表示,是一个全双工通道。

对于应用程序发来的数据,TCP 可以立即发送,也可以缓存一段时间以便一次发送更多的数据,这完全由 TCP 软件自己决定的。有时应用程序确实希望数据立即被发送出去,为了强迫数据发送,可以使用 PUSH 标记。

当一个具有高优先级数据的应用立即被处理时,例如:一个交互是用户输入 Ctrl+C 来中断一个已经开始运行的远程计算时,发送方应用程序把一些控制信息放到数据流中,并且将它连同 URGENT 标志一起交给 TCP。这一事件将导致 TCP 停止积累数据,将该连接上已有的所有数据立即发送出去。当接收方接收到紧急数据时,接收方应用程序被中断,它停止当

前正在做的工作，并且读入数据流以便找到紧急数据。

TCP 连接上的每个字节都有它自己独有的 32 位序列号。

发送方和接收方的 TCP 实体以数据段的形式交换数据。TCP 数据段包含一个 20 字节的头（选项部分另加）和随后的 0 个或多个数据字节。段的大小必须同时满足以下条件：

- 1) 每个数据段包括 TCP 头在内，要适合 IP 的 65515 字节净荷大小；
- 2) 每个网络都有一个最大传输单元（MTU），每个数据段必须适合 MTU。实践中，MTU 通常为 1500 字节（以太网的净荷大小）。

TCP 实体使用的基本协议是具有动态窗口大小的滑动窗口协议。当发送方传送一段时，它启动一个计时器。当该段到达接收方式，接收方的 TCP 实体返回一个携带了确认号和剩余窗口大小的段。确认号等于接收方希望接收的下一个序号。如果发送方的计时器在确认段到达之前超时，则发送方再次发送原来的段。

2. TCP 头

TCP 头部格式如图 2-23 所示。



图 2-23 TCP 头部格式

源端口和目的端口分别为发送方和接收方的端口，各占 16 位。

序列号表示该 TCP 数据段中的第一个 TCP 数据字节在从 TCP 发送方向 TCP 接收方发送的数据字节流中的位置。TCP 用序列号对每个字节进行计数。只有 SYN 标志置 1 时该字段才有效。占 32 位，以字节为单位编号。

确认号应当是上次已成功收到数据字节序号加 1。只有 ACK 标志置 1 时确认号字段才有效。占 32 位，以字节为单位编号。

TCP 头的长度给出首部中 32 位字的数目，即长度单位为 32 位字，包含可选项域，占 4 位。

保留域占 6 位。

6 位的标志位。1 表示有效。其中

URG 表示紧急指针有效，和紧急指针配合使用，发送紧急数据。

ACK 表示确认号是否有效。

PSH 表示指示接收方应该尽快将这个报文段交给应用层，不做缓存。

RST 表示重置一个已混乱的连接。

SYN 表示序列号是否有效，用来发起一个连接。

FIN 表示发端完成发送任务后，FIN 用于释放连接。

窗口大小用于基于可变滑动窗口的流控，指示发送方从确认号开始可以再发送窗口大小的字节流，窗口大小为字节数，占 16 位。

校验和为增加可靠性，校验的范围包括 TCP 头、TCP 数据以及与 UDP 一样的概念性伪头。除了伪头的协议号为 TCP (6)，并且校验和必需强制执行，占 16 位。

紧急指针用来指示紧急数据在当前数据段中的位置，和序列号字段中的值相加表示紧急数据最后一个字节的序号。当 URG 标志置 1 时紧急指针才有效，占 16 位。

可选项域占 0 或多个 32 位字。

TCP 报文封装成一个 IP 数据包的格式，如图 2-24 所示。

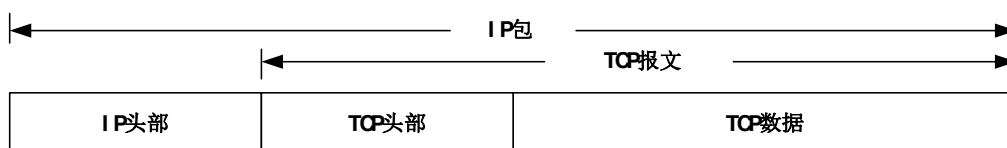


图 2-24 TCP 封装

3. TCP 连接建立

为了建立一个连接，服务器必须先执行 LISTEN 和 ACCEPT 原语，然后被动地等待连接请求的到来。

当客户执行 CONNECT 原语时，CONNECT 原语发送一个 TCP SYN 报文 (SYN 标志设置为 1)，这个报文指明客户打算连接的服务器的端口，以及初始序列号 (ISN)，然后等待服务器的响应。

当这个 TCP 报文到达服务器时，服务器的 TCP 实体检查是否有一个进程已经在目标端口上监听，如果没有，则它发送一个 TCP RST 报文，拒绝客户的连接请求。

如果某进程正在该端口上监听，则 TCP 实体将收到的 TCP 报文交给该进程处理。该进程可以接受或者拒绝这个连接请求。如果接受，则返回一个包含服务器的 ISN 的 TCP SYN 报文作为响应。同时，将确认号设置为客户的 ISN 加 1 以对客户发送的 TCP SYN 报文进行确认。一个 SYN 将占用一个序号。

客户必须将确认号设置为服务器的 ISN 加 1 以对服务器的 SYN 报文进行确认。

经过三次握手后连接就建立了。

正常情况下，TCP 连接建立如图 2-25 所示。

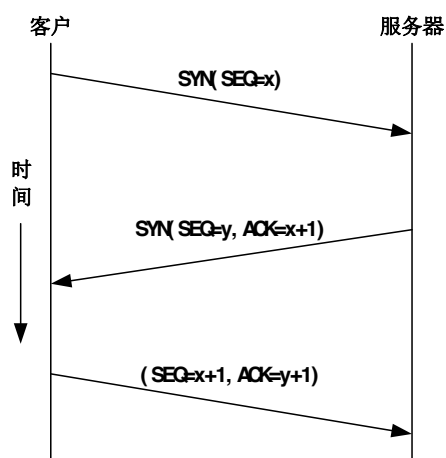


图 2-25 正常情况下的 TCP 连接建立

4. 释放连接

TCP 连接是全双工，因此每个方向必须单独地进行关闭。这原则就是当一方完成它的数据发送任务后就能发送一个 TCP FIN 报文来终止这个方向连接。当一端收到一个 TCP FIN 报文，它必须通知应用层另一端已经终止了那个方向的数据传送。发送 TCP FIN 报文通常是应用层进行关闭的结果。只有双向连接都关闭时，TCP 连接才会终止。TCP 连接释放如图 2-26 所示。

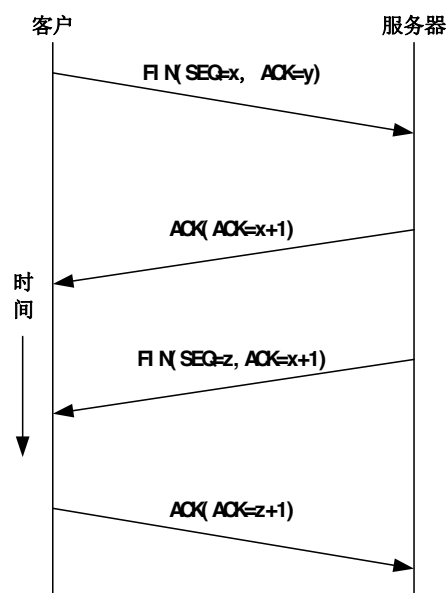


图 2-26 TCP 连接释放

2.7.4 课程设计步骤

访问某个 Web 网站，如：www.huanqiu.com。使用 Wireshark 捕获网络中 TCP 报文。

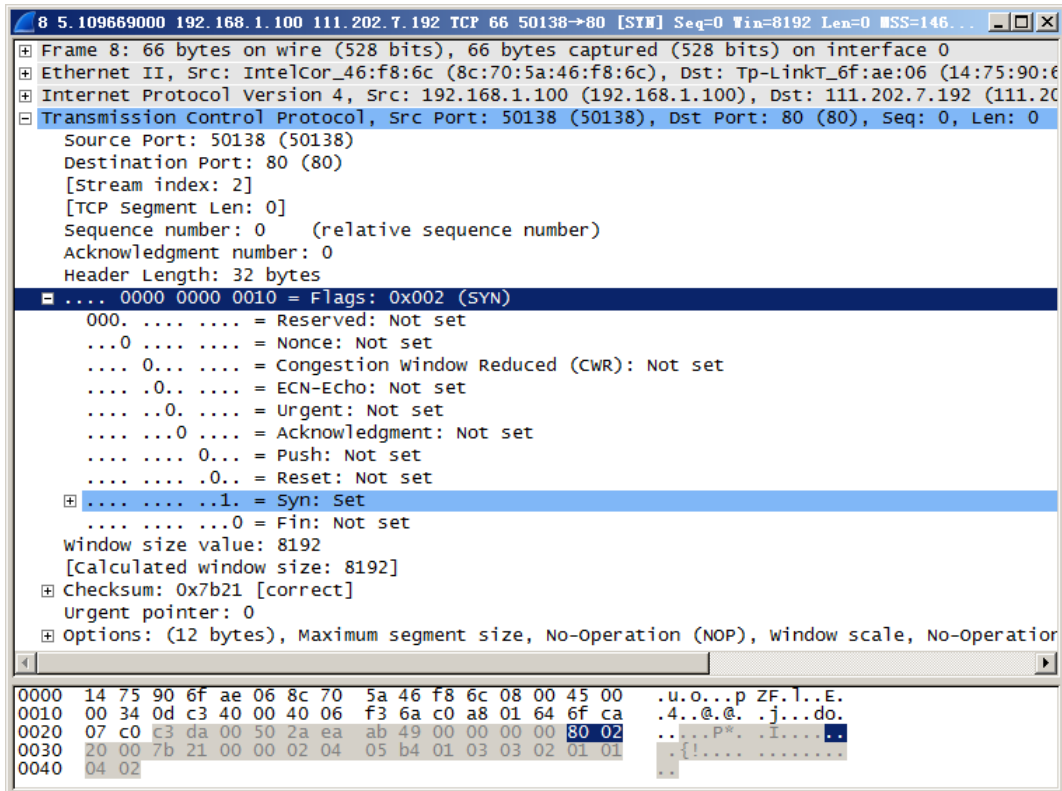
1. 捕获和分析 TCP 连接建立的 TCP 报文

TCP 连接建立过程如图 2-27 所示。从图中可以看出，第一个 TCP 报文是 192.168.1.100 向 111.202.7.192 (www.huanqiu.com) 发送的 SYN 设置为 1 的连接请求；第二个 TCP 报文是 111.202.7.192 返回给 192.168.1.100 的 ACK 设置为 1 的连接确认，同时 SYN 也设置为 1；第三个 TCP 报文是 192.168.1.100 收到连接确认后向 111.202.7.192 发送的一个连接确认。经过三次握手后连接就建立了。

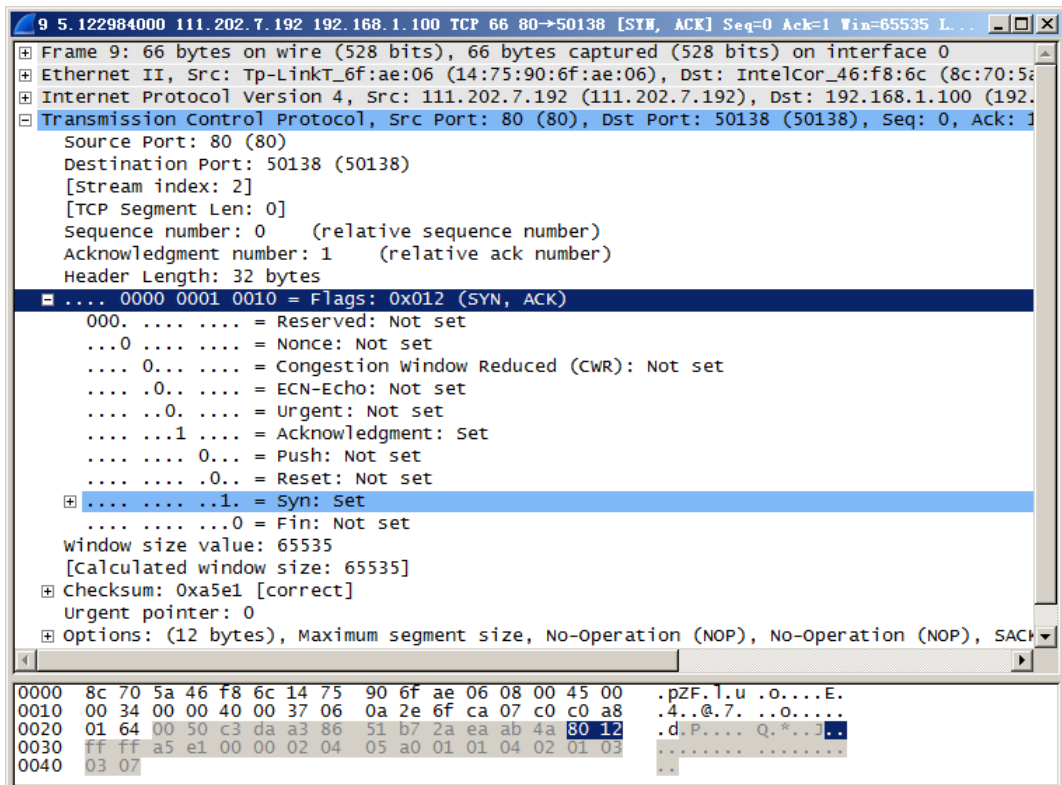
No.	Time	Source	Destination	Protocol	Length	Info
8	5.10966900	192.168.1.100	111.202.7.192	TCP	66	50138->80 [SYN] Seq=0 win=8192 Len=0 MSS=1460 wS=4 SACK=0
9	5.12298400	111.202.7.192	192.168.1.100	TCP	66	80->50138 [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1460 wS=4 SACK=0
10	5.12304400	192.168.1.100	111.202.7.192	TCP	54	50138->80 [ACK] Seq=1 Ack=1 win=66240 Len=0

图 2-27 TCP 连接建立过程

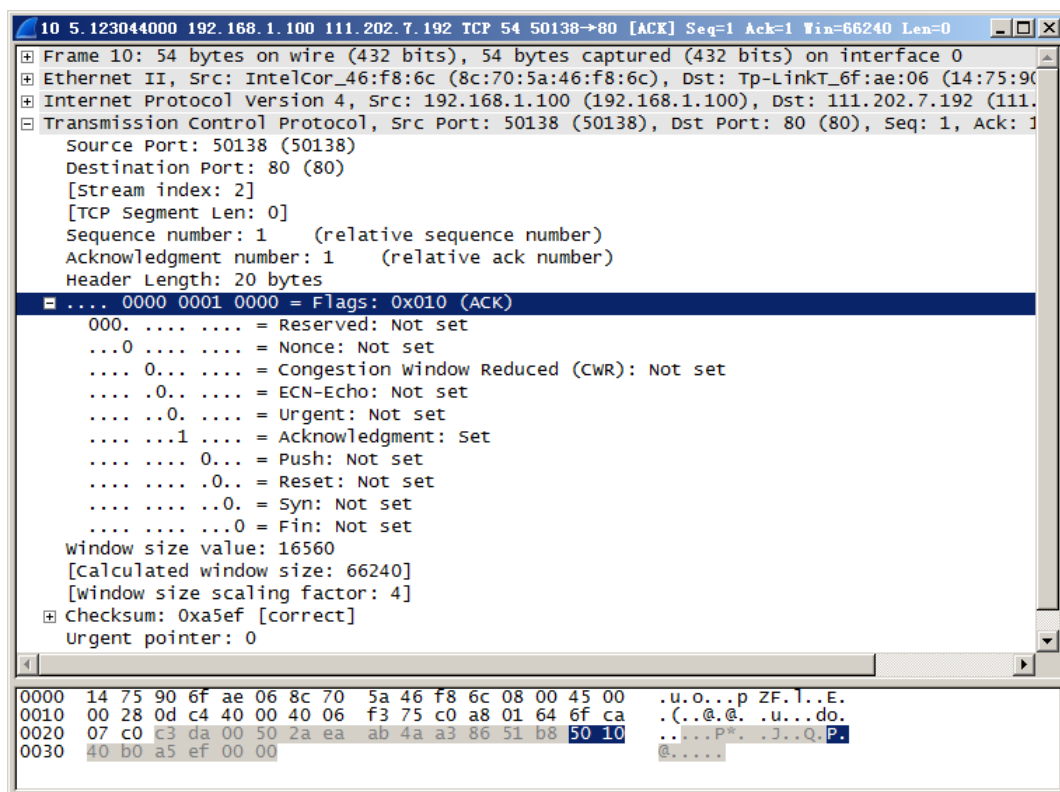
TCP 连接建立中的 TCP 报文格式如图 2-28 所示。图 2-28 (a) 是 TCP 连接建立中的 TCP SYN 报文，该报文的源端口和目的端口分别为 50138 和 80，初始序列号为 0，标志中 SYN 设置为 1，窗口大小为 8192，校验和正确。图 2-28 (b) 是 TCP 连接建立中的 TCP SYN+ACK 报文，该报文的源端口和目的端口分别为 80 和 50138，初始序列号为 0，确认号为 1，标志中 SYN 和 ACK 都设置为 1，窗口大小为 65535，校验和正确。图 2-28 (c) 是 TCP 连接建立中的 TCP ACK 报文，该报文的源端口和目的端口分别为 50138 和 80，确认号为 1，标志中 ACK 设置为 1，窗口大小为 16560，校验和正确。



(a)



(b)



(c)

图 2-28 TCP 连接建立过程中的 TCP 报文

(a) SYN 报文 (b) SYN+ACK 报文 (c) ACK 报文

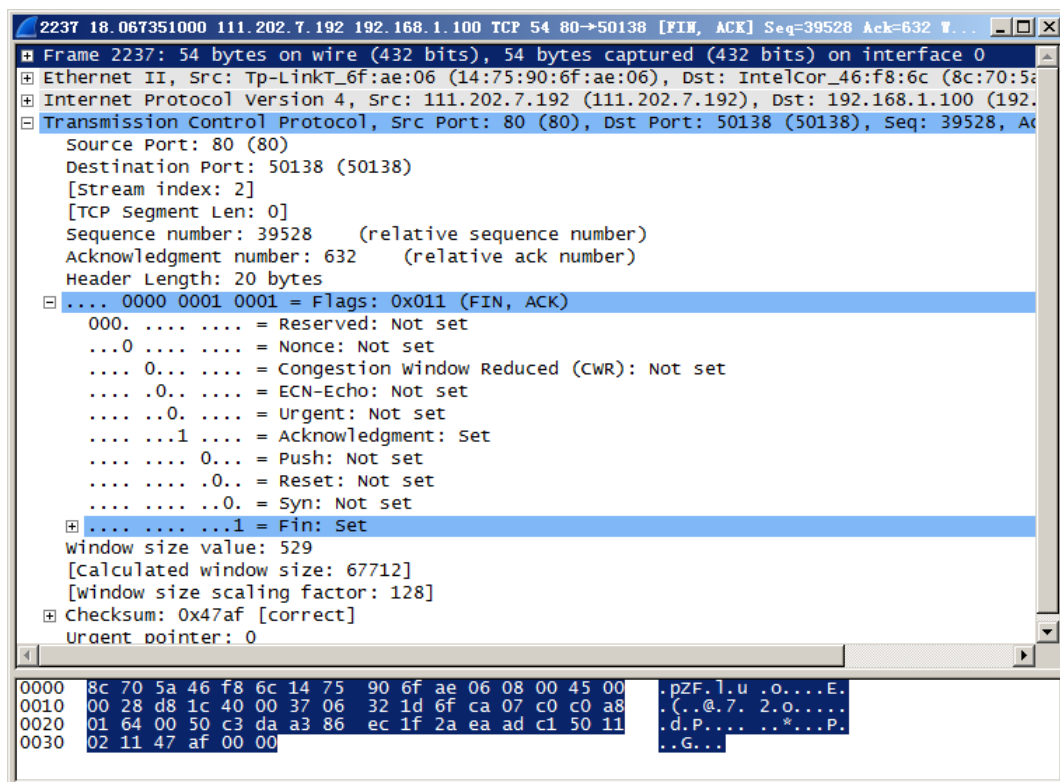
2. 捕获 TCP 连接释放的 TCP 报文并分析

TCP 连接释放过程如图 2-29 所示。从图中可以看出，111.202.7.192(www.huanqiu.com) 向 192.168.1.100 发送的 FIN 设置为 1 的释放连接请求（图 2-29 中第一个报文）；192.168.1.100 收到这个 FIN 报文后，返回 ACK 设置为 1 的确认报文（图 2-29 中第二个报文）。另一个方向的 TCP 连接释放过程与此类似。两个方向连接都释放后，整个 TCP 连接被成功释放。

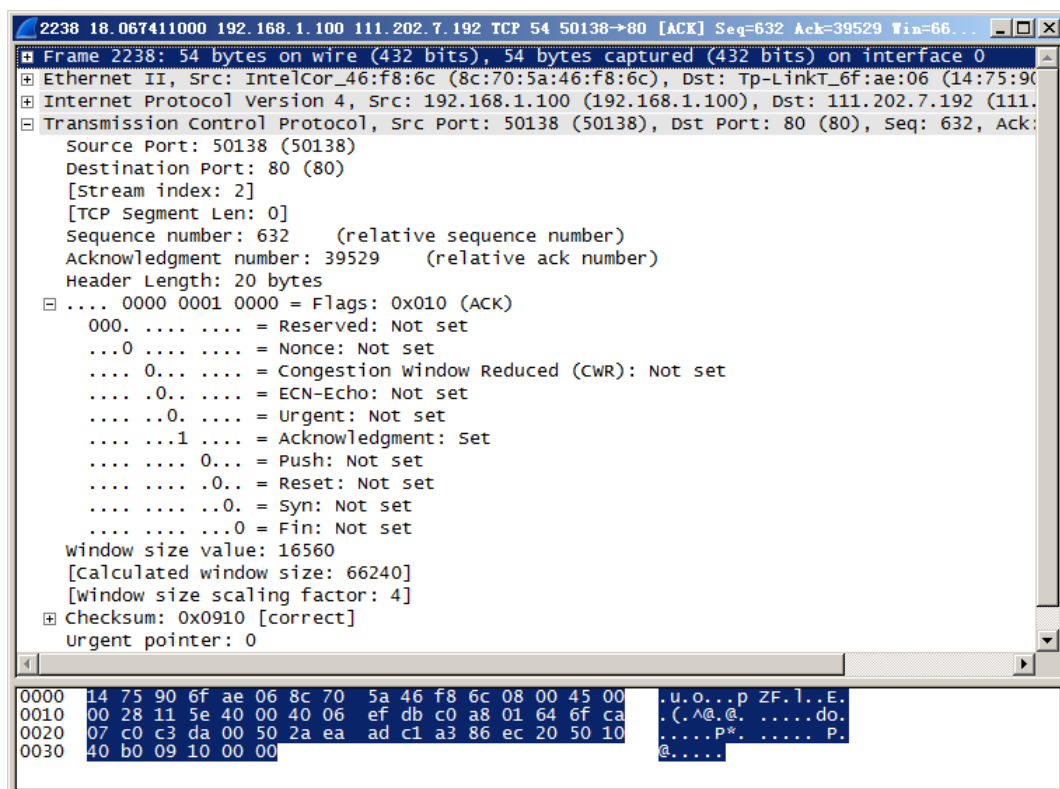
No.	Time	Source	Destination	Protocol	Length	Info
2237	18.0673510	111.202.7.192	192.168.1.100	TCP	54	80→50138 [FIN, ACK] Seq=39528 Ack=632 win=67712 Len=0
2238	18.0674110	192.168.1.100	111.202.7.192	TCP	54	50138→80 [ACK] Seq=632 Ack=39529 win=66240 Len=0
2239	18.0675080	192.168.1.100	111.202.7.192	TCP	54	50138→80 [FIN, ACK] Seq=632 Ack=39529 win=66240 Len=0
2240	18.0832180	111.202.7.192	192.168.1.100	TCP	54	80→50138 [ACK] Seq=39529 Ack=633 win=67712 Len=0

图 2-29 TCP 连接释放过程

一个方向的 TCP 连接释放中的 TCP 报文格式如图 2-30 所示。图 2-30 (a) 是 TCP 连接建立中的 TCP FIN 报文，该报文的源端口和目的端口分别为 80 和 50138，序列号为 39528，标志中 FIN 设置为 1，窗口大小为 529，校验和显示正确。图 2-30 (b) 是 TCP 连接释放中的 TCP ACK 报文，该报文的源端口和目的端口分别为 50138 和 80，序列号为 632，确认号为 39529，标志中 ACK 都设置为 1，窗口大小为 16560，校验和显示正确。另一个方向的 TCP 连接释放中的 TCP 报文与图 2-30 类似。



(a)



(b)

图 2-30 一个方向的 TCP 连接释放过程中的 TCP 报文

(a) FIN 报文 (b) ACK 报文

3. TCP 数据传输中的确认机制

TCP 部分数据传输过程如图 2-31 所示。图 2-31 中每个 TCP 报文格式如图 2-32 所示。

No.	Time	Source	Destination	Protocol	Length	Info
1026	6.64420900	111.202.7.192	192.168.1.100	HTTP	1460	HTTP/1.1 200 OK (image/x-icon)
1027	6.64427500	192.168.1.100	111.202.7.192	TCP	54	50068→80 [ACK] Seq=632 Ack=39908 Win=66240 Len=0
1050	6.67412400	111.202.7.192	192.168.1.100	TCP	1460	[TCP Retransmission] 80→50068 [PSH, ACK] Seq=38502
1051	6.67417100	192.168.1.100	111.202.7.192	TCP	66	[TCP Dup ACK 1027#1] 50068→80 [ACK] Seq=632 Ack=39908

图 2-31 TCP 部分数据传输过程

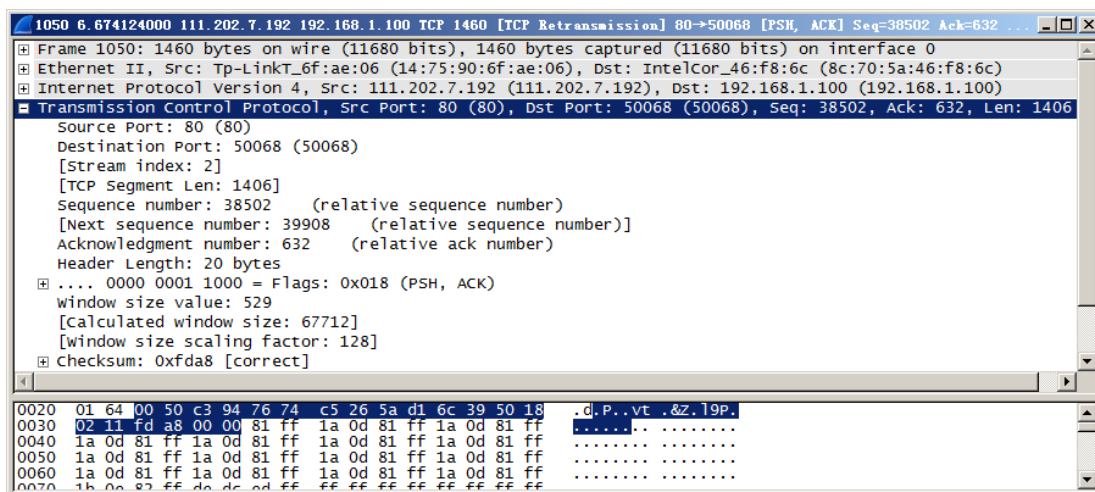
图 2-32 (a) 中的 TCP 报文是 111.202.7.192 向 192.168.1.100 发送的序列号为 38502、确认号为 632、长度为 1406 字节的 TCP 报文；图 2-32 (b) 中 TCP 报文是 192.168.1.100 返回给 111.202.7.192 的序列号为 632、确认号为 39908 (39908=38502+1406)、长度为 0 字节的 TCP 报文；图 2-32 (c) 中的报文是 111.202.7.192 向 192.168.1.100 发送的序列号为 38502 的重复 TCP 报文；当 192.168.1.100 收到这个重复报文后，丢弃这个报文，并重新发送确认号为 39908 的 TCP 报文，表明期望接收序列号为 39908 的 TCP 报文，如图 2-32 (d) 所示。

1026 6.644209000 111.202.7.192 192.168.1.100 HTTP 1460 HTTP/1.1 200 OK (image/x-icon)	
Frame 1026: 1460 bytes on wire (11680 bits), 1460 bytes captured (11680 bits) on interface 0	
Ethernet II, Src: Tp-LinkT_6f:ae:06 (14:75:90:6f:ae:06), Dst: IntelCor_46:f8:6c (8c:70:5a:46:f8:6c)	
Internet Protocol Version 4, Src: 111.202.7.192 (111.202.7.192), Dst: 192.168.1.100 (192.168.1.100)	
Transmission Control Protocol, Src Port: 80 (80), Dst Port: 50068 (50068), Seq: 38502, Ack: 632, Len: 1406	
Source Port: 80 (80)	
Destination Port: 50068 (50068)	
[Stream index: 2]	
[TCP Segment Len: 1406]	
Sequence number: 38502 (relative sequence number)	
[Next sequence number: 39908 (relative sequence number)]	
Acknowledgment number: 632 (relative ack number)	
Header Length: 20 bytes	
.... 0000 0001 1000 = Flags: 0x018 (PSH, ACK)	
Window size value: 529	
[Calculated window size: 67712]	
[Window size scaling factor: 128]	
Checksum: 0xfda8 [correct]	
0020 01 64 00 50 c3 94 76 74 c5 26 5a d1 6c 39 50 18 .d.P..vt.&z.l9p.	
0030 02 11 fd a8 00 00 81 ff 1a 0d 81 ff 1a 0d 81 ffdo.....	
0040 1a 0d 81 ff 1a 0d 81 ff 1a 0d 81 ff 1a 0d 81 ffdo.....	
0050 1a 0d 81 ff 1a 0d 81 ff 1a 0d 81 ff 1a 0d 81 ffdo.....	
Frame (1460 bytes) Reassembled TCP (4625 bytes)	

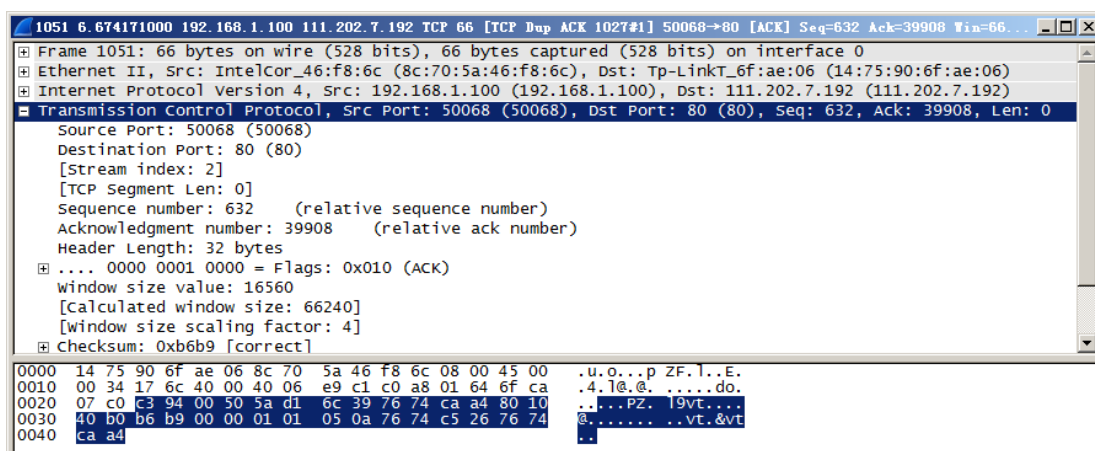
(a)

1027 6.644275000 192.168.1.100 111.202.7.192 TCP 54 50068→80 [ACK] Seq=632 Ack=39908 Win=66240 Len=0	
Frame 1027: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0	
Ethernet II, Src: IntelCor_46:f8:6c (8c:70:5a:46:f8:6c), Dst: Tp-LinkT_6f:ae:06 (14:75:90:6f:ae:06)	
Internet Protocol Version 4, Src: 192.168.1.100 (192.168.1.100), Dst: 111.202.7.192 (111.202.7.192)	
Transmission Control Protocol, Src Port: 50068 (50068), Dst Port: 80 (80), Seq: 632, Ack: 39908, Len: 0	
Source Port: 50068 (50068)	
Destination Port: 80 (80)	
[Stream index: 2]	
[TCP Segment Len: 0]	
Sequence number: 632 (relative sequence number)	
Acknowledgment number: 39908 (relative ack number)	
Header Length: 20 bytes	
.... 0000 0001 0000 = Flags: 0x010 (ACK)	
Window size value: 16560	
[Calculated window size: 66240]	
[Window size scaling factor: 4]	
Checksum: 0x6985 [correct]	
0000 14 75 90 6f ae 06 8c 70 5a 46 f8 6c 08 00 45 00 .u.o..p ZF.l..E.	
0010 00 28 17 67 40 00 40 06 e9 d2 c0 a8 01 64 6f ca .(.g@.@.....do.	
0020 07 c0 c3 94 00 50 5a d1 6c 39 76 74 ca a4 50 10 ...PZ..l9vt..P.	
0030 40 b0 69 85 00 00 @.l....	

(b)



(c)



(d)

图 2-32 TCP 数据传输过程中的 TCP 报文

(a) TCP 数据报文 (b) TCP 确认报文 (c) 重复 TCP 数据报文 (d) 重复 TCP 确认报文

2.8 HTTP 协议分析

2.8.1 课程设计目的

本课程设计的主要目的是通过捕获和分析 HTTP 报文，理解 HTTP 报文格式，掌握 HTTP 原理。

2.8.2 课程设计内容

利用 Wireshark 捕获和分析 HTTP 报文，课程设计内容如下：捕获 HTTP 报文，观察 HTTP 报文格式和 HTTP 工作过程。

2.8.3 相关知识

1. HTTP 协议

HTTP 协议是一个简单的请求/响应协议，它通常运行在 TCP 上。它指定了客户可能发送给服务器什么样的消息及得到什么样的响应。请求和响应报文的头以 ASCII 码形式给出，而报文的内容则具有一个类似 MIME 的格式。

浏览器与服务器联系最常用的方法是与服务器上的端口 80 建立一个 TCP 连接。在 HTTP1.0 中，连接建立起来后浏览器只发送一个请求，一个响应报文发回来后，TCP 连接就释放了。HTTP1.1 支持持续连接。有了持续连接，就有可能建立一个 TCP 连接，在其上发送一个请求并得到一个响应，然后再发送请求并得到响应，而且还可以发送流水线请求。

2. HTTP 请求

HTTP 请求的第一行必须是请求行，用来说明请求类型、要访问的资源 and 使用的 HTTP 版本。之后是头部，用来说明服务器要使用的附加信息。头部之后是一个空行，空行后可以添加任意的其他数据，称为主体。

请求行的第一个词是请求的方法名。常用的方法如下：

1) GET 方法请求服务器发送页面，该页面被适当编码成 MIME。大部分发给 Web 服务器的请求都是 GET 方法。GET 的通用形式是：

GET filename HTTP/1.1

其中 filename 是预取的页面名字，1.1 是协议版本号。

2) POST 方法用于提交表单。它不是简单地检索一个页面，而是上载数据到服务器，然后服务器利用这些数据做某件事，具体取决于 URL，概念上是将数据附加到对象上，最后方法返回一个指向结果的页面。

3) PUT 方法不是读取页面，而是写入页面。通过这个方法可以在远程服务器上建立起一组 Web 页面。这个请求的主体包含了页面。页面可以利用 MIME 编码，这种情况下，跟在 PUT 后面的行可能包含了认证头，以便证明调用者的确有权执行所请求的操作。

3. HTTP 响应

HTTP 响应由一个状态行及可能的头部和主体组成。

状态行包括一个 3 位数字的状态码，该状态码指明了这个请求是否被满足；如果没满足，则原因是什么。状态码的第一个数字把响应分成 5 大组。

1XX 码实际上很少使用。如 100 表示服务器同意处理客户请求。

2XX 码表示这个请求被成功处理，并且如果有的话，返回了相应的内容。如：200 表示请求成功，204 表示没有内容。

3XX 码告诉客户应该检查其他地方。使用另一个不同的 URL，或在它自己的缓存中查找。如：301 表示移动页面，304 缓存的页面仍然有效。

4XX 码表示由于客户错误而导致请求失败。如：403 表示禁止页面，404 页面没找到。

5XX 表示服务器自身出现内部问题，有可能是服务器代码中有错误，也可能是临时负载过重。如：500 表示服务器内部错误，503 表示稍后再试。

4. 头部

请求行后面可能还有额外的行，其中包含了更多的信息，它们统称为头部。响应报文也有头部。以下是常用的 HTTP 头部：

1) User-Agent 头允许客户将它的浏览器实现告诉服务器（如：Mozilla/5.0）。服务器可据此裁剪给浏览器的响应；

2) 如果客户对可接受的信息有一定的限制，可以用 4 个 Accept 头告诉服务器愿意接受

什么。第一个头指定了哪些 MIME 类型是可以接受的（如：text/html）。第二个头给出了字符集（如：ISO-8859-5）。第三个头处理压缩方法（如：gzip）。第四个头指明了一种自然语言（如：汉语）。如果服务器有多个页面可供选择的话，则它可以利用这些信息向客户提供所需的页面。如果客户的请求不能满足，则返回一个错误码，且请求失败；

3) If-Modified-Since 和 If-None-Match 头用于缓存。它们允许客户在页面的缓存副本不再有效时请求该页面；

4) Host 头是服务器的名字，它取自 URL。这个头是强制性质的；

5) 对于那些被保护的页面，Authorization 头是必需的。这种情况下，客户可能需要证明自己有权查看被请求的页面；

6) Cookie 头给服务器发回 Cookie 的先前 URL；

7) Set-Cookie 头客户存储的 Cookie；

8) Server 头服务器信息；

9) Content-Encoding 头指明要对资源采用的附加内容译码方式，因而要获得 Content-Type 中提及的介质类型，必须采用与译码方式一致的解码机制。例如：Content-Encoding: x-gzip。从 Content-Encoding 开始允许服务器描述它发送的页面的属性；

10) Content-Language 头描述了页面使用的自然语言；

11) Content-Length 头指明了页面以字节计的长度。例如：Content-Length: 3495；

12) Content-Type 头指明了页面的 MIME 类型。例：Content-Type: text/html；

13) Last-Modified 头说明了页面最后被修改的时间；

14) Expires 头说明了页面能保持有效多长时间；

15) Location 头被服务器用来通知客户，它应该尝试另外一个不同的 URL；

16) Accept-Ranges 头声明了服务器愿意处理这种页面请求；

17) Date 头可用在两个方向上，包含了发送报文的日期和时间。这个头可以在两个方向上同时使用；

18) Range 头告诉响应报文提供的页面的字节范围，这个头可以在两个方向上同时使用。

2.8.4 课程设计步骤

访问某个 Web 网站，使用 Wireshark 捕获网络 HTTP 报文，为了只捕获 HTTP 报文，在 Capture Options 对话框中单击 Capture Filter，选择 HTTP TCP port (80)。

1. 分析 HTTP 请求/响应

清空 Firefox 的缓存，启动 Wireshark 开始捕获数据包，在 Firefox 的地址栏输入 <http://www.ietf.org/download/rfc-index.txt>。

Wireshark 捕获的 HTTP 数据包如图 2-33 所示。

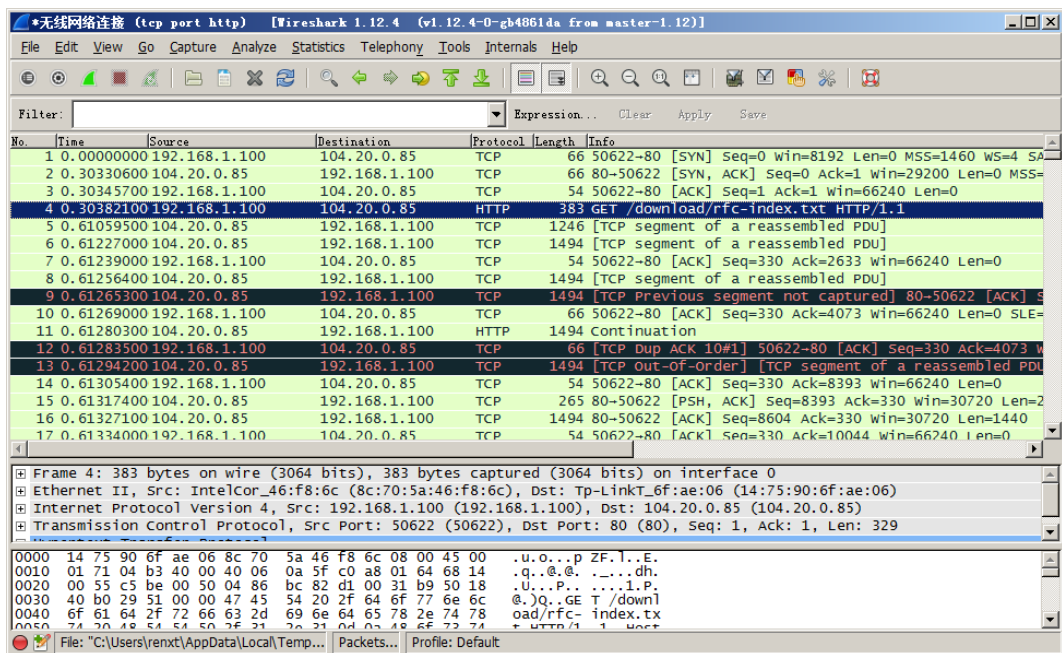


图 2-33 HTTP 数据包

选择 Analyze 菜单中的 Follow TCP Stream 按钮，显示 HTTP 请求和响应如图 2-34 所示。

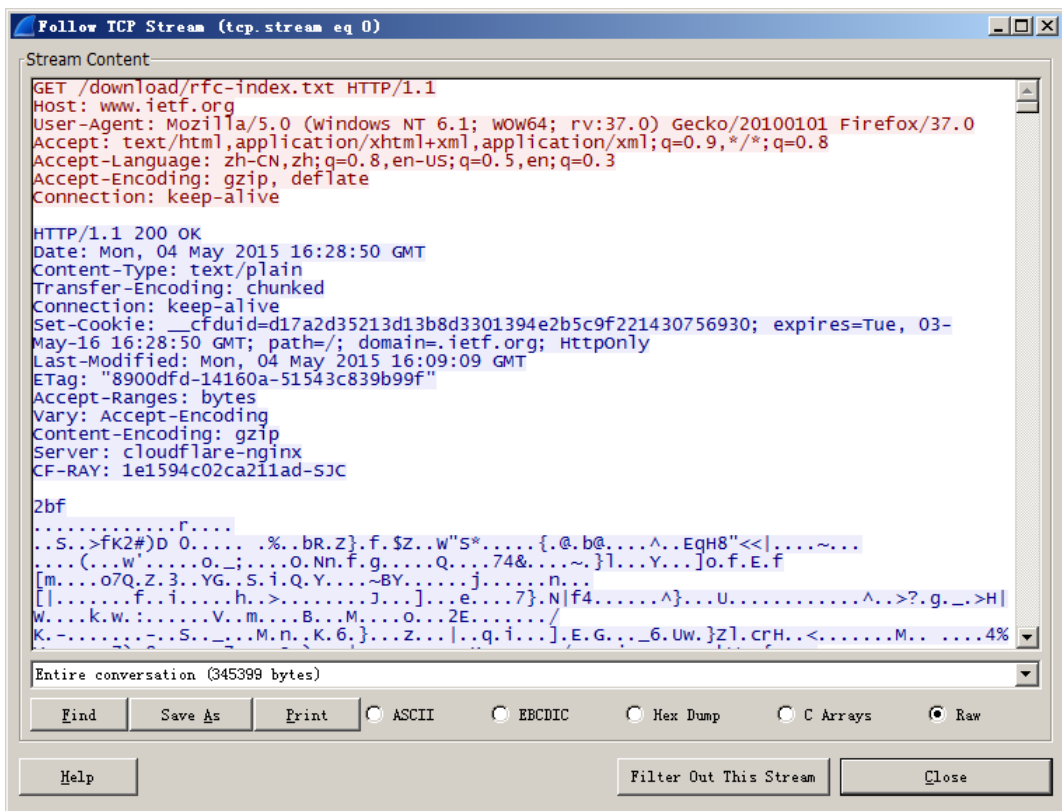


图 2-34 HTTP 请求和响应

从图 2-34 可以看出，浏览器运行的和服务器运行的都是 HTTP/1.1；浏览器向服务器指出它能接收的语言版本的对象为简体中文、美国英语和英语；从服务器向浏览器返回的状态码是 200。

2. 浏览器缓存的页面有效时 HTTP 请求/响应

在 Firefox 的地址栏输入 <http://www.ietf.org/download/rfc-index.txt>。启动

Wireshark，单击 Firefox 中的刷新按钮，Wireshark 捕获的 HTTP 请求和响应如图 2-35 所示。

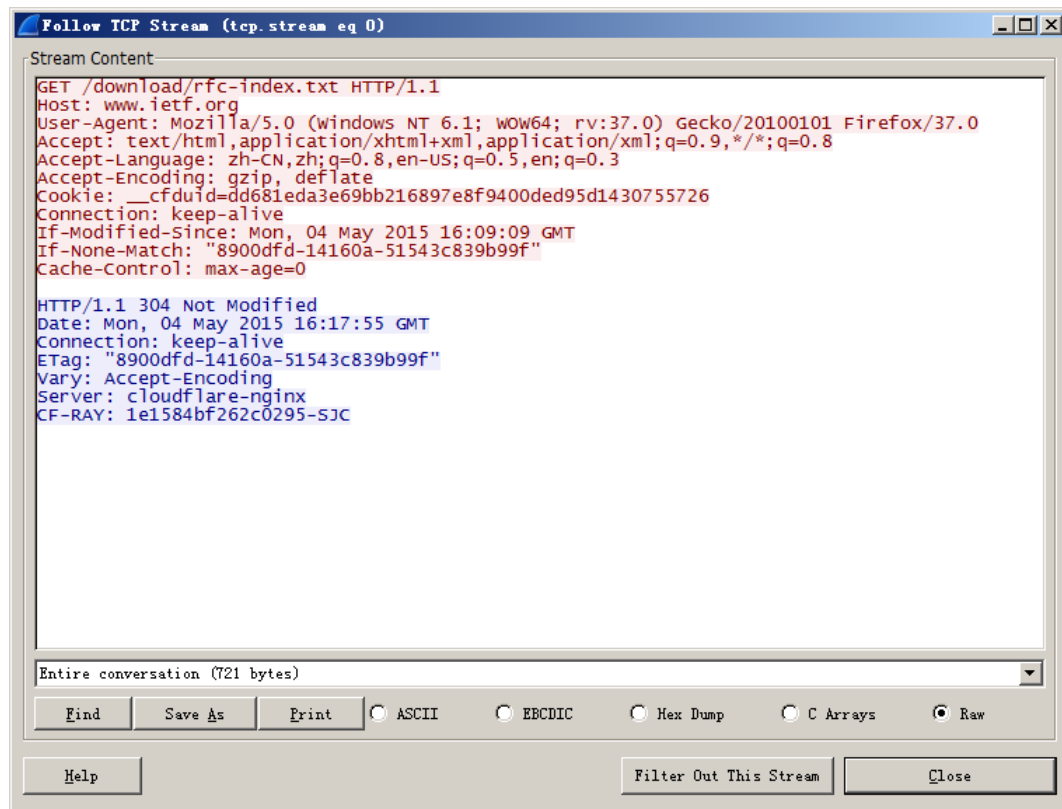


图 2-35 浏览器缓存的页面有效时 HTTP 请求/响应

从图 2-35 可以看出，浏览器向服务器发出的第一个 HTTP 请求中有一行是 If-Modified-Since，其后的内容为 Mon, 04 May 2015 16:09:09 GMT；从服务器返回的 HTTP 响应中的状态行为 HTTP/1.1 304 Not Modified，可以断定服务器没有返回文件的内容。

第三章 网络编程

本章主要是设计和实现数据链路层、网络层、传输层和应用层的常用协议，加深理解计网络协议的工作原理，掌握网络编程的基本流程和基本方法，培养编程开发能力。

本章的主要内容包括以太网 ARP 请求发送程序、以太网 ARP 响应分析程序、ping 程序、多播客户/服务器程序、UDP 报文发送程序、TCP SYN 报文发送程序、UDP 客户/服务器程序、TCP 客户/服务器程序、服务器接口监测程序、邮件客户端程序和聊天客户/服务器程序等。其中前 8 个程序采用 C 语言实现的，后 3 个程序采用 Java 语言实现的。

3.1 以太网 ARP 请求发送程序

3.1.1 课程设计目的

ARP 协议为 IP 地址到对应的硬件地址之间提供动态映射。本课程设计的主要目的是通过构造与发送以太网 ARP 请求，掌握 ARP 协议的工作原理，理解以太网帧和 ARP 请求的格式。

3.1.2 课程设计内容

本课程设计要求设计与实现 Linux 下以太网 ARP 请求发送程序，具体内容如下：

- 1) 该程序能够实现以下功能：能够获取指定接口的 MAC 地址、IP 地址和接口索引；构造以太网帧头部和 ARP 请求，并向本地网络广播这个 ARP 请求；
- 2) 设计和实现以太网 ARP 请求发送程序；
- 3) 能够用 Wireshark 分析发送的 ARP 请求。

3.1.3 相关知识

本课程设计需要的 ARP 协议知识，请参考第二章第 3 节 ARP 协议分析。

本课程设计需要的 Linux 网络编程的知识，请参考附录 B。

3.1.4 课程设计的分析

1. 详细设计

以太网 ARP 请求发送程序的流程图如图 3-1 所示。

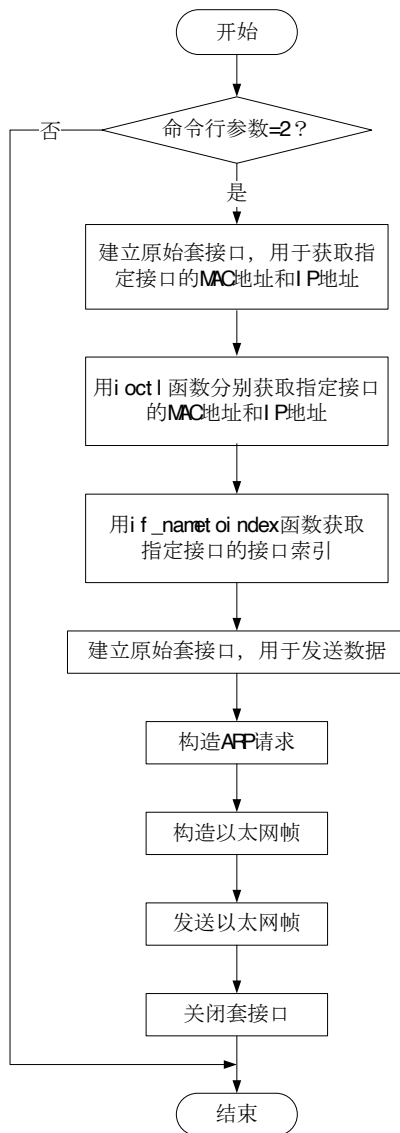


图 3-1 以太网 ARP 请求发送程序的流程图

构造 ARP 请求的流程图如图 3-2 所示。

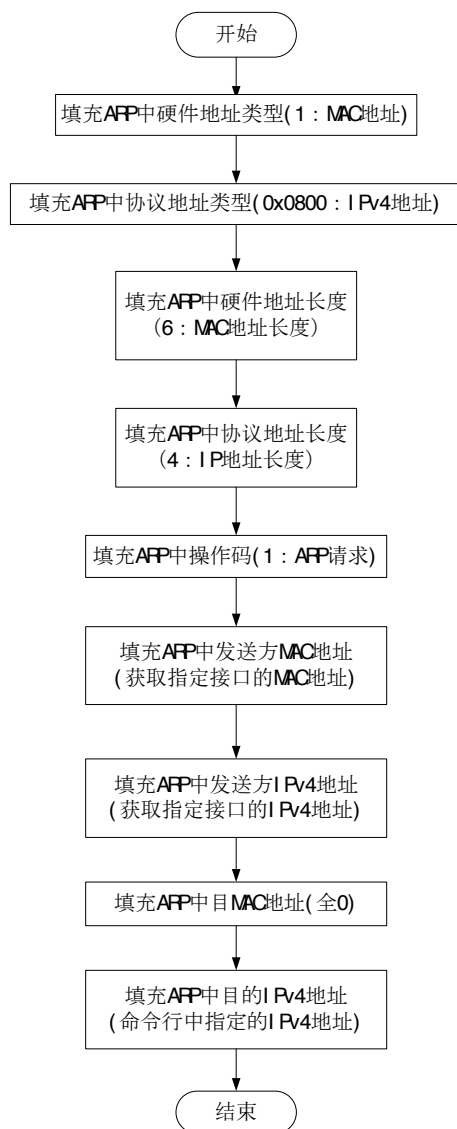


图 3-2 构造 ARP 请求流程图

构造以太网帧的流程图如图 3-3 所示。

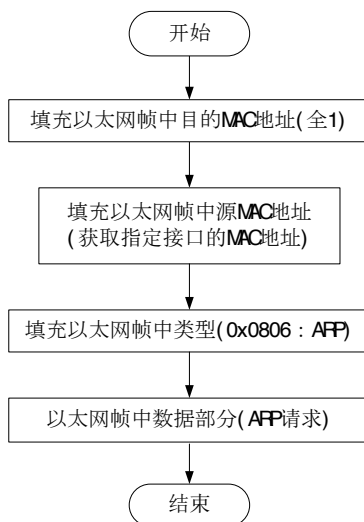


图 3-3 构造以太网帧的流程图

2. 核心代码

1) ARP 结构定义。ARP 结构（ether_arp 结构）在<netinet/if_ether.h>中定义，其中 arphdr 在< linux/if_arp.h >中定义。

ether_arp 结构

```
struct ether_arp {
    struct arphdr ea_hdr;      // 见“struct arphdr”
    u_int8_t arp_sha[ETH_ALEN]; // 发送方MAC地址
    u_int8_t arp_spa[4];       // 发送方IP地址
    u_int8_t arp_tha[ETH_ALEN]; // 目的MAC地址
    u_int8_t arp_tpa[4];       // 目的IP地址
};
#define arp_hrd ea_hdr.ar_hrd
#define arp_pro ea_hdr.ar_pro
#define arp_hln ea_hdr.ar_hln
#define arp_pln ea_hdr.ar_pln
#define arp_op ea_hdr.ar_op
```

arphdr 结构

```
struct arphdr
{
    __be16      ar_hrd;        // 硬件类型。值为 1 表示以太网地址
    __be16      ar_pro;        // 协议类型。值为 0x0800 表示 IP 地址
    unsigned char ar_hln;      // 硬件地址长度。值为 6 表示 MAC 地址长度
    unsigned char ar_pln;      // 协议地址长度。值为 4IP 地址长度
    __be16      ar_op;        // 操作码。值为 1 表示 ARP 请求，值为 2 表示 ARP 响应

    #if 0
        //
        * Ethernet looks like this : This bit is variable sized however...

        unsigned char      ar_sha[ETH_ALEN]; // sender hardware address
        unsigned char      ar_sip[4];        // sender IP address
        unsigned char      ar_tha[ETH_ALEN]; // target hardware address
        unsigned char      ar_tip[4];        // target IP address
    #endif
};
```

2) 构造以太网头部和 ARP 请求，发送 ARP 请求。

// 说明的变量

```
char *if_name, *tgt_ip, *src_ip;
uint8_t *src_mac, *tgt_mac, *ether_frame;
struct ether_arp arp_hdr; // ARP 头部
struct sockaddr_ll phy_layer_addr;
struct ifreq ifr;
...;
```

```

// 建立一个原始套接口，用于获取指定接口的 MAC 地址和 IPv4 地址
sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
// 用 ioctl() 函数获取名为 if_name 接口的 MAC 地址
memset(&ifr, 0, sizeof(ifr));
snprintf(ifr.ifr_name, sizeof(ifr.ifr_name), "%s", if_name);
ioctl(sd, SIOCGIFHWADDR, &ifr);
// 用 ioctl() 函数获取名为 if_name 接口的 IPv4 地址
ioctl(sd, SIOCGIFADDR, &ifr);
...;
// 建立一个原始套接口，用于发送以太网帧
sd = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
// 以下是构造 ARP
// 填 ARP 中硬件地址类型。它的值为 1 表示以太网
arp_hdr.arp_hrd = htons(1);
// ARP 中协议地址类型。它的值为 0x0800 表示 IP
arp_hdr.arp_pro = htons(ETH_P_IP);
// ARP 中硬件地址长度。它的值为 6 表示 MAC 地址长度
arp_hdr.arp_hln = 6;
// ARP 中的协议地址长度。它的值为 4 表示 IPv4 地址
arp_hdr.arp_pln = 4;
// ARP 中操作码。它的值为 1 表示 ARP 请求
arp_hdr.arp_op = htons(ARPOP_REQUEST);
// ARP 中发送方 MAC 地址
memcpy(&arp_hdr.arp_sha, src_mac, 6 * sizeof(uint8_t));
// ARP 中发送方 IPv4 地址
status = inet_pton(AF_INET, src_ip, &arp_hdr.arp_spa);
// ARP 中目的 MAC 地址。将它设为 0
memset(&arp_hdr.arp_tha, 0, 6 * sizeof(uint8_t));
// ARP 中目的 IPv4 地址
status = inet_pton(AF_INET, tgt_ip, &arp_hdr.arp_tpa);
// 以下是构造以太网帧
// 以太网帧中目的 MAC 地址
memcpy(ether_frame, tgt_mac, 6 * sizeof(uint8_t));
// 以太网帧中的源 MAC 地址
memcpy(ether_frame + 6, src_mac, 6 * sizeof(uint8_t));
// 以太网帧中类型。对于 ARP 请求或响应来说，该字段的值为 0x0806
ether_frame[12] = ETH_P_ARP / 256;
ether_frame[13] = ETH_P_ARP % 256;
// 向套接口 sd 中发送构造的以太网帧
sendto(sd, ether_frame, frame_length, 0, (struct sockaddr *) &phy_layer_addr,
sizeof(phy_layer_addr));
...;

```

3.1.5 进一步的扩展

通过对目标主机或网关发送伪造的 ARP 请求报文，欺骗目标的 ARP 高速缓存，使目标主机的数据发送到错误的地方，达到使对方无法正常上网的效果。

1) 定时向默认网关发送伪造的 ARP 请求，伪装成目标主机，修改网关的 ARP 高速缓存，使网关发送到目标主机的信息都发送到本机，实现目标主机的断网效果。向默认网关发送伪造的 ARP 请求的主要字段见表 3-1。

表 3-1 向默认网关发送伪造的 ARP 请求的主要字段

ARP 请求的字段	填入内容
目的以太网地址	网关 MAC 地址
源以太网地址	本机 MAC 地址
操作码	1
发送方 MAC 地址	本机 MAC 地址
发送方 IP 地址	目的主机 IP 地址
目的 MAC 地址	网关 MAC 地址
目的 IP 地址	网关 IP 地址

2) 定时向目标主机发送伪造的 ARP 请求，伪装成网关，修改目标主机的 ARP 高速缓存，使目标主机发送到网关的信息都发送到本机，实现目标主机的断网效果。向目标主机发送伪造的 ARP 请求的主要字段见表 3-2。

表 3-2 向目标主机发送伪造的 ARP 请求的主要字段

ARP 请求的字段	填入内容
目的以太网地址	目标主机 MAC 地址
源以太网地址	本机 MAC 地址
操作值	1
发送方 MAC 地址	本机 MAC 地址
发送方 IP 地址	网关 IP 地址
目的 MAC 地址	目标主机 MAC 地址
目的 IP 地址	目标主机 IP 地址

3.2 以太网 ARP 响应分析程序

3.2.1 课程设计目的

本课程设计的主要目的是通过接收与解析以太网 ARP 响应，掌握 ARP 协议的工作原理，理解以太网帧和 ARP 请求的格式。

3.2.2 课程设计内容

本课程设计要求设计与实现 Linux 下以太网 ARP 响应分析程序，具体内容如下：

- 1) 该程序能够实现以下功能：从网络中读取数据包，若 ARP 响应，则解析这个响应的各字段并输出；
- 2) 设计和实现以太网 ARP 响应分析程序；
- 3) 能够用 Wireshark 分析收到的 ARP 响应。

3.2.3 相关知识

本课程设计需要的 ARP 协议知识，请参考第二章第 3 节 ARP 协议分析。

本课程设计需要的 Linux 网络编程知识，请参考附录 B。

3.2.4 课程设计分析

1. 详细设计

以太网 ARP 响应分析程序的流程图如图 3-4 所示。

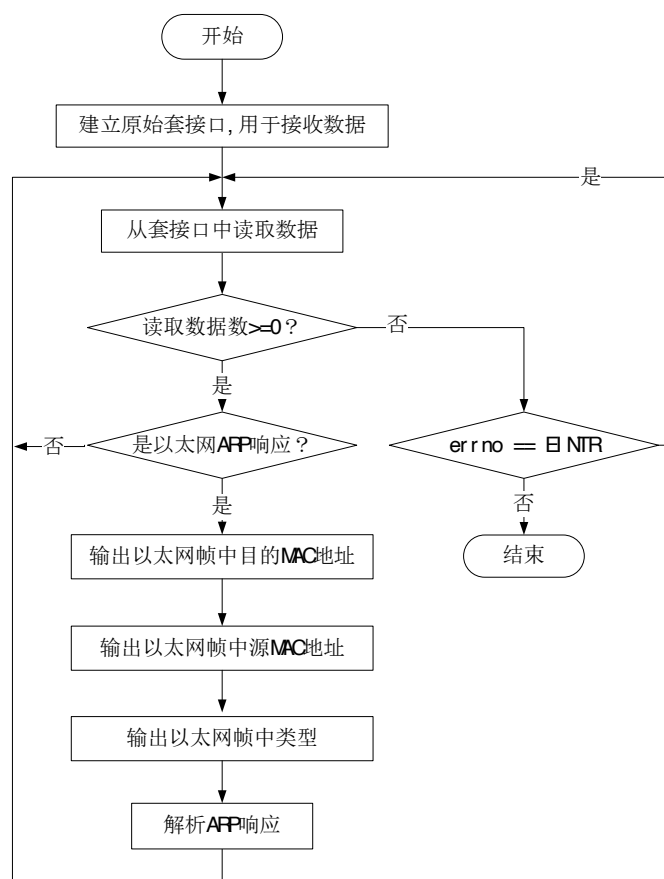


图 3-4 以太网 ARP 响应分析程序的流程图

解析 ARP 响应的流程图如图 3-5 所示。

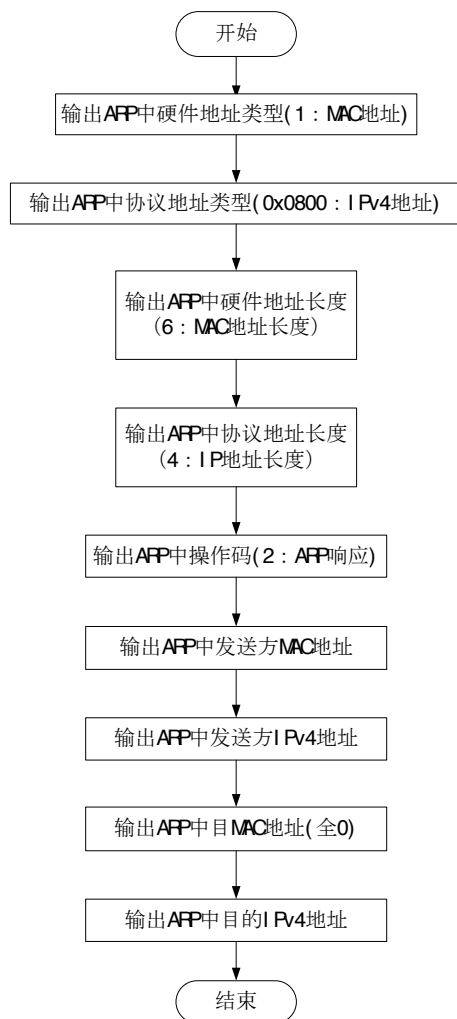


图 3-5 解析 ARP 响应流程图

2. 核心代码

1) ARP 结构定义。请参考太网 ARP 请求发送程序。

2) 以太网 ARP 接收与解析。

// 说明变量

```
uint8_t *ether_frame;
```

```
struct ether_arp *arp_hdr;
```

```
...;
```

// 建立一个原始套接口，用于接收数据

```
sd = socket(PF_PACKET, SOCK_RAW, htons (ETH_P_ALL));
```

```
...;
```

//从套接口 sd 读取以太网帧

```
if((status = recv(sd, ether_frame, IP_MAXPACKET, 0)) < 0) {
    if(errno == EINTR) {
        memset(ether_frame, 0, IP_MAXPACKET * sizeof (uint8_t));
        continue;
    } else { ...;}
}
```

// 如果它是 ARP 响应，则输出这个 ARP 以太网帧

```

if((((ether_frame[12]) << 8) + ether_frame[13]) == ETH_P_ARP) &&
    (ntohs(arp_hdr->arp_op) == ARPOP_REPLY)) {
    // 输出以太网帧中的头部
    printf("%02x:%02x:%02x:%02x:%02x:%02x\n", ether_frame[0], ether_frame[1],
        ether_frame[2], ether_frame[3], ether_frame[4], ether_frame[5]);
    printf("%02x:%02x:%02x:%02x:%02x:%02x\n", ether_frame[6], ether_frame[7],
        ether_frame[8], ether_frame[9], ether_frame[10], ether_frame[11]);
    printf("Ethernet type code (0x0806 for ARP): 0x%02x%02x\n",
        ether_frame[12], ether_frame[13]);
    // 输出以太网帧中的数据 (ARP 响应头部)
    printf("\nEthernet data (ARP reply header):\n");
    printf("Hardware address type (1 for ethernet): %u\n",
        ntohs(arp_hdr->arp_hrd));
    printf("Protocol address type (0x0800 for IPv4 addresses): 0x%04x\n",
        ntohs(arp_hdr->arp_pro));
    printf("Hardware (MAC) address length: %u bytes\n", arp_hdr->arp_hln);
    ...;
}

```

3.2.5 进一步的扩展

通过对目标主机或网关发送伪造的 ARP 响应，欺骗目标的 ARP 高速缓存，使目标主机的数据发送到错误的地方，达到使对方无法正常上网的效果。

1) 定时向默认网关发送伪造的 ARP 响应，伪装成目标主机，修改网关的 MAC 高速缓存，使网关发送到目标主机的信息都发送到本机，实现目标主机的断网效果。向默认网关发送伪造的 ARP 响应的主要字段见表 3-3。

表 3-3 向默认网关发送伪造的 ARP 响应的主要字段

ARP 响应字段	填入内容
目的以太网地址	网关 MAC 地址
源以太网地址	本机 MAC 地址
操作值	2 (响应)
发送方 MAC 地址	本机 MAC 地址
发送方 IP 地址	目的主机 IP 地址
目的 MAC 地址	网关 MAC 地址
目的 IP 地址	网关 IP 地址

2) 定时向目标主机发送伪造的 ARP 响应报文，伪装成网关，修改目标主机的 MAC 表，使目标主机发送到网关的信息都发送到本机，实现目标主机的断网效果。向目标主机发送伪造的 ARP 响应的主要字段见表 3-4。

表 3-4 向目标主机发送伪造的 ARP 响应的主要字段

ARP 响应字段	填入内容
目的以太网地址	目标主机 MAC 地址
源以太网地址	本机 MAC 地址
操作值	2（响应）
发送方 MAC 地址	本机 MAC 地址
发送方 IP 地址	网关 IP 地址
目的 MAC 地址	目标主机 MAC 地址
目的 IP 地址	目标主机 IP 地址

3.3 ping 程序

3.3.1 课程设计目的

本课程设计主要目的是通过构造和发送 ICMP 回射请求报文,接收和解析 ICMP 响应报文,使学生掌握 ICMP 协议的工作原理,理解 ICMP 回射报文格式。

3.3.2 课程设计内容

本课程设计要求设计与实现 Linux 下 ping 程序,具体如下:

- 1) 该程序能够实现以下功能:能够构造 ICMP 回射请求报文,通过信号量定时发送回射请求报文,接收和解析 ICMP 回射响应报文;
- 2) 设计和实现 ping 程序;
- 3) 能够用 Wireshark 分析的 ICMP 报文。

3.3.3 相关知识

ping 程序

ping 程序发送一份 ICMP 回射请求 (echo request) 报文给指定主机,并等待返回 ICMP 回射响应 (echo reply);该程序会按时间和反应成功的次数,估计往返时延和丢包率。

ICMP 回射报文的格式如图 3-6 所示:

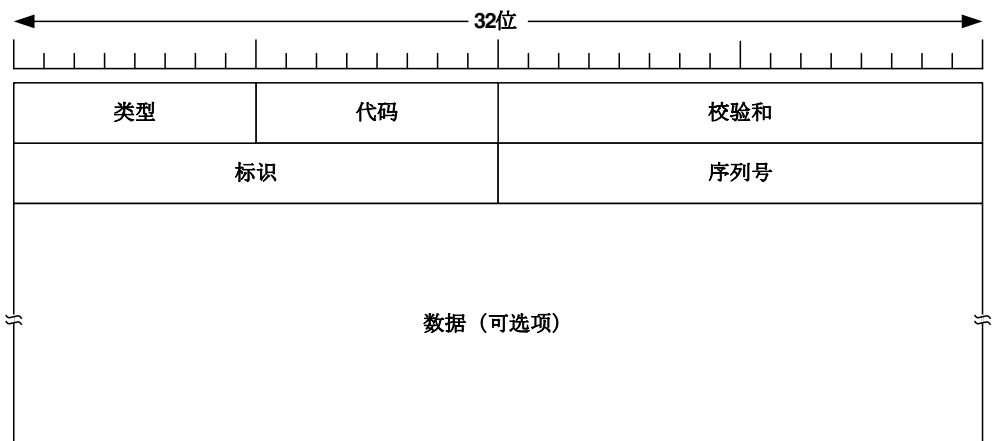


图 3-6 ICMP 回射报文的格式

类型的值为 8 表示回射请求，类型的值为 1 表示回射响应。

代码为 0。

校验和为从 TYPE 开始到 IP 包结束的校验和。

如果代码的值为 0，标识用来匹配回射请求和回射响应报文

ICMP 回射报文封装成一个 IP 数据包的格式如图 3-7 所示。

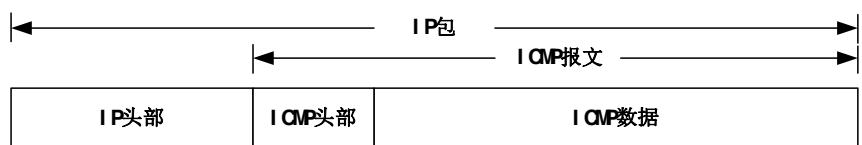


图 3-7 ICMP 回射报文封装

本课程设计需要的 Linux 网络编程知识，请参考附录 B。

3.3.4 课程设计分析

1. 详细设计

main 函数如图 3-8 所示。

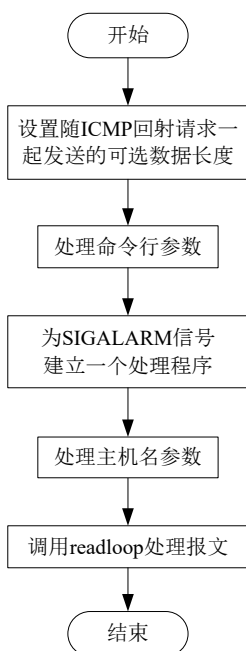


图 3-8 main 函数流程图

readloop() 函数流程图如图 3-9 所示。

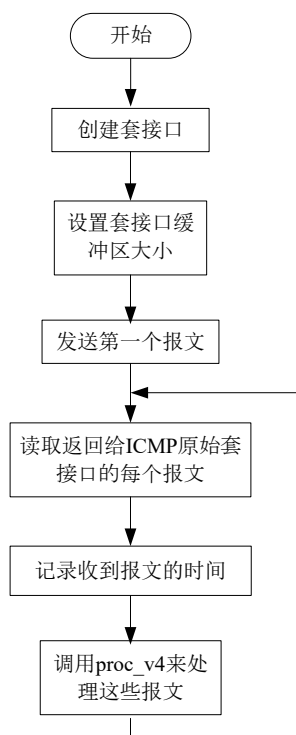


图 3-9 readloop() 函数流程图

proc_v4() 函数流程图如图 3-10 所示。

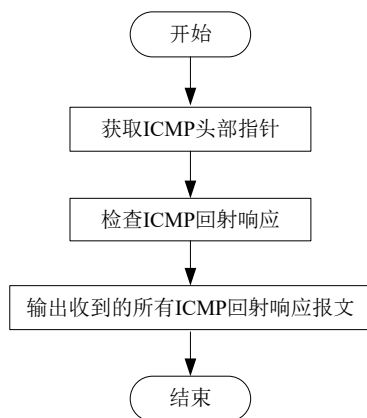


图 3-10 proc_v4() 函数流程图

send_v4() 函数流程图如图 3-11 所示。

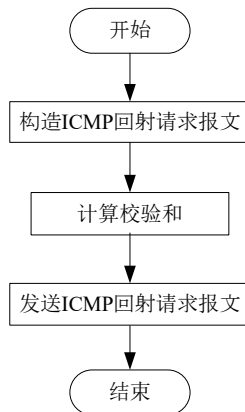


图 3-11 send_v4() 函数流程图

2. 核心代码

1) IPv4 头结构在<netinet/ip.h>中定义。IPv4 头结构如下所示。

```

struct ip
{
#if __BYTE_ORDER == __LITTLE_ENDIAN //Intel x86 机器采用 little-endian 字节序
    unsigned int ip_hl:4;           // 头部长度
    unsigned int ip_v:4;           // 版本
#endif
#if __BYTE_ORDER == __BIG_ENDIAN
    unsigned int ip_v:4;
    unsigned int ip_hl:4;
#endif
    u_int8_t ip_tos;                // tos
    u_short ip_len;                 // 总长度
    u_short ip_id;                  // 标识
    u_short ip_off;                 // 标志和分段偏移
#define IP_RF 0x8000                // 保留分段标志位
#define IP_DF 0x4000                // 不允许分段标志位
#define IP_MF 0x2000                // 更多分段标志位
#define IP_OFFMASK 0x1fff           // 分段位移的掩码
    u_int8_t ip_ttl;                // TTL
    u_int8_t ip_p;                  // 协议
    u_short ip_sum;                 // 检验和
    struct in_addr ip_src, ip_dst;  // 源地址和目的地址
};
  
```

ICMP 头结构在<netinet/ip_icmp.h>中定义。ICMP 头结构如下所示。

```

struct icmp
{
    u_int8_t icmp_type; // 类型
    u_int8_t icmp_code; // 代码
    u_int16_t icmp_cksum; // 校验和
    union
  
```

```

{
    u_char ih_pptr;
    struct in_addr ih_gwaddr;
    struct ih_idseq
    {
        u_int16_t icd_id;
        u_int16_t icd_seq;
    } ih_idseq;
    u_int32_t ih_void;

    // ICMP_UNREACH_NEEDFRAG -- Path MTU Discovery (RFC1191)
    struct ih_pmtu
    {
        u_int16_t ipm_void;
        u_int16_t ipm_nextmtu;
    } ih_pmtu;

    struct ih_rtradv
    {
        u_int8_t irt_num_addrs;
        u_int8_t irt_wpa;
        u_int16_t irt_lifetime;
    } ih_rtradv;
} icmp_hun;

#define icmp_pptr    icmp_hun.ih_pptr
#define icmp_gwaddr  icmp_hun.ih_gwaddr
#define icmp_id      icmp_hun.ih_idseq.icd_id //标识
#define icmp_seq     icmp_hun.ih_idseq.icd_seq //序列号
#define icmp_void    icmp_hun.ih_void
#define icmp_pmvoid  icmp_hun.ih_pmtu.ipm_void
#define icmp_nextmtu icmp_hun.ih_pmtu.ipm_nextmtu
#define icmp_num_addrs icmp_hun.ih_rtradv.irt_num_addrs
#define icmp_wpa     icmp_hun.ih_rtradv.irt_wpa
#define icmp_lifetime icmp_hun.ih_rtradv.irt_lifetime

union
{
    struct
    {
        u_int32_t its_otime;
        u_int32_t its_rtime;
        u_int32_t its_ttime;
    } id_ts;
    struct
    {

```

```

    struct ip idi_ip;
    // options and then 64 bits of data
} id_ip;
struct icmp_ra_addr id_radv;
u_int32_t id_mask;
u_int8_t id_data[1];
} icmp_dun;
#define icmp_otime icmp_dun.id_ts.its_otime
#define icmp_rtime icmp_dun.id_ts.its_rtime
#define icmp_ttime icmp_dun.id_ts.its_ttime
#define icmp_ip icmp_dun.id_ip.idi_ip
#define icmp_radv icmp_dun.id_radv
#define icmp_mask icmp_dun.id_mask
#define icmp_data icmp_dun.id_data //ICMP数据
};

```

2) 函数的部分实现

readloop() 函数的部分实现:

```

// 建立一个 ICMP 原始套接口
sockfd = socket(sasend->sa_family, SOCK_RAW, icmpproto);
// 为安全性考虑, 将进程的有效用户 ID 设成进程的实际用户 ID
setuid(getuid());
// 将套接口接收缓冲区设为 61440 字节, 主要为了减少接收缓冲区溢出的可能性
size = 60 * 1024;
setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &size, sizeof(size));
// 调用 SIGALRM 信号处理程序来发送一个数据包, 并调度下一次 SIGALRM 为一秒之后
sig_alm(SIGALRM);
for(;;) {
    len = salen;
    n = recvfrom(sockfd, recvbuf, sizeof(recvbuf), 0, sarecv, &len);
    // 记录数据包收到的时间
    gettimeofday(&tval, NULL);
    // 处理 ICMP
    proc_v4(recvbuf, n, &tval);
}

```

sig_alm(int signo) 函数的实现:

```

// SIGALRM 信号处理函数
sig_alm(int signo)
{
    send_v4();
    // 设置信号 SIGALRM 在经过 1 秒数后发送给目前的进程
    alarm(1);
    return;
}

```

proc_v4 函数的部分实现:


```

// IP 头部指针
ip = (struct ip *) ptr;    // start of IP header
// IP 头部长度。它是以 4 字节为一个单位计数的，将它乘以 4 才是以字节为单位的头
部长度
hlenl = ip->ip_hl << 2;
// ICMP 头部指针
icmp = (struct icmp *) (ptr + hlenl);
// 如果是 ICMP 回射响应，那么检查标识符字段，这个响应是不是这个进程发出的请求
的响应
if(icmp->icmp_type == ICMP_ECHOREPLY) {
    if(icmp->icmp_id != pid)
        return;
    if(icmplen < 16)
        err_quit("icmplen (%d) < 16", icmplen);
    // 将请求发出时间（包含在 ICMP 响应的可选数据部分内）与当前时间相减，可计算
    出 RTT（以毫秒为单位）
    tvsend = (struct timeval *) icmp->icmp_data;
    tv_sub(tvrecv, tvsend);
    rtt = tvrecv->tv_sec * 1000.0 + tvrecv->tv_usec / 1000.0;
    ...;
}
send_v4() 函数的部分实现：
// 指向 ICMP 头
icmp = (struct icmp *) sendbuf;
// ICMP 中类型
icmp->icmp_type = ICMP_ECHO;
...;
// ICMP 中数据。它的值为当前时间
gettimeofday((struct timeval *) icmp->icmp_data, NULL);
...;
// 发送 ICMP。由于没使用 IP_HDRINCL 选项，内核将构造 IPv4 头部并将其放在缓冲区
前面
sendto(sockfd, sendbuf, len, 0, sasend, salen);

```

3.3.5 进一步的扩展

Ping 命令实现的扩充。在给定的 Ping 程序的基础上做如下功能扩充：

- 1) -h 显示帮助信息。
- 2) -b 允许 ping 一个广播地址，只用于 IPv4。
- 3) -t 设置 ttl 值，只用于 IPv4。
- 4) -q 安静模式。不显示每个收到的包的分析结果，只在结束时，显示汇总结果。

3.4 多播客户/服务器程序

3.4.1 课程设计目的

本课程设计主要目的是通过服务器向指定多播地址发送多播报文和客户加入多播组后接收多播报文，使学生掌握多播的工作原理，理解多播组地址到以太网地址的转换，在局域网中加入和离开多播组过程。

3.4.2 课程设计内容

本课程设计要求设计与实现 Linux 下多播客户/服务器程序，具体内容如下：

- 1) 该程序能够实现以下功能：多播服务器能每 2 秒向指定多播地址发送多播报文，本局域网中的多播客户能够加入指定多播组，并能接收 8 个多播报文后离开多播组；
- 2) 设计和实现多播客户/服务器程序；
- 3) 能够用 Wireshark 分析的多播报文。

3.4.3 相关知识

IP 多播是一种允许一台或多台主机（多播源）发送单一数据包到多台主机的 TCP/IP 网络技术。多播作为一点对多点的通信，是节省网络带宽的有效方法之一。多播能使一个或多个多播源只把数据包发送给特定的多播组，而只有加入该多播组的主机才能接收到数据包。目前 IP 多播技术被广泛应用在直播体育比赛、向复制服务器池更新程序、网络视频会议、多媒体远程教育等方面。

1. IP 多播地址

范围从 224.0.0.0 到 239.255.255.255 的 IPv4 地址被划分为局部链接多播地址、预留多播地址和管理权限多播地址三类。其中局部链接多播地址范围在 224.0.0.0~224.0.0.255，这是为路由协议和其它用途保留的地址，路由器并不转发属于这个范围的 IP 包；预留多播地址为 224.0.1.0~238.255.255.255，可用于全球范围（如 Internet）或网络协议；管理权限多播地址为 239.0.0.0~239.255.255.255，可供组织内部使用，类似于私有 IP 地址，不能用于 Internet，可限制多播范围。

2. 多播组

使用同一个 IP 多播地址接收多播数据包的所有主机构成了一个主机组，也称为多播组。一个多播组的成员是随时变动的，一台主机可以随时加入或离开多播组，多播组成员的数目和所在的地理位置也不受限制，一台主机也可以属于几个多播组。同时不属于某一个多播组的主机也可以向该多播组发送数据包。一些多播组地址被 IANA 确定为知名地址，它们被当作永久主机组。如：224.0.0.1 代表“子网内的所有支持多播的主机组”，224.0.0.2 代表“子网内的所有支持多播的路由器组”，224.0.1.1 用作网络时间协议 NTP。

3. 多播组地址到以太网地址的转换

IP 多播数据报传送依赖于低层物理网络，应用最广泛的以太网数据链路层支持多播功能。以太网的物理地址的第 1 个字节的最低位表示单地址或组地址：0 为单地址，1 为组地址，如：多播地址，01:00:5E:00:00:01。IANA 拥有高位 24bit 为 00:00:5e 的以太网地址

块，其中用于多播的以太网地址范围从 01:00:5e:00:00:00 到 01:00:5e:7f:ff:ff。当 IP 多播数据报交到底层以太网进行传送时，多播地址到以太网地址的映射关系如图 3-12 所示。

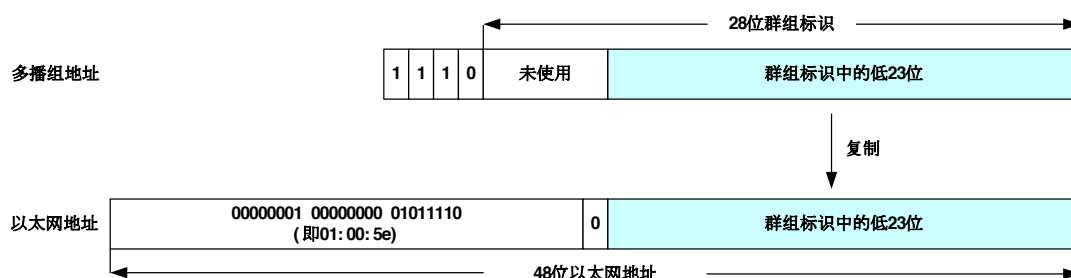


图 3-12 多播组地址到以太网地址映射

多播组号中的 5 bit 在映射过程中被忽略，地址映射是不唯一的，设备驱动程序或 IP 层就必须对数据报进行过滤。

4. 多播的工作过程

在局域网中，主机的网络接口将到目的主机的数据包发送到高层，这些数据包中的目的地址是物理接口地址或广播地址。如果主机已经加入到一个多播组中，主机的网络接口就会识别出发送到该组成员的数据包。因此，如果主机接口的物理地址为 8C:70:5A:46:F8:6C，其加入的多播组为 224.1.2.3，则发送给主机的数据包中的目的地址必是下面三种类型之一：接口地址：8C:70:5A:46:F8:6C，广播地址：FF:FF:FF:FF:FF:FF:FF:FF，多播地址：01:00:5E:01:10:11。

广域网中，路由器必须支持多播路由。当主机中运行的进程加入到某个多播组中时，主机向子网中的所有多播路由器发送 IGMP (Internet 组管理协议) 报文，告诉路由器凡是发送到这个多播组的多播报文都必须发送到本地的子网中，这样主机的进程就可以接收到报文了。子网中的路由器再通知其它的路由器，这些路由器就知道该将多播报文转发到哪些子网中去。

子网中的路由器也向 224.0.0.1 发送一个 IGMP 报文，要求组中的主机提供组的相关信息。组中的主机收到这个报文后，经过一段随机时间，再向路由器发送响应。这样可防止了组中所有的主机同时向路由器发送响应，造成网络拥塞。主机向多播地址发送一个报文做为对路由器的响应，组中的其它主机一旦看到这个响应报文，就不再发送响应报文了，因为组中的主机向路由器提供的都是相同的信息。

如果组中的主机都退出了，路由器就收不到响应，路由器认为该组目前没有主机加入，于是停止到该子网报文的路由。IGMPv2 的解决方案是组中的主机在退出时向 224.0.0.2 发送报文通知多播路由器。

本课程设计的多播客户和服务在同一局域网中。

本课程设计需要的 Linux 网络编程知识，请参考附录 B。

3.4.4 课程设计分析

1. 详细设计

多播客户程序流程图如图 3-13 所示。

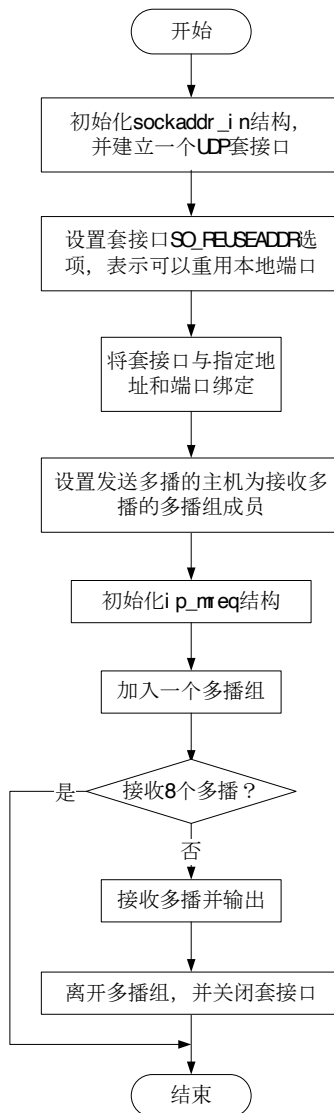


图 3-13 多播客户程序流程图

多播服务器程序流程图如图 3-14 所示。

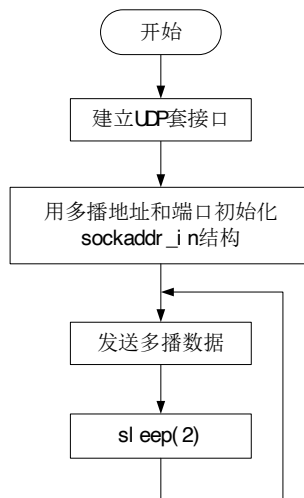


图 3-14 多播服务器程序流程图

2. 核心代码

1) ip_mreq 结构在< >中定义。ip_mreq 结构如下所示。

ip_mreq 结构在<netinet/in.h>定义:

```
#if defined __USE_MISC || defined __USE_GNU
// IPv4 多播请求
struct ip_mreq
{
    struct in_addr imr_multiaddr; // IP 多播地址
    struct in_addr imr_interface; // 本地接口的 IP 地址
};
```

2) 多播客户和服务程序的部分实现

多播客户程序的部分实现:

// 建立一个 UDP 套接口

```
sd = socket(PF_INET, SOCK_DGRAM, 0);
```

// 设定 SO_REUSEADDR, 允许多个应用绑定同一个本地端口接收数据包

```
loop = 1;
```

```
setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &loop, sizeof(loop));
```

// 用 bind 绑定本地端口, IP 为 INADDR_ANY, 从而能接收多播数据包

```
bind(sd, (struct sockaddr *)&sin, sizeof(sin));
```

// 设置发送多播的主机为接收多播的多播组成员

```
loop = 1;
```

```
setsockopt(sd, IPPROTO_IP, IP_MULTICAST_LOOP, &loop, sizeof(loop));
```

// 将所加入的多播组的 IP 多播地址复制到一个 ip_mreq 结构中

```
command.imr_multiaddr.s_addr = inet_addr("224.1.2.3");
```

// 所要加入多播组的那个本地接口的 IP 地址复制到一个 ip_mreq 结构中

```
command.imr_interface.s_addr = htonl(INADDR_ANY);
```

// 加入一个多播组。进一步告诉 Linux 内核, 特定的套接口即将接收多播数据

```
setsockopt(sd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &command, sizeof(command))
```

// 接收 8 个多播

```
while(iter++ < 8) {
```

```
    sin_len = sizeof(sin);
```

```
    recvfrom(sd, message, 256, 0, (struct sockaddr *)&sin, &sin_len);
```

```
}
```

// 退出多播组

```
setsockopt(sd, IPPROTO_IP, IP_DROP_MEMBERSHIP, &command, sizeof(command));
```

多播服务器程序的部分实现:

// 建立 UDP 套接口

```
socket_descriptor = socket(AF_INET, SOCK_DGRAM, 0);
```

// 用多播地址和端口初始化 sockaddr_in 结构

```
memset(&address, 0, sizeof(address));
```

```
address.sin_family = AF_INET;
```

```
address.sin_addr.s_addr = inet_addr("224.1.2.3");
```

```
address.sin_port = htons(port);
```

// 开始 IP 多播

```

while(1) {
    sendto(socket_descriptor, "test from multicast",
           sizeof("test from multicast"), 0, (struct sockaddr *)&address,
           sizeof(address));
    sleep(2);
}

```

3.4.5 进一步的扩展

无。

3.5 UDP 报文发送程序

3.5.1 课程设计目的

UDP 协议是一个无连接的传输协议。本课程设计的主要目的是通过构造与发送 UDP 报文，掌握 UDP 协议的工作原理，理解 UDP 报文格式。

3.5.2 课程设计内容

本课程设计要求设计与实现 Linux 下 UDP 报文发送程序，具体内容如下：

- 1) 该程序能够实现以下功能：能够获取指定接口的索引；能够构造 IP 头部和 UDP 报文，并向指定地址发送这个 UDP 报文；
- 2) 设计和实现 UDP 报文发送程序；
- 3) 能够用 Wireshark 分析发送的 UDP 报文。

3.5.3 相关知识

本课程设计需要的 IP 协议知识，请参考 2.4 IP 协议分析。

本课程设计需要的 UDP 协议知识，请参考 2.6 UDP 协议分析。

本课程设计需要的 Linux 网络编程知识，请参考附录 B。

3.5.4 课程设计分析

1. 详细设计

UDP 报文发送程序的流程图如图 3-15 所示。

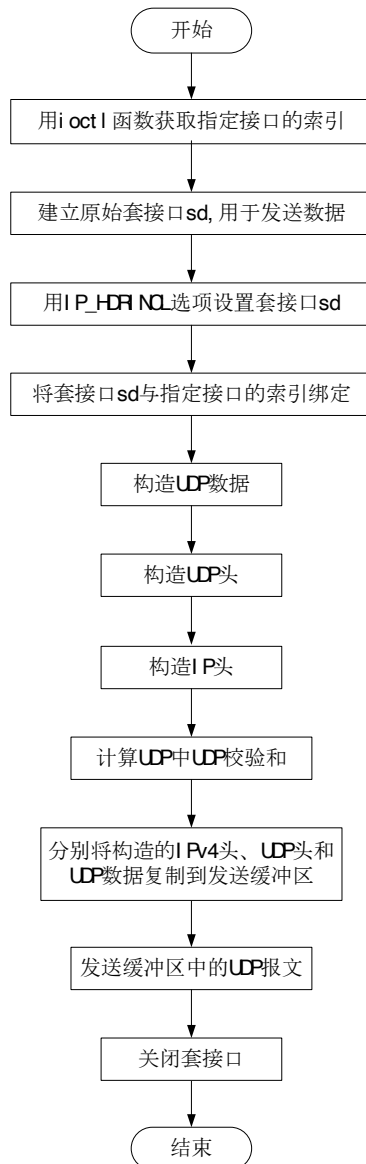


图 3-15 UDP 报文发送程序流程图

构造 UDP 头的流程图如图 3-16 所示。

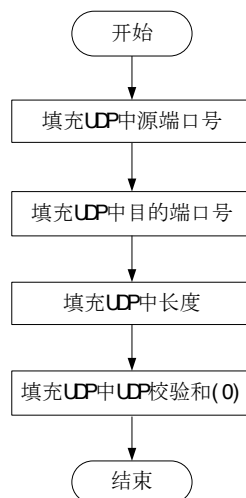


图 3-16 构造 UDP 头流程图

2. 核心代码

1) IPv4 头结构在<netinet/ip.h>中定义。UDP 头结构在<netinet/udp.h>中定义。IPv4 结构如下所示。

```
// UDP header as specified by RFC 768, August 1980.
#ifdef __FAVOR_BSD
struct udphdr
{
    u_int16_t uh_sport;        // 源端口号
    u_int16_t uh_dport;        // 目的端口号
    u_int16_t uh_ulen;         // UDP长度
    u_int16_t uh_sum;          // UDP检验和
};
#else
struct udphdr
{
    u_int16_t source;
    u_int16_t dest;
    u_int16_t len;
    u_int16_t check;
};
#endif
```

2) UDP 报文构造和发送

// 说明变量

```
char *if_name, *tgt_ip, *src_ip;
struct ip ip_hdr;
struct udphdr udp_hdr;
char *data, *packet;
struct sockaddr_in sin;
struct ifreq ifr;
...;
// 建立一个原始套接口，用于获取指定接口的索引
sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
// 用 ioctl() 函数获取名为 if_name 接口的索引
// 用于将发送数据包的套接口与这个接口索引绑定
memset(&ifr, 0, sizeof(ifr));
snprintf(ifr.ifr_name, sizeof(ifr.ifr_name), "%s", if_name);
ioctl(sd, SIOCGIFINDEX, &ifr);
// 发送数据包时，用户需要指定要发送数据包的目的 IPv4 地址
memset(&sin, 0, sizeof(struct sockaddr_in));
sin.sin_family = AF_INET;
status = inet_pton(AF_INET, tgt_ip, &(sin.sin_addr));
// 建立一个原始套接口，用于发送数据包
sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
// 使用套接口 IP_HDRINCL 选项设置套接口。在之后发送数据时，内核将构造
```



```

// 以太网帧头部数据并将其放到 IP 头部前，用户需要设置 IP 头部数据
setsockopt(sd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on));
// 将套接口 sd 与指定接口的索引绑定
setsockopt(sd, SOL_SOCKET, SO_BINDTODEVICE, &ifr, sizeof(ifr));
// 构造 UDP 中数据
strcpy(data, "Sending an IPv4 UDP packet through a raw socket.");
datalen = strlen("Sending an IPv4 UDP packet through a raw socket.");
// 构造 UDP 头
// UDP 中源端口号
udp_hdr.uh_sport = htons(9866);
// UDP 中目的端口号
udp_hdr.uh_dport = htons(161);
// UDP 中 UDP 长度 (32 bit words)，包括 UDP 头和 UDP 数据
udp_hdr.uh_ulen = htons(UDP_HDRLEN + datalen);
// UDP 中校验和，先设为 0，构造 IP 头后再计算
udp_hdr.uh_sum = 0;
// 构造 IPv4 包头部。IPv4 包头见<netinet/ip.h>
// IPv4 包中头部长度。它的值为 5
ip_hdr.ip_hl = IP4_HDRLEN >> 2;
// IPv4 包中版本号。它的值为 4 表示 IPv4
ip_hdr.ip_v = 4;
// IPv4 包中类型
ip_hdr.ip_tos = 0;
// IPv4 包中总长度。包括 IPv4 头部长度、UDP 头部长度和 UDP 数据
ip_hdr.ip_len = htons(IP4_HDRLEN + UDP_HDRLEN + datalen);
// IPv4 包中标识
ip_hdr.ip_id = htons(getpid());
// IPv4 包中保留分段标志位、不允许分段标志位、更多分段标志位和分段偏移都设为
0
ip_hdr.ip_off = 0;
// IPv4 包中 TTL。默认设为最大值 255
ip_hdr.ip_ttl = 255;
// IPv4 包中协议。它的值为 17 表示 UDP
ip_hdr.ip_p = IPPROTO_UDP;
// IPv4 包中源 IPv4 地址
status = inet_pton(AF_INET, src_ip, &(ip_hdr.ip_src));
// IPv4 包中目的 IPv4 地址
ip_hdr.ip_dst.s_addr = sin.sin_addr.s_addr;
// IPv4 包中头部校验和
ip_hdr.ip_sum = 0;
ip_hdr.ip_sum = checksum((uint16_t *) &ip_hdr, IP4_HDRLEN);
// 计算 UDP 中校验和
udp_hdr.uh_sum = udp4_checksum(ip_hdr, udp_hdr, data, datalen);
// 将构造的 IPv4 头部复制到发送缓冲区中

```

```

memcpy(packet, &ip_hdr, IP4_HDRLEN * sizeof(uint8_t));
// 将构造的 UDP 头部复制到发送缓冲区中
memcpy((packet + IP4_HDRLEN), &udp_hdr, UDP_HDRLEN * sizeof(uint8_t));
// 将构造的 UDP 数据复制到发送缓冲区中
memcpy(packet + IP4_HDRLEN + UDP_HDRLEN, data, datalen * sizeof(uint8_t));
// 发送数据包
sendto(sd, packet, IP4_HDRLEN + UDP_HDRLEN + datalen, 0,
        (struct sockaddr *) &sin, sizeof(struct sockaddr));

```

3.5.5 进一步的扩展

详细了解 UDP 报文格式和 DNS 报文格式基础上，构造 DNS 查询。可以设置 DNS 报文中标识为 1234，标志为 0，问题数为 1，然后把回答资源记录数、权威资源记录数和附加资源记录数都设置为 0。

3.6 TCP SYN 报文发送程序

3.6.1 课程设计目的

TCP 协议是面向连接的、可靠的、端到端的和基于字节流的传输层协议。本课程设计的主要目的是通过构造与发送 TCP SYN 报文，掌握 TCP 协议的工作原理，理解 TCP 报文格式。

3.6.2 课程设计要求

本课程设计要求设计与实现 Linux 下 TCP SYN 报文发送程序，具体内容如下：

- 1) 该程序能够实现以下功能：能够获取指定接口的索引；能够构造 IP 头部和 TCP 报文，并向指定地址发送这个 TCP SYN 报文；
- 2) 设计和实现 TCP SYN 报文发送程序；
- 3) 能够用 Wireshark 分析发送的 TCP SYN 报文。

3.6.3 相关知识

本课设需要的 IP 协议知识，请参考 2.4 IP 协议分析。

本课设需要的 TCP 协议知识，请参考 2.7 TCP 协议分析。

本课设需要的 Linux 网络编程知识，请参考附录 B。

3.6.4 课程设计要求

1. 详细设计

TCP SYN 报文发送程序的流程图如图 3-17 所示。

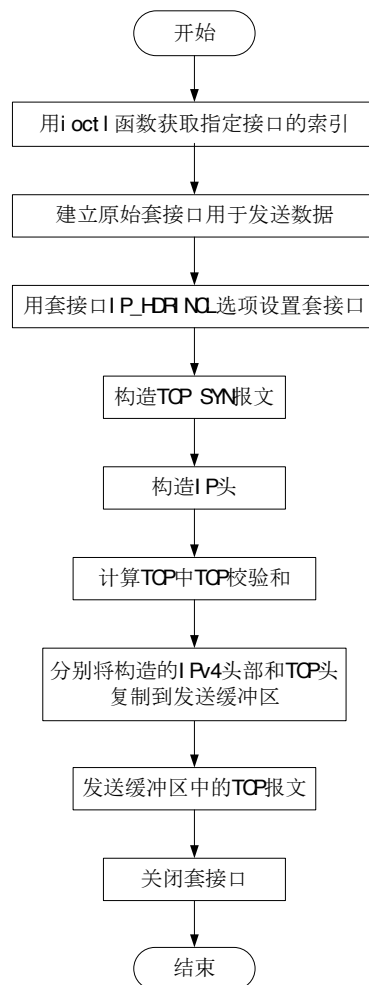


图 3-17 TCP SYN 报文发送程序流程图

构造 TCP SYN 报文的流程图如图 3-18 所示。

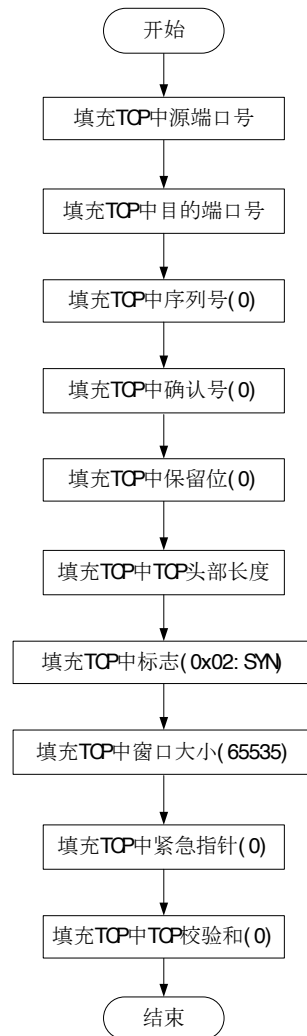


图 3-18 构造 TCP SYN 报文流程图

构造 IP 头的流程图如图 3-19 所示。

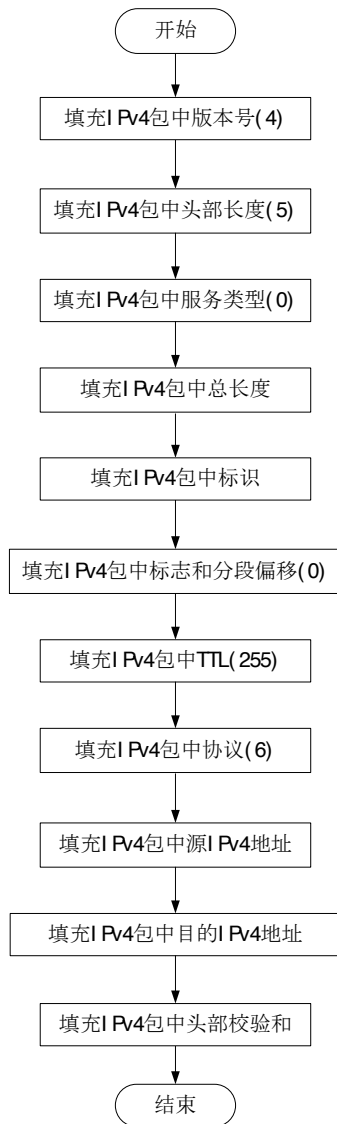


图 3-19 构造 IP 头流程图

2. 核心代码

1) IPv4 头结构在< netinet/ip.h >中定义。IPv4 头结构如下所示。

```

struct ip
{
    #if __BYTE_ORDER == __LITTLE_ENDIAN //Intel x86 机器采用 little-endian 字节序
        unsigned int ip_hl:4;           // 头部长度
        unsigned int ip_v:4;           // 版本
    #endif
    #if __BYTE_ORDER == __BIG_ENDIAN
        unsigned int ip_v:4;
        unsigned int ip_hl:4;
    #endif
    u_int8_t ip_tos;                   // tos
    u_short ip_len;                    // 总长度

```

```

    u_short ip_id;                // 标识
    u_short ip_off;               // 标志和分段偏移
#define IP_RF 0x8000             // 保留分段标志位
#define IP_DF 0x4000             // 不允许分段标志位
#define IP_MF 0x2000             // 更多分段标志位
#define IP_OFFMASK 0x1fff        // 分段位移的掩码
    u_int8_t ip_ttl;              // TTL
    u_int8_t ip_p;                // 协议
    u_short ip_sum;               // 检验和
    struct in_addr ip_src, ip_dst; // 源地址和目的地址
};

```

TCP 头结构在<netinet/tcp.h>中定义。TCP 头结构如下所示。

```

# ifdef __FAVOR_BSD                // 本课程设计采用这种TCP头部结构
typedef u_int32_t tcp_seq;
//
* TCP header.
* Per RFC 793, September, 1981.

struct tcphdr
{
    u_int16_t th_sport;            // 源端口
    u_int16_t th_dport;            // 目的端口
    tcp_seq th_seq;                // 序列号
    tcp_seq th_ack;                // 确认号
# if __BYTE_ORDER == __LITTLE_ENDIAN // Intel x86机器采用little-endian字节序
    u_int8_t th_x2:4;              // 保留
    u_int8_t th_off:4;              // TCP头部长度（32位字）
# endif
# if __BYTE_ORDER == __BIG_ENDIAN
    u_int8_t th_off:4;
    u_int8_t th_x2:4;
# endif
    u_int8_t th_flags;
# define TH_FIN    0x01
# define TH_SYN    0x02
# define TH_RST    0x04
# define TH_PUSH   0x08
# define TH_ACK    0x10
# define TH_URG    0x20
    u_int16_t th_win;               // 窗口大小
    u_int16_t th_sum;               // TCP校验和
    u_int16_t th_urp;               // 紧急指针
};

```

```

# else // !__FAVOR_BSD
struct tcphdr
{
    u_int16_t source;
    u_int16_t dest;
    u_int32_t seq;
    u_int32_t ack_seq;
# if __BYTE_ORDER == __LITTLE_ENDIAN
    u_int16_t res1:4;
    u_int16_t doff:4;
    u_int16_t fin:1;
    u_int16_t syn:1;
    u_int16_t rst:1;
    u_int16_t psh:1;
    u_int16_t ack:1;
    u_int16_t urg:1;
    u_int16_t res2:2;
# elif __BYTE_ORDER == __BIG_ENDIAN
    u_int16_t doff:4;
    u_int16_t res1:4;
    u_int16_t res2:2;
    u_int16_t urg:1;
    u_int16_t ack:1;
    u_int16_t psh:1;
    u_int16_t rst:1;
    u_int16_t syn:1;
    u_int16_t fin:1;
# else
#   error "Adjust your <bits/endian.h> defines"
# endif
    u_int16_t window;
    u_int16_t check;
    u_int16_t urg_ptr;
};# endif // __FAVOR_BSD

```

2) TCP SYN 报文构造和发送。

// 说明变量

```

char *if_name, *tgt_ip, *src_ip;
struct ip ip_hdr; //IPv4 头
struct tcphdr tcp_hdr; //TCP 头
char *packet;
struct sockaddr_in sin;
struct ifreq ifr;
...;

```

// 建立一个原始套接口，用于获取指定接口的索引

```

sd = socket (AF_INET, SOCK_RAW, IPPROTO_RAW));
//用 ioctl() 函数获取名为 if_name 接口的索引
ioctl(sd, SIOCGIFINDEX, &ifr);
// 发送数据包时, 用户需要指定要发送数据包的目的 IPv4 地址
memset(&sin, 0, sizeof(struct sockaddr_in));
sin.sin_family = AF_INET;
status = inet_pton (AF_INET, tgt_ip, &(sin.sin_addr));
// 建立一个原始套接口, 用于发送数据包
sd = socket (AF_INET, SOCK_RAW, IPPROTO_RAW));
// 使用套接口 IP_HDRINCL 选项设置套接口。在之后发送数据时, 内核将构造
// 以太网帧头部数据并将其放到 IP 头部前, 用户需要设置 IP 头部数据
setsockopt(sd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on));
// 将套接口 sd 与指定接口的索引绑定
setsockopt(sd, SOL_SOCKET, SO_BINDTODEVICE, &ifr, sizeof(ifr));
// 构造 TCP 头部
// TCP 中源端口号
tcp_hdr.th_sport = htons(9866);
// TCP 中目的端口号
tcp_hdr.th_dport = htons(80);
// TCP 中序列号
tcp_hdr.th_seq = htonl(0);
// TCP 中确认号
tcp_hdr.th_ack = htonl(0);
// TCP 中保留位 (4 位)
tcp_hdr.th_x2 = 0;
// TCP 中 TCP 头部长度 (32-bit words)
tcp_hdr.th_off = TCP_HDRLEN >> 2;
// TCP 中标志。TH_SYN=0x02
tcp_hdr.th_flags = TH_SYN;
// TCP 中窗口大小
tcp_hdr.th_win = htons(65535);
// TCP 中紧急指针
tcp_hdr.th_urp = htons(0);
// TCP 中 TCP 校验和。先设为 0, 构造 IP 头后再计算它的校验和
tcp_hdr.th_sum = 0;
// 构造 IPv4 包头部。请参考 UDP 报文发送程序中构造 IPv4 包头部
...;
// 将构造的 IPv4 头部复制到发送缓冲区中
memcpy(packet, &ip_hdr, IP4_HDRLEN * sizeof(uint8_t));
// 将构造的 TCP 头部复制到发送缓冲区中
memcpy((packet + IP4_HDRLEN), &tcp_hdr, TCP_HDRLEN * sizeof(uint8_t));
// 发送数据包
sendto (sd, packet, IP4_HDRLEN + TCP_HDRLEN, 0, (struct sockaddr *) &sin,
        sizeof(struct sockaddr));

```


3.6.5 进一步的扩展

向指定服务器发送 TCP SYN 报文，接收到服务器返回的 TCP SYN|ACK 报文后，再向服务器发送 TCP ACK 报文，完成 TCP 连接的三次握手。

3.7 UDP 客户/服务器程序

3.7.1 课程设计目的

本课程设计主要目的是使用 UDP 传输协议，实现一个简单的 UDP 客户/服务器程序，以了解传输层所提供的 UDP 服务的特点，应用层和传输层之间的软件接口风格，熟悉数据报套接口机制和 UDP 客户/服务器程序的结构。

3.7.2 课程设计内容

本课程设计要求设计与实现 Linux 下 UDP 客户/服务器程序，具体内容如下：

- 1) 该程序能够实现以下功能：客户从标准输入读一行文本，写到 UDP 套接口中；服务器从 UDP 套接口中读取该行，并回射给客户；客户读该回射行，并将其输出到标准输出；
- 2) 设计和实现 UDP 客户/服务器程序；
- 3) 能够用 Wireshark 分析的 UDP 报文。

3.7.3 相关知识

本课设需要的 Linux 网络编程知识，请参考附录 B。

3.7.4 课程设计分析

1. 详细设计

UDP 客户 / 服务器程序所用的套接口函数如图 3-20 所示。

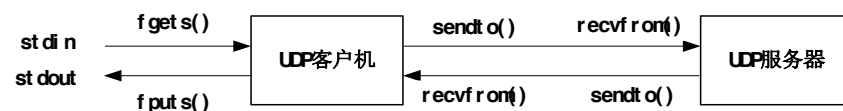


图 3-20 UDP 客户 / 服务器程序所用的套接口函数

客户 main() 函数流程图如图 3-21 所示。

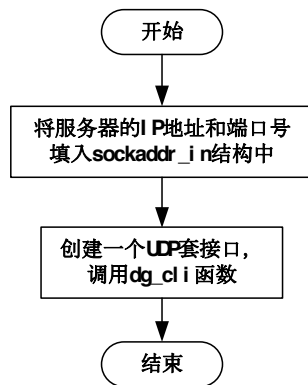


图 3-21 客户 main() 函数流程图

客户 dg_cli() 函数流程图如图 3-22 所示。

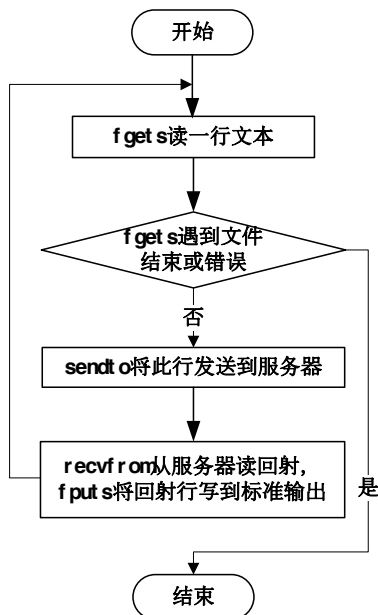


图 3-22 客户 dg_cli() 函数流程图

服务器 main() 函数流程图如图 3-23 所示。

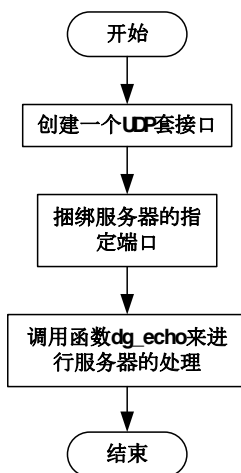


图 3-23 服务器 main() 函数流程图

服务器 dg_echo() 函数流程图如图 3-24 所示。

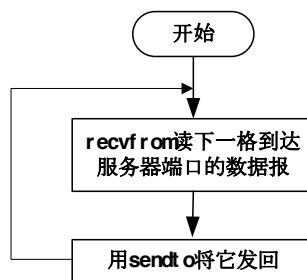


图 3-24 服务器 dg_echo() 函数流程图

2. 核心代码

1) UDP 客户程序。

main() 函数部分实现:

```
// 将服务器的 IP 地址和端口号填入 sockaddr_in 结构中
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(SERV_PORT);
inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
// 建立一个 UDP 套接口
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
// 调用 dg_cli() 函数
dg_cli(stdin, sockfd, (SA *)&servaddr, sizeof(servaddr));
dg_cli() 函数部分实现:
```

```
while (fgets(sendline, MAXLINE, fp) != NULL) {
    // 将数据写入套接口
    sendto(sockfd, sendline, strlen(sendline), 0, pservaddr, servlen);
    // 从套接口中读取数据
    n = recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);
    recvline[n] = 0; /* null terminate */
    fputs(recvline, stdout);
}
```

2) UDP 服务器程序。

main() 函数部分实现:

```
// 建立一个 UDP 套接口
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
// 将 INADDR_ANY 地址和端口号填入 sockaddr_in 结构中
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(SERV_PORT);
// 绑定指定端口号
bind(sockfd, (SA *)&servaddr, sizeof(servaddr));
// 调用 dg_echo() 函数
dg_echo(sockfd, (SA *)&cliaddr, sizeof(cliaddr));
dg_cli() 函数部分实现:
for ( ; ; ) {
```

```

len = clilen;
// 从套接口中读数据
n = recvfrom(sockfd, mesg, MAXLINE, 0, pcliaddr, &len);
// 将数据写入套接口
sendto(sockfd, mesg, n, 0, pcliaddr, len);
}

```

3.7.5 进一步的扩展

无。

3.8 TCP 客户/服务器程序

3.8.1 课程设计目的

本课程设计主要目的是使用 TCP 传输协议，实现一个简单的 TCP 客户/服务器程序，以了解传输层所提供的 TCP 服务的特点，应用层和传输层之间的软件接口风格，熟悉 TCP 套接口机制和 TCP 客户/服务器程序的结构。

3.8.2 课程设计要求

本课程设计要求设计与实现 Linux 下 TCP 客户/服务器程序，具体内容如下：

- 1) 该程序能够实现以下功能：客户从标准输入读一行文本，写到 TCP 套接口中；服务器从 TCP 套接口中读取该行，并回射给客户；客户读该回射行，并将其写到标准输出；
- 2) 设计和实现 TCP 客户/服务器程序；
- 3) 能够用 Wireshark 分析的 TCP 报文。

3.8.3 相关知识

本课设需要的 Linux 网络编程知识，请参考附录 B。

3.8.4 课程设计要求

1. 详细设计

TCP 客户/服务器程序所用套接口函数如图 3-25 所示。

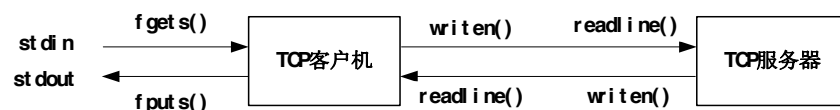


图 3-25 TCP 客户/服务器程序所用套接口函数

客户 main() 函数流程图如图 3-26 所示。

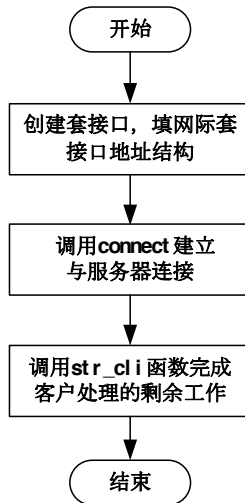


图 3-26 客户 main() 函数流程图

客户 str_echo() 函数流程图如图 3-27 所示。

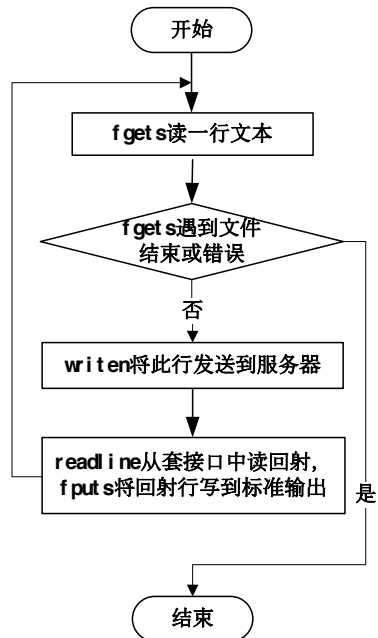


图 3-27 客户 str_echo() 函数流程图

服务器 main() 函数流程图如图 3-28 所示。

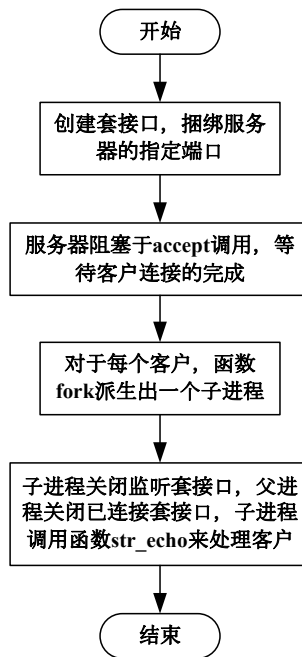


图 3-28 服务器 main() 函数流程图

服务器 str_echo() 函数流程图如图 3-29 所示。

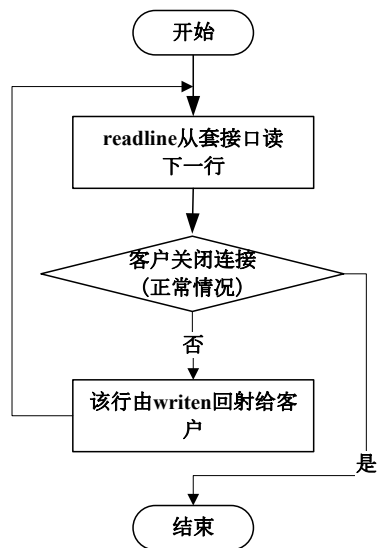


图 3-29 服务器 str_echo() 函数流程图

2. 核心代码

1) TCP 客户程序

main() 函数的部分实现:

// 建立一个 TCP 套接口

```
sd = socket(AF_INET, SOCK_STREAM, 0);
```

// 将服务器的 IP 地址和端口号填入 sockaddr_in 结构

```
bzero(&servaddr, sizeof(servaddr));
```

```
servaddr.sin_family = AF_INET;
```

```
servaddr.sin_port = htons(SERV_PORT);
```

```
inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
```

// 调用 connect() 建立与服务器连接

```

connect(sd, (SA *)&servaddr, sizeof(servaddr))
// 调用 str_cli 函数
str_cli(stdin, sd);
str_cli () 函数的部分实现:
while(fgets(sendline, MAXLINE, fp) != NULL) {
    // 将数据写入 TCP 套接口
    write(sd, sendline, strlen(sendline));
    // 从套接口中读数据
    readline(sd, recvline, MAXLINE);
    fputs(recvline, stdout);
}
2) TCP 服务器程序
main() 函数的部分实现:
// 建立一个 TCP 套接口
listenfd = socket(AF_INET, SOCK_STREAM, 0)
// 将 INADDR_ANY 地址和端口号填入 sockaddr_in 结构中
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(SERV_PORT);
// 绑定到指定的端口号
bind(listenfd, (SA *)&servaddr, sizeof(servaddr));
// 建立监听套接口
listen(listenfd, LISTENQ)
for( ; ;) {
    // 建立已连接套接口, 并阻塞
    clilen = sizeof(cliaddr);
    connfd = accept(listenfd, (SA *) &cliaddr, &clilen));
    // 若是子进程, 关闭监听套接口, 并调用 str_echo()
    if((childpid = fork()) == 0) {
        close(listenfd) ;
        str_echo(connfd);
        exit(0);
    }
    close(connfd);
}
str_echo() 函数的部分实现:
for( ; ; ) {
    // 从套接口中读数据
    if((n = readline(sockfd, line, MAXLINE)) == 0)
        return;
    // 将数据写入套接口
    write(sockfd, line, n);
}

```

3.8.5 进一步的扩展

在给定的 TCP 客户/服务器程序的基础上做如下功能扩充：

- 1) 在客户上显示服务器的当前目录；
- 2) 在客户上改变服务器的当前目录；
- 3) 将服务器上指定的文件下载到客户；
- 4) 将客户上指定的文件上传到服务器；
- 5) 退出返回。

3.9 服务器接口流量监测程序

3.9.1 课程设计目的

本课程设计主要目的是实现一个基于 SNMP 的服务器接口流量监测程序，以掌握 SNMP 协议，理解 SNMP 报文格式。

3.9.2 课程设计内容

本课程设计要求设计与实现基于 SNMP 的服务器接口流量监测程序，具体内容如下：

- 1) 该程序能够实现以下功能：向指定的服务器发送 SNMP 请求；获取该服务器的指定接口输入和输出流量，并计算其利用率；
- 2) 设计和实现基于 SNMP 的服务器接口流量监测程序；
- 3) 能够用 Wireshark 分析的 SNMP 报文。

3.9.3 相关知识

简单网络管理协议（SNMP）包含两个部分：管理站（也叫管理进程，Manager）和被管设备。被管设备种类繁多。例如，路由器、交换机、服务器和打印机等，被管设备中与网络管理相关的软件叫做代理进程（Agent）。管理站可以显示所有被管设备的状态。例如，接口的流量状况等。

管理进程和代理进程之间的通信可以有两种方式。一种方式是管理进程向代理进程发出请求，询问或设置一个具体的参数值。另外一种方式是代理进程主动向管理进程报告有某些重要的事件发生。例如，一个接口掉线了。

SNMP 包含三个组成部分：

- 1) 一个管理信息库（MIB）。MIB 包含所有代理进程的所有可被查询和修改的参数。RFC 1213 定义了 MIB-II。
- 2) 关于 MIB 的一套公用的结构和表示符号，叫做管理信息结构（SMI）。这个在 RFC 1155 中定义。例如，SMI 定义计数器是一个非负整数，它的计数范围是 $0 \sim 2^{32}-1$ ，当达到最大值时，又从 0 开始计数。
- 3) 管理进程和代理进程之间的通信协议，叫做简单网络管理协议（SNMP）。该协议已经发展了三代，现在的版本是 SNMPv3，以前的版本是 SNMPv1 和 SNMPv2。SNMP 包括数据报交

换的格式等。在 SNMP 中传输层主要使用的协议是 UDP。

1. SNMP

关于管理进程和代理进程之间的交互信息，SNMP 定义了 5 种报文：

- 1) get-request 操作：从代理进程获取一个或多个参数值；
- 2) get-next-request 操作：从代理进程获取一个或多个参数的下一个参数值；
- 3) set-request 操作：设置代理进程的一个或多个参数值；
- 4) get-response 操作：返回一个或多个参数值，这是代理进程发出的，是前 3 个操作的响应操作；
- 5) trap：代理进程主动发出报文，通知管理站某些事情发生。

这些操作前 4 种操作都是简单的请求/响应方式，而且在 SNMP 经常使用 UDP 协议传输，可能发生数据报丢失的情况，因此要有一定的超时和重传机制。

管理进程发出的前面 3 种操作采用 UDP 的 161 端口。代理进程发出的 trap 操作采用 UDP 的 162 端口。由于收发采用了不同的端口，一个系统可以同时作为管理进程和代理进程。

封装成 UDP 数据包 5 种操作的 SNMP 报文格式如图 3-30 所示。

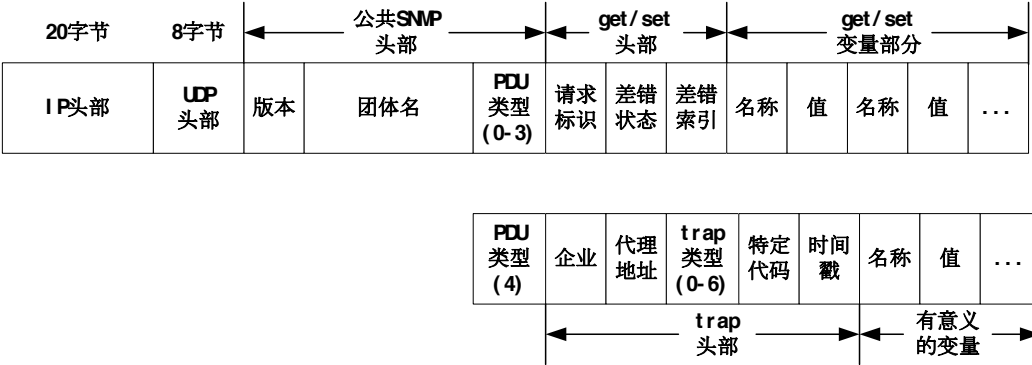


图 3-30 SNMP 报文

SNMP 版本。SNMPv1 取值为 0，SNMPv2 取值为 1，SNMPv3 取值为 3。

团体名是 SNMP 的一个安全机制，只有相同团体名的设备可以互相访问，作为认证 SNMP 报文的口令。

PDU 类型指明此 PDU 是什么。get-request 操作、get-next-request 操作、set-request 操作、get-response 操作和 trap 操作的类型分别为 0、1、2、3 和 4。

对于 get-request 操作、get-next-request 操作和 set-request 操作，请求标识由管理进程设置，然后由代理进程在 get-response 操作中返回。这个字段的作用是使管理进程能够将代理进程发出的响应和管理进程发出的请求进行匹配。这个字段允许管理进程对一个或多个代理进程发出多个请求，并且从返回的众多响应中进行分类。

差错状态是一个整数，它是由代理进程标注的，指明有差错发生。

差错索引是一个整数偏移量，指明当有差错发生时，差错发生在哪个参数。它是由代理进程标注的，并且只有在发生 noSuchName、readOnly 和 badValue 差错时才进行标注。

在 get-request 操作、get-next-request 操作和 set-request 操作的请求报文中，包含变量名称和变量值的一张表。对于 get-request 操作和 get-next-request 操作，变量值部分不需要填写。

对于 trap 操作 SNMP 报文格式有所变化。

PDU 类型为 4，表示该 SNMP PDU 是一个 trap。

企业表示产生该 trap 的设备的类型，它的值为 sysObject 的值。

代理地址是生成该 trap 的设备的 IP 地址。

trap 类型为 0-5 的为通用 trap，trap 类型为 6 的为私有 trap。
时间戳是生成该 trap 时 sysUpTime 的值。

2. SMI

为了使网络管理协议简单化，SMI 对 MIB 中允许的变量类型做出限制，指明对这些变量的命名规则，并创建定义变量类型的规则。例如，SMI 标准包括对一些术语的定义。如 IPAddress 定义为 4 字节长度的字符串，Counter 定义为 $0-2^{32}-1$ 的整数，并指明它们是用于定义 MIB 变量的一些术语。更为重要的是，SMI 中的规则描述了 MIB 如何指向由许多值构成的表格。例如路由表。

3. 对象标识符

对象标识符 (OID) 是一个整数序列，以点 (“.”) 分隔。这些整数构成一个树型结构。对象标识符从树的根开始，根没有名字，但有三个直接后代，分别由 ISO 和 ITU 管理，或由 ISO 和 ITU 联合管理。

图 3-31 显示了在 SNMP 中用到的这种树型结构。所有的 MIB 变量都从 1.3.6.1.2.1 这个标识符开始。为了阅读方便，树上的每个结点同时还有一个文字名，例如，标识符 1.3.6.1.2.1 就和 iso.org.dod.internet.mgmt.mib 对应。在管理进程和代理进程进行通信时，MIB 变量名是以对象标识符来标识的，都是以 1.3.6.1.2.1 开头的。

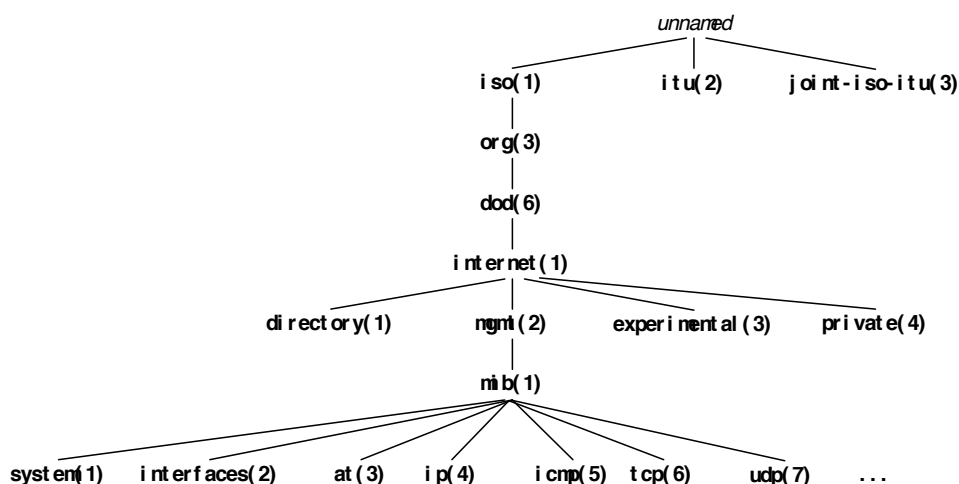


图 3-31 管理信息库中的对象标识符

4. MIB

MIB 是所有代理进程包含的，并且能够被管理进程进行查询和设置的信息的集合。在 RFC 1213 中定义的 MIB-II。

MIB 被划分为若干个组。例如，system 组、interfaces 组、at（地址转换）组、ip 组、icmp 组、tcp 组和 udp 组等，如图 3-31 所示。下面讨论 interfaces 组中的变量，如图 3-32 所示。

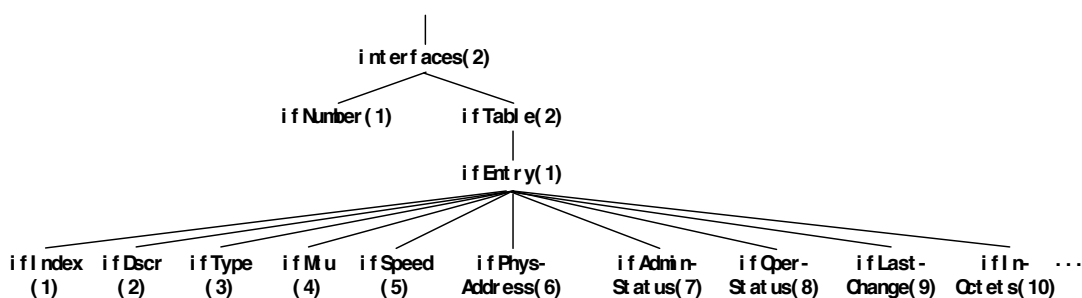


图 3-32 interfaces 组

interfaces 组包含一个变量和一个表格。表 3-5 描述了这些变量。

表 3-5 interfaces 组对象

对象名	类型	说明
ifNumber	INTEGER	设备的接口数
ifTable	SEQUENCE OF IfEntry	接口表
ifEntry	SEQUENCE	接口表项
ifIndex	INTEGER	接口表索引
ifDescr	DisplayString	接口的描述，包括厂商，产品名称等
ifType	INTEGER	接口的类型
ifMtu	INTEGER	接口的最大传输包字节数
ifSpeed	Gauge	接口每秒可传送的最大位数，单位：bit/s
ifPhysAddress	PhysAddress	接口的物理地址
ifAdminStatus	INTEGER	接口配置状态。1-正常，2-关闭，3-测试
ifOperStatus	INTEGER	接口的当前状态
ifLastChange	TimeTicks	接口进入当前操作状态时 sysUpTime 的值
ifInOctets	Counter	接口输入的字节数
ifInUcastPkts	Counter	接口输入的单播包数
ifInNUcastPkts	Counter	接口输入的非单播包数
ifInDiscards	Counter	接口输入包丢弃数
ifInErrors	Counter	接口输入包错误数
ifInUnkuownProtos	Counter	接口输入未知协议的包数
ifOutOctets	Counter	接口输出的字节数
ifOutUcastPkts	Counter	接口输出的单播包数
ifOutNUcastPkts	Counter	接口输出的非单播包数
ifOutDiscards	Counter	接口输出包丢弃数
ifOutErrors	Counter	接口输出包错误数
ifOutQlen	Gauge	接口输出队列中的包数
ifSpecific	OBJECT IDENTIFIER	参考 MIB 对实现接口的媒体的定义

interfaces 组中的 ifDescr、ifType、ifMtu、ifSpeed 和 ifAdminStatus 等对象可用于配置管理。ifType 标示接口的类型，枚举值到接口类型的映射在 IANAifType-MIB 中维护，这个 MIB 定期更新，见 <http://www.iana.org/assignments/ianaiftype-mib>，在实际应用中，最好将该整数转换成更易于理解的字符串。例如一个接口的 ifDescr 是 ethernet1，则它的 ifType 值是 6，这时可以用 ethernetCsmacd 来表示。ifMtu 是在接口上可以发送或接收的最大包的大小，太小的 MTU 值会导致包分段太多，使设备和网络效率低。ifspeed 就是一个接口每秒可以传送的最大位数。例如 100000000 表示接口的带宽为 100Mbps，网络管理员可以根据网络流量的变化情况重新设置 ifSpeed，调整接口的传输能力。ifAdminStatus 显示接口的配置状态，网络管理员根据一个接口是否需要使用，将该接口的 ifAdminStatus 值设置为开启（up）或关闭（down）。

interfaces 组对象主要包括 ifInOctets、ifOutOctets、ifInUcastPkts、ifOutUcastPkts、ifInNUcastPkts 和 ifOutNUcastPkts 等对象可用于计费管理。

interfaces 组中的 ifInOctets、ifInUcastPkts、ifInNUcastPkts、ifInDiscards、ifInErrors、ifInUnkuownProtos、ifOutOctets、ifOutUcastPkts、ifOutNUcastPkts、ifOutDiscards、ifOutErrors 和 ifOutQlen 等对象可用于性能管理。根据这些对象可以了

解接口的性能特性。例如接口的利用率、错误率和丢包率等。以下是接口的一些性能特性计算公式。

对于半双工接口，接口利用率计算公式

$$\text{接口利用率} = \frac{(\Delta \text{InOctets} + \Delta \text{OutOctets}) * 8 * 100}{(y - x) * \text{ifSpeed}}$$

对于全双工接口，要分别计算接口输入和输出的利用率、错误率和丢包率等性能特性。

接口输入利用率计算公式

$$\text{接口输入利用率} = \frac{\Delta \text{InOctets} * 8 * 100}{(y - x) * \text{ifSpeed}}$$

接口输出利用率计算公式

$$\text{接口输出利用率} = \frac{\Delta \text{OutOctets} * 8 * 100}{(y - x) * \text{ifSpeed}}$$

接口输入错误率计算公式

$$\text{接口输入错误率} = \frac{\Delta \text{InErrors} * 100}{\Delta \text{InUcastPkts} + \Delta \text{InNUcastPkts}}$$

接口输出错误率计算公式

$$\text{接口输出错误率} = \frac{\Delta \text{OutErrors} * 100}{\Delta \text{OutUcastPkts} + \Delta \text{OutNUcastPkts}}$$

接口输入丢包率计算公式

$$\text{接口输入丢包率} = \frac{\Delta \text{InDiscards} * 100}{\Delta \text{InUcastPkts} + \Delta \text{InNUcastPkts}}$$

接口输出丢包率计算公式

$$\text{接口输出丢包率} = \frac{\Delta \text{OutDiscards} * 100}{\Delta \text{OutUcastPkts} + \Delta \text{OutNUcastPkts}}$$

其中 x, y 表示两个采集时间点，表示某对象值在两个采集点间变化的差值。

5. AdventNet SNMP API

AdventNet SNMP API 为基于 SNMP 的网络管理应用提供了一个全面的开发工具包。它包含一系列强大的 Java SNMP 库，用来为监控和跟踪网络元素创建实时的、可靠、可伸展且独立于操作系统的网络管理应用程序。

开发人员可以利用 AdventNet 的 SNMP 库来创建基于 SNMP 的网络管理应用程序。Java SNMP 库除了提供基本的 SNMP 操作。例如，SNMP GET、SNMP GETNEXT、SNMP GETBULK 和 SNMP SET，还为陷阱和表格处理提供了现成的组件。这些组件便于简单和快速地开发和部署 SNMPv1、SNMPv2c 和 SNMPv3 管理应用程序。在开发中所用到的 AdventNet SNMP 包的目标类常包含在 com.adventnet.snmp.beans 包中。

本课程设计中用到了 SnmpTarget 类。它的 snmpGetList() 方法实现 GetRequest 请求；snmpGetNextList() 方法实现 GetnextRequest 请求；snmpGetAllList() 方法结合 MibOperations、SnmpOID 和 MibNode 对象，通过连续执行 GetNextRequest 操作取得一张 MIB 表的所有数据，返回到一个二维字符串数组。SnmpTarget 通过 ResultListen 监听器监听是否有响应消息到达。

AdventNet SNMP API 可以到 <https://www.webnms.com/snmp/> 进行下载，使用 AdventNet SNMP API 需要将 AdventNetSnmp.jar 和 AdventNetLogging.jar 加入到 lib 中去。

3.9.4 课程设计分析

核心代码

1) main() 方法

```
public static void main(String[] args) {
    ...;
    try {
        ...;
        // 多个设备时
        for(int i = 0; i < did.length; i++) {
            Timer d5minTimer = new Timer();
            // 300,000 毫秒=5 分钟, IP 地址为 192.168.1.1, 团体名为 public,
            // OID 分别为 eth0 接口的 ifInOctets 和 ifOutOctets
            d5minTimer.scheduleAtFixedRate(new FiveMinutesTask(did[i], ipAddr[i],
                oid, publicKey[i], t_value), 0, 300000);
        }
    } catch(Exception e) { ...;}
}
```

2) FiveMinutesTask() 构造类

```
protected FiveMinutesTask(int did, String ipAddr, String[] oid,
    String publicKey, double[] t_value) {
    ...;
    // 建立一个 SnmpTarget 对象
    target = new SnmpTarget();
    // 设置主机地址
    target.setTargetHost(ipAddr);
    // 设置团体名
    target.setCommunity(publicKey);
    // 设置 OID
    target.setObjectIDList(oid);
}
```

3) run() 方法

```
public void run() {
    // 执行 get 操作
    result = target.snmpGetList();
    if((result == null) || result.length == 0) {
        System.out.println("result null");
        isFirst = true;
    } else {
        String currrtime = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(
            Calendar.getInstance().getTime());
        for(int i = 0; i < n_value.length; i++) {
            try {
```

```

        n_value[i] = Long.parseLong(result[i]);
    } catch (NumberFormatException nfe) { ...;}
}
// 如果 5 分钟前取到数据, 则计算接口输入/输出利用率, 并判断是否超过阈值
if(!isFirst) {
    int ifspeed = 100000000;
    // 计算 5 分钟接口输入的字节数
    long diff = n_value[0] - o_value[0];
    // 如果 ifInOctets 的值达到最大值后归 0, 则差值加上 2 *
(Integer.MAX_VALUE+1)
    if(diff < 0)
        diff = diff + (long)2 * ((long)Integer.MAX_VALUE + (long)1);
    // 接口输入利用率 = (ifInOctets * 8) / ((秒数)* IfSpeed)
    double utilization = (diff * 8.0) / (300.0 * ifspeed);
    // 如果接口输入利用率大于指定的阈值, 则输出告警信息
    if(utilization > t_value[0])
        System.out.println("Alarm: " + currtime +
            ": Utilization of the input interface = " + utilization +
            ">" + String.valueOf(t_value[0]) + ").");
    // 计算 5 分钟接口输出的字节数
    diff = n_value[1] - o_value[1];
    // 若 ifOutOctets 的值达到最大值后归 0, 则差值加上 2*
(Integer.MAX_VALUE+1)
    if(diff < 0)
        diff = diff + (long)2 * ((long)Integer.MAX_VALUE + (long)1);
    // 接口输出利用率 = (ifOutOctets * 8) / ((秒数)* IfSpeed)
    utilization = (diff * 8.0) / (300.0 * ifspeed);
    // 如果接口输出利用率大于指定的阈值, 则输出告警信息
    if(utilization > t_value[1])
        System.out.println("Alarm: " + currtime +
            ": Utilization of the Output interface = " + utilization +
            ">" + String.valueOf(t_value[1]) + ").");
    }
}
for (int i = 0; i < n_value.length; i++) {
    o_value[i] = n_value[i];
    isFirst = false;
}
}

```

3.9.5 进一步的扩展

在给定的服务器接口流量监测程序的基础上做如下功能扩充:

- 1) 能够从配置文件中读取配置信息, 根据配置信息监测被管对象;

2) 能够监测接口输入/输出丢包率、接口输入/输出错误率、IP 输入丢包率、IP 包转发速率、IP 分段率、IP 需要重组率、ICMP 报文的输入速率、ICMP 报文的输出速率、接收到的 ICMP 重定向报文的速率、TCP 报文输入/输出速率等参数;

3) 当发现设备出现异常时, 能够通过 Email 将异常情况报告给网络管理员。

3.10 邮件客户端程序

3.10.1 课程设计目的

本课程设计主要目的是实现一个基于 JavaMail 的电子邮件客户端, 以掌握电子邮件相关协议 (SMTP 协议和 POP3 协议), 理解邮件头和邮件内容。

3.10.2 课程设计内容

本课程设计要求设计与实现基于 JavaMail 的邮件客户端程序, 具体内容如下:

- 1) 该程序能够实现以下功能: 程序能够设置邮件服务器地址、收信人、抄送、主题、正文和附件等信息; 能够接收邮件、写邮件和发送邮件;
- 2) 设计和实现邮件客户端程序;
- 3) 能够用 Wireshark 分析的 SMTP 报文和 POP 报文。

3.10.3 相关知识

1. SMTP

简单邮件传输协议 (SMTP) 最早由 RFC 821 说明, 现在已被 RFC 5321 取代。它通过一个连接来发送邮件、返回发送状态和任何错误的报告。

邮件发送过程: 发送电子邮件的客户端首先与邮件服务器的 25 端口建立一个 TCP 连接, 客户端等待邮件服务器响应。邮件服务器开始发送一个文本行给客户端, 该行表明了自己的标识, 并告诉客户端是否已经准备好接收邮件。如果服务器没有这么做, 那么客户端就释放连接, 稍后再次尝试与服务器联系。如果服务器愿意接收电子邮件, 则客户端声明这封电子邮件来自于谁以及将要发给谁。如果确实存在这样的收件人, 则服务器指示客户端发送邮件; 然后客户端发送邮件, 服务器予以确认。

邮件发送过程中, 客户端向服务器发送 SMTP 命令, 服务器进行判断后做出相应的响应。SMTP 命令与响应都由 7 位 ASCII 字符组成, 发送时采用 8 位字符并把最高位填充为 0。

SMTP 命令由命令和其后的参数组成。命令由四个字母组成的, 不区别大小写; 参数是长度不定的字符串并以 CRLF 表示结束, 接收方在收到 CRLF 前不会对命令做任何处理。一个完整的发送邮件过程必须按顺序发送 HELO、MAIL、RCPT、DATA 和 QUIT 这五个 SMTP 命令, 而其它命令则不是必须的, 可以根据用户的需求进行选择。

SMTP 响应确定了在邮件传输过程中请求和处理的情况, 也保证了发送方知道接收方接收 SMTP 命令的状态。SMTP 响应由三位数字组成, 其后跟一些文本信息。这三位数字有各自的表示意义, 共同帮助客户端决定下一个进入的状态, 而文本信息则介绍了这三位数字所代表的具体意义。例如, 250 sender ok。命令的发送方根据响应决定下一步操作情况。一般

来说，每条命令可能的响应分为三种情况：操作成功，操作失败和操作发生错误，有的命令还具有中间过程。操作失败和操作发生错误响应的区别在于发送方可以重新发送响应为操作失败的命令请求重新处理，而当收到操作发生错误的响应后，客户端将结束此次发送过程。

2. ESMTP

基本 SMTP 运行良好，但是它存在以下方面的不足：它不包括认证，这意味着 FROM 命令可以给出任何发件人的地址；SMTP 传输的是 ASCII 报文而不是二进制数据；SMTP 发送的邮件以明文形式出现。

为了处理这些以及其它与邮件处理相关的问题，SMTP 增加了一个扩展机制，这个机制是 RFC 5321 标准的强制执行部分。带有扩展功能的 SMTP 就称为扩展 SMTP (ESMTP)。

想要使用 ESMTP 版本的客户端必须发送 EHLO 报文，而不是原来的 HELO。如果该报文遭到了服务器的拒绝。那么说明对方是一个普通的 SMTP 服务器，客户端应该以普通的 SMTP 与其交互。如果 EHLO 报文被服务器接受，服务器就用它支持的扩展予以响应。

SMTP 扩展方案增加了新的命令 EHLO，并对命令 MAIL 和 RCPT 的参数域进行扩展；对 SMTP 服务也进行了认证扩展。

3. POP3

邮局协议第三版 (POP3) 由 RFC 1939 说明，它用于电子邮件的接收。

接收邮件过程：邮件服务器通过侦听 TCP 端口 110 开始 POP3 服务。当客户端需要使用 POP3 服务时，客户端将与服务器主机建立 TCP 连接。当连接建立后，POP3 服务器发送确认消息。客户端和 POP3 服务器相互交换命令和响应，这一过程一直持续到连接终止。在生命周期中，POP3 会话有几个不同的状态。一旦 TCP 连接被打开，而且 POP3 服务器发送确认信息，就进入确认状态。在此状态中，客户端必须向 POP3 服务器确认自己是其客户。一旦确认成功，服务器就获取与客户邮件相关的资源，就进入操作状态。在操作状态中，客户端提出服务，当客户端发出 QUIT 命令时，进入更新状态。在更新状态中，POP3 服务器释放在操作状态中取得的资源，并发送消息，终止连接。

邮件接收过程中，客户端向服务器发送 POP3 命令，服务器进行判断后做出相应的响应。POP3 命令有 USER、PASS、APOP、STAT、UIDL、LIST、RETR、DELE、RSET、NOOP、QUIT 和 TOP 等。

4. 邮件格式

RFC 5322 是邮件格式的最新版本，最初的 Internet 邮件格式由 RFC 822 描述。邮件由一个基本的信封（作为 SMTP 的一部分由 RFC 5321 描述）、数个头字段、一个空行和邮件体组成。头的每个字段由一行 ASCII 文本组成，其中包括域名、一个冒号，对于多数头的字段来说还包括一个值。最初的 RFC 822 没有明确区分信封字段和头字段。在一般的用法中，用户代理创建一个邮件并把它传递给邮件传输代理，然后邮件传输代理利用某些头字段来构造出实际的信封。

邮件主要的头字段。“To:” 字段给出了主要收件人的地址，可以有多个收件人。“Cc:” 字段给出了所有次要收件人的地址。“Bcc:” 字段与 “Cc:” 字段类似，只是在发送给主收件人和次收件人的所有副本中，这一行被删除了。“From:” 和 “Sender:” 字段分别指出了邮件的撰写人和发送人。如果 “Sender:” 字段与 “From:” 相同，可以省略 “Sender:” 字段。“Received:” 字段给出了邮件传输代理的标识、接收到此邮件的日期和时间，以及其它一些可用来查找路由系统中错误的信息。“Reply-To:” 字段给出了回复邮件应采用的电子邮件地址。“Message-Id:” 字段给出了稍后引用本邮件的唯一编号。

5. MIME

多用途互联网邮件扩展 (MIME) 扩展了电子邮件标准，使其能够支持非 ASCII 字符、二

进制格式附件等多种格式的邮件消息。MIME 由 RFC 2045-2047、4288、4289 和 2049 描述。MIME 的基本思想是继续使用 RFC 822 格式，但是在邮件体中增加了结构性，并且为传送非 ASCII 码的邮件定义了编码规则。

MIME 邮件格式比 RFC 822 邮件格式增加的邮件头有 MIME-Version，Content-Description，Content-ID，Content-Transfer-Encoding 和 Content-Type 等。可以占一行或多行。

“MIME-Version:”是第一个邮件头，它告诉接收邮件的用户代理，正在处理一条 MIME 报文，以及该报文使用的是哪个版本的 MIME。任何不包含“MIME-Version:”邮件头的报文都被假定是英语明文报文（或至少只使用 ASCII 字符），并按照这种假定情况下的方式来进行处理。

“Content-Description:”头是一个 ASCII 字符串，它指明了邮件包含什么内容。这个头是必需的，这样收件人才能知道是否值得解码和阅读该消息。

“Content-ID:”头标识了邮件的内容，它采用了与标准的“Message-Id:”头相同的格式。

“Content-Transfer-Encoding:”头指出了如何包装邮件体，以便通过网络进行传输。当开发 MIME 时，SMTP 只能处理一行不超过 1000 个字符的 ASCII 邮件。ASCII 字符占了每 8 位字节中的 7 位，而二进制数据则使用了每个字符的全部 8 位作为扩展的字符集。如果没有任何保障措施，是无法安全传送这些数据的。需要某种方式使得携带二进制数据的邮件看起来就像常规的 ASCII 邮件报文一样。自从 MIME 开发出来后，SMTP 也做了相应的扩展，允许邮件正文传送 8 位的二进制数据。

RFC 1521 定义了 7 种 MIME 类型。每种类型都有一个或多个子类型。类型和子类型由一个斜线隔开，如 Content-Type:text/plain。类型包括文本类型（text），图像类型（image），音频类型（audio），视频类型（video），应用类型（application），邮件类型（message）和多个类型的组合类型（multipart）等，除标准类型和子类型，MIME 允许发信人和收信人使用以“X-”开头的字符串定义专用内容类型。

text 类型。“text/plain”可用于表达普通邮件，显示的就是收到的，无须编码的，也不需要进一步处理。这个选项允许以 MIME 方式传输普通邮件。“text/html”允许在 RFC822 电子邮件中发送网页。

image 类型可传输静态图象。

audio 和 video 类型分别用于表示声音和运动图象。

application 类型对那些未被其它类型覆盖但又需要应用程序来解释的未知格式的统称。

message 类型使得一个邮件可以被完全封装在另一个邮件中。这种方案对转发电子邮件很有用。

multipart 类型允许一个邮件包含多个部分，并明确地分出每个部分开始和结束的界限。mixed 子类型允许每个部分类型都不相同，而且无须强制任何额外结构。许多电子邮件程序允许用户在一个文本报文中提供一个或多个附件，这些附件可以 multipart 类型发送。

alternative 子类型允许同一报文被包含多次，但被表示成两种或多种不同的媒体。例如，一个报文可以按照 3 种形式来发送：纯粹的 ASCII 文本、HTML 和 PDF。

multipart 类型是 MIME 邮件的精髓。邮件体被分为多个部分，每个部分又包含部分头和部分体两部分，这两部分之间也以空行分隔。multipart 诸类型的共同特征是，在部分头指定“boundary”参数字符串，部分体内的每个子部分以此串定界。所有的子部分都以“--”

+boundary 行开始，父部分则以 “--” +boundary+ “--” 行结束。部分与部分之间也以空行分隔。

6. JavaMail

JavaMail 提供给开发者处理电子邮件相关的编程接口，可以到 <http://www.oracle.com/technetwork/java/index-138643.html> 进行下载，使用 JavaMail 需要将 mail.jar 加入到 lib 中去。JavaMail 需要 JavaBeans Activation Framework (JAF) 来支持任意数据块的输入及相应处理。activation.jar 文件就是 JAF 的 jar 包，使用 JavaMail 时应与 mail.jar 一起加入到 lib 中去，JAF 可以到 <http://www.oracle.com/technetwork/java/javase/index-jsp-136939.html> 进行下载。

3.10.4 课程设计分析

核心代码

1) MailClient 类是邮件客户端类。以下是其主要的方法：

main() 建立 MailClient 的对象。

```
public static void main(String args[]) throws Exception {  
    MailClient mc = new MailClient();  
}
```

MailClient() 是 MailClient 类的构造方法。

```
public MailClient() {  
    try {  
        jbInit();  
    } catch (Exception e) {...;}  
}
```

jbInit() 方法设置邮件客户端窗口，接收邮件并将其显示在邮件列表中。

```
private void jbInit() throws Exception {  
    // 建立邮件客户端窗口  
    ...  
    // 建立一个写邮件对象  
    newMailPanel = new NewMailPanel(this);  
    // 建立一个收发邮件对象  
    MyMail jm = new MyMail(this);  
    // 设置收发邮件  
    setMyMail(jm);  
    // 从邮件服务器接收邮件，并将邮件存放到成员变量 message 中  
    jm.jackProcess();  
    // 建立一个邮件列表对象  
    MailListPanel ml = new MailListPanel(this, jm.getMessage());  
    // 设置邮件列表  
    setMailListPanel(ml);  
    // 将邮件列表对象加入邮件客户端对象中  
    getContentPane().add(ml);  
    ...  
}
```

```
}
```

2) MyMail 类是收发邮件类。能够接收和解析邮件，以及构造和发送邮件。以下是其主要的功能：

MyMail() 是 MyMail 类的构造方法。

```
public MyMail(MailClient mailClient) {  
    this.mailClient = mailClient;  
}
```

jackProcess() 方法从邮件服务器接收邮件，并将邮件存放到成员变量 message 中。

```
public void jackProcess() throws Exception {  
    Properties prop = new Properties();  
    close();  
    // 获取邮件会话属性  
    if (MailConstant.INCOMING_PORT != null &&  
        !MailConstant.INCOMING_PORT.equals("")) {  
        prop.setProperty("mail.pop3.port", MailConstant.INCOMING_PORT);  
        prop.setProperty("mail.pop3.socketFactory.port",  
            MailConstant.INCOMING_PORT);  
        prop.setProperty("mail.pop3.socketFactory.class",  
            "javax.net.ssl.SSLSocketFactory");  
        prop.setProperty("mail.pop3.socketFactory.fallback", "false");  
    }  
    prop.put("mail.pop3.host", MailConstant.MAIL_SERVER_INCOMING);  
    prop.put("mail.store.protocol", "pop3");  
    // 如果需要身份认证，则根据邮件会话属性和创建的密码验证器构造一个发送邮件  
    // 的 Session 对象，否则根据邮件会话属性构造一个发送邮件的 Session 对象  
    Session session = null;  
    if (MailConstant.MAIL_SERVER_AUTH != null  
        && MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {  
        session = Session.getInstance(prop, authen);  
    } else {  
        session = Session.getInstance(prop);  
    }  
    // 是否设置调试标志  
    session.setDebug(MailConstant.MAIL_DEBUG != null ? MailConstant.MAIL_DEBUG  
        .equalsIgnoreCase("true") : false);  
    try {  
        // 利用 Session 对象获得 Store 对象，并连接 pop3 邮件服务器  
        store = session.getStore(MailConstant.POP_MAIL);  
        store.connect(MailConstant.MAIL_SERVER_INCOMING, -1, MailConstant.  
            MAIL_SERVER_USER, MailConstant.MAIL_SERVER_PASSWORD);  
    } catch (Exception ex) {  
        ...;  
    }  
    // 获得邮箱内的邮件夹 Folder 对象
```

```

folder = store.getDefaultFolder();
if (folder == null) {
    JOptionPane.showMessageDialog(null,
        "There is no default folder in your mail server",
        "Cant load Mail folder", JOptionPane.ERROR_MESSAGE);
    runtime.exec("notepad mail.properties");
    System.exit(-1);
}
folder = folder.getFolder(MailConstant.INBOX);
if (folder == null) {
    JOptionPane.showMessageDialog(null,
        "No default folder to view from your mail Client",
        "Cant load Mail folder", JOptionPane.ERROR_MESSAGE);
    runtime.exec("notepad mail.properties");
    System.exit(-1);
}
// 将获取邮件夹 Folder 对象以“读写”方式打开
folder.open(Folder.READ_WRITE);
// 获取邮件夹 Folder 内的邮件个数
int totalMessages = folder.getMessageCount();
System.out.println("totalMessages = " + totalMessages);
mailClient.getProgressDialog().getLblProgress().setText(
    "Number of messages: " + totalMessages);
mailClient.getStatusBar().setText(" [ " + totalMessages +
    " messages in Inbox ] " + " [ Today : " + new Date() + " ]");
if (totalMessages == 0) {
    return;
}
// 获取邮件夹 Folder 内的所有邮件
Message[] messages = folder.getMessages();
this.messages = messages;
}

```

readMessage() 方法解析成员变量 messages 中指定邮件，将邮件附件内容保存为文件，在阅读窗格中显示邮件头和邮件内容。

```

public void readMessage(final ProgressDialog pd, final JEditorPane
editor, final int msg_id) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            try {
                StringBuffer buffer = new StringBuffer("");
                // 如果邮件内容不是 html 类型的且是未读邮件，则处理邮件内容
                if (messages[msg_id] != null &&
                    !messages[msg_id].getContentType().equals("text/html") &&
                    !messages[msg_id].isSet(Flags.Flag.SEEN)) {

```

```

// 解析指定邮件的邮件头，将其保存到 buffer
buffer.append("From: " + messages[msg_id].getFrom()[0] + "\n");
Address[] add = messages[msg_id].getRecipients(RecipientType.TO);
buffer.append("Sent To: ");
for (int j = 0; j < add.length; j++) {
    buffer.append(add[j].toString() + ", ");
}
buffer.append("\n");
Address[] add2 = messages[msg_id].getRecipients(RecipientType.CC);
buffer.append("CC To: ");
if (add2 != null) {
    for (int j = 0; j < add2.length; j++) {
        buffer.append(add2[j].toString() + ", ");
    }
}
buffer.append("\n");
buffer.append("Subject: " + messages[msg_id].getSubject() + "\n");
// 解析邮件附件内容并将其保存为文件。
boolean contentFlag = doJack(messages[msg_id]);
// 解析指定邮件的发送时间，将其保存到 buffer
buffer.append("Sent Date : " +
    messages[msg_id].getSentDate() + "\n");
buffer.append("\n");
buffer.append("\n");
buffer.append("\n");
if (!contentFlag) { //如果邮件内容是 Multipart 对象
    BufferedReader is = new BufferedReader(
        new InputStreamReader(messages[msg_id].getInputStream()));
    String line = "";
    boolean flag = false;
    int counter = 0;
    while ((line = is.readLine()) != null) {
        if (line.startsWith("Content-Transfer-")) {
            flag = true;
            counter++;
        }
        if (flag == true && line.indexOf("_NextPart_") > 0) {
            flag = false;
            counter++;
        }
        if (counter == 1 && !line.startsWith("Content-Transfer-")) {
            buffer.append(line + "\n");
        }
    }
}
}

```

```

        editor.setContentType("text/plain");
    } else {
        buffer.append(" " + messages[msg_id].getContent());
    }
    // 否则如果是 html 类型的且是未读邮件, 则将 html 类型邮件内容添加到
buffer
    } else if (messages[msg_id] != null &&
        messages[msg_id].getContentType().equals("text/html")) {
        buffer.append(" " + messages[msg_id].getContent());
        editor.setContentType("text/html");
    }
    // 将 buffer 中的内容放到阅读窗格
    editor.setText(buffer.toString());
    pd.dispose();
} catch (Exception e) {
    ...;
}
}
});
}

```

doJack() 方法解析邮件附件内容并将其保存为文件。如果邮件内容是 Multipart 对象, 则解析 Multipart 对象并将其中邮件附件内容保存为文件后返回 false, 否则返回 true。

```

private boolean doJack(Message m) {
    String filepath = "";
    Multipart multipart;
    try {
        // 如果邮件内容是 Multipart 对象, 则解析 Multipart 对象并保存其中的附件后
        返回 false, 否则返回 true
        if (m.getContent() instanceof Multipart) {
            // 获取邮件内容
            multipart = (Multipart) m.getContent();
            // 遍历 Multipart 对象, 并保存 Multipart 对象中的附件
            for (int z = 0; z < multipart.getCount(); z++) {
                BodyPart part = multipart.getBodyPart(z);
                // 获取邮件内容的配置
                String disposition = part.getDisposition();
                // 如果是邮件附件内容, 则指定附件的保存位置后保存附件
                if ((disposition != null) && (disposition.equals(Part.ATTACHMENT) ||
                    (disposition.equals(Part.INLINE)))) {
                    // 指定保存附件的位置
                    JFileChooser chooser = new JFileChooser();
                    chooser.setCurrentDirectory(new
                        java.io.File(System.getProperty("user.home")));
                    chooser.setDialogTitle("Select The Folder where you want to save");
                }
            }
        }
    } catch (Exception e) {
        ...;
    }
}

```

```

        chooser.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
        chooser.setSelectedFile(new File(part.getFileName()));
        chooser.setAcceptAllFileFilterUsed(false);
        // 将附件保存到指定的位置
        if(chooser.showSaveDialog(mailClient) ==
            JFileChooser.APPROVE_OPTION) {
            filepath = chooser.getSelectedFile().toString();
            String filename = filepath + "/" + part.getFileName();
            File file = new File(filename);
            file.createNewFile();
            InputStream in = part.getInputStream();
            FileOutputStream fos = new FileOutputStream(file);
            byte[] buf = new byte[1024];
            int c = 0;
            while ((c = in.read(buf)) != -1) {
                fos.write(buf, 0, c);
            }
            in.close(); fos.close();
        }
        else {
            continue;
        }
    }
    else if (m.getContent() instanceof String) {
        return true;
    }
} catch (IOException exception) {
    ...;
} catch (MessagingException exception) {
    ...;
}
return false;
}

sendMailWithAtt()方法构造邮件的邮件头和邮件内容，并发送邮件。
public boolean sendMailWithAtt(String sender_name, String email_from,
    String email_to, String email_cc, String email_bcc,
    String email_subject, String email_body, File file_name) {
    boolean result = true;
    try {
        // 获取邮件会话属性
        Properties props = new Properties();
        props.setProperty("mail.transport.protocol", "smtp");
        props.setProperty("mail.host", MailConstant.MAIL_SERVER_OUTGOING);
        if (MailConstant.MAIL_SERVER_AUTH != null &&

```

```

        MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {
            props.put("mail.smtp.auth", MailConstant.MAIL_SERVER_AUTH);
        }
        if (MailConstant.OUTGOING_PORT != null &&
            !MailConstant.OUTGOING_PORT.trim().equals("")) {
            props.put("mail.smtp.port", MailConstant.OUTGOING_PORT);
            props.put("mail.smtp.socketFactory.port",
                MailConstant.OUTGOING_PORT);
            props.put("mail.smtp.socketFactory.class",
                "javax.net.ssl.SSLSocketFactory");
            props.put("mail.smtp.socketFactory.fallback", "false");
            props.setProperty("mail.smtp.quitwait", "false");
        }
        // 如果需要身份认证, 则根据邮件会话属性和创建的密码验证器构造一个发送邮件
        件的 session, 否则根据邮件会话属性构造一个发送邮件的 session
        Session session = null;
        if (MailConstant.MAIL_SERVER_AUTH != null &&
            MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {
            session = Session.getInstance(props, authen);
        } else {
            session = Session.getInstance(props);
        }
        session.setDebug(MailConstant.MAIL_DEBUG != null ?
            MailConstant.MAIL_DEBUG.equalsIgnoreCase("true") : false);
        // 根据 session 创建一个邮件对象
        Message message = new MimeMessage(session);
        // 设置邮件发送者地址
        message.setFrom(new InternetAddress(email_from));
        // 设置邮件接收者地址
        if (email_to != null && !email_to.trim().equals("")) {
            String tos[] = email_to.trim().split(",");
            Address toAdd[] = new Address[tos.length];
            for (int i = 0; i < toAdd.length; i++) {
                toAdd[i] = new InternetAddress(tos[i]);
            }
            message.addRecipients(Message.RecipientType.TO, toAdd);
        }
        // 设置邮件抄送者地址
        if (email_cc != null && !email_cc.trim().trim().equals("")) {
            String ccs[] = email_cc.split(",");
            Address ccAdd[] = new Address[ccs.length];
            for (int i = 0; i < ccAdd.length; i++) {
                ccAdd[i] = new InternetAddress(ccs[i]);
            }
        }

```



```

        message.addRecipients(Message.RecipientType.CC, ccAdd);
    }
    // 设置邮件密送者地址
    if ( email_bcc != null && !email_bcc.trim().trim().equals( "" )) {
        String bccs[] = email_bcc.split(",");
        Address bccAdd[] = new Address[bccs.length];
        for ( int i = 0; i < bccAdd.length; i++ ) {
            bccAdd[i] = new InternetAddress(bccs[i]);
        }
        message.addRecipients(Message.RecipientType.BCC, bccAdd);
    }
    // 设置邮件主题
    message.setSubject(email_subject);
    // 设置邮件发送日期
    message.setSentDate(new Date());
    // 将邮件文本内容加入 multipart
    BodyPart messageBodyPart = new MimeBodyPart();
    messageBodyPart.setText(email_body);
    Multipart multipart = new MimeMultipart();
    multipart.addBodyPart(messageBodyPart);
    // 若有附件，则将邮件附件内容加入 multipart
    messageBodyPart = new MimeBodyPart();
    if (file_name != null) {
        // 根据附件的文件名获取数据源
        DataSource source = new FileDataSource(file_name);
        // 将附件的内容加入 messageBodyPart
        messageBodyPart.setDataHandler(new DataHandler(source));
        // 将附件的文件名加入 messageBodyPart
        messageBodyPart.setFileName(file_name.getName());
        multipart.addBodyPart(messageBodyPart);
    }
    // 设置邮件内容，包括邮件文本内容和邮件附件内容
    message.setContent(multipart);
    // 发送邮件
    Transport.send(message);
} catch (MessagingException mex) { // Trap the MessagingException Error
    result = false;
}
return result;
}

```

3.10.5 进一步的扩展

进一步的扩展可以参考成熟的电子邮件客户端，如 FoxMail 和 Outlook 等。

3.11 聊天客户/服务器程序

3.11.1 课程设计目的

本课程设计主要目的是设计与实现一个简单的聊天客户/服务器程序。通过聊天客户/服务器程序的设计与实现，掌握网络编程的知识和技能，熟悉网络软件的开发过程。

3.11.2 课程设计内容

本课程设计要求设计与实现一个简单的聊天客户/服务器程序，具体内容如下：

- 1) 该程序能够实现以下功能：能够支持多人聊天，聊天的内容可以仅仅支持文本信息；客户端程序具有图形化用户界面；
- 2) 能够用 Wireshark 分析的聊天程序的报文。

3.11.3 相关知识

本课程设计需要 Java 网络编程的基本知识，请参见附录 C。

3.11.4 课程设计的分析

聊天客户/服务器程序包括客户程序和服务器程序。

1. 客户程序核心代码

1) ChatClient 类聊天客户端窗口类。以下是其主要的方法：
main() 方法建立 ChatClient 的对象，并对其进行设置。

```
public static void main(String args[]) {  
    ChatClient client = new ChatClient();  
    client.setTitle(client.appName);  
    client.setSize(450, 500);  
    client.setLocation(100, 100);  
    client.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    client.setVisible(true);  
    client.msgWindow.requestFocus();  
}
```

ChatClient() 是 ChatClient 类的构造方法。

```
public ChatClient() {  
    uiInit();  
    txtHost.setText("127.0.0.1");  
    txtPort.setText("3500");  
}
```

uiInit() 方法建立聊天客户端窗口。

```
public void uiInit() {
```

```

// 建立服务器和用户信息窗口
...;
// 建立发送窗口
...;
// 建立聊天记录窗口
...;
}

```

keyTyped() 方法在用户昵称和发送信息等文本框键入回车键时分别登录服务器和发送信息，在服务器地址、端口和用户昵称文本框键入回车键时光标分别定位在端口、用户昵称和发送消息文本框。

```

public void keyTyped(KeyEvent e) {
    if (e.getKeyChar() == KeyEvent.VK_ENTER) {
        if (e.getSource() == msgWindow)
            send();
        if (e.getSource() == txtNick) {
            connect();
            msgWindow.requestFocus();
        }
        if (e.getSource() == txtHost)
            txtPort.requestFocus();
        if (e.getSource() == txtPort)
            txtNick.requestFocus();
    }
}

```

actionPerformed() 方法功能是单击 Connect 和 Send 按钮时分别登录服务器和发送信息。

```

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == buttonConnect)
        connect();
    if (e.getSource() == buttonSend)
        send();
}

```

send() 方法将发送窗口的信息发送给服务器。

```

private void send() {
    String toSend = msgWindow.getText();
    ck.sendMessage(toSend);
    lastMsg = "" + toSend;
    msgWindow.setText("");
}

```

connect() 方法功能是用户登录服务器。

```

private void connect() {
    try {
        if (ck != null)
            ck.dropMe();
    }
}

```

```

        ck = new ClientKernel(txtHost.getText(), Integer.parseInt(
                                                                    txtPort.getText()));

        ck.setNick(txtNick.getText());
        if (ck.isConnected()) {
            ck.addClient(this);
            addMsg("<font color=\"#00ff00\">connected! Local Port:" +
                    ck.getLocalPort() + "</font>");
        } else {
            addMsg("<font color=\"#ff0000\">connect failed! </font>");
        }
    } catch (Exception e) {...; }
}

```

2) ClientKernel 是客户端发送和接收信息的类。以下是其主要的方法：
ClientKernel () 是 ClientKernel 类的构造方法。

```

public ClientKernel(String server, int port) {
    this.port = port;
    nick = "" + port;
    serverAd = server;
    clients = new LinkedList();
    connect();
    if (isConnected) {
        cms = new ClientMsgSender(this, sock);
        cml = new ClientMsgListener(this, sock);
    }
}

```

connect() 方法建立与服务器的连接。

```

public void connect() {
    try {
        sock = new Socket(serverAd, port);
        isConnected = true;
    } catch (IOException ioe) {...;}
}

```

setNick() 方法将指定的字符串前添加 0xFD 和 "nick " 后向服务器发送。

```

public boolean setNick(String nick) {
    sendMessage("" + ClientKernel.COMMAND + "nick " + nick);
    return true;
}

```

sendMessage() 方法的功能是在线时，如果发送的信息是命令（首字符为 '/'），则将其首字符替换为 0xFD 后向服务器发送，否则向服务器直接发送。

```

public void sendMessage(String str) {
    if (!dropMe) {
        if (str.charAt(0) == '/')
            cms.addMessage("" + ClientKernel.COMMAND + str.substring(1));
        else

```

```

        cms.addMessage(str);
    }
}

```

storeMsg() 方法将信息添加到每个用户的聊天窗口中。

```

public synchronized void storeMsg(String str) {
    Object[] client = clients.toArray();
    for (int i = 0; i < client.length; i++)
        ((ChatClient) (client[i])).addMsg(str);
}

```

3) ClientMsgSender 是向服务器发送信息的类。以下是其主要的方法：

ClientMsgSender() 是 ClientMsgSender 类的构造方法。

```

public ClientMsgSender(ClientKernel ck, Socket s) {
    this.ck = ck;
    this.s = s;
    msgList = new LinkedList();
    start();
}

```

run() 方法向服务器发送信息列表中的信息。

```

public void run() {
    try {
        DataOutputStream dataOut = new DataOutputStream(s.getOutputStream());
        while (running) {
            while (msgList.size() > 0) {
                String msg = ((String) (msgList.removeFirst()));
                char[] data = msg.toCharArray();
                for (int i = 0; i < data.length; i++)
                    dataOut.write((int) data[i]);
                dataOut.write(ClientKernel.MSGENDCHAR);
            }
            sleep(10);
        }
        dataOut.write(ClientKernel.EXIT);
        dataOut.close();
    } catch (Exception ioe) {...};
    } finally {
        hasStoped = true;
    }
}

```

4) ClientMsgListener 是接收服务器信息的类。以下是其主要的方法：

ClientMsgListener() 是 ClientMsgListener 类的构造方法。

```

public ClientMsgListener(ClientKernel ck, Socket s) {
    this.ck = ck;
    this.s = s;
    start();
}

```

```

}
run()方法接收服务器信息，并将其添加到聊天窗口中。
public void run() {
    try {
        BufferedInputStream buffIn = new BufferedInputStream(s.getInputStream());
        DataInputStream dataIn = new DataInputStream(buffIn);
        while (running) {
            StringBuffer strBuff = new StringBuffer();
            int c;
            while ((c = dataIn.read()) != ClientKernel.MSGENDCHAR) {
                strBuff.append((char) c);
            }
            // 将收到的信息添加到信息列表中
            ck.storeMsg("" + strBuff.toString());
        }
        dataIn.close();
        buffIn.close();
    } catch (IOException ioe) {...;
    } finally {
        hasStoped = true;
    }
}

```

2. 服务器程序核心代码

1) MainServer 是服务器的主类。 建立服务器套接口，接收客户端的连接请求。以下是其主要的方法：

main()方法建立FileDataSource、BroadcastCommandParser和MainServer等类的对象。

```

public static void main(String arg[]) {
    int port = 0;
    MainServer ms;
    MainServer.ds = new FileDataSource();
    MainServer.cp = new BroadcastCommandParser();
    MainServer.cp.setDataSource(MainServer.ds);
    if(arg.length!=1) {
        ms = new MainServer(3500);
        System.out.println("Usage: java jchat.server.MainServer <port>");
        System.out.println("Attempting to use default port 3500");
    } else {
        try {
            port = Integer.parseInt(arg[0]);
        } catch (NumberFormatException nfe) {...;
        } finally {
            ms = new MainServer(port);
        }
    }
}

```

```
}
```

MainServer() 是 MainServer 类的构造方法。

```
public MainServer(int port) {  
    this.port = port;  
    ck = new ConnectionKeeper(MainServer.cp);  
    MainServer.uptime = System.currentTimeMillis();  
    start();  
}
```

run() 方法建立服务器套接口，等待客户端的连接请求；当接收到客户端的连接请求时建立 ConnectedClient 的对象，并将其加入用户列表中。

```
public void run() {  
    while(true) {  
        try {  
            sSock = new ServerSocket(port);  
            if(newPort) {  
                System.out.println("Server Listening at port: " +  
                                    sSock.getLocalPort());  
                newPort = false;  
            }  
            sock = sSock.accept();  
            ck.add(sock);  
            sSock.close();  
        } catch(Exception e) {...;}  
    }  
}
```

2) FileDataSource 是用户信息处理类。以下是其主要的方法：

FileDataSource() 是 FileDataSource 类的构造方法。

```
public FileDataSource() {  
    System.out.println("FileDataSource init");  
    initFileSystem();  
    if(allWrite) {  
        readFiles();  
    }  
}
```

initFileSystem() 方法初始化文件系统。判断是否存在 fsroot 目录，不存在则建立该目录；然后在该目录下分别建立用户文件（文件名为 users）和信息文件（文件名为 info）。

```
private void initFileSystem() {  
    File file = new File(fsroot);  
    // 判断是否存在 fsroot 目录，不存在则建立该目录  
    if(file.exists() && file.isDirectory()) {  
        if(debug) System.out.println("FileDataSource home " + fsroot);  
        basicWrite = true;  
    } else {  
        file.mkdir();  
    }  
}
```

```

        if(!(file.exists() && file.isDirectory())) {
            System.out.println("FileDataSource is unable to create home @ " +
                                fsroot + " ex " + file.exists() + " isDir " +
                                file.isDirectory());
        }else {
            System.out.println("FileDataSource home created " + fsroot);
            basicWrite = true;
        }
    }
}
// 在该目录下分别建立用户文件（文件名为 users）和信息文件（文件名为 info）。
if(basicWrite) {
    userFile = new File(fsroot + "users");
    infoFile = new File(fsroot + "info");
    ...;
}
}

```

readFiles() 方法是从用户文件中读取用户名和密码，将其保存到名为用户列表中；从信息文件中读取用户、域和数据保存到信息列表中。

```

private void readFiles() {
    Object[] ObjUsers = readLines(userFile);
    Object[] ObjInfo = readLines(infoFile);
    users = new LinkedList();
    infoList = new LinkedList();
    try {
        for(int i = 0;i<ObjUsers.length;i++) {
            String str = "" + ObjUsers[i];
            String name = str.substring(0, str.indexOf(" "));
            String pass = str.substring(str.indexOf(" ")+1);
            users.add(new User(name, pass));
        }
        for(int i = 0;i<ObjInfo.length;i++) {
            String str = "" + ObjInfo[i];
            StringTokenizer strTok = new StringTokenizer(str);
            infoList.add(new Info(strTok.nextToken(),
                                strTok.nextToken(), strTok.nextToken()));
        }
    } catch(Exception e) {...; }
}

```

3) BroadcastCommandParser 是解析客户端命令的类。以下是其主要的方法：
runCommand() 方法解析从客户端接收的命令。

```

public void runCommand(ConnectedClient cc, String str) {
    try {
        if(ds == null) {

```



```

        cc.sendMessage("Server: Your command didn't get parsed, The Server Admin
knows why ;)");
    } else {
        StringTokenizer strTok = new StringTokenizer(str);
        String command = strTok.nextToken();
        if(command.equalsIgnoreCase(NICK))
            if(strTok.hasMoreTokens())
                setNick(cc, strTok.nextToken());
            else
                cc.sendMessage("usage: /nick <newNick>");
        else if (command.equalsIgnoreCase(USERS))
            users(cc);
        else if (command.equalsIgnoreCase(EXIT))
            exit(cc);
        else if (command.equalsIgnoreCase(VERIFY))
            verifyNick(cc, strTok.nextToken());
        else if (command.equalsIgnoreCase(REGISTER))
            registerNick(cc, strTok.nextToken(), strTok.nextToken());
        else if (command.equalsIgnoreCase(WHO_AM_I))
            whoAmI(cc);
        else if (command.equalsIgnoreCase(MSG))
            msg(cc, strTok.nextToken(), strTok);
        else if (command.equalsIgnoreCase(STATS))
            stats(cc);
    }
} catch (Exception e) {...;}
}

```

setNick() 方法设置昵称。当昵称和密码都有效时如果指定的昵称未被占用，则设置为指定昵称，否则通知客户已占用；当昵称或密码无效时提醒用户。

```

private void setNick(ConnectedClient cc, String str) {
    cc.sendMessage("Server: You are now known as " + str);*/
    cc.verifiedBoolean = false;
    boolean verify = ds.verifyUser(str, "");
    if(verify) {
        if(isNickFree(cc, str)) {
            cc.nick = str;
            cc.verifiedBoolean = true;
        } else {
            cc.sendMessage("nick " + str + " was already taken");
        }
    } else {
        cc.verifiedCount = 5;
        cc.tmpNick = str;
        cc.sendMessage("Nick " + str + " is registered so you have to " +

```

```

        "verify that this nick is yours");
    }
}
users()方法将获取的用户列表发送给用户。
private void users(ConnectedClient cc) {
    LinkedList users =
        (LinkedList)((cc.getConnectionKeeper().users()).clone());
    String msg = "Current Connected Users: <br>";
    while(users.size()>0)
        msg += "*" + ((ConnectedClient)(users.removeFirst())).getNick() + "<br>";
    cc.sendMessage(msg);
}

```

verifyNick()方法验证用户和密码是否有效。

```

private void verifyNick(ConnectedClient cc, String password) {
    if(ds.verifyUser(cc.tmpNick, password)) {
        cc.nick = cc.tmpNick;
        cc.verifyedBoolean = true;
    } else {
        cc.nick = "" + cc.portNumber;
        cc.sendMessage("Invalid user/pass, your nick is set to " + cc.nick);
    }
}

```

registerNick()方法注册用户。

```

private void registerNick(ConnectedClient cc, String nick, String pass) {
    if(pass.length()<4 || nick.length()<4) {
        cc.sendMessage("Your nick/password needs to be atleast 4 chars long");
    } else {
        if(ds.addUser(nick, pass)) {
            cc.sendMessage("User " + nick + " is now registered and set as your own");
            cc.nick = nick;
            cc.verifyedBoolean = true;
        } else {
            cc.sendMessage("The username is allready taken");
        }
    }
}

```

msg()方法向指定用户发送指定信息。

```

private void msg(ConnectedClient cc, String user, StringTokenizer strTok) {
    StringBuffer strBuff = new StringBuffer();
    while(strTok.hasMoreTokens())
        strBuff.append(strTok.nextToken() + " ");
    cc.sendTo(user, cc.nick + ":" + strBuff.toString());
}

```

4) ConnectedClient 。以下是其主要的方法：

ConnectedClient() 是 ConnectedClient 类的构造方法。

```
public ConnectedClient(Socket sock, ConnectionKeeper ck) {
    this.ck = ck;
    ipNumber = sock.getInetAddress().getHostAddress();
    portNumber = sock.getPort();
    this.sock = sock;
    msgSend = new ServerMsgSender(this.sock, this);
    msgList = new ServerMsgListener(this.sock, this);
    nick = "" + portNumber;
}
```

sendMessage() 向用户发送信息。

```
public void sendMessage(String str) {
    msgSend.addMessage(str);
}
```

sendTo() 方法向用户列表中指定用户发送指定信息。

```
public void sendTo(String user, String msg) {
    ck.sendTo(this, user, msg);
}
```

broadcastMessage() 方法向用户列表中所有用户发送信息。

```
public void broadcastMessage(String str) {
    if(!isSpam(str)) ck.broadcast(str);
}
```

runCommand() 方法功能是如果是命令（手字符为 0xFD），则解析命令。

```
public void runCommand(String str) {
    if(str.charAt(0)==0xFD) {
        String str1 = str.substring(1);
        ck.runCommand(this, str1);
    }
}
```

5) ServerMsgSender 类是服务器将信息列表中的信息发送给客户端的类。以下是其主要的方法：

ServerMsgSender () 是 ServerMsgSender 类的构造方法。

```
public ServerMsgSender(Socket sock, ConnectedClient cc) {
    this.sock = sock;
    this.cc = cc;
    msgList = new LinkedList();
    start();
}
```

run() 方法将信息列表中的内容发送到客户端。

```
public void run() {
    try {
        DataOutputStream dataOut = new DataOutputStream(sock.getOutputStream());
        while(running) {
            while(msgList.size()>0) {
```

```

        String toSend = (String) (msgList.removeFirst());
        dataOut.writeBytes("" + toSend + MainServer.MSGENDCHAR);
        if(cc.printMsg) System.out.println("MsgSender.run: Sending: " +
                                           toSend);

        sleep(10);
    }
    sleep(10);
}
} catch(Exception e) {...;}
}

```

6) ServerMsgListener 类是接收和处理客户端信息或命令的类。以下是其主要的方法：
ServerMsgListener () 是 ServerMsgListener 类的构造方法。

```

public ServerMsgListener(Socket s, ConnectedClient cc) {
    msgList = new LinkedList();
    sock = s;
    this.cc = cc;
    start();
}

```

run() 方法从客户端读取信息，并判断是否是命令（首字符为 0xFD）；如果不是命令，则将信息广播给在线所有用户，否则处理命令。

```

public void run() {
    try {
        BufferedInputStream buffIn =
            new BufferedInputStream(sock.getInputStream());
        DataInputStream dataIn = new DataInputStream(buffIn);
        while(running) {
            ...;
            StringBuffer strBuff = new StringBuffer();
            sleep(10);
            while( (c=dataIn.read()) != 0xff) {
                strBuff.append((char)c);
                if(!didRun) didRun=true;
                // 判断类型是否是 Command
                if(c==0xFD) isCommand = true;
            }
            if(cc.verifyedCount>0 && !cc.verifyedBoolean && !isCommand) {
                cc.verifyedCount--;
                if(cc.verifyedCount==1) {
                    cc.sendMessage("You have failed to verify your nick");
                    cc.nick = "" + cc.portNumber;
                    cc.sendMessage("Your nick is " + cc.nick);
                } else {
                    cc.sendMessage("type: \"/verify &lt;password&gt\" to verify your
nick");

```

```

    }
}
if(didRun) {
    String toSend = "" + cc.nick + ":" + strBuff.toString();
    if(cc.printMsg)
        System.out.println("MsgListenet.run Sending msg: " + toSend);
    // 如果不是命令，则广播信息，否则处理命令
    if(!isCommand)
        cc.broadcastMessage(toSend);
    else
        cc.runCommand(strBuff.toString());
}
}
} catch(SocketException se) { ...;
} catch(Exception e) {...;
}
}

```

7) ConnectionKeeper。以下是其主要的方法：

ConnectionKeeper ()是 ConnectionKeeper 类的构造方法。

```

public ConnectionKeeper(CommandParser parser) {
    this.cp = parser;
    clientList = new LinkedList();
}

```

runCommand() 方法功能是解析命令。

```

public void runCommand(ConnectedClient cc, String str) {
    cp.runCommand(cc, str);
}

```

sendTo() 向用户列表中指定用户发送指定信息。

```

public void sendTo(ConnectedClient sender, String user, String msg) {
    boolean found = false;
    for(int i =0;i<clientList.size();i++) {
        ConnectedClient receiver = (ConnectedClient)(clientList.get(i));
        if(user.equalsIgnoreCase(receiver.nick)) {
            receiver.sendMessage(msg);
            found = true;
            break;
        }
    }
    if(!found)
        sender.sendMessage("Unable to find user " + user);
}

```

broadcast() 方法向用户列表中所有用户发送信息。

```

public void broadcast(String msg) {
    for(int i =0;i<clientList.size();i++) {

```

```
        ConnectedClient cc = (ConnectedClient) (clientList.get(i));  
        cc.sendMessage(msg);  
    }  
}
```

3.11.5 进一步的扩展

参考 QQ，在给定的参考程序的基础上，对功能做出扩充，不局限于以下的范围：

- 1) 在客户端界面上显示所有联入聊天服务器的用户；
- 2) 支持两个用户之间的聊天；
- 3) 支持增加好友的功能，好友上线时如果该用户在线，则做出提醒；
- 4) 增加用户的个人信息修改、保存和查询；
- 5) 在聊天内容中支持中文；
- 6) 在聊天内容中支持图片等多媒体信息；
- 7) 支持在用户之间传输文件等附件等。

附录

附录 A 网络编程的参考答案

A.1 以太网 ARP 请求发送程序参考答案

本程序代码如下：

// 通过原始套接口发送以太网 ARP 报文。需要构造以太网头部和 ARP 报文。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/if_ether.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>
#include <bits/ioctls.h>
#include <net/if.h>
#include <linux/if_ether.h>    // ETH_P_ARP = 0x0806
#include <linux/if_packet.h>
#include <net/ethernet.h>
#include <errno.h>
#define ETH_HLEN 14          // 以太网帧头长度
#define IP_HLEN 20           // IPv4 头长度
#define ARP_HLEN 28          // ARP 头长度
#define ARPOP_REQUEST 1      // 1 为 ARP 请求
int main(int argc, char **argv) {
    int status, frame_length;
    int sd;
    char *if_name;            // 接口名
    char *src_ip, *tgt_ip;    // 发送方 IPv4 地址和目的 IPv4 地址
    uint8_t *src_mac, *tgt_mac; // 发送方 MAC 地址和目的 MAC 地址
    uint8_t *ether_frame;     // 以太网帧
    struct ether_arp arp_hdr;  // ARP 头部
    struct sockaddr_ll phy_layer_addr;
    struct ifreq ifr;
```

```

if (argc != 2) {
    printf("usage:arp_send <IPaddress>\n");
    exit(1);
}
// 分配各种缓冲区
if ((src_mac = (uint8_t *)malloc(6 * sizeof(uint8_t))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(src_mac, 0, 6 * sizeof(uint8_t));
if ((tgt_mac = (uint8_t *)malloc(6 * sizeof(uint8_t))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(tgt_mac, 0, 6 * sizeof(uint8_t));
if ((ether_frame = (uint8_t *)malloc(IP_MAXPACKET * sizeof(uint8_t))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(ether_frame, 0, IP_MAXPACKET * sizeof(uint8_t));
if ((if_name = (char *)malloc(20 * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(if_name, 0, 20 * sizeof(char));
if ((tgt_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(tgt_ip, 0, INET_ADDRSTRLEN * sizeof(char));
if ((src_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(src_ip, 0, INET_ADDRSTRLEN * sizeof(char));
// 设定发送数据包的接口名
strcpy(if_name, "eth0");
// 建立一个原始套接口，用于获取指定接口的 MAC 地址和 IPv4 地址
if ((sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0) {
    perror("socket ");
    exit(1);
}
// 用 ioctl()函数获取名为 if_name 接口的 MAC 地址
memset(&ifr, 0, sizeof(ifr));

```



```

snprintf(ifr.ifr_name, sizeof(ifr.ifr_name), "%s", if_name);
if (ioctl(sd, SIOCGIFHWADDR, &ifr) < 0) {
    perror("ioctl ");
    return(1);
}
// 复制名为 if_name 接口的 MAC 地址。将它作为 ARP 请求的发送方 MAC 地址
memcpy(src_mac, ifr.ifr_hwaddr.sa_data, 6 * sizeof(uint8_t));
// 输出名为 if_name 接口的 MAC 地址
printf("MAC address for interface %s is ", if_name);
printf("%02x:%02x:%02x:%02x:%02x:%02x\n", src_mac[0], src_mac[1],
        src_mac[2], src_mac[3], src_mac[4], src_mac[5]);
// 用 ioctl() 函数获取名为 if_name 接口的 IPv4 地址
if (ioctl(sd, SIOCGIFADDR, &ifr) < 0) {
    perror("ioctl ");
    return(1);
}
// 复制名为 if_name 接口的 IPv4 地址。将它作为 ARP 请求的发送方 IPv4 地址
strcpy(src_ip, inet_ntoa(((struct sockaddr_in*)&(ifr.ifr_addr))->sin_addr));
// 输出名为 if_name 接口的 IPv4 地址
printf("IPv4 address for interface %s is ", if_name);
printf("%s\n", src_ip);
close(sd);
// 发送数据包时，需指定 sockaddr_ll 结构的 sll_family、物理层地址及它的长度、接口索引等字段，其它字段置零
memset(&phy_layer_addr, 0, sizeof(phy_layer_addr));
// 获取名为 if_name 的接口索引
if((phy_layer_addr.sll_ifindex = if_nametoindex(if_name)) == 0) {
    perror("if_nametoindex ");
    exit(1);
}
printf("Index for interface %s is %i\n", if_name, phy_layer_addr.sll_ifindex);
phy_layer_addr.sll_family = AF_PACKET;
memcpy(phy_layer_addr.sll_addr, src_mac, 6 * sizeof(uint8_t));
phy_layer_addr.sll_halen = 6;
// 以太网帧中的目的 MAC。将它设为广播地址
memset(tgt_mac, 0xff, 6 * sizeof(uint8_t));
// ARP 请求中的目的 IPv4 地址
strcpy(tgt_ip, argv[1]);
// 建立一个原始套接口，用于发送以太网帧
if ((sd = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL))) < 0) {
    perror("socket ");
    exit(1);
}
// 以下是构造 ARP 头

```

```

// ARP 中硬件地址类型。它的值为 1 表示以太网
arp_hdr.arp_hrd = htons(1);
// ARP 中协议地址类型。它的值为 0x0800 表示 IP
arp_hdr.arp_pro = htons(ETH_P_IP);
// ARP 中硬件地址长度。它的值为 6 表示 MAC 地址长度
arp_hdr.arp_hln = 6;
// ARP 中的协议地址长度。它的值为 4 表示 IPv4 地址
arp_hdr.arp_pln = 4;
// ARP 中操作码。它的值为 1 表示 ARP 请求
arp_hdr.arp_op = htons(ARPOP_REQUEST);
// ARP 中发送方 MAC 地址
memcpy(&arp_hdr.arp_sha, src_mac, 6 * sizeof(uint8_t));
printf("\nSender MAC address in ARP: %02x:%02x:%02x:%02x:%02x:%02x\n",
        arp_hdr.arp_sha[0], arp_hdr.arp_sha[1], arp_hdr.arp_sha[2],
        arp_hdr.arp_sha[3], arp_hdr.arp_sha[4], arp_hdr.arp_sha[5]);
// ARP 中发送方 IPv4 地址
if ((status = inet_pton(AF_INET, src_ip, &arp_hdr.arp_spa)) != 1) {
    fprintf(stderr, "inet_pton failed for Sender IP address\n");
    exit(1);
}
printf("Sender IPv4 address in ARP: %u.%u.%u.%u\n", arp_hdr.arp_spa[0],
        arp_hdr.arp_spa[1], arp_hdr.arp_spa[2], arp_hdr.arp_spa[3]);
// ARP 中目的 MAC 地址。将它设为 0
memset(&arp_hdr.arp_tha, 0, 6 * sizeof(uint8_t));
printf("target MAC address in ARP: %02x:%02x:%02x:%02x:%02x:%02x\n",
        arp_hdr.arp_tha[0], arp_hdr.arp_tha[1], arp_hdr.arp_tha[2],
        arp_hdr.arp_tha[3], arp_hdr.arp_tha[4], arp_hdr.arp_tha[5]);
// ARP 中目的 IPv4 地址
if ((status = inet_pton(AF_INET, tgt_ip, &arp_hdr.arp_tpa)) != 1) {
    fprintf(stderr, "inet_pton failed for target IP address\n");
    exit(1);
}
printf("target IPv4 address in ARP: %u.%u.%u.%u\n", arp_hdr.arp_tpa[0],
        arp_hdr.arp_tpa[1], arp_hdr.arp_tpa[2], arp_hdr.arp_tpa[3]);
// 以下是构造以太网帧
// 以太网帧长度=以太网头部长+以太网数据长度（即 ARP 请求长度）
frame_length = 6 + 6 + 2 + ARP_HDRLLEN;
// 以太网帧中目的 MAC 地址
memcpy(ether_frame, tgt_mac, 6 * sizeof(uint8_t));
// 以太网帧中的源 MAC 地址
memcpy(ether_frame + 6, src_mac, 6 * sizeof(uint8_t));
// 以太网帧中类型。对于 ARP 请求或响应来说，该字段的值为 0x0806
ether_frame[12] = ETH_P_ARP / 256;
ether_frame[13] = ETH_P_ARP % 256;

```

```

// 以太网帧中数据部分（即 ARP 请求）
memcpy(ether_frame + ETH_HDRLEN, &arp_hdr, ARP_HDRLEN * sizeof(uint8_t));
// 向套接口 sd 中发送构造的以太网帧
if(sendto(sd, ether_frame, frame_length, 0, (struct sockaddr *) &phy_layer_addr,
          sizeof(phy_layer_addr)) <= 0) {
    perror("sendto");
    exit (1);
}
close(sd);
free(src_mac);
free(tgt_mac);
free(ether_frame);
free(if_name);
free(tgt_ip);
free(src_ip);
return (0);
}

```

A.2 以太网 ARP 响应分析程序参考答案

本程序代码如下：

// 接收和解析以太网 ARP 响应报文

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <signal.h>
#include <netinet/ip.h>
#include <netinet/if_ether.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <linux/if_ether.h> // ETH_P_ARP = 0x0806, ETH_P_ALL = 0x0003
#include <net/ethernet.h>
#include <errno.h>
#define ARPOP_REPLY 2
int i;
int main(int argc, char **argv) {
    int status, sd;
    uint8_t *ether_frame;
    struct ether_arp *arp_hdr;
    // 建立一个原始套接口，用于接收数据
    if((sd = socket(PF_PACKET, SOCK_RAW, htons (ETH_P_ALL))) < 0) {
        perror ("socket ");
        exit (1);
    }
}

```

```

}
// 分配缓冲区
if((ether_frame = (uint8_t *)malloc(IP_MAXPACKET * sizeof(uint8_t))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(ether_frame, 0, IP_MAXPACKET * sizeof(uint8_t));
// 指向 ARP 头
arp_hdr = (struct ether_arp *) (ether_frame + 6 + 6 + 2);
i=0;
while(1) {
    //从套接口 sd 读取以太网帧
    if((status = recv(sd, ether_frame, IP_MAXPACKET, 0)) < 0) {
        if(errno == EINTR) {
            memset(ether_frame, 0, IP_MAXPACKET * sizeof(uint8_t));
            continue; // Something weird happened, but let's try again.
        } else {
            perror("recv");
            exit(1);
        }
    }
    // 如果它是 ARP 响应，则输出这个 ARP 以太网帧
    // ARP 以太网帧格式： 6 字节 MAC+6 字节 MAC+2 字节类型+28 字节 ARP 头部
    if((((ether_frame[12] << 8) + ether_frame[13]) == ETH_P_ARP) &&
        (ntohs(arp_hdr->arp_op) == ARPOP_REPLY)) {
        i++;
        // 输出以太网帧中的头部
        printf("\nEthernet frame header %u:\n", i);
        printf("Destination MAC: ");
        printf("%02x:%02x:%02x:%02x:%02x:%02x\n", ether_frame[0], ether_frame[1],
            ether_frame[2], ether_frame[3], ether_frame[4], ether_frame[5]);
        printf("Source MAC: ");
        printf("%02x:%02x:%02x:%02x:%02x:%02x\n", ether_frame[6], ether_frame[7],
            ether_frame[8], ether_frame[9], ether_frame[10], ether_frame[11]);
        printf("Ethernet type code (0x0806 for ARP): 0x%02x%02x\n",
            ether_frame[12], ether_frame[13]);
        // 输出以太网帧中的数据（ARP 响应头部）
        printf("\nEthernet data (ARP reply header):\n");
        printf("Hardware address type (1 for ethernet): %u\n", ntohs(arp_hdr->arp_hrd));
        printf("Protocol address type (0x0800 for IPv4 addresses): 0x%04x\n",
            ntohs(arp_hdr->arp_pro));
        printf("Hardware(MAC) address length: %u bytes\n", arp_hdr->arp_hln);
        printf("Protocol(MAC) address length: %u bytes\n", arp_hdr->arp_pln);
        printf("Opcode (2 for ARP reply): %u\n", ntohs(arp_hdr->arp_op));
    }
}

```

```

printf("Sender hardware(MAC) address: ");
printf("%02x:%02x:%02x:%02x:%02x:%02x\n",
        arp_hdr->arp_sha[0], arp_hdr->arp_sha[1], arp_hdr->arp_sha[2],
        arp_hdr->arp_sha[3], arp_hdr->arp_sha[4], arp_hdr->arp_sha[5]);
printf("Sender protocol(IPv4) address: %u.%u.%u.%u\n", arp_hdr->arp_spa[0],
        arp_hdr->arp_spa[1], arp_hdr->arp_spa[2], arp_hdr->arp_spa[3]);
printf("Target hardware(MAC) address: ");
printf("%02x:%02x:%02x:%02x:%02x:%02x\n",
        arp_hdr->arp_tha[0], arp_hdr->arp_tha[1], arp_hdr->arp_tha[2],
        arp_hdr->arp_tha[3], arp_hdr->arp_tha[4], arp_hdr->arp_tha[5]);
printf("Target protocol(IPv4) address: %u.%u.%u.%u\n", arp_hdr->arp_tpa[0],
        arp_hdr->arp_tpa[1], arp_hdr->arp_tpa[2], arp_hdr->arp_tpa[3]);
    }
}
close(sd);
free(ether_frame);
return(0);
}

```

A.3 ping 程序参考答案

本程序代码如下：

```

// 简单 ping 程序
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <netinet/in_systm.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <sys/un.h>
#include <unistd.h>
#include <stdarg.h>
#include <syslog.h>
#include <errno.h>

#define BUFSIZE 1500 // 缓冲区的最大长度
#define MAXLINE 4096 // 一行的最大长度
char recvbuf[BUFSIZE]; // 接收缓冲区
char sendbuf[BUFSIZE]; // 发送缓冲区
int datalen = 56; // 随 ICMP 回射请求一起发送的可选数据长度
char *host; // 目的主机地址
int nsent; // 每发送一个数据包加一
pid_t pid; // 进程号

```

```

int sockfd; // 套接口描述字
int verbose; // 是否详尽输出
int daemon_proc; // daemon_init()将其设为非 0
struct sockaddr *sasend;
struct sockaddr *sarecv;
socklen_t salen;
int icmpproto;
// 函数原型
void readloop(void);
void tv_sub(struct timeval *, struct timeval *);
void proc_v4(char *, ssize_t, struct timeval *);
void sig_alm(int);
void send_v4(void);
unsigned short in_cksum(unsigned short *, int);
struct addrinfo* host_serv(const char *host, const char *serv, int family, int socktype);
char *Sock_ntop_host(const struct sockaddr *sa, socklen_t salen);
static void err_doit(int errnoflag, int level, const char *fmt, va_list ap);
void err_quit(const char *fmt, ...);
void err_sys(const char *fmt, ...);
// main()函数
int main(int argc, char **argv) {
    int c;
    struct addrinfo *ai;
    opterr = 0;
    while((c = getopt(argc, argv, "v")) != -1) {
        switch (c) {
            case 'v': // 详尽输出
                verbose++;
                break;
            case '?':
                err_quit("unrecognized option: %c", c);
        }
    }
    if(optind != argc-1)
        err_quit("usage: ping [ -v ] <hostname>");
    host = argv[optind];
    pid = getpid();
    // 为 SIGALRM 信号建立了一个处理程序
    signal(SIGALRM, sig_alm);
    ai = host_serv(host, NULL, 0, 0);
    printf("ping %s (%s): %d data bytes\n", ai->ai_canonname,
        Sock_ntop_host(ai->ai_addr, ai->ai_addrlen), datalen);
    if(ai->ai_family != AF_INET)
        err_quit("unknown address family %d", ai->ai_family);
}

```

```

    sasend = ai->ai_addr;
    sarecv = calloc(1, ai->ai_addrlen);
    salen = ai->ai_addrlen;
    icmpproto = IPPROTO_ICMP;
    // 处理接收数据包
    readloop();
    exit(0);
}
// 处理接收数据包函数
void readloop(void) {
    int size;
    char recvbuf[BUFSIZE];
    socklen_t len;
    ssize_t n;
    struct timeval tval;
    // 建立一个 ICMP 原始套接口
    sockfd = socket(sasend->sa_family, SOCK_RAW, icmpproto);
    // 为安全性考虑, 将进程的有效用户 ID 设成进程的实际用户 ID
    setuid(getuid());
    // 将套接口接收缓冲区设为 61440 字节, 主要为了减少接收缓冲区溢出的可能性
    size = 60 * 1024;          // OK if setsockopt fails
    setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &size, sizeof(size));
    // 调用 SIGALRM 信号处理程序来发送一个数据包, 并调度下一次 SIGALRM 为一
秒之后
    sig_alm(SIGALRM);
    for(;;) {
        len = salen;
        n = recvfrom(sockfd, recvbuf, sizeof(recvbuf), 0, sarecv, &len);
        if(n < 0) {
            if(errno == EINTR)
                continue;
            else
                err_sys("recvfrom error");
        }
        // 记录数据包收到的时间
        gettimeofday(&tval, NULL);
        // 处理 ICMP
        proc_v4(recvbuf, n, &tval);
    }
}
// 将两个 timeval 结构相减, 并将结果存入第一个 timeval 结构中函数
void tv_sub(struct timeval *out, struct timeval *in) {
    if((out->tv_usec -= in->tv_usec) < 0) {      /* out -= in */
        --out->tv_sec;
    }
}

```

```

        out->tv_usec += 1000000;
    }
    out->tv_sec -= in->tv_sec;
}
// 处理 ICMP 回射响应函数
void proc_v4(char *ptr, ssize_t len, struct timeval *tvrecv) {
    int hlen1, icmplen;
    double rtt;
    struct ip *ip;
    struct icmp *icmp;
    struct timeval *tvsend;
    // IP 头部指针
    ip = (struct ip *) ptr;          // start of IP header
    // IP 头部长度。它是以 4 字节为一个单位计数的，将它乘以 4 才是以字节为单位的头
    部长度
    hlen1 = ip->ip_hl << 2;
    // ICMP 头部指针
    icmp = (struct icmp *) (ptr + hlen1);
    if((icmplen = len - hlen1) < 8)
        err_quit("icmplen (%d) < 8", icmplen);
    // 如果是 ICMP 回射响应，那么检查标识符字段，这个响应是不是这个进程发出的请
    求的响应
    if(icmp->icmp_type == ICMP_ECHOREPLY) {
        if(icmp->icmp_id != pid)
            return;
        if(icmplen < 16)
            err_quit("icmplen (%d) < 16", icmplen);
        // 将请求发出时间（包含在 ICMP 响应的可选数据部分内）与当前时间相减，可计
        算出 RTT（以毫秒为单位）
        tvsend = (struct timeval *) icmp->icmp_data;
        tv_sub(tvrecv, tvsend);
        rtt = tvrecv->tv_sec * 1000.0 + tvrecv->tv_usec / 1000.0;
        printf("%d bytes from %s: seq=%u, ttl=%d, rtt=%.3f ms\n",
            icmplen, Sock_ntop_host(sarecv, salen),
            icmp->icmp_seq, ip->ip_ttl, rtt);
    } else if(verbose) { //详尽输出
        printf(" %d bytes from %s: type = %d, code = %d\n",
            icmplen, Sock_ntop_host(sarecv, salen),
            icmp->icmp_type, icmp->icmp_code);
    }
}
// SIGALRM 信号处理函数。它发送一个 ICMP 回射请求，然后调度下一次 SIGALRM
在一秒后产生
void sig_alrm(int signo) {

```



```

    send_v4();
    // 设置信号 SIGALRM 在经过 1 秒数后发送给目前的进程
    alarm(1);
    return;
}
// 发送 ICMP 回显请求
void send_v4(void) {
    int len;
    struct icmp *icmp;
    // 指向 ICMP 头
    icmp = (struct icmp *) sendbuf;
    // ICMP 中类型
    icmp->icmp_type = ICMP_ECHO;
    // ICMP 中代码
    icmp->icmp_code = 0;
    // ICMP 中标识符。它的值为进程 ID
    icmp->icmp_id = pid;
    // ICMP 中序列号
    icmp->icmp_seq = nsent++;
    // ICMP 中数据。它的值为当前时间
    gettimeofday((struct timeval *) icmp->icmp_data, NULL);
    // ICMP 中校验和
    len = 8 + datalen;
    icmp->icmp_cksum = 0;
    icmp->icmp_cksum = in_cksum((u_short *) icmp, len);
    // 发送 ICMP。由于没使用 IP_HDRINCL 选项，内核将构造 IPv4 头部并将其放在缓
    冲区前面。
    sendto(sockfd, sendbuf, len, 0, sasend, salen);
}
// 计算网际校验和函数。被校验数据 16 位值的反码和，如果数据字节为奇数，则尾部
补一个字节的 0（详见 RFC 1071）
unsigned short in_cksum(unsigned short *addr, int len) {
    int nleft = len;
    int sum = 0;
    unsigned short *w = addr;
    unsigned short answer = 0;
    while(nleft > 1) {
        sum += *w++;
        nleft -= 2;
    }
    // 若字节总数是奇数，则增加一个字节的填充(全 0)
    if(nleft == 1) {
        *(unsigned char *)(&answer) = *(unsigned char *)w;
        sum += answer;
    }
}

```

```

    }
    // 把高 16 位的进位加到低 16 位上
    sum = (sum >> 16) + (sum & 0xffff);
    // 再加进位
    sum += (sum >> 16);
    // 将和取反码，截取其低 16 位
    answer = ~sum;
    return(answer);
}
// 将主机名或主机 IP 地址转换为 addrinfo 结构函数
struct addrinfo* host_serv(const char *host, const char *serv, int family, int socktype) {
    int n;
    struct addrinfo hints, *res;
    bzero(&hints, sizeof(struct addrinfo));
    hints.ai_flags = AI_CANONNAME;
    // AF_UNSPEC, AF_INET, AF_INET6 等
    hints.ai_family = family;
    // 0, SOCK_STREAM, SOCK_DGRAM 等
    hints.ai_socktype = socktype;
    if((n = getaddrinfo(host, serv, &hints, &res)) != 0)
        return(NULL);
    return(res);
}
// 将 sockaddr 结构的地址转换字符串函数
char* sock_ntop_host(const struct sockaddr *sa, socklen_t salen) {
    static char str[128];
    struct sockaddr_in *sin = (struct sockaddr_in *) sa;
    if(inet_ntop(AF_INET, &sin->sin_addr, str, sizeof(str)) == NULL)
        return(NULL);
    return(str);
}
char* Sock_ntop_host(const struct sockaddr *sa, socklen_t salen) {
    char *ptr;
    if((ptr = sock_ntop_host(sa, salen)) == NULL)
        err_sys("sock_ntop_host error");
    return(ptr);
}
// 输出错误提示并返回调用函数
static void err_doit(int errnoflag, int level, const char *fmt, va_list ap) {
    int  errno_save, n;
    char buf[MAXLINE];
    errno_save = errno;
#ifdef HAVE_VSNPRINTF
    vsnprintf(buf, sizeof(buf), fmt, ap);

```

```

#else
    vsprintf(buf, fmt, ap);
#endif
    n = strlen(buf);
    if(errnoflag)
        snprintf(buf+n, sizeof(buf)-n, ":%s", strerror(errno_save));
    strcat(buf, "\n");
    if(daemon_proc) {
        syslog(level, buf);
    } else {
        fflush(stdout);
        fputs(buf, stderr);
        fflush(stderr);
    }
    return;
}
// 遇到与系统调用无关的严重错误，输出错误提示并结束
void err_quit(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    err_doit(0, LOG_ERR, fmt, ap);
    va_end(ap);
    exit(1);
}
// 遇到与系统调用有关的严重错误，输出错误提示并结束
void err_sys(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    err_doit(1, LOG_ERR, fmt, ap);
    va_end(ap);
    exit(1);
}

```

A.4 多播客户/服务器程序参考答案

本程序分为多播客户程序和多播服务器程序。

多播客户程序代码如下：

```

// IP 多播客户程序
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

```

```

#include <stdlib.h>
#include <unistd.h>
int port = 9866;
int main(void) {
    int loop = 1; // 多播循环
    int iter = 0;
    int sin_len;
    int sd;
    char message[256];
    struct sockaddr_in sin;
    struct ip_mreq command;
    // 初始化 sockaddr_in 结构
    memset(&sin, 0, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_addr.s_addr = htonl(INADDR_ANY);
    sin.sin_port = htons(port);
    // 建立一个 UDP 套接口
    if((sd = socket(PF_INET, SOCK_DGRAM, 0)) == -1) {
        perror("socket");
        exit(1);
    }
    // 设定 SO_REUSEADDR, 允许多个应用绑定同一个本地端口接收数据包
    loop = 1;
    if(setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &loop, sizeof(loop)) < 0) {
        perror("setsockopt:SO_REUSEADDR");
        exit(1);
    }
    // 用 bind 绑定本地端口, IP 为 INADDR_ANY, 从而能接收多播数据包
    if(bind(sd, (struct sockaddr *)&sin, sizeof(sin)) < 0) {
        perror("bind");
        exit(1);
    }
    // 设置发送多播的主机为接收多播的多播组成员
    loop = 1;
    if(setsockopt(sd, IPPROTO_IP, IP_MULTICAST_LOOP, &loop, sizeof(loop)) < 0) {
        perror("setsockopt:IP_MULTICAST_LOOP");
        exit(1);
    }
    // 将所加入的多播组的 IP 多播地址复制到一个 ip_mreq 结构中
    command.imr_multiaddr.s_addr = inet_addr("224.1.2.3");
    // 所要加入多播组的那个本地接口的 IP 地址复制到一个 ip_mreq 结构中
    command.imr_interface.s_addr = htonl(INADDR_ANY);
    if(command.imr_multiaddr.s_addr == -1) {
        perror("224.1.2.3 not a legal multicast address");
    }
}

```

```

        exit(1);
    }
    // 加入一个多播组。进一步告诉 Linux 内核，特定的套接口即将接收多播数据
    if (setsockopt(sd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &command,
sizeof(command)) < 0) {
        perror("setsockopt:IP_ADD_MEMBERSHIP");
    }
    // 接收 8 个多播
    while(iter++ < 8) {
        sin_len = sizeof(sin);
        if(recvfrom(sd, message, 256, 0, (struct sockaddr *)&sin, &sin_len) == -1) {
            perror("recvfrom");
        }
        printf("Response #%-2d from server: %s\n", iter, message);
    }
    // 接收 8 个多播后退出多播组
    if(setsockopt(sd, IPPROTO_IP, IP_DROP_MEMBERSHIP, &command,
sizeof(command)) < 0) {
        perror("setsockopt:IP_DROP_MEMBERSHIP");
    }
    close(sd);
    exit(0);
}

```

多播服务器程序代码如下：

// IP 多播服务器程序。

```

#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <unistd.h>
#include <stdlib.h>
int port = 9866;
int main(void) {
    int socket_descriptor;
    struct sockaddr_in address;
    // 建立 UDP 套接口
    socket_descriptor = socket(AF_INET, SOCK_DGRAM, 0);
    if (socket_descriptor == -1) {
        perror("Opening socket");
        exit(1);
    }
    // 用多播地址和端口初始化 sockaddr_in 结构

```

```

memset(&address, 0, sizeof(address));
address.sin_family = AF_INET;
address.sin_addr.s_addr = inet_addr("224.1.2.3");
address.sin_port = htons(port);
// 开始 IP 多播
while(1) {
    if(sendto(socket_descriptor, "test from multicast", sizeof("test from multicast"), 0,
              (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("sendto");
        exit(1);
    }
    sleep(2);
}
exit(0);
}

```

A.5 UDP 报文发送程序参考答案

本程序代码如下：

// 通过原始套接口发送 UDP 报文。需要构造 IP 头、UDP 头和 UDP 数据..

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#define __FAVOR_BSD
#include <netinet/udp.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>
#include <bits/ioctls.h>
#include <net/if.h>
#include <errno.h>
#define IP4_HDRLEN 20 // IPv4 头部长度
#define UDP_HDRLEN 8  // UDP 头部长度
uint16_t checksum (uint16_t *, int);
uint16_t udp4_checksum (struct ip, struct udphdr, uint8_t *, int);
int main(int argc, char **argv) {
    const int on = 1;
    int status, datalen, sd, *ip_flags;
    char *if_name, *tgt_ip, *src_ip;
    struct ip ip_hdr;

```

```

struct udphdr udp_hdr;
char *data, *packet;
struct sockaddr_in sin;
struct ifreq ifr;
// 分配缓冲区
if((data = (char *)malloc(IP_MAXPACKET * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(data, 0, IP_MAXPACKET * sizeof(char));
if((packet = (char *)malloc(IP_MAXPACKET * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(packet, 0, IP_MAXPACKET * sizeof(char));
if((if_name = (char *)malloc(20 * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(if_name, 0, 20 * sizeof(char));
if((src_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(src_ip, 0, INET_ADDRSTRLEN * sizeof(char));
if((tgt_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(tgt_ip, 0, INET_ADDRSTRLEN * sizeof(char));
// 设定发送数据包的接口名
strcpy(if_name, "eth0");
// 建立一个原始套接口，用于获取指定接口的索引
if((sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0) {
    perror("socket");
    exit(1);
}
// 用 ioctl() 函数获取名为 if_name 接口的索引，用于将发送数据包的套接口与这个接口索引绑定
memset(&ifr, 0, sizeof(ifr));
snprintf(ifr.ifr_name, sizeof(ifr.ifr_name), "%s", if_name);
if(ioctl(sd, SIOCGIFINDEX, &ifr) < 0) {
    perror("ioctl");
    return(1);
}

```

```

}
close(sd);
printf("Index for interface %s is %i\n", if_name, ifr.ifr_ifindex);
// 设定 UDP 报文的源 IPv4 地址
strcpy(src_ip, "192.168.1.188");
// 设定 UDP 报文的目的 IPv4 地址
strcpy(tgt_ip, "192.168.1.1");
// 发送数据包时，用户需要指定要发送数据包的目的 IPv4 地址
memset(&sin, 0, sizeof(struct sockaddr_in));
sin.sin_family = AF_INET;
if((status = inet_pton(AF_INET, tgt_ip, &(sin.sin_addr))) != 1) {
    fprintf(stderr, "inet_pton failed for destination IP address\n");
    exit(1);
}
// 建立一个原始套接口，用于发送数据包
if((sd = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0) {
    perror("socket");
    exit(1);
}
// 使用套接口 IP_HDRINCL 选项设置套接口。在之后发送数据时，
// 内核将构造以太网帧头部数据并将其放到 IP 头部前，用户需要设置 IP 头部数据
if(setsockopt(sd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
    perror("setsockopt failed to set IP_HDRINCL ");
    exit(1);
}
// 将套接口 sd 与指定接口的索引绑定
if(setsockopt(sd, SOL_SOCKET, SO_BINDTODEVICE, &ifr, sizeof(ifr)) < 0) {
    perror("setsockopt failed to bind to interface ");
    exit(1);
}
// 构造 UDP 报文，包括 UDP 头和数据。UDP 头部见<netinet/udp.h>
// 构造 UDP 中数据
strcpy(data, "Sending an IPv4 UDP packet through a raw socket.");
datalen = strlen("Sending an IPv4 UDP packet through a raw socket.");
//构造 UDP 头
// UDP 中源端口号
udp_hdr.uh_sport = htons(9866);
printf("Source port in TCP header: %s\n", "9866");
// UDP 中目的端口号
udp_hdr.uh_dport = htons(161);
printf("Dest port in TCP header: %s\n", "161");
// UDP 中 UDP 长度（32 bit words），包括 UDP 头和 UDP 数据
udp_hdr.uh_ulen = htons(UDP_HDRLEN + datalen);
// UDP 中校验和，先设为 0，构造 IP 头后再计算

```



```

udp_hdr.uh_sum = 0;
// 构造 IPv4 包头部。IPv4 包头见<netinet/ip.h>
// IPv4 包中头部长度。它的值为 5
ip_hdr.ip_hl = IP4_HDRLEN >> 2;
// IPv4 包中版本号。它的值为 4 表示 IPv4
ip_hdr.ip_v = 4;
// IPv4 包中类型
ip_hdr.ip_tos = 0;
// IPv4 包中总长度。包括 IPv4 头部长度、UDP 头部长度和 UDP 数据
ip_hdr.ip_len = htons(IP4_HDRLEN + UDP_HDRLEN + datalen);
// IPv4 包中标识
ip_hdr.ip_id = htons(getpid());
printf("Identification in IP header: %u\n", getpid());
// IPv4 包中标志和分段偏移。
// IPv4 包中保留分段标志位、不允许分段标志位、更多分段标志位和分段偏移。都设
为 0
ip_hdr.ip_off = 0;
// IPv4 包中 TTL。默认设为最大值 255
ip_hdr.ip_ttl = 255;
// IPv4 包中协议。它的值为 17 表示 UDP
ip_hdr.ip_p = IPPROTO_UDP;
// IPv4 包中源 IPv4 地址
if ((status = inet_pton(AF_INET, src_ip, &(ip_hdr.ip_src))) != 1) {
    fprintf(stderr, "inet_pton failed\n");
    exit(1);
}
printf("Source addresss in IP header: %s\n", src_ip);
// IPv4 包中目的 IPv4 地址
ip_hdr.ip_dst.s_addr = sin.sin_addr.s_addr;
printf("Dest addresss in IP header: %s\n", tgt_ip);
// IPv4 包中头部校验和
ip_hdr.ip_sum = 0;
ip_hdr.ip_sum = checksum((uint16_t *) &ip_hdr, IP4_HDRLEN);
// 计算 UDP 中校验和
udp_hdr.uh_sum = udp4_checksum(ip_hdr, udp_hdr, data, datalen);
// 将构造的 IPv4 头部复制到发送缓冲区中
memcpy(packet, &ip_hdr, IP4_HDRLEN * sizeof(uint8_t));
// 将构造的 UDP 头部复制到发送缓冲区中
memcpy((packet + IP4_HDRLEN), &udp_hdr, UDP_HDRLEN * sizeof(uint8_t));
// 将构造的 UDP 数据复制到发送缓冲区中
memcpy(packet + IP4_HDRLEN + UDP_HDRLEN, data, datalen * sizeof(uint8_t));
// 发送数据包
if(sendto(sd, packet, IP4_HDRLEN + UDP_HDRLEN + datalen, 0, (struct sockaddr *)
&sin,

```

```

sizeof(struct sockaddr) < 0) {
    perror("sendto");
    exit(1);
}
close(sd);
free(data);
free(packet);
free(if_name);
free(tgt_ip);
free(src_ip);
return(0);
}
// 计算网际校验和函数。被校验数据 16 位值的反码和，如果数据字节为奇数，则尾部
补一个字节的 0（详见 RFC 1071）
uint16_t checksum(uint16_t *addr, int len) {
    register int nleft = len;
    register uint16_t *w = addr;
    register int sum = 0;
    uint16_t answer = 0;
    while(nleft > 1) {
        sum += *w++;
        nleft -= 2;
    }
    // 若字节总数是奇数，则增加一个字节的填充(全 0)
    if(nleft == 1) {
        *(u_char *)&answer = *(u_char *)w;
        sum += answer;
    }
    // 把高 16 位的进位加到低 16 位上
    sum = (sum >> 16) + (sum & 0xffff);
    // 再加进位
    sum += (sum >> 16);
    // 将和取反码，截取其低 16 位
    answer = ~sum;
    return(answer);
}
// 计算 UDP 校验和函数
uint16_t udp4_checksum(struct ip ip_hdr, struct udphdr udp_hdr, uint8_t *payload,
int payloadlen){
    char buf[IP_MAXPACKET];
    char *ptr;
    int chksumlen = 0;
    int i;
    // 指向缓冲区

```

```

ptr = &buf[0];
// 将 IP 包中源 IPv4 地址复制到 buf
memcpy(ptr, &ip_hdr.ip_src.s_addr, sizeof(ip_hdr.ip_src.s_addr));
ptr += sizeof(ip_hdr.ip_src.s_addr);
chksumlen += sizeof(ip_hdr.ip_src.s_addr);
// 将 IP 包中目的 IPv4 地址复制到 buf
memcpy(ptr, &ip_hdr.ip_dst.s_addr, sizeof(ip_hdr.ip_dst.s_addr));
ptr += sizeof(ip_hdr.ip_dst.s_addr);
chksumlen += sizeof(ip_hdr.ip_dst.s_addr);
// 将 0 字段复制到 buf (8 位)
*ptr = 0; ptr++;
chksumlen += 1;
// 将 IP 包中协议复制到 buf (8 位)
memcpy(ptr, &ip_hdr.ip_p, sizeof(ip_hdr.ip_p));
ptr += sizeof(ip_hdr.ip_p);
chksumlen += sizeof(ip_hdr.ip_p);
// 将 UDP 中长度复制到 buf (16 位)
memcpy(ptr, &udp_hdr.uh_ulen, sizeof(udp_hdr.uh_ulen));
ptr += sizeof(udp_hdr.uh_ulen);
chksumlen += sizeof(udp_hdr.uh_ulen);
// 将 UDP 中源端口号复制到 buf (16 位)
memcpy(ptr, &udp_hdr.uh_sport, sizeof(udp_hdr.uh_sport));
ptr += sizeof(udp_hdr.uh_sport);
chksumlen += sizeof(udp_hdr.uh_sport);
// 将 UDP 中目的端口号复制到 buf (16 位)
memcpy(ptr, &udp_hdr.uh_dport, sizeof(udp_hdr.uh_dport));
ptr += sizeof(udp_hdr.uh_dport);
chksumlen += sizeof(udp_hdr.uh_dport);
// 将 UDP 中长度复制到 buf (16 位)
memcpy(ptr, &udp_hdr.uh_ulen, sizeof(udp_hdr.uh_ulen));
ptr += sizeof(udp_hdr.uh_ulen);
chksumlen += sizeof(udp_hdr.uh_ulen);
// 将 UDP 中检验和复制到 buf (8 位)。先设为 0
*ptr = 0; ptr++;
*ptr = 0; ptr++;
chksumlen += 2;
// 将 UDP 中数据复制到 buf (8 位)
memcpy(ptr, payload, payloadlen);
ptr += payloadlen;
chksumlen += payloadlen;
return checksum ((uint16_t *) buf, chksumlen);
}

```

A.6 TCP SYN 报文发送程序参考答案

本程序代码如下：

// 通过原始套接口发送 UDP 报文。需要构造 IP 头和 TCP 头

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#define __FAVOR_BSD
#include <netinet/tcp.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>
#include <bits/ioctls.h>
#include <net/if.h>
#include <errno.h>
#define IP4_HDRLEN 20 // IPv4 头部长度的
#define TCP_HDRLEN 20 // TCP 头部长度的，不包括选项
uint16_t checksum(uint16_t *, int);
uint16_t tcp4_checksum(struct ip, struct tcphdr);
int main(int argc, char **argv) {
    const int on = 1;
    int i, status, sd;
    char *if_name, *tgt_ip, *src_ip;
    struct ip ip_hdr;
    struct tcphdr tcp_hdr;
    char *packet;
    struct sockaddr_in sin;
    struct ifreq ifr;
    // 分配缓冲区
    if((packet = (char *)malloc(IP_MAXPACKET * sizeof(char))) == NULL) {
        fprintf(stderr, "malloc error\n");
        exit(1);
    }
    memset(packet, 0, IP_MAXPACKET * sizeof(char));
    if((if_name = (char *)malloc(20 * sizeof(char))) == NULL) {
        fprintf(stderr, "malloc error\n");
        exit(1);
    }
    memset(if_name, 0, 20 * sizeof(char));
```

```

if((src_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(src_ip, 0, INET_ADDRSTRLEN * sizeof(char));
if((tgt_ip = (char *)malloc(INET_ADDRSTRLEN * sizeof(char))) == NULL) {
    fprintf(stderr, "malloc error\n");
    exit(1);
}
memset(tgt_ip, 0, INET_ADDRSTRLEN * sizeof(char));
// 设定发送数据包的接口名
strcpy(if_name, "eth0");
// 建立一个原始套接口，用于获取指定接口的索引
if((sd = socket (AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0) {
    perror ("socket");
    exit (1);
}
// 用 ioctl()函数获取名为 if_name 接口的索引，用于将发送数据包的套接口与这个
接口索引绑定
memset(&ifr, 0, sizeof(ifr));
snprintf(ifr.ifr_name, sizeof(ifr.ifr_name), "%s", if_name);
if(ioctl(sd, SIOCGIFINDEX, &ifr) < 0) {
    perror ("ioctl");
    return (1);
}
close(sd);
printf ("Index for interface %s is %i\n", if_name, ifr.ifr_ifindex);
// 设定 TCP SYN 报文的源 IPv4 地址
strcpy(src_ip, "192.168.1.188");
// 设定 TCP SYN 报文的目的 IPv4 地址
strcpy(tgt_ip, "192.168.1.1");
// 发送数据包时，用户需要指定要发送数据包的目的 IPv4 地址
memset(&sin, 0, sizeof(struct sockaddr_in));
sin.sin_family = AF_INET;
if((status = inet_pton (AF_INET, tgt_ip, &(sin.sin_addr))) != 1) {
    fprintf(stderr, "inet_pton failed for destination IP address\n");
    exit (1);
}
// 建立一个原始套接口，用于发送数据包
if((sd = socket (AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0) {
    perror ("socket");
    exit (1);
}
// 使用套接口 IP_HDRINCL 选项设置套接口。

```

// 在之后发送数据时，内核将构造以太网帧头部数据并将其放到 IP 头部前，用户需要设置 IP 头部数据

```
if(setsockopt(sd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
    perror("setsockopt failed to set IP_HDRINCL");
    exit(1);
}
// 将套接口 sd 与指定接口的索引绑定
if(setsockopt(sd, SOL_SOCKET, SO_BINDTODEVICE, &ifr, sizeof(ifr)) < 0) {
    perror("setsockopt failed to bind to interface");
    exit(1);
}
// 构造 TCP 头部。TCP 头部见<netinet/tcp.h>
// TCP 中源端口号
tcp_hdr.th_sport = htons(9866);
printf("Source port in TCP header: %s\n", "9866");
// TCP 中目的端口号
tcp_hdr.th_dport = htons(80);
printf("Dest port in TCP header: %s\n", "80");
// TCP 中序列号
tcp_hdr.th_seq = htonl(0);
// TCP 中确认号
tcp_hdr.th_ack = htonl(0);
// TCP 中保留位（4 位）
tcp_hdr.th_x2 = 0;
// TCP 中 TCP 头部长度（32-bit words）
tcp_hdr.th_off = TCP_HDRLEN >> 2;
// TCP 中标志。TH_SYN=0x02
tcp_hdr.th_flags = TH_SYN;
// TCP 中窗口大小
tcp_hdr.th_win = htons(65535);
// TCP 中紧急指针
tcp_hdr.th_urp = htons(0);
// TCP 中 TCP 校验和。先设为 0，构造 IP 头后再计算它的校验和
tcp_hdr.th_sum = 0;
// 构造 IPv4 包头部。IPv4 包头见<netinet/ip.h>
// IPv4 包中版本号。它的值为 4 表示 IPv4
ip_hdr.ip_v = 4;
// IPv4 包中头部长度。它的值为 5
ip_hdr.ip_hl = IP4_HDRLEN >> 2;
// IPv4 包中服务类型
ip_hdr.ip_tos = 0;
// IPv4 包中总长度。包括 IPv4 头部长度和 TCP 头部长度
ip_hdr.ip_len = htons(IP4_HDRLEN + TCP_HDRLEN);
// IPv4 包中标识。
```

```

ip_hdr.ip_id = htons(getpid());
printf("Identification in IP header: %u\n", getpid());
// IPv4 包中标志和分段偏移。
// IPv4 包中保留分段标志位、不允许分段标志位、更多分段标志位和分段偏移。都设
为 0

```

```

ip_hdr.ip_off = 0;
// IPv4 包中 TTL。默认设为最大值 255
ip_hdr.ip_ttl = 255;
// IPv4 包中协议。它的值为 6 表示 TCP
ip_hdr.ip_p = IPPROTO_TCP;
// IPv4 包中源 IPv4 地址
if((status = inet_pton(AF_INET, src_ip, &(ip_hdr.ip_src))) != 1) {
    fprintf(stderr, "inet_pton failed for source IP address\n");
    exit(1);
}
printf("Source addresss in IP header: %s\n", src_ip);
// IPv4 包中目的 IPv4 地址
ip_hdr.ip_dst.s_addr = sin.sin_addr.s_addr;
printf("Dest addresss in IP header: %s\n", tgt_ip);
// 计算 IPv4 包中头部校验和
ip_hdr.ip_sum = checksum((uint16_t *) &ip_hdr, IP4_HDRLEN);
// 计算 TCP 中 TCP 校验和
tcp_hdr.th_sum = tcp4_checksum(ip_hdr, tcp_hdr);
// 将构造的 IPv4 头部复制到发送缓冲区中
memcpy(packet, &ip_hdr, IP4_HDRLEN * sizeof(uint8_t));
// 将构造的 TCP 头部复制到发送缓冲区中
memcpy((packet + IP4_HDRLEN), &tcp_hdr, TCP_HDRLEN * sizeof(uint8_t));
// 发送数据包
if(sendto(sd, packet, IP4_HDRLEN + TCP_HDRLEN, 0, (struct sockaddr *) &sin,
    sizeof(struct sockaddr)) < 0) {
    perror("sendto() failed ");
    exit(1);
}
close(sd);
free(packet);
free(if_name);
free(tgt_ip);
free(src_ip);
return(0);
}

```

// 计算网际校验和函数。被校验数据 16 位值的反码和，如果数据字节为奇数，则尾部补一个字节的 0（详见 RFC 1071）

```

uint16_t checksum(uint16_t *addr, int len) {
    register int nleft = len;

```

```

    register uint16_t *w = addr;
    register int sum = 0;
    uint16_t answer = 0;
    while(nleft > 1) {
        sum += *w++;
        nleft -= 2;
    }
    // 若字节总数是奇数，则增加一个字节的填充(全 0)
    if(nleft == 1) {
        *(u_char *)&answer = *(u_char *)w;
        sum += answer;
    }
    // 把高 16 位的进位加到低 16 位上
    sum = (sum >> 16) + (sum & 0xffff);
    // 再加进位
    sum += (sum >> 16);
    // 将和取反码，截取其低 16 位
    answer = ~sum;
    return(answer);
}

// 计算 TCP 校验和函数
uint16_t tcp4_checksum(struct ip ip_hdr, struct tcphdr tcp_hdr) {
    uint16_t svalue;
    char buf[IP_MAXPACKET], cvalue;
    char *ptr;
    int chksumlen = 0;
    ptr = &buf[0];
    // 将 IP 包中源 IPv4 地址复制到 buf
    memcpy(ptr, &ip_hdr.ip_src.s_addr, sizeof(ip_hdr.ip_src.s_addr));
    ptr += sizeof(ip_hdr.ip_src.s_addr);
    chksumlen += sizeof(ip_hdr.ip_src.s_addr);
    // 将 IP 包中目的 IPv4 地址复制到 buf
    memcpy(ptr, &ip_hdr.ip_dst.s_addr, sizeof(ip_hdr.ip_dst.s_addr));
    ptr += sizeof(ip_hdr.ip_dst.s_addr);
    chksumlen += sizeof(ip_hdr.ip_dst.s_addr);
    // 将 0 字段复制到 buf (8 位)
    *ptr = 0; ptr++;
    chksumlen += 1;
    // 将 IP 包中协议复制到 buf (8 位)
    memcpy(ptr, &ip_hdr.ip_p, sizeof(ip_hdr.ip_p));
    ptr += sizeof(ip_hdr.ip_p);
    chksumlen += sizeof(ip_hdr.ip_p);
    // 将 TCP 中长度复制到 buf (16 位)
    svalue = htons(sizeof(tcp_hdr));

```



```

memcpy(ptr, &svalue, sizeof(svalue));
ptr += sizeof(svalue);
chksumlen += sizeof(svalue);
// 将 TCP 中源端口号复制到 buf (16 位)
memcpy(ptr, &tcp_hdr.th_sport, sizeof(tcp_hdr.th_sport));
ptr += sizeof(tcp_hdr.th_sport);
chksumlen += sizeof(tcp_hdr.th_sport);
// 将 TCP 中目的端口号复制到 buf (16 位)
memcpy(ptr, &tcp_hdr.th_dport, sizeof(tcp_hdr.th_dport));
ptr += sizeof(tcp_hdr.th_dport);
chksumlen += sizeof(tcp_hdr.th_dport);
// 将 TCP 中序列号复制到 buf (32 位)
memcpy(ptr, &tcp_hdr.th_seq, sizeof(tcp_hdr.th_seq));
ptr += sizeof(tcp_hdr.th_seq);
chksumlen += sizeof(tcp_hdr.th_seq);
// 将 TCP 中确认号复制到 buf (32 位)
memcpy(ptr, &tcp_hdr.th_ack, sizeof(tcp_hdr.th_ack));
ptr += sizeof(tcp_hdr.th_ack);
chksumlen += sizeof(tcp_hdr.th_ack);
// 将 TCP 中头部长度 (4 位) 和保留位 (4 位) 复制到 buf
cvalue = (tcp_hdr.th_off << 4) + tcp_hdr.th_x2;
memcpy(ptr, &cvalue, sizeof(cvalue));
ptr += sizeof(cvalue);
chksumlen += sizeof(cvalue);
// 将 TCP 中标志复制到 buf (8 位)
memcpy(ptr, &tcp_hdr.th_flags, sizeof(tcp_hdr.th_flags));
ptr += sizeof(tcp_hdr.th_flags);
chksumlen += sizeof(tcp_hdr.th_flags);
// 将 TCP 中窗口大小复制到 buf (16 位)
memcpy(ptr, &tcp_hdr.th_win, sizeof(tcp_hdr.th_win));
ptr += sizeof(tcp_hdr.th_win);
chksumlen += sizeof(tcp_hdr.th_win);
// 将 TCP 中检验和复制到 buf (8 位)。先设为 0
*ptr = 0; ptr++;
*ptr = 0; ptr++;
chksumlen += 2;
// 将 TCP 紧急指针复制到 buf (8 位)
memcpy(ptr, &tcp_hdr.th_urp, sizeof(tcp_hdr.th_urp));
ptr += sizeof(tcp_hdr.th_urp);
chksumlen += sizeof(tcp_hdr.th_urp);
return checksum ((uint16_t *) buf, chksumlen);
}

```

A.7 UDP 客户/服务器程序参考答案

本程序分为 UDP 客户程序和 UDP 服务器程序。

UDP 客户程序代码如下：

```
// UDP 客户程序
```

```
#include <netinet/in.h>
```

```
#include <errno.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <sys/socket.h>
```

```
#define MAXLINE 4096
```

```
#define LISTENQ 1024
```

```
#define SERV_PORT 9877
```

```
#define SA struct sockaddr
```

```
void dg_cli(FILE *, int, const SA *, socklen_t);
```

```
int main(int argc, char **argv) {
```

```
    int sockfd;
```

```
    struct sockaddr_in servaddr;
```

```
    if (argc != 2) {
```

```
        printf("usage:udpcli01 sigio <IPaddress>\n");
```

```
        exit(1);
```

```
    }
```

```
    // 将服务器的 IP 地址和端口号填入 sockaddr_in 结构中
```

```
    bzero(&servaddr, sizeof(servaddr));
```

```
    servaddr.sin_family = AF_INET;
```

```
    servaddr.sin_port = htons(SERV_PORT);
```

```
    inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
```

```
    // 创建一个 UDP 套接口
```

```
    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0))<0) {
```

```
        printf("socket error.\n");
```

```
        exit(1);
```

```
    }
```

```
    // 调用 dg_cli 函数
```

```
    dg_cli(stdin, sockfd, (SA *)&servaddr, sizeof(servaddr));
```

```
    exit(0);
```

```
}
```

// 从标准输入读一行文本，写到 UDP 套接口中；从 UDP 套接口读回射行，并将其输出到标准输出

```
void dg_cli(FILE *fp, int sockfd, const SA *pservaddr, socklen_t servlen) {
```

```
    int n;
```

```
    char sendline[MAXLINE], recvline[MAXLINE + 1];
```

```
    while (fgets(sendline, MAXLINE, fp) != NULL) {
```

```
        // 将数据写入套接口
```

```
        sendto(sockfd, sendline, strlen(sendline), 0, pservaddr, servlen);
```

```

        // 从套接口中读取数据
        n = recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);
        recvline[n] = 0; /* null terminate */
        fputs(recvline, stdout);
    }
}

UDP 服务器程序代码如下：
#include <netinet/in.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#define MAXLINE 4096
#define LISTENQ 1024
#define SERV_PORT 9877
#define SA struct sockaddr
static int sockfd;
void dg_echo(int, SA *, socklen_t);
int main(int argc, char ** argv) {
    struct sockaddr_in servaddr, cliaddr;
    // 建立一个 UDP 套接口
    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0))<0) {
        printf("socket error.\n");
        exit(1);
    }
    // 将 INADDR_ANY 地址和端口号填入 sockaddr_in 结构中
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);
    // 绑定指定端口号
    bind(sockfd, (SA *)&servaddr, sizeof(servaddr));
    // 调用 dg_echo()函数
    dg_echo(sockfd, (SA *)&cliaddr, sizeof(cliaddr));
}
// 从 UDP 套接口中读取该行，并回射给客户
void dg_echo(int sockfd, SA *pcliaddr, socklen_t clen) {
    int n;
    socklen_t len;
    char mesg[MAXLINE];
    for ( ; ) {
        len = clen;
        // 从套接口中读数据
        n = recvfrom(sockfd, mesg, MAXLINE, 0, pcliaddr, &len);
    }
}

```

```

        // 将数据写入套接口
        sendto(sockfd, mesg, n, 0, pcliaddr, len);
    }
}

```

A.8 TCP 客户/服务器程序

本程序分为 TCP 客户程序程序和 TCP 服务器程序。

TCP 客户程序代码如下：

```

#include <netinet/in.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#define MAXLINE 4096
#define LISTENQ 1024
#define SERV_PORT 9877
#define SA struct sockaddr
void str_cli(FILE *,int);
ssize_t readline(int, void *, size_t);
static ssize_t my_read(int, char *);
int main(int argc, char **argv) {
    int sd;
    struct sockaddr_in servaddr;
    if (argc != 2) {
        printf("usage:tcpcli01 <IPaddress>");
        exit(1);
    }
    // 建立一个 TCP 套接口
    if ((sd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("socket error.\n");
        exit(1);
    }
    // 将服务器的 IP 地址和端口号填入 sockaddr_in 结构
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(SERV_PORT);
    inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
    // 调用 connect 建立与服务器连接
    if (connect(sd, (SA *)&servaddr, sizeof(servaddr)) < 0) {
        printf("connect error.\n");
        exit(1);
    }
    // 调用 str_cli 函数

```

```

    str_cli(stdin, sd);
    exit(0);
}
// 从标准输入读一行文本，写到 TCP 套接口中；从 TCP 套接口读回射行，并将其输出
到标准输出

```

```

void str_cli(FILE *fp, int sd) {
    char sendline[MAXLINE], recvline[MAXLINE];
    while(fgets(sendline, MAXLINE, fp) != NULL) {
        // 将数据写入 TCP 套接口
        write(sd, sendline, strlen(sendline));
        // 从套接口中读数据
        if(readline(sd, recvline, MAXLINE) == 0) {
            printf("str_cli: server terminated prematurely.\n");
            exit(1);
        }
        fputs(recvline, stdout);
    }
}

```

```

ssize_t readline(int fd, void *vptr, size_t maxlen) {
    int n, rc;
    char c, *ptr;
    ptr = vptr;
    for(n = 1; n < maxlen; n++) {
        if((rc = my_read(fd, &c)) == 1) {
            *ptr++ = c;
            if(c == '\n')
                break; /* newline is stored, like fgets() */
        } else if (rc == 0) {
            if (n == 1)
                return(0); /* EOF, no data read */
            else
                break; /* EOF, some data was read */
        } else
            return(-1); /* error, errno set by read() */
    }
    *ptr = 0; /* null terminate like fgets() */
    return(n);
}

```

```

static ssize_t my_read(int fd, char *ptr) {
    static int      read_cnt = 0;
    static char     *read_ptr;
    static char     read_buf[MAXLINE];
    if(read_cnt <= 0) {
again:

```

```

        if((read_cnt = read(fd, read_buf, sizeof(read_buf))) < 0) {
            if(errno == EINTR)
                goto again;
            return(-1);
        } else if (read_cnt == 0)
            return(0);
        read_ptr = read_buf;
    }
    read_cnt--;
    *ptr = *read_ptr++;
    return(1);
}

```

TCP 服务器程序代码如下：

```

#include <netinet/in.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#define MAXLINE 4096
#define LISTENQ 1024
#define SERV_PORT 9877
#define SA struct sockaddr
void str_echo(int);
ssize_t readline(int, void *, size_t);
static ssize_t my_read(int, char *);
int main(int argc, char **argv) {
    int  listenfd, connfd;
    pid_t  childpid;
    socklen_t  clilen;
    struct sockaddr_in  cliaddr, servaddr;
    // 建立一个 TCP 套接口
    if((listenfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("socket error.\n");
        exit(1);
    }
    // 将 INADDR_ANY 地址和端口号填入 sockaddr_in 结构中
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);
    // 绑定到指定的端口号
    if (bind(listenfd, (SA *)&servaddr, sizeof(servaddr)) < 0) {
        printf("bind error.\n");
        exit(1);
    }
}

```

```

    }
    // 建立监听套接口
    if (listen(listenfd,LISTENQ) < 0) {
        printf("listen error.\n");
        exit(1);
    }
    for(;;) {
        // 建立已连接套接口，并阻塞
        clilen = sizeof(cliaddr);
        if((connfd = accept(listenfd, (SA *) &cliaddr, &clilen)) < 0) {
            printf("accept error.\n");
            exit(1);
        }
        // 若是子进程，关闭监听套接口，并调用 str_echo()
        if((childpid = fork()) == 0) {
            close(listenfd);
            str_echo(connfd);
            exit(0);
        }
        close(connfd);
    }
}
//从套接口中读取该行，并回射给客户
void str_echo(int sockfd) {
    ssize_t n;
    char line[MAXLINE];
    for(;;) {
        // 从套接口中读数据
        if((n = readline(sockfd, line, MAXLINE)) == 0)
            return;
        // 将数据写入套接口
        write(sockfd, line, n);
    }
}

```

A.9 服务器接口流量监测程序参考答案

本程序由 PerformanceMonitor.java 和 FiveMinutesTask.java 组成。
PerformanceMonitor.java 的代码如下：

```

package com.snmp;
import java.util.*;
import java.io.*;
import java.text.SimpleDateFormat;
import com.adventnet.snmp.beans.*;

```

// 服务器接口流量监测类. 指定被测设备地址、团体名和 OID 等, 针对每个设备每 5 分钟执行一次 FiveMinutesTask 类线程, 对被管对象进行监测.

```
public class PerformanceMonitor {
    // 被监测设备号. 用来标示设备
    private int did[];
    // 被监测设备的 IPv4 地址
    private String[] ipAddr;
    // 被监测设备的团体名
    private String[] snmpPublickey;
    // 被监测设备的 OID
    private String[] oid;
    // 设定的阈值
    private double t_value[];
    public PerformanceMonitor() {
        did = new int[1];
        ipAddr = new String[1];
        snmpPublickey = new String[1];
        oid = new String[2]; //有几个 oid, 就几个值
        t_value = new double[2]; //有几个 oid, 就几个值
        // 第 1 台被检测设备的信息
        did[0] = 1;
        ipAddr[0] = new String("192.168.1.166");
        snmpPublickey[0] = new String("public");
        // 这台设备的第 1 个接口的输入的字节数 (ifInOctets)
        //oid[0] = new String(".1.3.6.1.2.1.2.2.1.10.1"); //lo: 一般不检测该接口
        // 这台设备的第 2 个接口的输入的字节数 (ifInOctets)
        oid[0] = new String(".1.3.6.1.2.1.2.2.1.10.2"); //eth0
        // 这台设备的第 2 个接口的输出的字节数 (ifOutOctets)
        oid[1] = new String(".1.3.6.1.2.1.2.2.1.16.2"); //eth0
        // 这台设备的第 1 个接口的输入的字节数的阈值
        t_value[0] = 0.2;
        // 这台设备的第 2 个接口的输入的字节数的阈值
        t_value[1] = 0.4;
        // 第 2 台被检测设备的信息
        //did[1] = 2;
        //ipAddr[1] = new String("192.168.1.188");
        //snmpPublickey[1] = new String("public");
        //...;
    }
    public int[] getDid() {
        return this.did;
    }
    public String[] getIpAddr() {
        return this.ipAddr;
    }
}
```



```

    }
    public String[] getSnmppublickey() {
        return this.snmppublickey;
    }
    public String[] getOid() {
        return this.oid;
    }
    public double[] getT_value() {
        return this.t_value;
    }
    public static void main(String[] args) {
        int did[];
        String[] ipAddr;
        String[] publickey;
        String[] oid;
        double t_value[];
        try {
            PerformanceMonitor dParameter = new PerformanceMonitor();
            did = dParameter.getDid();
            ipAddr = dParameter.getIpAddr();
            publickey=dParameter.getSnmppublickey();
            oid = dParameter.getOid();
            t_value = dParameter.getT_value();
            for(int i=0;i<did.length;i++) {
                // Thread. get target: eth0 接口的 ifInOctets 和 ifOutOctets
                Timer d5minTimer=new Timer();
                // 300,000 毫秒=5 分钟, IP 地址为 192.168.1.1, 团体名为 public,
                // OID 为 eth0 接口的 ifInOctets 和 ifOutOctets
                d5minTimer.scheduleAtFixedRate(new FiveMinutesTask(did[i], ipAddr[i], oid,
                                                                    publickey[i], t_value), 0, 300000);
            }
        } catch(Exception e) {
            System.out.println("There are no devices in the device table " + e);
        }
    }
}

```

FiveMinutesTask.java 的代码如下:

// 每五分钟执行一次, 执行时读取指定设备的接口的输入/输出流量, 并计算接口输入/输出利用率.

```

class FiveMinutesTask extends TimerTask {
    private int did;
    private String ipAddr;
    private String[] oid;
    private String publickey;
}

```

```

//阈值
private double t_value[];
//上次读取的值
private int o_value[];
//本次读取的值
private int n_value[];
private SnmpTarget target;
private String[] result;
private boolean isFirst = true;
protected FiveMinutesTask(int did, String ipAddr, String[] oid, String publicKey,
                           double[] t_value) {
    this.ipAddr=ipAddr;
    this.oid= oid;
    this.publickey=publicKey;
    this.did=did;
    this.t_value = t_value;
    n_value =new int[2];
    o_value =new int[2];
    // 建立一个 SnmpTarget 对象
    target= new SnmpTarget();
    // 设置主机地址
    target.setTargetHost(ipAddr);
    // 设置团体名
    target.setCommunity(publicKey);
    // 设置 OID
    target.setObjectIDList(oid);
    //target.setRetries(3);
}
public void run() {
    System.out.println("Every 5 minute task start!");
    // 执行 SNMP get 操作
    result = target.snmpGetList();
    if((result == null) ||result.length==0) {
        System.out.println("result null");
        isFirst = true;
    } else {
        String currtime=new SimpleDateFormat("yyyy-MM-dd HH:mm:ss")
                                .format(Calendar.getInstance().getTime());

        for(int i=0;i<n_value.length;i++) {
            n_value[i] = Integer.parseInt(result[i]);
            //System.out.println("n_value: " + n_value[i]);
        }
        // 如果 5 分钟前取到数据，则计算接口输入/输出利用率，并判断是否超过阈值
        if(!isFirst) {

```

```

int ifspeed = 100000000;
// 计算 5 分钟接口输入的字节数
System.out.println("ifInOctets: n_value = " + n_value[0] +
    " o_value = " + o_value[0]);
int diff = n_value[0]-o_value[0];
// 接口输入利用率 = (ifInOctets * 8) / ((秒数)* IfSpeed)
double utilization = (diff * 8.0) / (300.0 * ifspeed);
// 如果接口输入利用率大于指定的阈值, 则输出告警信息
if(utilization > t_value[0])
    System.out.println("Alarm: " + currtime +
        ": Utilization of the input interface = " + utilization +
        ">" + String.valueOf(t_value[0]) + ").");
// 计算 5 分钟接口输出的字节数
System.out.println("ifOutOctets: n_value = " + n_value[1] +
    " o_value = " + o_value[1]);
diff = n_value[1]-o_value[1];
// 接口输出利用率 = (ifOutOctets * 8) / ((秒数)* IfSpeed)
utilization = (diff * 8.0) / (300.0 * ifspeed);
// 如果接口输出利用率大于指定的阈值, 则输出告警信息
if(utilization > t_value[1])
    System.out.println("Alarm: " + currtime +
        ": Utilization of the Output interface = " + utilization +
        ">" + String.valueOf(t_value[1]) + ").");
    }
}
// 保存刚读取的数据
for (int i=0;i<n_value.length;i++) {
    o_value[i] = n_value[i];
    isFirst = false;
}
}
}

```

A.10 邮件客户端程序参考答案

所附代码为开源软件 JackMail 的修改版本, 对该程序的使用和发放应该遵守相关规定。

本程序主要由 MailClient.java、MailListPanel.java、NewMailPanel.java、MyCaret.java、MyScrollBarUI.java、MyScrollPane.java、MyMail.java、MailConstant.java 和 util.java 文件组成。

MailClient.java 的代码如下:

```

package com.mail.client;
import java.awt.*;
import java.awt.event.*;
import java.io.File;

```

```

import javax.swing.*;
import javax.swing.border.Border;
import com.mail.MyMail;
import com.mail.util.MailConstant;
// 邮件客户端类.
public class MailClient extends JFrame {
    private BorderLayout layoutMain = new BorderLayout();
    private JMenuBar menuBar = new JMenuBar();
    private JMenu menuFile = new JMenu();
    private JMenuItem menuFileExit = new JMenuItem();
    private JMenu menuHelp = new JMenu();
    private JLabel statusBar = new JLabel();
    private JToolBar toolBar = new JToolBar();
    private JButton buttonchknewmail = new JButton();
    private JButton buttonCreateMail = new JButton();
    private JButton buttonSendMail = new JButton();
    private JButton buttonatt = new JButton();
    private ImageIcon imagechknewmail = new ImageIcon(MailClient.class.getResource(
                                                                    "checknewmail.gif"));
    private ImageIcon imagesendmail = new ImageIcon(MailClient.class.getResource(
                                                                    "send_icon.gif"));
    private ImageIcon imagewritemail = new ImageIcon(MailClient.class.getResource(
                                                                    "writemail.gif"));
    private ImageIcon imageatt = new ImageIcon(MailClient.class.getResource(
                                                                    "attach_icon.gif"));

    public static Color BACKGROUND_COLOR = null;
    public static Color FOREGROUND_COLOR = null;
    static {
        int br = Integer.parseInt(MailConstant.BACK_COLOR_R);
        int bg = Integer.parseInt(MailConstant.BACK_COLOR_G);
        int bb = Integer.parseInt(MailConstant.BACK_COLOR_B);
        int fr = Integer.parseInt(MailConstant.FORE_COLOR_R);
        int fg = Integer.parseInt(MailConstant.FORE_COLOR_G);
        int fb = Integer.parseInt(MailConstant.FORE_COLOR_B);
        BACKGROUND_COLOR = new Color(br, bg, bb);
        FOREGROUND_COLOR = new Color(fr, fg, fb);
    }
    private MyMail myMail = null;
    private MailListPanel mailListPanel = null;
    private File attachedFile = null;
    private NewMailPanel newMailPanel = null;
    public static Border LINE_BORDER = BorderFactory.createLineBorder(
                                                                    MailClient.FOREGROUND_COLOR, 2);

    public NewMailPanel getNewMailPanel() {

```

```

        return newMailPanel;
    }
    public void setNewMailPanel(NewMailPanel newMailPanel) {
        this.newMailPanel = newMailPanel;
    }
    public MyMail getMyMail() {
        return myMail;
    }
    public void setMyMail(MyMail myMail) {
        this.myMail = myMail;
    }
    public MailListPanel getMailListPanel() {
        return mailListPanel;
    }
    public void setMailListPanel(MailListPanel mailListPanel) {
        this.mailListPanel = mailListPanel;
    }
    // MailClient 类的构造方法.
    public MailClient() {
        try {
            jbInit();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    // 设置邮件客户端窗口，接收邮件并将其显示在邮件列表中.
    private void jbInit() throws Exception {
        this.setJMenuBar(menuBar);
        this.getContentPane().setLayout(layoutMain);
        this.getContentPane().setBackground(BACKGROUND_COLOR);
        this.setForeground(FOREGROUND_COLOR);
        this.setBackground(BACKGROUND_COLOR);
        this.setSize(new Dimension(400, 300));
        this.setTitle("Mail Client");
        menuFile.setText("File");
        menuFile.setForeground(FOREGROUND_COLOR);
        menuFile.setBackground(BACKGROUND_COLOR);
        menuFileExit.setText("Exit");
        menuFileExit.setForeground(FOREGROUND_COLOR);
        menuFileExit.setBackground(BACKGROUND_COLOR);
        menuFileExit.addActionListener(new ActionListener() { public void actionPerformed(
            ActionEvent ae) { fileExit_ActionPerformed(ae); } });
        menuHelp.setText("Help");
        menuHelp.setForeground(FOREGROUND_COLOR);
    }

```

```

menuHelp.setBackground(BACKGROUND_COLOR);
menuFile.add(menuFileExit);
menuBar.add(menuFile);
menuBar.add(menuHelp);
menuBar.setBackground(BACKGROUND_COLOR);
statusBar.setText("");
statusBar.setForeground(FOREGROUND_COLOR);
statusBar.setBackground(BACKGROUND_COLOR);
this.getContentPane().add(statusBar, BorderLayout.SOUTH);
buttonchknewmail.setToolTipText("Check new arrivals..");
buttonchknewmail.setIcon(imagechknewmail);
buttonchknewmail.setBackground(BACKGROUND_COLOR);
buttonchknewmail.addActionListener(new ActionListener() { public void
    actionPerformed(ActionEvent ae) { buttonchknewmail_ActionPerformed(ae); } });
buttonSendMail.setToolTipText("Send Mail");
buttonSendMail.setIcon(imagesendmail);
buttonSendMail.setBackground(BACKGROUND_COLOR);
buttonSendMail.addActionListener(new ActionListener() { public void
    actionPerformed(ActionEvent ae) { buttonSendMail_ActionPerformed(ae); } });
buttonatt.setToolTipText("Attached File");
buttonatt.setIcon(imageatt);
buttonatt.addActionListener(new ActionListener() { public void
    actionPerformed(ActionEvent ae) { buttonAtt_ActionPerformed(ae); } });
buttonCreateMail.setToolTipText("Create New Mail");
buttonCreateMail.setIcon(imagewritemail);
buttonCreateMail.setBackground(BACKGROUND_COLOR);
buttonCreateMail.addActionListener(new ActionListener() { public void
    actionPerformed(ActionEvent ae) { buttonCreateMail_ActionPerformed(ae); } });
toolBar.add(buttonchknewmail);
toolBar.add(buttonSendMail);
toolBar.add(buttonCreateMail);
toolBar.add(buttonatt);
toolBar.setBackground(BACKGROUND_COLOR);
this.getContentPane().add(toolBar, BorderLayout.NORTH);
// 建立一个写邮件对象
newMailPanel = new NewMailPanel(this);
// 建立一个收发邮件对象
MyMail jm = new MyMail(this);
setMyMail(jm);
// 从邮件服务器接收邮件，并将邮件存放到成员变量 message 中
jm.jackProcess();
// 建立一个邮件列表对象
MailListPanel ml = new MailListPanel(this, jm.getMessage());
setMailListPanel(ml);

```

```

// 将邮件列表对象加入邮件客户端对象中
getContentPane().add(ml);
int width = Toolkit.getDefaultToolkit().getScreenSize().width;
int height = Toolkit.getDefaultToolkit().getScreenSize().height;
setSize((int)(width*0.9), (int)(height/2));
setLocation(width/16, height/2);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// 禁用附件按钮和发送按钮
buttonatt.setEnabled(false);
buttonSendMail.setEnabled(false);
// 焦点在邮件列表时
addWindowFocusListener(new WindowAdapter() {
    public void windowGainedFocus(WindowEvent e) {
        getMailListPanel().getEditor().requestFocusInWindow();
    }
});
setVisible(true);
}
// 点击 Exit 按钮时退出程序.
void fileExit_ActionPerformed(ActionEvent e) {
    System.exit(0);
}
// 点击接收收件按钮时接收收件.
void buttonchknewmail_ActionPerformed(ActionEvent e) {
    try {
        setCursor(new Cursor(Cursor.WAIT_CURSOR));
        getMyMail().jackProcess();
        mailListPanel.reloadTableData(getMyMail().getMessage());
        mailListPanel.getSplitpane().add(mailListPanel.getScrollPane(), JSplitPane.TOP);
        mailListPanel.getSplitpane().setDividerLocation(
            mailListPanel.getSplitpane().getHeight()/2);
        setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
    } catch (Exception exception) {
    }
    buttonatt.setEnabled(false);
    buttonSendMail.setEnabled(false);
}
// 点击写邮件按钮时写邮件
void buttonCreateMail_ActionPerformed(ActionEvent e) {
    setCursor(new Cursor(Cursor.WAIT_CURSOR));
    buttonatt.setEnabled(true);
    buttonSendMail.setEnabled(true);
    MyScrollPane jsp = new MyScrollPane(new MailPanel,
        MyScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,

```

```

        MyScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);
mailListPanel.getSplitpane().add(jsp, JSplitPane.TOP);
mailListPanel.getSplitpane().setDividerLocation((
        mailListPanel.getSplitpane().getHeight()/2) - 50);
setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
}
// 点击发送按钮时发送邮件.
void buttonSendMail_ActionPerformed(ActionEvent e) {
    setCursor(new Cursor(Cursor.WAIT_CURSOR));
    String sender_name = MailConstant.MAIL_SERVER_USER;
    String email_from = MailConstant.MAIL_SERVER_USER;
    String email_to = getNewMailPanel().getTxt_A_To().getText();
    String email_cc = getNewMailPanel().getTxt_B_Cc().getText();
    String email_bcc = getNewMailPanel().getTxt_C_Bcc().getText();
    String subject = getNewMailPanel().getTxt_D_Sub().getText();
    String body = getMailListPanel().getEditor().getText();
    String msg = "Not able to send your mail. Please recheck all fields and retry.";
    boolean flag = false;
    if (email_to != null && !email_to.equals("")) {
        flag = getMyMail().sendMailWithAtt(sender_name, email_from, email_to, email_cc,
            email_bcc, subject, body, attachedFile);

        if(flag)
            msg = "Your message has been sent successfully.";
        JOptionPane.showMessageDialog(this, msg);
        mailListPanel.getSplitpane().add(mailListPanel.getScrollPane(), JSplitPane.TOP);
        mailListPanel.getSplitpane().setDividerLocation(
            mailListPanel.getSplitpane().getHeight()/2);
    } else {
        JOptionPane.showMessageDialog(this, msg);
    }
    buttonatt.setEnabled(false);
    buttonSendMail.setEnabled(false);
    setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
}
// 点击附件按钮时添加附件
void buttonAtt_ActionPerformed(ActionEvent e) {
    JFileChooser chooser = new JFileChooser();
    chooser.setCurrentDirectory(new java.io.File(System.getProperty("user.home")));
    chooser.setDialogTitle("Select the File for attachment");
    chooser.setFileSelectionMode(JFileChooser.FILES_ONLY);
    if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        attachedFile = chooser.getSelectedFile();
        getNewMailPanel().getJLblAttachment().setText(
            getNewMailPanel().getJLblAttachment().getText() +

```



```

        attachedFile.getName() + "    [Double click on file name to remove]");
    }
}
// 邮件客户端主方法
public static void main(String args[]) throws Exception {
    MailClient mc = new MailClient();
}
public File getAttachedFile() {
    return attachedFile;
}
public void setAttachedFile(File attachedFile) {
    this.attachedFile = attachedFile;
}
public JLabel getStatusBar() {
    return statusBar;
}
public void setStatusBar(JLabel statusBar) {
    this.statusBar = statusBar;
}
}
}

```

MailListPanel.java 的代码如下:

```

package com.mail.client;
import java.awt.*;
import java.awt.event.*;
import java.math.BigDecimal;
import javax.mail.*;
import javax.swing.*;
import javax.swing.JEditorPane;
import javax.swing.table.*;
import com.mail.util.Util;
// 客户端邮件列表类
public class MailListPanel extends JPanel implements MouseListener {
    private JTable table;
    private MyScrollPane scrollPane;
    private MyScrollPane scrollPane2;
    private JEditorPane editor;
    private JSplitPane splitpane = null;
    //邮件客户端的邮件列表中列标题
    private String columnNames[];
    //新邮件数组. 新邮件包括的 ID, 发件人, 主题, 接收时间, 大小等信息
    private String dataValues[][];
    //指定邮箱的所有邮件
    private Message[] messages;
    MailClient mailClient;
}

```

```

private DefaultTableModel defaultTableModel = null;
public DefaultTableModel getDefaultTableModel() {
    return defaultTableModel;
}
public void setDefaultTableModel(DefaultTableModel defaultTableModel) {
    this.defaultTableModel = defaultTableModel;
}
// MailListPanel 类的构造方法
public MailListPanel(MailClient mailClient, Message[] messages) {
    try {
        this.mailClient = mailClient;
        this.messages = messages;
        jbInit();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
// 将接收的新邮件加载邮件列表窗口，并建立空的阅读窗口
private void jbInit() throws Exception {
    setSize(300, 200);
    setBackground(MailClient.BACKGROUND_COLOR);
    setForeground(MailClient.FOREGROUND_COLOR);
    setLayout(new BorderLayout());
    // 邮件客户端重新加载邮件列表
    reloadTableData(this.messages);
    // 邮件客户端的阅读窗口 editor
    editor = new JEditorPane("test/html", "");
    scrollPane2 = new MyScrollPane(editor,
                                   MyScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
                                   MyScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
    splitpane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, scrollPane, scrollPane2);
    add(splitpane, BorderLayout.CENTER);
    splitpane.setDividerSize(5);
    splitpane.setDividerLocation(125);
    splitpane.setBackground(MailClient.BACKGROUND_COLOR);
    splitpane.setBackground(MailClient.FOREGROUND_COLOR);
    editor.setBackground(MailClient.BACKGROUND_COLOR);
    editor.setForeground(MailClient.FOREGROUND_COLOR);
    editor.setCursor(mailClient.getCursor());
    editor.setCaret(new MyCaret());
    editor.setName("ZZZ");
}
// 建立邮件客户端的邮件列表中列标题。列标题包括 ID, Att, From, Subject, Recieved
Date, Size[MB]

```

```

public void CreateColumns() {
    columnNames = new String[6];
    columnNames[0] = "ID";
    columnNames[1] = "Att";
    columnNames[2] = "From";
    columnNames[3] = "Subject";
    columnNames[4] = "Recieved Date";
    columnNames[5] = "Size[MB]";
}
// 解析所有新邮件的 ID, 发件人, 主题, 接收时间, 大小等信息
public synchronized void CreateData() {
    // Create data for each element
    dataValues = new String[messages.length][6];
    for (int i = 0; i < messages.length; i++) {
        try {
            // 如果是新邮件, 则解析邮件的 ID, 发件人, 主题, 接收时间, 大小等信息
            if (!messages[i].isSet(Flags.Flag.SEEN)) {
                dataValues[i][0] = "" + messages[i].getMessageNumber();
                dataValues[i][1] = "";
                dataValues[i][2] = messages[i].getFrom()[0].toString();
                dataValues[i][3] = messages[i].getSubject();
                dataValues[i][4] = messages[i].getSentDate() + "";
                BigDecimal mb = new BigDecimal(messages[i].getSize());
                BigDecimal divisor = new BigDecimal(1028 * 1028);
                mb = mb.setScale(2, BigDecimal.ROUND_HALF_UP);
                dataValues[i][5] = "" + mb.divide(divisor, BigDecimal.ROUND_UP);
            }
        } catch (MessagingException e) {
            e.printStackTrace();
        }
    }
}
// 在客户端邮件列表上单击某封邮件时, 调用解析指定邮件的邮件头和邮件内容的方法
public void mouseClicked(MouseEvent e) {
    if (e.getClickCount() > 1) {
        final int row = table.rowAtPoint(e.getPoint());
        int col = table.columnAtPoint(e.getPoint());
        try {
            Thread.sleep(500);
        } catch (InterruptedException e1) {
            e1.printStackTrace();
        }
        SwingUtilities.invokeLater(new Runnable() {

```

```

        public void run() {
            mailClient.getMyMail().readMessage(editor, row);
        }
    });
    return;
}
}

public String[][] getDataValues() {
    return dataValues;
}

public void setDataValues(String[][] dataValues) {
    this.dataValues = dataValues;
}

public JEditorPane getEditor() {
    return editor;
}

public void setEditor(JEditorPane editor) {
    this.editor = editor;
}

public Message[] getMessages() {
    return messages;
}

public void setMessages(Message[] messages) {
    this.messages = messages;
}

public JTable getTable() {
    return table;
}

public void setTable(JTable table) {
    this.table = table;
}

public String[] getColumnNames() {
    return columnNames;
}

public void setColumnNames(String[] columnNames) {
    this.columnNames = columnNames;
}

// 邮件客户端重新加载邮件列表
public void reloadTableData(Message[] messages) {
    this.messages = messages;
    CreateColumns();
    if (messages != null && messages.length > 0)
        synchronized( messages ) {
            CreateData();
        }
    }
}

```

```

    }
    // 构造一个 DefaultTableModel，并通过将 dataValues 和 columnNames 传递到
setDataVector 方法来初始化该表
    defaultTableModel = new DefaultTableModel(dataValues, columnNames) {
        public boolean isCellEditable(int row, int col) {
            return false;
        }
    };
    table = new JTable(defaultTableModel);
    table.setBackground(MailClient.BACKGROUND_COLOR);
    // 设置邮件客户端邮件列表各列的列宽
    TableColumn col1 = table.getColumnModel().getColumn(0);
    col1.setMaxWidth(35);
    TableColumn col2 = table.getColumnModel().getColumn(1);
    col2.setMaxWidth(35);
    TableColumn col3 = table.getColumnModel().getColumn(2);
    col3.setPreferredWidth(200);
    TableColumn col4 = table.getColumnModel().getColumn(3);
    col4.setPreferredWidth(200);
    TableColumn col5 = table.getColumnModel().getColumn(4);
    col5.setPreferredWidth(100);
    TableColumn col6 = table.getColumnModel().getColumn(5);
    col6.setMaxWidth(100);
    table.getTableHeader().setFont(new Font("Verdana", Font.BOLD, 13));
    table.setFont(new Font("Aerial", Font.BOLD, 13));
    table.setShowVerticalLines(false);
    table.setSelectionBackground(MailClient.FOREGROUND_COLOR);
    table.setSelectionForeground(MailClient.BACKGROUND_COLOR);
    // 监听鼠标动作
    table.addMouseListener(this);
    table.getTableHeader().setBackground(MailClient.BACKGROUND_COLOR);
    table.setBackground(MailClient.BACKGROUND_COLOR);
    table.setForeground(MailClient.FOREGROUND_COLOR);
    table.getTableHeader().setForeground(MailClient.FOREGROUND_COLOR);
    col6.setCellRenderer(new DefaultTableCellRenderer() {
        public Component getTableCellRendererComponent(JTable table, Object value,
            boolean isSelected, boolean hasFocus, int row, int column) {
            JLabel ret = (JLabel) super.getTableCellRendererComponent(table, value,
                isSelected, hasFocus, row, column);
            ret.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
            return ret;
        }
    });
    scrollPane = new MyScrollPane(table,

```

```

        MyScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
        MyScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);
    }
    public JSplitPane getSplitpane() {
        return splitpane;
    }
    public void setSplitpane(JSplitPane splitpane) {
        this.splitpane = splitpane;
    }
    public MyScrollPane getScrollPane() {
        return scrollPane;
    }
    public void setScrollPane(MyScrollPane scrollPane) {
        this.scrollPane = scrollPane;
    }
    public MyScrollPane getScrollPane2() {
        return scrollPane2;
    }
    public void setScrollPane2(MyScrollPane scrollPane2) {
        this.scrollPane2 = scrollPane2;
    }
}

```

NewMailPanel.java 的代码如下:

```

package com.mail.client;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.EtchedBorder;
import com.mail.util.Util;
// 写邮件窗口类
public class NewMailPanel extends JPanel {
    private JTextArea txt_A_To = new JTextArea();//createArea("", true, 0, null);
    private JTextArea txt_B_Cc = new JTextArea();
    private JTextArea txt_C_Bcc = new JTextArea();
    private JTextArea txt_D_Sub = new JTextArea();
    private JLabel jLabel1 = new JLabel();
    private JLabel jLabel2 = new JLabel();
    private JLabel jLabel3 = new JLabel();
    private JLabel jLabel4 = new JLabel();
    private JLabel jLabel5 = new JLabel();
    private JLabel jLabelAttachment = new JLabel();
    private GridBagLayout gridBagLayout1 = new GridBagLayout();
    private MailClient mailClient = null;
    // NewMailPanel 类的构造方法

```

```

public NewMailPanel(MailClient mailClient) {
    this.mailClient = mailClient;
    try {
        jbInit();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
// 写邮件窗口
private void jbInit() throws Exception {
    this.setLayout(gridBagLayout1);
    this.setSize(new Dimension(990, 108));
    this.setBackground(MailClient.BACKGROUND_COLOR);
    txt_A_To.setBackground(MailClient.BACKGROUND_COLOR);
    txt_A_To.setForeground(MailClient.FOREGROUND_COLOR);
    txt_A_To.setBorder(MailClient.LINE_BORDER);
    txt_A_To.setCursor(mailClient.getCursor());
    txt_A_To.setCaret(new MyCaret());
    txt_A_To.setName("A");
    txt_B_Cc.setBackground(MailClient.BACKGROUND_COLOR);
    txt_B_Cc.setForeground(MailClient.FOREGROUND_COLOR);
    txt_B_Cc.setBorder(MailClient.LINE_BORDER);
    txt_B_Cc.setCursor(mailClient.getCursor());
    txt_B_Cc.setCaret(new MyCaret());
    txt_B_Cc.setName("B");
    txt_C_Bcc.setBackground(MailClient.BACKGROUND_COLOR);
    txt_C_Bcc.setForeground(MailClient.FOREGROUND_COLOR);
    txt_C_Bcc.setBorder(MailClient.LINE_BORDER);
    txt_C_Bcc.setCursor(mailClient.getCursor());
    txt_C_Bcc.setCaret(new MyCaret());
    txt_C_Bcc.setName("C");
    txt_D_Sub.setBackground(MailClient.BACKGROUND_COLOR);
    txt_D_Sub.setForeground(MailClient.FOREGROUND_COLOR);
    txt_D_Sub.setBorder(MailClient.LINE_BORDER);
    txt_D_Sub.setCursor(mailClient.getCursor());
    txt_D_Sub.setCaret(new MyCaret());
    txt_D_Sub.setName("D");
    jLabel1.setText("To...");
    jLabel1.setFont(new Font("Arial", 1, 11));
    jLabel1.setForeground(MailClient.FOREGROUND_COLOR);
    jLabel2.setText("Cc...");
    jLabel2.setFont(new Font("Arial", 1, 11));
    jLabel2.setForeground(MailClient.FOREGROUND_COLOR);
    jLabel3.setText("Bcc...");

```

```

jLabel3.setFont(new Font("Arial", 1, 11));
jLabel3.setForeground(MailClient.FOREGROUND_COLOR);
jLabel4.setText("Subject:");
jLabel4.setFont(new Font("Arial", 1, 11));
jLabel4.setForeground(MailClient.FOREGROUND_COLOR);
jLblAttachment.setText("");
jLblAttachment.setFont(new Font("Arial", 1, 11));
jLblAttachment.setForeground(MailClient.FOREGROUND_COLOR);
jLblAttachment.setToolTipText("Double click to remove the file attached");
jLblAttachment.addMouseListener(new JackMouseAdapter());
jLabel5.setText("Attachment:");
jLabel5.setFont(new Font("Arial", 1, 11));
jLabel5.setForeground(MailClient.FOREGROUND_COLOR);
this.add(jLabel4, new GridBagConstraints(0, 6, 1, 1, 0.0, 0.0,
    GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(0, 5, 0, 0), 21, 1));
this.add(jLabel3, new GridBagConstraints(0, 4, 1, 1, 0.0, 0.0,
    GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(0, 5, 0, 0), 37, 1));
this.add(jLabel2, new GridBagConstraints(0, 2, 1, 1, 0.0, 0.0,
    GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(0, 5, 0, 0), 42, 1));
this.add(jLabel1, new GridBagConstraints(0, 0, 1, 1, 0.0, 0.0,
    GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(5, 5, 0, 0), 42, 1));
this.add(jLabel5, new GridBagConstraints(0, 8, 1, 1, 0.0, 0.0,
    GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(0, 5, 0, 0), 12, 1));
this.add(txt_D_Sub, new GridBagConstraints(1, 6, 1, 2, 1.0, 1.0,
    GridBagConstraints.CENTER, GridBagConstraints.BOTH,
        new Insets(0, 10, 8, 10), 750, 0));
this.add(txt_C_Bcc, new GridBagConstraints(1, 4, 1, 2, 1.0, 1.0,
    GridBagConstraints.CENTER, GridBagConstraints.BOTH,
        new Insets(0, 10, 0, 10), 750, 0));
this.add(txt_B_Cc, new GridBagConstraints(1, 2, 1, 2, 1.0, 1.0,
    GridBagConstraints.CENTER, GridBagConstraints.BOTH,
        new Insets(0, 10, 0, 10), 750, 0));
this.add(txt_A_To, new GridBagConstraints(1, 0, 1, 2, 1.0, 1.0,
    GridBagConstraints.CENTER, GridBagConstraints.BOTH,
        new Insets(5, 10, 0, 10), 750, 0));
this.add(jLblAttachment, new GridBagConstraints(1, 8, 1, 2, 1.0, 1.0,
    GridBagConstraints.CENTER, GridBagConstraints.BOTH,
        new Insets(0, 10, 8, 10), 750, 0));
txt_D_Sub.setFocusTraversalKeys(
    KeyboardFocusManager.FORWARD_TRAVERSAL_KEYS, null);
txt_D_Sub.setFocusTraversalKeys(
    KeyboardFocusManager.BACKWARD_TRAVERSAL_KEYS, null);
txt_A_To.setFocusTraversalKeys(
    KeyboardFocusManager.FORWARD_TRAVERSAL_KEYS, null);

```



```

txt_A_To.setFocusTraversalKeys(
    KeyboardFocusManager.BACKWARD_TRAVERSAL_KEYS, null);
txt_B_Cc.setFocusTraversalKeys(
    KeyboardFocusManager.FORWARD_TRAVERSAL_KEYS, null);
txt_B_Cc.setFocusTraversalKeys(
    KeyboardFocusManager.BACKWARD_TRAVERSAL_KEYS, null);
txt_C_Bcc.setFocusTraversalKeys(
    KeyboardFocusManager.FORWARD_TRAVERSAL_KEYS, null);
txt_C_Bcc.setFocusTraversalKeys(
    KeyboardFocusManager.BACKWARD_TRAVERSAL_KEYS, null);
mailClient.pack();
mailClient.pack();
}
public JTextArea getTxt_C_Bcc() {
    return txt_C_Bcc;
}
public void setTxt_C_Bcc(JTextArea txtBcc) {
    this.txt_C_Bcc = txtBcc;
}
public JTextArea getTxt_B_Cc() {
    return txt_B_Cc;
}
public void setTxt_B_Cc(JTextArea txtCc) {
    this.txt_B_Cc = txtCc;
}
public JTextArea getTxt_D_Sub() {
    return txt_D_Sub;
}
public void setTxt_D_Sub(JTextArea txtSub) {
    this.txt_D_Sub = txtSub;
}
public JTextArea getTxt_A_To() {
    return txt_A_To;
}
public void setTxt_A_To(JTextArea txtTo) {
    this.txt_A_To = txtTo;
}
public JLabel getJLblAttachment() {
    return jLblAttachment;
}
public void setJLblAttachment(JLabel lblAttachment) {
    jLblAttachment = lblAttachment;
}
}
// 双击附件删除附件

```

```

class JackMouseAdapter extends MouseAdapter {
    public void mouseClicked(MouseEvent me) {
        if (me.getClickCount() > 1) {
            jLblAttachment.setText("");
            mailClient.setAttachedFile(null);
        }
    }
}
}
}
}

MyCaret.java 的代码如下:
package com.mail.client;
import java.awt.*;
import javax.swing.Timer;
import javax.swing.plaf.TextUI;
import javax.swing.text.*;
// 插入符的实现类
public class MyCaret extends DefaultCaret {
    // MyCaret 类的构造方法
    public MyCaret() {
        super();
        this.setBlinkRate(500);
    }
    public void paint(Graphics g) {
        if (isVisible()) {
            JTextComponent component = this.getComponent();
            TextUI textUI = component.getUI();
            Rectangle rect = null;
            try {
                rect = textUI.modelToView(component, this.getDot());
            } catch (BadLocationException exc) {
            }
            // 画水平线
            int x = rect.x;
            int y = rect.y + rect.height - 4;
            g.drawLine(x, y, x + 5, y);
            g.drawLine(x, y - 1, x + 5, y - 1);
        }
    }
}
}
}
}

```

MyScrollBarUI.java 的代码如下:

```

package com.mail.client;
import java.awt.*;
import javax.swing.plaf.basic.BasicScrollBarUI;
public class MyScrollBarUI extends BasicScrollBarUI {

```

```

protected void paintThumb(Graphics g, JComponent c, Rectangle thumbBounds) {
    g.setColor(MailClient.BACKGROUND_COLOR);
    g.fillRect((int) thumbBounds.getX(), (int) thumbBounds.getY(),
        (int) thumbBounds.getWidth(), (int) thumbBounds.getHeight());
}

protected void paintTrack(Graphics g, JComponent c, Rectangle trackBounds) {
    g.setColor(MailClient.FOREGROUND_COLOR);
    g.fillRect((int) trackBounds.getX(), (int) trackBounds.getY(),
        (int) trackBounds.getWidth(), (int) trackBounds.getHeight());
}

protected JButton createDecreaseButton(int orientation) {
    JButton button = super.createDecreaseButton(orientation);
    button.setBackground(MailClient.BACKGROUND_COLOR);
    button.setBorder(BorderFactory.createLineBorder(
        MailClient.FOREGROUND_COLOR, 1));
    return button;
}

protected JButton createIncreaseButton(int orientation) {
    JButton button = super.createIncreaseButton(orientation);
    button.setBackground(MailClient.BACKGROUND_COLOR);
    button.setBorder(BorderFactory.createLineBorder(
        MailClient.FOREGROUND_COLOR, 1));
    return button;
}
}

```

MyScrollPane.java 的代码如下：

```

package com.mail.client;
import java.awt.Component;
import javax.swing.JScrollPane;

public class MyScrollPane extends JScrollPane {
    public MyScrollPane(Component view, int vsbPolicy, int hsbPolicy) {
        super(view, vsbPolicy, hsbPolicy);
        setJackUI();
    }

    public MyScrollPane(Component view) {
        super(view);
        setJackUI();
    }

    public MyScrollPane(int vsbPolicy, int hsbPolicy) {
        super(vsbPolicy, hsbPolicy);
        setJackUI();
    }

    public MyScrollPane() {
        super();
    }
}

```

```

        setJackUI();
    }
    private void setJackUI() {
        getVerticalScrollBar().setUI(new MyScrollBarUI());
        getHorizontalScrollBar().setUI(new MyScrollBarUI());
        setBackground(MailClient.BACKGROUND_COLOR);
        getViewport().setBackground(MailClient.BACKGROUND_COLOR);
        setForeground(MailClient.FOREGROUND_COLOR);
        getViewport().setForeground(MailClient.FOREGROUND_COLOR);
    }
}

```

MyMail.java 的代码如下：

```

package com.mail;
import java.io.*;
import java.util.*;
import javax.activation.*;
import javax.mail.*;
import javax.mail.internet.*;
import javax.mail.internet.MimeMessage.RecipientType;
import javax.swing.*;
import com.mail.client.MailClient;
import com.mail.util.MailConstant;
// 收发邮件类. 能够接收和解析邮件, 以及构造和发送邮件
public class MyMail {
    private Message[] messages = null; // 邮件数组
    private Folder folder = null;      // 邮件夹
    private Store store = null;
    private MailClient mailClient;
    private Runtime runtime = Runtime.getRuntime();
    Authenticator authen = new MyAuthenticator();
    // MyMail 类的构造方法
    public MyMail(MailClient mailClient) {
        this.mailClient = mailClient;
    }
    // 从邮件服务器接收邮件, 并将邮件存放到成员变量 message 中
    public void jackProcess() throws Exception {
        Properties prop = new Properties();
        close();
        // 获取邮件会话属性
        if (MailConstant.INCOMING_PORT != null
            && !MailConstant.INCOMING_PORT.equals("")) {
            prop.setProperty("mail.pop3.port", MailConstant.INCOMING_PORT);
            prop.setProperty("mail.pop3.socketFactory.port",
                MailConstant.INCOMING_PORT);

```

```

        prop.setProperty("mail.pop3.socketFactory.class",
            "javax.net.ssl.SSLSocketFactory");
        prop.setProperty("mail.pop3.socketFactory.fallback", "false");
    }
    prop.put("mail.pop3.host", MailConstant.MAIL_SERVER_INCOMING);
    prop.put("mail.store.protocol", "pop3");
    // 如果需要身份认证, 则根据邮件会话属性和创建的密码验证器构造一个发送邮件的 Session 对象, 否则根据邮件会话属性构造一个发送邮件的 Session 对象
    Session session = null;
    if (MailConstant.MAIL_SERVER_AUTH != null
        && MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {
        session = Session.getInstance(prop, authen);
    } else {
        session = Session.getInstance(prop);
    }
    // 是否设置调试标志
    session.setDebug(MailConstant.MAIL_DEBUG != null ? MailConstant.MAIL_DEBUG
        .equalsIgnoreCase("true") : false);
    try {
        // 利用 Session 对象获得 Store 对象, 并连接 pop3 邮件服务器
        store = session.getStore(MailConstant.POP_MAIL);
        store.connect(MailConstant.MAIL_SERVER_INCOMING, -1, MailConstant.
            MAIL_SERVER_USER, MailConstant.MAIL_SERVER_PASSWORD);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(null, "Connetcion problem: Check mail.properties
for 'MAIL_SERVER_HOST', 'MAIL_SERVER_USER', 'MAIL_SERVER_PASSWORD',
'POP_MAIL' and 'SMTP_MAIL' are correct", "Failed to Connect",
JOptionPane.ERROR_MESSAGE);
        runtime.exec("notepad mail.properties");
        System.exit(-1);
    }
    // 获得邮箱内的邮件夹 Folder 对象
    folder = store.getDefaultFolder();
    if (folder == null) {
        JOptionPane.showMessageDialog(null,
            "There is no default folder in your mail server",
            "Cant load Mail folder", JOptionPane.ERROR_MESSAGE);
        runtime.exec("notepad mail.properties");
        System.exit(-1);
    }
    folder = folder.getFolder(MailConstant.INBOX);
    if (folder == null) {
        JOptionPane.showMessageDialog(null,
            "No default folder to view from your mail Client",

```

```

        "Cant load Mail folder", JOptionPane.ERROR_MESSAGE);
runtime.exec("notepad mail.properties");
System.exit(-1);
}
// 将获取邮件夹 Folder 对象以"读写"方式打开
folder.open(Folder.READ_WRITE);
// 获取邮件夹 Folder 内的邮件数
int totalMessages = folder.getMessageCount();
System.out.println("totalMessages =" + totalMessages);
mailClient.getStatusBar().setText(" [ " + totalMessages + " messages in Inbox ] " +
    " [ Today : " + new Date() + " ]");
if (totalMessages == 0) {
    return;
}
// 获取邮件夹 Folder 内的所有邮件
Message[] messages = folder.getMessages();
this.messages = messages;
}
// 关闭 Folder 和 Store
public void close() {
    try {
        if ( folder != null )
            folder.close(true);
        if ( store != null )
            store.close();
    } catch (MessagingException e) {
    }
}
// 获取邮件
public Message[] getMessage() {
    return messages;
}
// 解析成员变量 messages 中指定邮件，将邮件附件内容保存为文件，在阅读窗格中
// 显示邮件头和邮件内容
public void readMessage(final JEditorPane editor, final int msg_id) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            try {
                StringBuffer buffer = new StringBuffer("");
                // 如果邮件内容不是 html 类型的且是未读邮件，则处理邮件内容
                if (messages[msg_id] != null &&
                    !messages[msg_id].getContentType().equals("text/html") &&
                    !messages[msg_id].isSet(Flags.Flag.SEEN)) {
                    // 解析指定邮件的邮件头，将其保存到 buffer

```

```

buffer.append("From: " + messages[msg_id].getFrom()[0] + "\n");
Address[] add = messages[msg_id].getRecipients(RecipientType.TO);
buffer.append("Sent To: ");
for (int j = 0; j < add.length; j++) {
    buffer.append(add[j].toString() + ", ");
}
buffer.append("\n");
Address[] add2 = messages[msg_id].getRecipients(RecipientType.CC);
buffer.append("CC To: ");
if (add2 != null) {
    for (int j = 0; j < add2.length; j++) {
        buffer.append(add2[j].toString() + ", ");
    }
}
buffer.append("\n");
buffer.append("Subject: " + messages[msg_id].getSubject() + "\n");
// 解析指定邮件的邮件附件内容并将其保存为文件
boolean contentFlag = doJack(messages[msg_id]);
// 解析指定邮件的发送时间，将其保存到 buffer
buffer.append("Sent Date :" + messages[msg_id].getSentDate() + "\n");
buffer.append("\n");
buffer.append("\n");
buffer.append("\n");
// 解析指定邮件的邮件内容，将其保存到 buffer
if (!contentFlag) { //如果邮件内容是 Multipart 对象
    BufferedReader is = new BufferedReader(
        new InputStreamReader(messages[msg_id].getInputStream()));
    String line = "";
    boolean flag = false;
    int counter = 0;
    while ((line = is.readLine()) != null) {
        if (line.startsWith("Content-Transfer-")) {
            flag = true;
            counter++;
        }
        if (flag == true
            && line.indexOf("_NextPart_") > 0) {
            flag = false;
            counter++;
        }
        if (counter == 1
            && !line.startsWith("Content-Transfer-")) {
            buffer.append(line + "\n");
        }
    }
}

```

```

        }
        editor.setContentType("text/plain");
    } else {
        buffer.append("" + messages[msg_id].getContent());
    }
    // 否则如果是 html 类型的且是未读邮件, 则将 html 类型邮件内容添加到
buffer
    } else if (messages[msg_id] != null &&
        messages[msg_id].getContentType().equals( "text/html")) {
        buffer.append("" + messages[msg_id].getContent());
        editor.setContentType("text/html");
    }
    // 将 buffer 中的内容放到阅读窗格
    editor.setText(buffer.toString());
} catch (Exception e) {
    JOptionPane.showMessageDialog(null, "IO exception:" +
        e.getMessage().substring(0, 20), "Unable to read message this time",
        JOptionPane.ERROR_MESSAGE);
}
}
});
}
// 解析邮件附件内容并将其保存为文件. 如果邮件内容是 Multipart 对象, 则解析
Multipart 对象并将其中邮件附件内容保存为文件后返回 false, 否则返回 true
private boolean doJack(Message m) {
    String filepath = "";
    Multipart multipart;
    try {
        // 如果邮件内容是 Multipart 对象, 则解析 Multipart 对象并保存其中的附件后返
回 false, 否则返回 true
        if (m.getContent() instanceof Multipart) {
            // 获取邮件内容
            multipart = (Multipart) m.getContent();
            // 遍历 Multipart 对象, 并保存 Multipart 对象中的附件
            for (int z = 0; z < multipart.getCount(); z++) {
                BodyPart part = multipart.getBodyPart(z);
                // 获取邮件内容的配置
                String disposition = part.getDisposition();
                // 如果是邮件附件内容, 则指定附件的保存位置后保存附件
                if ((disposition != null) && (disposition.equals(Part.ATTACHMENT) ||
                    (disposition.equals(Part.INLINE)))) {
                    // 指定保存附件的位置
                    JFileChooser chooser = new JFileChooser();
                    chooser.setCurrentDirectory(new java.io.File(

```



```

        System.getProperty("user.home")));
chooser.setDialogTitle("Select The Folder where you want to save");
chooser.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
chooser.setSelectedFile(new File(part.getFileName()));
chooser.setAcceptAllFileFilterUsed(false);
// 将附件保存到指定的位置
if (chooser.showSaveDialog(mailClient) ==
        JFileChooser.APPROVE_OPTION) {
    filepath = chooser.getSelectedFile().toString();
    String filename = filepath + "/" + part.getFileName();
    File file = new File(filename);
    file.createNewFile();
    InputStream in = part.getInputStream();
    FileOutputStream fos = new FileOutputStream(file);
    byte[] buf = new byte[1024];
    int c = 0;
    while ((c = in.read(buf)) != -1) {
        fos.write(buf, 0, c);
    }
    in.close();
    fos.close();
} else {
    continue;
}
} else if (m.getContent() instanceof String) {
    return true;
}
} catch (IOException exception) {
    JOptionPane.showMessageDialog(null, "IO exception:"
        + exception.getMessage().substring(0, 20),
        "Unable to save attachemtnt", JOptionPane.ERROR_MESSAGE);
} catch (MessagingException exception) {
    JOptionPane.showMessageDialog(null, "Message exception:"
        + exception.getMessage().substring(0, 20),
        "Unable to save attachemtnt", JOptionPane.ERROR_MESSAGE);
}
return false;
}
// 构造邮件的邮件头和邮件内容，并发送邮件
public boolean sendMailWithAtt(String sender_name, String email_from,
    String email_to, String email_cc, String email_bcc,
    String email_subject, String email_body, File file_name) {

```

```

boolean result = true;
try {
    // 获取邮件会话属性
    Properties props = new Properties();
    props.setProperty("mail.transport.protocol", "smtp");
    props.setProperty("mail.host", MailConstant.MAIL_SERVER_OUTGOING);
    if (MailConstant.MAIL_SERVER_AUTH != null
        && MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {
        props.put("mail.smtp.auth", MailConstant.MAIL_SERVER_AUTH);
    }
    if (MailConstant.OUTGOING_PORT != null
        && !MailConstant.OUTGOING_PORT.trim().equals("")) {
        props.put("mail.smtp.port", MailConstant.OUTGOING_PORT);
        props.put("mail.smtp.socketFactory.port",
            MailConstant.OUTGOING_PORT);
        props.put("mail.smtp.socketFactory.class",
            "javax.net.ssl.SSLSocketFactory");
        props.put("mail.smtp.socketFactory.fallback", "false");
        props.setProperty("mail.smtp.quitwait", "false");
    }
    // 如果需要身份认证，则根据邮件会话属性和创建的密码验证器构造一个发送邮件的 session，
    // 否则根据邮件会话属性构造一个发送邮件的 session
    Session session = null;
    if (MailConstant.MAIL_SERVER_AUTH != null &&
        MailConstant.MAIL_SERVER_AUTH.equalsIgnoreCase("true")) {
        session = Session.getInstance(props, authen);
    } else {
        session = Session.getInstance(props);
    }
    session.setDebug(MailConstant.MAIL_DEBUG != null ?
        MailConstant.MAIL_DEBUG.equalsIgnoreCase("true") : false);
    // 根据 session 创建一个邮件对象
    Message message = new MimeMessage(session);
    // 设置邮件发送者地址
    message.setFrom(new InternetAddress(email_from));
    // 设置邮件接收者地址
    if (email_to != null && !email_to.trim().equals("")) {
        String tos[] = email_to.trim().split(",");
        Address toAdd[] = new Address[tos.length];
        for (int i = 0; i < toAdd.length; i++) {
            toAdd[i] = new InternetAddress(tos[i]);
        }
        message.addRecipients(Message.RecipientType.TO, toAdd);
    }
}

```

```

// 设置邮件抄送者地址
if ( email_cc != null && !email_cc.trim().trim().equals( "" ) ) {
    String ccs[] = email_cc.split(",");
    Address ccAdd[] = new Address[ccs.length];
    for ( int i = 0; i < ccAdd.length; i++ ) {
        ccAdd[i] = new InternetAddress(ccs[i]);
    }
    message.addRecipients(Message.RecipientType.CC, ccAdd);
}
// 设置邮件密送者地址
if ( email_bcc != null && !email_bcc.trim().trim().equals( "" ) ) {
    String bccs[] = email_bcc.split(",");
    Address bccAdd[] = new Address[bccs.length];
    for ( int i = 0; i < bccAdd.length; i++ ) {
        bccAdd[i] = new InternetAddress(bccs[i]);
    }
    message.addRecipients(Message.RecipientType.BCC, bccAdd);
}
// 设置邮件主题
message.setSubject(email_subject);
// 设置邮件发送日期
message.setSentDate(new Date());
// 将邮件文本内容加入 multipart
BodyPart messageBodyPart = new MimeBodyPart();
messageBodyPart.setText(email_body);
Multipart multipart = new MimeMultipart();
multipart.addBodyPart(messageBodyPart);
// 若有附件，则将邮件附件内容加入 multipart
messageBodyPart = new MimeBodyPart();
if (file_name != null) {
    // 根据附件的文件名获取数据源
    DataSource source = new FileDataSource(file_name);
    // 将附件的内容加入 messageBodyPart
    messageBodyPart.setDataHandler(new DataHandler(source));
    // 将附件的文件名加入 messageBodyPart
    messageBodyPart.setFileName(file_name.getName());
    multipart.addBodyPart(messageBodyPart);
}
// 设置邮件内容，包括邮件文本内容和邮件附件内容
message.setContent(multipart);
// 发送邮件
Transport.send(message);
} catch (MessagingException mex) { // Trap the MessagingException Error
    result = false;
}

```

```

    }
    return result;
}
// 自定义邮件授权类
private class MyAuthenticator extends javax.mail.Authenticator {
    // 使用 Authenticator 抽象类必须实现 getPasswordAuthentication()方法。该方法用于
    // 保存认证时要用到的用户名和密码。
    // 并且要在 Session 中进行注册，使 Session 能了解在认证时该使用哪个类。
    public PasswordAuthentication getPasswordAuthentication() {
        return new PasswordAuthentication(MailConstant.MAIL_SERVER_USER,
            MailConstant.MAIL_SERVER_PASSWORD);
    }
}
}
}

```

MailConstant.java 的代码如下：

```

package com.mail.util;
public class MailConstant {
    public final static String MAIL_SERVER_INCOMING =
        Util.getString("MAIL_SERVER_INCOMING");
    public final static String MAIL_SERVER_OUTGOING =
        Util.getString("MAIL_SERVER_OUTGOING");
    public final static String INCOMING_PORT = Util.getString("INCOMING_PORT");
    public final static String OUTGOING_PORT = Util.getString("OUTGOING_PORT");
    public final static String MAIL_SERVER_AUTH =
        Util.getString("MAIL_SERVER_AUTH");
    public final static String MAIL_SERVER_USER =
        Util.getString("MAIL_SERVER_USER");
    public final static String MAIL_SERVER_PASSWORD =
        Util.getString("MAIL_SERVER_PASSWORD");
    public final static String MAIL_DEBUG = Util.getString("MAIL_DEBUG");
    public final static String MAIL_CONTENT_TYPE =
        Util.getString("MAIL_CONTENT_TYPE");
    public final static String INBOX = "INBOX";
    public final static String POP_MAIL = "pop3";
    public final static String SMTP_MAIL = "smtp";
    public final static String BACK_COLOR_R = Util.getString("BACK_COLOR_R");
    public final static String BACK_COLOR_G = Util.getString("BACK_COLOR_G");
    public final static String BACK_COLOR_B = Util.getString("BACK_COLOR_B");
    public final static String FORE_COLOR_R = Util.getString("FORE_COLOR_R");
    public final static String FORE_COLOR_G = Util.getString("FORE_COLOR_G");
    public final static String FORE_COLOR_B = Util.getString("FORE_COLOR_B");
}

```

util.java 的代码如下：

```

package com.mail.util;

```

```

import java.io.*;
import java.util.Properties;
import javax.swing.JOptionPane;
public class Util {
    private static Properties props = new Properties();
    static {
        try {
            props.load(new FileInputStream("mail.properties"));
        } catch (FileNotFoundException e) {
            JOptionPane.showMessageDialog(null,
                "Check your Property file, it should be in application home dir, Error:" +
                e.getCause(), "Cant load mail.properties", JOptionPane.ERROR_MESSAGE);
            System.exit(-1);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null,
                "Check your Property file, it should be in application home dir, Error:" +
                e.getCause(), "Cant load mail.properties",JOptionPane.ERROR_MESSAGE);
            System.exit(-1);
        }
    }
    public static String getString(String key) {
        return props.getProperty(key);
    }
}

```

A.11 聊天客户/服务器程序参考答案

所附代码为开源软件 JChat 的修改版本，对该程序的使用和发放应该遵守 GPL 的规定。

本程序由聊天客户程序和聊天服务器程序组成。

聊天客户程序主要由 ChatClient.java 和 ClientKernel.java 文件组成。

ChatClient.java 的代码如下：

```

package com.chat.client;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
// 聊天客户端窗口类
public class ChatClient extends JFrame implements KeyListener, ActionListener,
    FocusListener {
    public static final String appName = "Chat Tool";
    public static final String serverText = "127.0.0.1";
    public static final String portText = "3500";
    public static final String nickText = "YourName";
    JPanel northPanel, southPanel, centerPanel;

```

```

JTextField txtHost, txtPort, msgWindow, txtNick;
JButton buttonConnect, buttonSend;
JScrollPane sc;
ClientKernel ck;
ClientHistory historyWindow;
private String lastMsg = "";
// ChatClient 类的构造方法
public ChatClient() {
    uiInit();
    txtHost.setText("127.0.0.1");
    txtPort.setText("3500");
}
// 建立聊天客户端窗口
public void uiInit() {
    setLayout(new BorderLayout());
    // 建立服务器和用户信息窗口
    northPanel = new JPanel(new GridLayout(0, 2));
    northPanel.add(new JLabel("Host address:"));
    northPanel.add(txtHost = new JTextField(ChatClient.serverText));
    northPanel.add(new JLabel("Port:"));
    northPanel.add(txtPort = new JTextField(ChatClient.portText));
    northPanel.add(new JLabel("Nick:"));
    northPanel.add(txtNick = new JTextField(ChatClient.nickText));
    northPanel.add(new JLabel(""));
    northPanel.add(new JLabel(""));
    northPanel.add(new JLabel(""));
    northPanel.add(buttonConnect = new JButton("Connect"));
    txtHost.addKeyListener(this);
    txtHost.addFocusListener(this);
    txtNick.addFocusListener(this);
    txtNick.addKeyListener(this);
    txtPort.addKeyListener(this);
    txtPort.addFocusListener(this);
    buttonConnect.addKeyListener(this);
    buttonConnect.addActionListener(this);
    this.add(northPanel, BorderLayout.NORTH);
    // 建立发送窗口
    southPanel = new JPanel();
    southPanel.add(msgWindow = new JTextField(20));
    southPanel.add(buttonSend = new JButton("Send"));
    buttonSend.addActionListener(this);
    msgWindow.addKeyListener(this);
    add(southPanel, BorderLayout.SOUTH);
    // 建立聊天记录窗口

```

```

        historyWindow = new ClientHistory();
        sc = new JScrollPane(historyWindow);
        sc.setAutoscrolls(true);
        this.add(sc, BorderLayout.CENTER);
    }
    // 建立 ChatClient 的对象，并对其进行设置
    public static void main(String args[]) {
        ChatClient client = new ChatClient();
        client.setTitle(client.appName);
        client.setSize(450, 500);
        client.setLocation(100, 100);
        client.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        client.setVisible(true);
        client.msgWindow.requestFocus();
    }
    // 将信息添加到聊天记录窗口
    public void addMsg(String str) {
        historyWindow.addText(str);
    }
    // 用户登录服务器
    private void connect() {
        try {
            if (ck != null)
                ck.dropMe();
            ck = new ClientKernel(txtHost.getText(), Integer.parseInt(txtPort.getText()));
            ck.setNick(txtNick.getText());
            if (ck.isConnected()) {
                ck.addClient(this);
                addMsg("<font color=\"#00ff00\">connected! Local Port:" +
                    ck.getLocalPort() + "</font>");
            } else {
                addMsg("<font color=\"#ff0000\">connect failed! </font>");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    // 将发送窗口的信息发送给服务器
    private void send() {
        String toSend = msgWindow.getText();
        ck.sendMessage(toSend);
        lastMsg = "" + toSend;
        msgWindow.setText("");
    }

```

```
// 在发送窗口中按下上箭头键时显示最后发送的内容
public void keyReleased(KeyEvent e) {
    if (e.getSource() == msgWindow && e.getKeyCode() == KeyEvent.VK_UP)
        msgWindow.setText(lastMsg);
}
// 在用户昵称和发送信息等文本框键入回车键时分别登录服务器和发送信息，在服务器地址、端口和用户昵称文本框键入回车键时光标分别定位在端口、用户昵称和发送消息文本框
```

```
public void keyTyped(KeyEvent e) {
    if (e.getKeyChar() == KeyEvent.VK_ENTER) {
        if (e.getSource() == msgWindow)
            send();
        if (e.getSource() == txtNick) {
            connect();
            msgWindow.requestFocus();
        }
        if (e.getSource() == txtHost)
            txtPort.requestFocus();
        if (e.getSource() == txtPort)
            txtNick.requestFocus();
    }
}
// 单击 Connect 和 Send 按钮时分别登录服务器和发送信息
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == buttonConnect)
        connect();
    if (e.getSource() == buttonSend)
        send();
}
```

内容

```
// 光标在服务器地址、端口、用户昵称和发送信息等文本框时清空相应文本框中的
```

```
public void focusGained(FocusEvent e) {
    if (e.getSource() == txtHost
        && txtHost.getText().equals(ChatClient.serverText))
        txtHost.setText("");
    if (e.getSource() == txtPort
        && txtPort.getText().equals(ChatClient.portText))
        txtPort.setText("");
    if (e.getSource() == txtNick
        && txtNick.getText().equals(ChatClient.nickText))
        txtNick.setText("");
}
// 光标离开文本框后恢复其中的信息
public void focusLost(FocusEvent e) {
```



```

        if (e.getSource() == txtPort && txtPort.getText().equals(""))
            txtPort.setText(ChatClient.portText);
        if (e.getSource() == txtHost && txtHost.getText().equals(""))
            txtHost.setText(ChatClient.serverText);
        if (e.getSource() == txtNick
            && txtNick.getText().equals(ChatClient.nickText))
            txtNick.setText(ChatClient.nickText);
    }
    // 聊天记录窗口类
    class ClientHistory extends JEditorPane {
        public ClientHistory() {
            super("text/html", "" + ChatClient.appName);
            setEditable(false);
            setAutoscrolls(true);
        }
        // 将字符串添加到聊天记录的末尾
        public void addText(String str) {
            String html = getText();
            int end = html.lastIndexOf("</body>");
            String startStr = html.substring(0, end);
            String endStr = html.substring(end, html.length());
            String newHtml = startStr + "<br>" + str + endStr;
            setText(newHtml);
            setSelectionStart(newHtml.length() - 1);
            setSelectionEnd(newHtml.length());
        }
        // 清除聊天记录
        public void clear() {
            setText("");
        }
    }
}

```

ClientKernel.java 的代码如下:

```

package com.chat.client;
import java.io.*;
import java.net.*;
import java.util.*;
// 客户端发送和接收信息的类
public class ClientKernel {
    public static final char MSGENDCHAR = 0xff;
    public static final char EXIT = 0xFE;
    public static final char NICK = 0xFD;
    public static final char COMMAND = 0xFD;
    private String serverAd;

```

```

private int port;
private Socket sock;
private boolean isConnected = false;
private boolean dropMe = false;
private LinkedList clients; // 用户列表
public String nick; // 昵称
public boolean printMsg = true;
private ClientMsgSender cms;
private ClientMsgListener cml;
// ClientKernel 类的构造方法
public ClientKernel(String server, int port) {
    this.port = port;
    nick = "" + port;
    serverAd = server;
    clients = new LinkedList();
    connect();
    if (isConnected) {
        cms = new ClientMsgSender(this, sock);
        cml = new ClientMsgListener(this, sock);
    }
}
// 建立与服务器的连接
public void connect() {
    try {
        sock = new Socket(serverAd, port);
        isConnected = true;
    } catch (IOException ioe) {
        ioe.printStackTrace();
    }
}
public int getPort() {
    return port;
}
// 将指定的字符串前添加 0xFD 和 "nick " 后向服务器发送
public boolean setNick(String nick) {
    sendMessage("" + ClientKernel.COMMAND + "nick " + nick);
    return true;
}
public int getLocalPort() {
    return sock.getLocalPort();
}
// 下线
public void dropMe() {
    System.out.println("Drop ME!!!");
}

```

```

        cms.drop();
        cml.drop();
        dropMe = true;
        while (cml.hasStoped() && cms.hasStoped())
            pause(5);
    }
    // 在线时，如果发送的信息是命令（首字符为'/'），则将其首字符替换为 0xFD 后向服务器发送，否则向服务器直接发送
    public void sendMessage(String str) {
        if (!dropMe) {
            if (str.charAt(0) == '/')
                cms.addMessage("'" + ClientKernel.COMMAND + str.substring(1));
            else
                cms.addMessage(str);
        }
    }
    // 将用户添加到用户列表中
    public void addClient(ChatClient c) {
        clients.add(c);
    }
    // 将用户从用户列表删除
    public void removeClient(ChatClient c) {
        clients.remove(c);
    }
    public void pause(int time) {
        try {
            Thread.sleep(time);
        } catch (Exception e) {
        }
    }
    // 将信息添加到每个用户的聊天窗口中
    public synchronized void storeMsg(String str) {
        Object[] client = clients.toArray();
        for (int i = 0; i < client.length; i++)
            ((ChatClient) (client[i])).addMsg(str);
    }
    public boolean isConnected() {
        return isConnected;
    }
}
// 向服务器发送信息的类
class ClientMsgSender extends Thread {
    private Socket s;
    private ClientKernel ck;

```

```

private LinkedList msgList; // 信息列表
private boolean running = true;
private boolean hasStoped = false;
// ClientMsgSender 类的构造方法
public ClientMsgSender(ClientKernel ck, Socket s) {
    this.ck = ck;
    this.s = s;
    msgList = new LinkedList();
    start();
}
// 将发送的内容添加到信息列表中
public synchronized void addMessage(String msg) {
    msgList.addLast(msg);
}
public void drop() {
    running = false;
}
public boolean hasStoped() {
    return hasStoped;
}
// 向服务器发送信息列表中的信息
public void run() {
    try {
        DataOutputStream dataOut = new DataOutputStream(s.getOutputStream());
        while (running) {
            while (msgList.size() > 0) {
                String msg = ((String) (msgList.removeFirst()));
                char[] data = msg.toCharArray();
                for (int i = 0; i < data.length; i++)
                    dataOut.write((int) data[i]);
                dataOut.write(ClientKernel.MSGENDCHAR);
            }
            sleep(10);
        }
        dataOut.write(ClientKernel.EXIT);
        dataOut.close();
    } catch (Exception ioe) {
        ioe.printStackTrace();
    } finally {
        hasStoped = true;
    }
}
}
// 接收服务器信息的类

```

```

class ClientMsgListener extends Thread {
    private ClientKernel ck;
    private Socket s;
    private boolean running = true;
    private boolean hasStoped = false;
    // ClientMsgListener 类的构造方法
    public ClientMsgListener(ClientKernel ck, Socket s) {
        this.ck = ck;
        this.s = s;
        start();
    }
    public void drop() {
        running = false;
    }
    public boolean hasStoped() {
        return hasStoped;
    }
    // 接收服务器信息，并将其添加到聊天窗口中
    public void run() {
        try {
            BufferedInputStream buffIn = new BufferedInputStream(s.getInputStream());
            DataInputStream dataIn = new DataInputStream(buffIn);
            while (running) {
                StringBuffer strBuff = new StringBuffer();
                int c;
                while ((c = dataIn.read()) != ClientKernel.MSGENDCHAR) {
                    strBuff.append((char) c);
                }
                // 将收到的信息添加到信息列表中
                ck.storeMsg("'" + strBuff.toString());
            }
            dataIn.close();
            buffIn.close();
        } catch (IOException ioe) {
            ioe.printStackTrace();
        } finally {
            hasStoped = true;
        }
    }
}

```

聊天服务器程序主要由 MainServer.java、ConnectedClient.java、ConnectionKeeper.java、DataSource.java、FileDataSource.java、CommandParser.java 和 BroadcastCommandParser.java 文件组成。

MainServer.java 的代码如下：

```

package com.chat.server;
import java.net.*;
import java.io.*;
// 服务器的主类. 建立服务器套接口, 接收客户端的连接请求
public class MainServer extends Thread {
    public static final String DISCONNECTED = "Software caused connection abort";
    public static final String DISCONNECTED_CLIENT = "Socket closed";
    public static final String PORT_USED_ERROR = "Address already in use: JVM_Bind";
    public static long uptime = 0;
    public static long connects = 0;
    public static final char MSGENDCHAR = 0xff;
    int port = 3500;
    int clients = 8;
    private boolean newPort = true;
    private ServerSocket sSock;
    private Socket sock;
    public ConnectionKeeper ck;
    public static DataSource ds;
    public static CommandParser cp;
    // MainServer 类的构造方法
    public MainServer(int port) {
        this.port = port;
        ck = new ConnectionKeeper(MainServer.cp);
        MainServer.uptime = System.currentTimeMillis();
        start();
    }
    // 建立服务器套接口, 等待客户端的连接请求; 当接收到客户端的连接请求时建立
    // ConnectedClient 的对象, 并将其加入用户列表中.
    public void run() {
        while(true) {
            try {
                sSock = new ServerSocket(port);
                if(newPort) {
                    System.out.println("Server Listening at port: " + sSock.getLocalPort());
                    newPort = false;
                }
                sock = sSock.accept();
                ck.add(sock);
                sSock.close();
            } catch(Exception e) {
                String message = e.getMessage();
                if(message.startsWith(MainServer.PORT_USED_ERROR)) {
                    System.out.println("Port " + port + " is already used,
                        Attempting to use " + (port+=1));
                }
            }
        }
    }
}

```

```

        newPort = true;
    } else {
        e.printStackTrace();
        System.exit(1);
    }
}
}
}
// 在指定的毫秒数内让当前正在执行的线程休眠
public static void sleep(int time) {
    try {
        Thread.sleep(time);
    } catch (Exception e) {}
}
// 主函数
public static void main(String arg[]) {
    int port = 0;
    MainServer ms;
    MainServer.ds = new FileDataSource();
    MainServer.cp = new BroadcastCommandParser();
    MainServer.cp.setDataSource(MainServer.ds);
    if(arg.length!=1) {
        ms = new MainServer(3500);
        System.out.println("Usage: java jchat.server.MainServer <port>\nAttempting to use
default port 3500");
    } else {
        try {
            port = Integer.parseInt(arg[0]);
        } catch (NumberFormatException nfe) {
            System.out.println("Attempting to use default port 3500");
            port = 3500;
        } finally {
            ms = new MainServer(port);
        }
    }
}
}

```

ConnectedClient.java 的代码如下:

```

package com.chat.server;
import java.io.*;
import java.net.*;
import java.util.*;
// 接收和处理客户端信息或命令，并将信息列表中的信息发送给客户端的类
public class ConnectedClient {

```

```

private ConnectionKeeper ck;
public String nick;
public Date connectedTime;
public String ipNumber;
public int portNumber;
public boolean verifiedBoolean = false;
public int verifiedCount = 0;
public String tmpNick = "";
private ServerMsgSender msgSend;
private ServerMsgListener msgList;
private Socket sock;
public boolean printMsg = false;
// ConnectedClient 类的构造方法
public ConnectedClient(Socket sock, ConnectionKeeper ck) {
    this.ck = ck;
    ipNumber = sock.getInetAddress().getHostAddress();
    portNumber = sock.getPort();
    this.sock = sock;
    msgSend = new ServerMsgSender(this.sock, this);
    msgList = new ServerMsgListener(this.sock, this);
    nick = "" + portNumber;
}
public ConnectionKeeper getConnectionKeeper() {
    return ck;
}
// 获取用户昵称
public String getNick() {
    return nick;
}
// 向用户发送信息
public void sendMessage(String str) {
    msgSend.addMessage(str);
}
// 向用户列表中指定用户发送指定信息
public void sendTo(String user, String msg) {
    ck.sendTo(this, user, msg);
}
// 向用户列表中所有用户发送信息
public void broadcastMessage(String str) {
    if(!isSpam(str)) ck.broadcast(str);
}
// 用户下线
public void dropClient() {
    msgList.closeConnection();
}

```



```

        msgSend.closeConnection();
        ck.remove(this);
    }
    // 如果是命令（手字符为 0xFD），则解析命令
    public void runCommand(String str) {
        if(str.charAt(0)==0xFD) {
            String str1 = str.substring(1);
            ck.runCommand(this, str1);
        }
    }
    private boolean isSpam(String str) {
        return false;
    }
    public static void main(String arg[]) {
        MainServer ms = new MainServer(1984);
    }
    // 查询自己
    public void whoAmI() {
        String str = "<br/>Connected Port: " + portNumber + "<br/>" +
            "Nick: " + nick + "<br/>";
        sendMessage(str);
    }
}
// 服务器将信息列表中的信息发送给客户端的类
class ServerMsgSender extends Thread {
    private Socket sock;
    private LinkedList msgList; //信息列表
    private ConnectedClient cc;
    private boolean running = true;
    // ServerMsgSender 类的构造方法
    public ServerMsgSender(Socket sock, ConnectedClient cc) {
        this.sock = sock;
        this.cc = cc;
        collectInfo();
        msgList = new LinkedList();
        start();
    }
    // 将发送内容添加到信息列表中
    public synchronized void addMessage(String str) {
        if(cc.printMsg) System.out.println("MsgSender.addMessage: " +str);
        msgList.addLast(str);
    }
    private void collectInfo() { }
    // 将信息列表中的内容发送到客户端

```

```

public void run() {
    try {
        DataOutputStream dataOut = new DataOutputStream(sock.getOutputStream());
        while(running) {
            while(msgList.size()>0) {
                String toSend = (String)(msgList.removeFirst());
                dataOut.writeBytes("" + toSend + MainServer.MSGENDCHAR);
                if(cc.printMsg) System.out.println("MsgSender.run: Sending: " + toSend);
                sleep(10);
            }
            sleep(10);
        }
    } catch(Exception e) {
        String msg = e.getMessage();
        if(msg.startsWith(MainServer.DISCONNECTED) ||
            msg.startsWith(MainServer.DISCONNECTED_CLIENT)) {
            System.out.println("MsgSender.run Client disconnected nick: " + cc.nick);
            cc.dropClient();
        } else {
            System.out.println("MsgSender.run: Msg: " + msg);
            e.printStackTrace();
            cc.dropClient();
        }
    }
}

// 将发送的连接状态设置断开
public void closeConnection() {
    running = false;
}

}

// 接收和处理客户端信息或命令的类
class ServerMsgListener extends Thread {
    private LinkedList msgList; //信息列表
    private Socket sock;
    private ConnectedClient cc;
    private boolean running = true;
    // ServerMsgListener 类的构造方法
    public ServerMsgListener(Socket s, ConnectedClient cc) {
        msgList = new LinkedList();
        sock = s;
        this.cc = cc;
        start();
    }
}

// 将接收的连接状态设置断开

```

```

public void closeConnection() {
    running = false;
}
// 从客户端读取信息，并判断是否是命令（首字符为 0xFD）；如果不是命令，则将
// 信息广播给在线所有用户，否则处理命令
public void run() {
    try {
        BufferedInputStream buffIn = new BufferedInputStream(sock.getInputStream());
        DataInputStream dataIn = new DataInputStream(buffIn);
        while(running) {
            int c;
            boolean didRun = false;
            boolean isCommand = false;
            StringBuffer strBuff = new StringBuffer();
            sleep(10);
            while( (c=dataIn.read()) != 0xff) {
                strBuff.append((char)c);
                if(!didRun) didRun=true;
                // 判断类型是否是 Command
                if(c==0xFD) isCommand = true;
            }
            if(cc.verifiedCount>0 && !cc.verifiedBoolean && !isCommand) {
                cc.verifiedCount--;
                if(cc.verifiedCount==1) {
                    cc.sendMessage("You have failed to verify your nick");
                    cc.nick = "" + cc.portNumber;
                    cc.sendMessage("Your nick is " + cc.nick);
                } else {
                    cc.sendMessage("type: \"/verify &lt;password>" to verify your nick");
                }
            }
            if(didRun) {
                String toSend = "" + cc.nick + ":" + strBuff.toString();
                if(cc.printMsg)
                    System.out.println("MsgListenet.run Sending msg: " + toSend);
                // 如果不是命令，则广播信息，否则处理命令
                if(!isCommand)
                    cc.broadcastMessage(toSend);
                else
                    cc.runCommand(strBuff.toString());
            }
        }
    } catch(SocketException se) {
        if(se.getMessage().startsWith("Connection reset"))

```

```

        cc.dropClient();
    } catch(Exception e) {
        e.printStackTrace();
        cc.dropClient();
    }
}
}

```

ConnectionKeeper.java 的代码如下:

```

package com.chat.server;
import java.io.*;
import java.net.*;
import java.util.*;
// 管理用户列表和客户端命令解析的类
public class ConnectionKeeper {
    @SuppressWarnings("rawtypes")
    private LinkedList clientList; //用户列表
    private CommandParser cp;
    // ConnectionKeeper 类的构造方法
    public ConnectionKeeper(CommandParser parser) {
        this.cp = parser;
        clientList = new LinkedList();
    }
    // 将建立的 ConnectedClient 的对象加入用户列表中
    public void add(Socket s) {
        MainServer.connects++;
        clientList.addLast(new ConnectedClient(s, this));
    }
    // 将指定的 ConnectedClient 的对象从用户列表中删除
    public void remove(ConnectedClient cc) {
        clientList.remove(cc);
        cc = null;
    }
    // 获取用户列表
    public LinkedList users() {
        return clientList;
    }
    // 解析命令
    public void runCommand(ConnectedClient cc, String str) {
        cp.runCommand(cc, str);
    }
    // 向用户列表中指定用户发送指定信息
    public void sendTo(ConnectedClient sender, String user, String msg) {
        boolean found = false;
        for(int i =0;i<clientList.size();i++) {

```

```

        ConnectedClient receiver = (ConnectedClient)(clientList.get(i));
        if(user.equalsIgnoreCase(receiver.nick)) {
            receiver.sendMessage(msg);
            found = true;
            //i = clientList.size()+5; // Stop the loop.
            break;
        }
    }
    if(!found) {
        sender.sendMessage("Unable to find user " + user);
    }
}
// 向用户列表中所有用户发送信息
public void broadcast(String msg) {
    for(int i =0;i<clientList.size();i++) {
        ConnectedClient cc = (ConnectedClient)(clientList.get(i));
        cc.sendMessage(msg);
    }
}
}

```

DataSource.java 的代码如下：

```

package com.chat.server;
// FileDataSource 的接口
public interface DataSource {
    public boolean verifyUser(String user, String pass);
    public boolean addUser(String user, String pass);
    public boolean removeUser(String user, String pass);
    public boolean addInfo(String user, String field, String info);
    public boolean removeInfo(String user, String field, String info);
    public String[] getAllUserInfo(String user);
    public String[] getUserList();
}

```

FileDataSource.java 的代码如下：

```

package com.chat.server;
import java.util.*;
import java.security.*;
import java.io.*;
// 用户信息处理类
public class FileDataSource implements DataSource {
    String[] us = {"no connected users"}; // An array of all users.
    boolean userListChanged = true;
    boolean debug = false;
    LinkedList users = new LinkedList(); //用户列表
    LinkedList infoList = new LinkedList(); //信息列表
}

```

```

String pathSep = System.getProperty("file.separator");
String fsroot = System.getProperty("user.home") + pathSep + ".mihalychat" + pathSep;
File userFile, infoFile; //用户文件和信息文件
boolean basicWrite = false;
boolean allWrite = false;
private final String[] infoComments = {"# Info file for Chat Server",
    "# Used when the server is running in FileDataSource mode",
    "# <username> <field> <data>",
    "# username is the name of the user which the info belongs to"};
private final String[] userComments = {"# User file for Mihaly Chat Server",
    "# <username> <password>",
    "# usernames are in clear text",
    "# passwords are MD5 hashes"};
// FileDataSource 类的构造方法
public FileDataSource() {
    System.out.println("FileDataSource init");
    initFileSystem();
    if(allWrite) {
        readFiles();
    }
}

public void printAll() {
    if(debug)System.out.println("\nStarting to print");
    Iterator it = users.iterator();
    while(it.hasNext())
        ((User)(it.next())).printUser();
    it = infoList.iterator();
    while(it.hasNext())
        ((Info)(it.next())).printInfo();
    if(debug)System.out.println("Ending print\n");
}

// 初始化文件系统. 判断是否存在 fsroot 目录, 不存在则建立该目录; 然后在该目录
// 下分别建立用户文件 (文件名为 users) 和信息文件 (文件名为 info)
private void initFileSystem() {
    File file = new File(fsroot);
    // 判断是否存在 fsroot 目录, 不存在则建立该目录
    if(file.exists() && file.isDirectory()) {
        if(debug)System.out.println("FileDataSource home " + fsroot);
        basicWrite = true;
    } else {
        System.out.println("FileDataSource is unable to access home" +
            "\nAttempting to create home");
        file.mkdir();
        if(!(file.exists() && file.isDirectory()))

```

```

        System.out.println("FileDataSource is unable to create home @" +
            + fsroot + " ex " + file.exists() + " isDir "
            + file.isDirectory());
    else {
        System.out.println("FileDataSource home created " + fsroot);
        basicWrite = true;
    }
}
// 在该目录下分别建立用户文件（文件名为 users）和信息文件（文件名为 info）。
if(basicWrite) {
    userFile = new File(fsroot + "users");
    infoFile = new File(fsroot + "info");
    if(userFile.exists() && infoFile.exists()) {
        if(userFile.canRead() && userFile.canWrite()
            && infoFile.canRead() && infoFile.canWrite()) {
            if(debug) System.out.println("FileDataSource userfile " + userFile +
                "\nFileDataSource infofile " + infoFile);
            allWrite = true;
        } else {
            System.out.println("FileDataSource files exist but" +
                "userFile.read " + userFile.canRead() +
                "userFile.write " + userFile.canWrite() +
                "infoFile.read " + infoFile.canRead() +
                "userFile.write " + userFile.canWrite());
        }
    } else {
        try {
            userFile.createNewFile();
            infoFile.createNewFile();
            if(userFile.exists() && infoFile.exists()) {
                if(userFile.canWrite() && infoFile.canWrite()) {
                    if(debug) System.out.println("FileDataSource userfile "
                        + userFile + "\nFileDataSource infofile "
                        + infoFile);
                    createFiles();
                    allWrite = true;
                } else {
                    System.out.println("FileDataSource user/info files " +
                        "we just created are not writeable :(");
                }
            }
        } else {
            System.out.println("FileDataSource you shouldn't get " +
                "here but you did. The user/info file that we just" +
                "created are gone missing");
        }
    }
}

```

```

        }
    } catch(IOException ioe) {
        System.out.println("FileDataSource " + ioe.getMessage());
    }
}
}

// 将用户注释和信息注释分别添加到用户文件和信息文件
private void createFiles() {
    for(int i=0;i<userComments.length;i++)
        appendToUserFile(userComments[i]);
    for(int i=0;i<infoComments.length;i++)
        appendToInfoFile(infoComments[i]);
}

// 从用户文件中读取用户名和密码，将其保存到名为用户列表中；从信息文件中读
// 取用户、域和数据保存到信息列表中
private void readFiles() {
    Object[] ObjUsers = readLines(userFile);
    Object[] ObjInfo = readLines(infoFile);
    users = new LinkedList();
    infoList = new LinkedList();
    try {
        for(int i = 0;i<ObjUsers.length;i++) {
            String str = "" + ObjUsers[i];
            String name = str.substring(0, str.indexOf(" "));
            String pass = str.substring(str.indexOf(" ")+1);
            users.add(new User(name,pass));
        }
        for(int i = 0;i<ObjInfo.length;i++) {
            String str = "" + ObjInfo[i];
            StringTokenizer strTok = new StringTokenizer(str);
            infoList.add(new Info(strTok.nextToken(),
                                strTok.nextToken(),
                                strTok.nextToken()));
        }
    } catch(Exception e) {
        errorInFiles(e);
    }
}

// 将用户添加到用户文件中
private void appendToUserFile(String str) {
    appendToFile(userFile, str);
}

```



```

// 将信息添加到信息文件中
private void appendToInfoFile(String str) {
    appendToFile(infoFile, str);
}
// 将指定字符串添加到指定文件
private void appendToFile(File e, String str) {
    try {
        FileOutputStream fileOut = new FileOutputStream(e,true);
        BufferedOutputStream buffOut = new BufferedOutputStream(fileOut);
        DataOutputStream dataOut = new DataOutputStream(buffOut);
        dataOut.writeBytes(str + "\n");
        dataOut.close();
        buffOut.close();
        fileOut.close();
    } catch(IOException ioe) {
        System.out.println("FileDataSource alot of checks was done when initing " +
            " " + e + " still it got messed up! This time no checks!");
        ioe.printStackTrace();
    }
}
// 从用户文件中删除指定用户
private void removeUserFromFile(String user) {
    try {
        if(allWrite) {
            File f = new File(fsroot + "newUser");
            FileOutputStream fo = new FileOutputStream(f);
            BufferedOutputStream buffOut = new BufferedOutputStream(fo);
            DataOutputStream dataOut = new DataOutputStream(buffOut);
            for(int i=0;i<userComments.length;i++)
                dataOut.writeBytes(userComments[i] + "\n");
            Iterator it = users.iterator();
            while(it.hasNext()) {
                User s = (User)it.next();
                if(!user.equals(s.getName())) // 150524
                    dataOut.writeBytes("'" + s.getName() + " " + s.getPass() + "\n");
            }
            dataOut.close();
            buffOut.close();
            fo.close();
            f.renameTo(userFile);
        }
    } catch(IOException ioe) {
        System.out.println("FileDataSource.removeUserFromFile user " + user);
        ioe.printStackTrace();
    }
}

```

```

    }
}
private void errorInFiles(Exception e) {
    System.out.println("FileDataSoure errorInFiles ");
    e.printStackTrace();
}
// 读取指定文件中所有非注释信息行，并以对象数组的形式返回
private Object[] readLines(File f) {
    LinkedList list = new LinkedList();
    try {
        FileInputStream fileIn = new FileInputStream(f);
        InputStreamReader isr = new InputStreamReader(fileIn);
        BufferedReader dataIn = new BufferedReader(isr);
        String str = "";
        while((str = dataIn.readLine()) != null)
            if(str.charAt(0) != '#')
                list.add(str);
    } catch(IOException ioe) {
        System.out.println("FileDataSource.readLines(File " + f + "): " + ioe.getMessage());
    } finally {
        return list.toArray();
    }
}
// 获取指定字符串的 MD5
public String getMD5(String str) {
    MessageDigest md = null;
    try {
        md = MessageDigest.getInstance("MD5");
        byte[] by = str.getBytes("UTF-8");
        md.update(by);
    } catch(Exception e) {e.printStackTrace();}
    byte[] b = md.digest();
    String strOut = "";
    for(int i=0;i<b.length;i++) strOut += b[i];
    return strOut;
}
public boolean addInfo(String user, String field, String data) {
    return true;
}
// 向用户列表和用户文件中添加用户
public boolean addUser(String user, String pass) {
    if(user == null || pass==null
        || user.equalsIgnoreCase("") || pass.equalsIgnoreCase(""))
        return false;
}

```

```

        if(verifyUser(user,"")) {
            String md5 = getMD5(pass);
            User tmpUser = new User(user, md5);
            users.add(tmpUser);
            appendToUserFile(user + " " + md5);
            userListChanged = true;
            return true;
        }
        return false;
    }
    // 从用户列表和用户文件中删除指定用户
    public boolean removeUser(String user, String pass) {
        if(user == null || pass==null
            || user.equalsIgnoreCase("") || pass.equalsIgnoreCase(""))
            return false;
        Iterator it = users.iterator();
        boolean found = false;
        User s = null;
        while(it.hasNext() && !found) {
            s = (User)(it.next());
            if(s.getName().equalsIgnoreCase(user)) {
                found = true;
                if(s.getPass().equalsIgnoreCase(pass)) {
                    users.remove(s);
                    removeUserFromFile(user);
                    return (userListChanged = true);
                }
            }
        }
        return false;
    }
    // 验证用户和密码是否有效
    public boolean verifyUser(String user, String pass) {
        Iterator it = users.iterator();
        boolean found = false;
        User s = null;
        while(!found && it.hasNext()) {
            s = (User)(it.next());
            if(s.getName().equalsIgnoreCase(user)) {
                found = true;
            }
        }
        if(found) {
            if(s.getPass().equalsIgnoreCase(pass))

```

```

        return true;
    else
        return false;
    }
    return true;
}
// 用户类
class User {
    private String n;
    private String p;
    public User(String name, String password) {
        n = name;
        p = password;
    }
    public String getName() {
        return n;
    }
    public String getPass() {
        return p;
    }
    public void printUser() {
        System.out.println("FileDataSource.User registered user " + n);
    }
}
// 信息类
class Info {
    String user;
    String field;
    String data;
    public Info(String user, String field, String data) {
        this.user = user;
        this.field = field;
        this.data = data;
    }
    public void printInfo() {
        System.out.println("FileDataSource.Info registered info " +
            "\n\tuser " + user + " field " + field + " data " + data);
    }
}
// 获取用户列表中所有用户的用户名
public String[] getUserList() {
    int usSize = users.size();
    if(userListChanged && usSize>0) {
        us = new String[usSize];
    }
}

```



```

    } else {
        StringTokenizer strTok = new StringTokenizer(str);
        String command = strTok.nextToken();
        if(command.equalsIgnoreCase(NICK))
            if(strTok.hasMoreTokens())
                setNick(cc, strTok.nextToken());
            else
                cc.sendMessage("usage: /nick <newNick>");
        else if (command.equalsIgnoreCase(USERS))
            users(cc);
        else if (command.equalsIgnoreCase(EXIT))
            exit(cc);
        else if (command.equalsIgnoreCase(VERIFY))
            verifyNick(cc, strTok.nextToken());
        else if(command.equalsIgnoreCase(REGISTER))
            registerNick(cc, strTok.nextToken(), strTok.nextToken());
        else if(command.equalsIgnoreCase(WHO_AM_I))
            whoAmI(cc);
        else if(command.equalsIgnoreCase(MSG))
            msg(cc, strTok.nextToken(), strTok);
        else if(command.equalsIgnoreCase(STATS))
            stats(cc);
    }
} catch(Exception e) {
    System.out.println("CommandParser: " + e.getMessage());
    cc.sendMessage("Invalid Command: " + str);
}
}
// 将用户的状态信息发送给用户
private void stats(ConnectedClient cc) {
    long runningTime = System.currentTimeMillis() - MainServer.uptime;
    String str = "Server has been running for " + printTime(runningTime) + "<br>" +
        "User connects since uptime " + MainServer.connects + "<br>";
    cc.sendMessage(str);
}
// 将毫秒转化时、分、秒和毫秒
private String printTime(long time) {
    String str = "";
    if(time<sec) {
        str+=" " + time + "ms";
        return str;
    }
    if(time>sec && time<min) {
        long t = time%sec;

```

```

        str+=" " + (time/sec) + "sec " + printTime(t);
        return str;
    }
    if(time>min && time<hours) {
        long t = time%min;
        str+= " " + (time/min) + "min " + printTime(t);
        return str;
    }
    if(time>hours && time<days) {
        long t= time%hours;
        str+= " " + (time/hours) + "hours " + printTime(t);
    }
    return str;
}
// 向指定用户发送指定信息
private void msg(ConnectedClient cc, String user, StringTokenizer strTok) {
    StringBuffer strBuff = new StringBuffer();
    while(strTok.hasMoreTokens())
        strBuff.append(strTok.nextToken() + " ");
    cc.sendTo(user, cc.nick + ":" + strBuff.toString());
}
// 将获取的用户列表发送给用户
private void users(ConnectedClient cc) {
    LinkedList users = (LinkedList)((cc.getConnectionKeeper().users()).clone());
    String msg = "Current Connected Users: <br>";
    while(users.size()>0)
        msg += "*" + ((ConnectedClient)(users.removeFirst())).getNick() + "<br>";
    cc.sendMessage(msg);
}
// 设置昵称. 当昵称和密码都有效时如果指定的昵称未被占用, 则设置为指定昵称,
// 否则通知客户已占用; 当昵称或密码无效时提醒用户
private void setNick(ConnectedClient cc, String str) {
    cc.verifiedBoolean = false;
    boolean verify = ds.verifyUser(str, "");
    if(verify) {
        if(isNickFree(cc, str)) {
            cc.nick = str;
            cc.verifiedBoolean = true;
        } else
            cc.sendMessage("nick " + str + " was already taken");
    } else {
        cc.verifiedCount = 5;
        cc.tmpNick = str;
        cc.sendMessage("Nick " + str + " is registered so you have to " +

```

```

        "verify that this nick is yours");
    }
}
// 检验用户是否已被占用
private boolean isNickFree(ConnectedClient cc, String nick) {
    LinkedList users = (LinkedList)((cc.getConnectionKeeper().users()).clone());
    Iterator it = users.iterator();
    while(it.hasNext()) {
        ConnectedClient comp = ((ConnectedClient)(it.next()));
        String compNick = comp.getNick();
        if(nick.equalsIgnoreCase(compNick))
            return false;
    }
    return true;
}
// 查询自己
private void whoAmI(ConnectedClient cc) {
    cc.whoAmI();
}
// 注册用户
private void registerNick(ConnectedClient cc, String nick, String pass) {
    if(pass.length()<4 || nick.length()<4) {
        cc.sendMessage("Your nick/password needs to be atleast 4 chars long");
    } else {
        if(ds.addUser(nick, pass)) {
            cc.sendMessage("User " + nick + " is now registered and set as your own");
            cc.nick = nick;
            cc.verifyedBoolean = true;
        } else {
            cc.sendMessage("The username is allready taken");
        }
    }
}
// 验证用户和密码是否有效
private void verifyNick(ConnectedClient cc, String password) {
    if(ds.verifyUser(cc.tmpNick, password)) {
        cc.nick = cc.tmpNick;
        cc.verifyedBoolean = true;
    } else {
        cc.nick = "" + cc.portNumber;
        cc.sendMessage("Invalid user/pass, your nick is set to " + cc.nick);
    }
}
// 50 毫秒后用户下线

```



```

private void exit(ConnectedClient cc) {
    cc.sendMessage("Server: You are being disconnected!");
    try { Thread.sleep(50); } catch(Exception e) {}
    cc.dropClient();
}
public void setDataSource(DataSource ds) {
    this.ds = ds;
}
}

```

附录 B Linux 网络编程基础知识

B.1 地址结构

Linux 系统的套接口可以支持多种协议，每种不同的协议都是用不同的地址结构。sockaddr 结构在<bits/socket.h> 中定义，是一种通用的套接口地址。它常用于 bind、connect、recvfrom、sendto 等函数的参数，指明地址信息。sockaddr 结构如下所示。

```

struct sockaddr
{
    __SOCKADDR_COMMON (sa_); // 指定地址族。等价于 sa_family_t sa_family
    // 地址数据。对于 Internet 地址族，包括 sin_port、sin_addr、sin_zero 等数据
    char sa_data[14];
};

```

socketaddr_in 结构在<netinet/in.h> 中定义，该结构把地址和端口号分别存放，在网络编程中常用来设置或获取地址信息。socketaddr 结构定义为：

```

struct sockaddr_in
{
    __SOCKADDR_COMMON (sin_); // 指定地址族，等价于 sa_family_t sin_family
    in_port_t sin_port; // 端口号。必须是网络字节序
    struct in_addr sin_addr; // IP 地址，必须为网络字节序

    // 为与通用套接口地址结构(struct sockaddr)保持大小一致而填充的数据
    unsigned char sin_zero[sizeof (struct sockaddr) -
        __SOCKADDR_COMMON_SIZE -
        sizeof (in_port_t) -
        sizeof (struct in_addr)];
};

```

其中的 in_addr 结构定义如下：

```

struct in_addr
{
    uint32_t s_addr; //为 IPv4 地址
};

```

其中__SOCKADDR_COMMON(sa_prefix)和__SOCKADDR_COMMON_SIZE 宏在<bits/sockaddr.h>中定义，它们的定义如下：

```
...
// POSIX.1g specifies this type name for the `sa_family' member.
typedef unsigned short int sa_family_t;

// This macro is used to declare the initial common members
// of the data types used for socket addresses, `struct sockaddr',
// `struct sockaddr_in', `struct sockaddr_un', etc.

#define __SOCKADDR_COMMON(sa_prefix) \
    sa_family_t sa_prefix##family

#define __SOCKADDR_COMMON_SIZE (sizeof (unsigned short int))
...
```

sockaddr_ll 结构是一个独立于设备的物理层地址，在<linux/if_packet.h>中定义。sockaddr_ll 结构定义如下：

```
struct sockaddr_ll {
    unsigned short sll_family;    // 通常为AF_PACKET
    unsigned short sll_protocol; // 物理层协议
    int            sll_ifindex;   // 接口索引
    unsigned short sll_hatype;   // 头部类型
    unsigned char  sll_pkttype;  // 包类型
    unsigned char  sll_halen;    // 物理层（如：IEEE802.3）地址长度
    unsigned char  sll_addr[8];  // 物理层地址
};
```

B.2 相关函数

1) socket() 函数

函数原型：

```
int socket(int domain, int type, int protocol);
```

头文件：

```
#include<sys/types.h>
```

```
#include<sys/socket.h>
```

函数说明：

socket() 用来建立一个协议族为 domain、协议类型为 type、协议编号为 protocol 的套接口描述字。

参数 domain 用于设置网络通信的域，socket() 根据这个参数选择通信协议族。通信协

议族在 `sys/socket.h` 中定义，以下是常见的协议：

名称	含义
PF_UNIX/PF_LOCAL/AF_UNIX/AF_LOCAL	UNIX 进程通信协议
PF_INET/AF_INET	IPv4 网络协议
PF_INET6/AF_INET6	IPv6 网络协议
PF_IPX/AF_IPX	IPX-Novell 协议
PF_NETLINK/AF_NETLINK	内核用户界面设备
PF_X25/AF_X25	ITU-T X.25/ISO-8208 协议
PF_AX25/AF_AX25	业余无线 AX.25 协议
PF_ATMPVC/AF_ATMPVC	原始 ATM PVC 访问
PF_APPLETALK/AF_APPLETALK	Appletalk (DDP) 协议
PF_PACKET/AF_PACKET	底层包访问

参数 `type` 用于设置套接口通信的类型，以下是参数 `type` 的值及含义

名称	含义
SOCK_STREAM	TCP 连接，提供可靠的、双向连接的字节流。支持带外数据传输。在数据传送前客户必须使用 <code>connect()</code> 来建立连接。
SOCK_DGRAM	UDP 连接。
SOCK_SEQPACKET	序列化包，提供一个序列化、可靠的、双向的基于连接的数据传输通道，数据长度定常。每次调用读系统调用时需将全部数据读出。
SOCK_RAW	提供原始网络协议访问。
SOCK_RDM	提供可靠的数据报文，不过可能数据会有乱序。
SOCK_PACKET	提供和网络驱动程序直接通信。

类型为 `SOCK_STREAM` 的套接口表示一个双向的字节流，与管道类似。流式套接口在收发数据前必须是已连接的，连接使用 `connect()` 函数进行。一旦连接，可使用 `read()` 或 `write()` 进行数据传输。流式通信方式保证数据不会丢失或重复接收，当数据在一段时间内没有接收完毕，可认为这个连接已死掉。

类型为 `SOCK_DGRAM` 和 `SOCK_RAW` 的套接口可使用 `sendto()` 函数发送数据，`recvfrom()` 函数接收数据，`recvfrom()` 接收来自指定 IP 地址发送方的数据。

参数 `protocol` 用来指定某个协议的特定类型。通常某个协议中只有一个特定类型，可将其设为 0；但是有些协议有多种特定的类型，就需要设置这个参数来选择特定的类型。

返回值：

成功则返回一个表示这个套接口的描述字，失败返回 -1。

以下为 `errno` 的值及含义

值	含义
EPROTONOSUPPORT	参数 <code>domain</code> 指定的类型不支持参数 <code>type</code> 或 <code>protocol</code> 指定的协议
ENFILE	核心内存不足，无法建立新的 socket 结构
EMFILE	进程文件表溢出，无法再建立新的 socket
EACCESS	权限不足，无法建立 <code>type</code> 或 <code>protocol</code> 指定的协议
ENOBUFS/ENOMEM	内存不足

EINVAL	传给系统调用的参数不正确
--------	--------------

2) bind() 函数

把一个本地地址赋予一个套接口。对于网际地址，协议地址是 32 位的 IPv4 地址或 128 位的 IPv6 地址与 16 位的 TCP 或 UDP 端口号的组合。

函数原型：

```
int bind(int s, const struct sockaddr *myaddr, socklen_t addrlen);
```

头文件：

```
#include <sys/socket.h>
```

函数说明：

参数 s 由 socket 函数返回的套接口描述字。参数 myaddr 是一个指向套接口地址结构的指针。参数 addrlen 为该结构的大小，可使用 sizeof(struct sockaddr) 获得。

对于 TCP，调用 bind 函数可以指定一个端口号，指定一个 IP 地址，可以两者都指定，也可以都不指定

返回值：

成功则返回 0，失败返回-1，错误原因存于 errno。

3) listen() 函数

函数原型：

```
int listen(int s, int backlog);
```

头文件：

```
#include <sys/types.h>
```

```
#include<sys/socket.h>
```

函数说明：

参数 s 的套接口描述字必需先经过 socket() 和 bind() 函数处理，然后才可调用 accept()。调用 listen() 后并未开始接收客户的连接，只是套接口成为正在监听套接口。参数 backlog 指定同时能处理的最大连接要求，如果连接数目达此上限则 client 端将收到 ECONNREFUSED 的错误。如果 socket 为 AF_INET 则参数 backlog 最大值可设至 128。listen() 只适用 SOCK_STREAM 或 SOCK_SEQPACKET 的 socket 类型。

返回值：

成功则返回 0，失败返回-1，错误原因存于 errno。errno 的值及含义见下表：

值	含义
EADDRINUSE	另一个socket已经在同一端口侦听
EBADF	参数s不是合法描述字
ENOTSOCK	参数s为一文件描述字，非socket

4) accept() 函数

当一个客户的连接请求到达服务器监听的端口时，客户的连接会在队列中等待，直到使用服务器处理接收请求。

函数原型：

```
int accept(int s, struct sockaddr *addr, int *addrlen);
```

头文件：

```
#include<sys/types.h>
#include<sys/socket.h>
```

函数说明：

参数 s 的套接口描述字必需先经过 bind() 和 listen() 函数处理，当服务器成功处理客户的连接请求后，accept() 会返回一个新的套接口描述字，以后的数据发送就经过新的套接口描述字来处理，而原来的套接口描述字能继续使用 accept() 来接收新的连线要求。客户连线成功时，参数 addr 所指的结构会被系统填入客户的 IP 地址、端口和协议族等信息。参数 addrlen 表示参数 addr 所指向的结构长度，可使用 sizeof(struct sockaddr) 获得。

返回值：

成功则返回新连接的客户套接口描述字，与客户间的通信是通过 accept() 返回的新套接口描述字来进行的。失败返回-1，错误原因存于 errno 中。errno 值及含义见下表：

值	含义
EBADF	参数s不是合法描述字。
EFAULT	参数addr指向无法存取的内存空间。
ENOTSOCK	参数s为一文件描述字，非socket。
EOPNOTSUPP	指定的socket并非SOCK_STREAM。
EPERM	防火墙拒绝此连线
ENOBUFS	系统的缓冲内存不足
ENOMEM	核心内存不足。

5) connect() 函数

TCP 客户用 connect() 函数建立与 TCP 服务器的连接。

函数原型：

```
int connect(int s, const struct sockaddr *servaddr, socklen_t addrlen);
```

头文件：

```
#include <sys/socket.h>
```

函数说明：

参数 s 由 socket() 函数返回的套接口描述字。参数 servaddr 是一个指向套接口地址结构的指针，套接口地址结构须含有服务器的 IP 地址和端口号。参数 addrlen 为这个结构的大小。

返回值：

成功则返回 0，失败返回-1，错误原因存于 errno。errno 值及含义见下表：

值	含义
EACCES	在AF_UNIX族协议中，使用路径名作为标识。EACCES表示目录不可写或不可访问
EADDRINUSE	本地地址已经在使用
EAFNOSUPPORT	参数servaddr的域sa_family不正确
EBADF	参数s不是合法描述字
ECONNREFUSED	连接要求被服务器端拒绝。
EFAULT	参数servaddr指向无法存取的内存空间
ENOTSOCK	参数s为一文件描述字，非socket

EISCONN	参数s的socket已连接
ENETUNREACH	网络不可达
ENOTSOCK	参数s为一文件描述字，非socket。
ETIMEDOUT	连接超时。

6) sendto()/send() 函数

当客户成功建立了一个套接口描述字，并构建了相应 struct sockaddr 结构后，或服务器成功将套接口描述字和地址结构绑定后，可使用 send() 或 sendto() 函数向某个主机发送数据。

函数原型：

```
ssize_t send(int s, const void *msg, size_t len, int flags);
ssize_t sendto (int s, const void *msg, size_t len, int flags, const struct
sockaddr *to, socklen_t tolen);
```

头文件：

```
#include<sys/types.h>
#include<sys/socket.h>
```

函数说明：

参数 s 为正在监听端口的套接口描述字，通过 socket() 获得。参数 msg 指向发送数据缓冲区。参数 len 为发送数据缓冲区的大小。参数 flags 一般设 0。参数 to 为指向目的主机数据结构 sockaddr_in 的指针，接收数据的主机地址信息放在这个结构中。参数 tolen 为参数 to 所指向数据结构的长度，可使用 sizeof (struct sockaddr_in) 来获得。

返回值：

成功则返回实际发送出去的字符数，数据长度可为 0。失败返回-1，错误原因存于 errno 中。errno 值及含义见下表：

值	含义
EBADF	参数 s 不是合法描述字。
EFAULT	参数中有一指针指向无法存取的内存空间。
ENOTSOCK	参数 s 为一文件描述字，非 socket。
EINTR	被信号所中断。
EAGAIN	此动作会令进程阻断，但参数 s 的 socket 为不可阻断的。
ENOBUFS	系统的缓冲内存不足。
EINVAL	传给系统调用的参数不正确。

7) recvfrom()/recv() 函数

当客户成功建立了一个套接口描述字，并构建了相应 struct sockaddr 结构后，或服务器成功将套接口描述字和地址结构绑定后，可使用 recv() 或 recvfrom() 函数接收到达这个套接口描述字上的数据或在这个套接口描述字上等待数据的到来。

函数原型：

```
int recv(int s, void *buf, size_t len, int flags);
int recvfrom(int s, void *buf, size_t len, int flags, struct sockaddr *from,
socklen_t *fromlen);
```

头文件：

```
#include<sys/types.h>
#include<sys/socket.h>
```

函数说明：

参数 *s* 为正在监听端口的套接口描述字，它由 `socket()` 生成。参数 *buf* 指向接收数据缓冲区，接收到的数据将放在这个指针所指向的内存空间。参数 *len* 为接收数据缓冲区的大小，系统根据这个值来防止接收缓冲区溢出。参数 *flags* 一般设 0，其他数值定义如下：

MSG_OOB	接收以 out-of-band 送出的数据。
MSG_PEEK	返回来的数据并不会在系统内删除，如果再调用 <code>recv()</code> 会返回相同的数据内容。
MSG_WAITALL	强迫接收到 <i>len</i> 大小的数据后才能返回，除非有错误或信号产生。
MSG_NOSIGNAL	此操作不愿被 SIGPIPE 信号中断。

参数 *from* 指向数据结构 `sockaddr_in` 的指针，发送数据时发送方的地址信息放在这个结构中。参数 *fromlen* 为 `sockaddr_in` 的结构长度，可使用 `sizeof (struct sockaddr)` 来获得。

返回值：

成功则返回接收到的字符数，失败则返回 -1，错误原因存于 `errno` 中。`errno` 值及含义见下表：

EBADF	参数 <i>s</i> 不是合法描述字。
EFAULT	参数中有一指针指向无法存取的内存空间。
ENOTSOCK	参数 <i>s</i> 为一文件描述字，非 <code>socket</code> 。
EINTR	被信号所中断。
EAGAIN	此动作会令进程阻断，但参数 <i>s</i> 的 <code>socket</code> 为不可阻断的。
ENOBUFS	系统的缓冲内存不足。
ENOMEM	核心内存不足。
EINVAL	传给系统调用的参数不正确。

8) `setsockopt()` 函数

`setsockopt()` 用来设置与某个套接口关联的选项。选项可能存在于多层协议中，它们总会出现在最上面的套接口层。当对套接口选项进行操作时，必须给出选项所处的层和选项的名称。为了操作套接口层的选项，应该将层的值指定为 `SOL_SOCKET`。为了操作其它层的选项，必须给出控制选项的协议类型号。

函数原型：

```
int setsockopt(int s, int level, int optname, const void *optval, socklen_t
optlen);
```

头文件：

```
#include<sys/types.h>
#include<sys/socket.h>
```

函数说明：

参数 *s* 为将要设置的套接口描述字，它由 `socket()` 生成。参数 *level* 表示选项所在协议层，一般设成 `SOL_SOCKET` 以操作套接口层的选项。参数 *optname* 为选项名。参数 *optval* 指向用于获取返回选项值的缓冲区。*optlen* 为 *optval* 的长度。

返回值：

成功则返回 0，若有错误则返回-1，错误原因存于 errno。errno 值及含义见下表

值	含义
EBADF	参数 s 不是合法描述字。
EFAULT	参数中有一指针指向无法存取的内存空间。
ENOTSOCK	参数 s 为一文件描述字，非 socket。
ENOPROTOOPT	参数 optname 指定的选项不正确。

套接口的选项及含义见下表：

level	optname	含义	optval
SOL_SOCKET	SO_BROADCAST	允许发送广播数据包	int
	SO_DEBUG	调试跟踪	int
	SO_DONTROUTE	发送出的数据包不要利用路由设备来传输	int
	SO_KEEPALIVE	保持连接	int
	SO_LINGER	延迟关闭连接	struct linger
	SO_OOBINLINE	带外数据放入正常数据流	int
	SO_RCVBUF	接收缓冲区大小	int
	SO_SNDBUF	发送缓冲区大小	int
	SO_RCVLOWAT	接收缓冲区下限	int
	SO_SNDLOWAT	发送缓冲区下限	int
	SO_RCVTIMEO	接收超时	struct timeval
	SO_SNDTIMEO	发送超时	struct timeval
	SO_REUSEADDR	允许地址重用	int
	SO_BSDCOMPAT	BSD 系统兼容	int
	IP_ADD_MEMBERSHIP	加入指定的多播组	struct ip_mreq
	IP_DROP_MEMBERSHIP	离开指定的多播组	struct ip_mreq
	IP_HDRINCL	数据包中包含 IP 头部	int
	IP_OPTIONS	IP 头部选项	struct ip_options
	IP_RECVDSTADDR	目的 IP 地址	int
	IP_RECVIF	接收数据的网络接口	int
	IP_TOS	TOS	int
	IP_TTL	TTL	int
	IP_MULTICAST_IF	设置发送的网络接口	in_addr
	IP_MULTICAST_TTL	设置发送 TTL	uchar
	IP_MULTICAST_LOOP	设置是否环回接口	uchar
	TCP_KEEPALIVE	保持连接时间	int
	TCP_MAXRT	TCP 最大重传时间	int
	TCP_MAXSEG	TCP 最大分段大小	int

IPPROTO_TCP	TCP_NODELAY	禁止 Nagle 算法	int
	TCPSTDURG	紧急指针	int

9) getaddrinfo() 函数

函数原型:

```
int getaddrinfo( const char *hostname, const char *service, const struct
addrinfo *hints, struct addrinfo **result);
```

头文件:

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

函数说明:

参数 hostname 为一个主机名或者地址串 (IPv4 的点分十进制或者 IPv6 的 16 进制串)。参数 service 为一个服务名或者 10 进制数的端口号字符串。参数 hints 为一个空指针, 或指向一个 addrinfo 结构的指针, 调用者在这个结构中填入关于所期望返回的信息类型的线索。例: 如果某服务既可以用 TCP 也可以用 UDP 提供, 调用者可以把 hints 结构中的 ai_socktype 成员设置成 SOCK_DGRAM, 返回的信息就只适用于数据报套接口。

本函数通过 result 指针参数返回一个指向 addrinfo 结构链表的指针, 而 addrinfo 结构定义在头文件 netdb.h 中

```
struct addrinfo{
    int ai_flags;
    int ai_family;
    int ai_socktype;
    int ai_protocol;
    socklen_t ai_addrlen;
    char *ai_canonname;
    struct sockaddr *ai_addr;
    struct addrinfo *ai_next;
};
```

返回值:

成功则返回 0, 失败返回 -1, 错误代码存于 errno。

10) inet_pton() 函数

将字符串类型的 IP 地址转换为二进制类型。

函数原型:

```
int inet_pton(int af, const char *src, void *dst)
```

头文件:

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

函数说明:

参数 af 是网络类型的协议族, 在 IPv4 中的值为 AF_INET。参数 src 指向要转换的字符

串。参数 dst 为转换之后的结果。在 IPv4 中，dst 指向结构 struct in_addr 的指针。

返回值：

转换成功时返回正值；返回-1 时，通常是用于 af 所指定的协议族不支持造成的，此时 errno 的返回值位 EAFNOSUPPORT；当返回 0 时，表示 src 指向的值不是合法的 IP 地址。

11) inet_ntop() 函数

将二进制的 IP 地址转换字符串。

函数原型：

```
const char *inet_ntop(int af, const void *src, char *dst, socklen_t cnt)
```

头文件：

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

函数说明：

参数 af 是网络类型的协议族，在 IPv4 中的值为 AF_INET；参数 src 指向要转换的二进制 IP 地址，在 IPv4 中，src 指向结构 struct in_addr 的指针；参数 dst 指向转换之后结果的指针；参数 cnt 为 dst 缓冲区的大小，避免溢出，如果缓存区太小无法存储地址的值，则返回一个空指针，并将 errno 置为 ENOSPC。

返回值：

成功则返回非空指针，指向 dst。失败则返回空指针，错误代码存于 errno。

12) signal() 函数

函数原型：

```
void (*signal(int signum, void(* handler)(int)))(int);
```

头文件：

```
#include <signal.h>
```

函数说明：

signal() 会根据参数 signum 指定的信号编号来设置该信号的处理函数。当指定的信号到达时就会跳转到参数 handler 指定的函数执行。

如果参数 handler 不是函数指针，则必须是下列两个常数之一：

SIG_IGN 忽略参数 signum 指定的信号。

SIG_DFL 将参数 signum 指定的信号重设为核心预设的信号处理方式。

返回值：

返回先前的信号处理函数指针，如果有错误则返回 SIG_ERR (-1)。

附加说明：

在信号发生跳转到自定的 handler 处理函数执行后，系统会自动将此处理函数换回原来系统预设的处理方式，如果要改变此操作请改用 sigaction()。

12) ioctl() 函数

Linux 网络程序与内核交互的方法是通过 ioctl 来实现的，ioctl 与网络协议栈进行交互，可得到网络接口的信息，网卡设备的映射属性和配置网络接口。并且还可以查看、修改、

删除 ARP 高速缓存的信息。

函数原型：

```
int ioctl(int s, int request, ....);
```

头文件：

```
#include <signal.h>
```

函数说明：

参数 *s* 为套接口描述字。参数 *request* 为请求码。省略的部分对应不同的内存缓冲区，而具体的内存缓冲区是由请求码 *request* 来决定的。

请求码值及含义见下表：

类型	Request	说明	数据类型
IO	FIOGETOWN	获取文件所属的进程 ID	int
	SIOCSTAMP	获取时间戳	int
文件	FIONBIO	设置/清除非阻塞标志	int
	FIOASYNC	设置/清除异步 I/O 标志	int
	FIONREAD	获取接收缓存区中的字节数	int
接口	SIOCGIFNAME	获取网络接口名称	struct ifreq
	SIOCGIFLINK	获取网络接口频道	struct ifreq
	SIOCGIFCONF	获取所有接口清单	struct ifconf
	SIOCGIFFLAGS	获取接口标志	struct ifreq
	SIOCSIFFLAGS	设置接口标志	struct ifreq
	SIOCGIFADDR	获取接口 IP 地址	struct ifreq
	SIOCSIFADDR	设置接口 IP 地址	struct ifreq
	SIOCGIFDSTADDR	获取目的地址	struct ifreq
	SIOCSIFDSTADDR	设置目的地址	struct ifreq
	SIOCGIFBRDADDR	获取广播目的地址	struct ifreq
	SIOCSIFBRDADDR	设置广播目的地址	struct ifreq
	SIOCGIFNETMASK	获取子网掩码	struct ifreq
	SIOCSIFNETMASK	设置子网掩码	struct ifreq
	SIOCGIFMETRIC	获取接口的测度	struct ifreq
	SIOCSIFMETRIC	设置接口的测度	struct ifreq
	SIOCGIFMEM	获取内存地址	struct ifreq
	SIOCSIFMEM	设置内存地址	struct ifreq
	SIOCGIFMTU	获取接口 MTU	struct ifreq
	SIOCSIFMTU	获取接口 MTU	struct ifreq
	SIOCGIFHWADDR	获取硬件地址	struct ifreq
	SIOCSIFHWADDR	设置硬件地址	struct ifreq
	SIOCxxx	(还有很多取决于系统的实现)	struct ifreq
ARP	SIOCSARP	设置 ARP 表项	struct arpreq
	SIOCGARP	获取 ARP 表项	struct arpreq
	SIOCDEAR	删除 ARP 表项	struct arpreq
路由	SIOCADDRT	增加路径	struct rtenr
	SIOCDELRT	删除路径	struct rtenr

附录 C Java 网络编程基础知识

套接口 (Socket) 用于表达两台机器间一个连接的两端。针对一个连接, 每台机器上都有一个套接口。在 Java 中, 创建一个套接口, 用它建立与其他机器的连接, 得到的结果是一个 `InputStream` 以及 `OutputStream`, 从而将连接作为一个 IO 流对象对待。有两个基于流的套接口类 `java.net.Socket` 和 `java.net.ServerSocket`, 分别用于客户和服务器建立套接口连接。

C.1 客户套接口

使用客户套接口进行编程的一般步骤: 构造一个新的套接口对象; 使用新创建的套接口对象连接远程服务器; 连接成功, 从套接口获得输入流和输出流, 进而开始发送和接受数据; 数据传输完毕, 关闭连接。

下面开始针对上面的每一个步骤进行说明。

1) 新建一个套接口对象

客户套接口总共提供了 9 个构造函数, 其中一个是受保护类型 (protected), 只能用于派生类提供自己的套接口实现时使用; 两个用于构造 UDP 传输的构造函数已经废弃, 采用新的数据报套接口 (`DatagramSocket`) 替代; 还有两个用于创建未连接的客户套接口对象的构造函数, 在基于客户套接口类派生新的子类实现一种特殊的套接口类的时候使用; 剩余的 4 个构造函数经常用于创建套接口对象, 这里给予详细介绍。

```
public Socket(String host, int port) throws UnknownHostException,
IOException
```

这个构造函数用于创建一个指定主机上的指定端口的套接口, 并且尝试连接远程主机。`host` 是用字符串表示的主机名, `port` 是 0~65535 之间的整数值, 表示端口。因为在该构造函数不仅仅是创建套接口对象, 还尝试连接远程的主机, 所以当给定的主机名无法解析时会抛出 `UnknownHostException` 异常; 其他原因导致无法连接时会抛出 `IOException` 异常。通常的用法如下:

```
try {
    //创建套接口对象
    Socket s = new Socket("news.sina.com.cn", 80);
    //进行后续处理
    ...;
} catch(UnknownHostException uhex) {
    //进行异常处理
    ...;
} catch(IOException ioex) {
    //进行异常处理
    ...;
}

public Socket(InetAddress address, int port) throws IOException
```

这个构造函数与上一个构造函数类似, 同样是构造一个指定主机上的指定端口的套接口, 并且尝试连接远程主机, 不同的是传入的参数不同, 它使用 `InetAddress` 对象指定主机, 而不是使用字符串形式的主机名。如果在连接主机的过程中出现无法连接主机的错误, 会抛

出 IOException 异常。通常的用法如下：

```
try {
    //创建 InetAddress 对象
    InetAddress ia = InetAddress.getByName(“news.sina.com.cn”);
    //创建套接口对象
    Socket s = new Socket(ia, 80);
    //后续处理
    ...;
} catch(UnknownHostException uhex) {
    //进行异常处理
    ...
} catch(IOException ioex) {
    //进行异常处理
    ...;
}

public Socket(String host, int port, InetAddress localAddr, int localPort)
throws IOException
```

这个构造函数同样是创建一个指定主机上指定端口的套接口对象，并且尝试进行连接。不同的是，它连接到前面两个参数指定的主机和端口，并且从后面两个参数指定的本地网络地址和端口进行连接。如果在连接主机的过程中出现无法连接主机的错误，会抛出 IOException 异常。

```
public Socket(InetAddress address, int port, InetAddress localAddr, int
localPort) throws IOException
```

这个构造函数与前面的构造函数功能相同，不同的是传入的参数不同，它使用 InetAddress 对象指定主机，而不是使用字符串形式的主机名。如果在连接主机的过程中出现无法连接主机的错误，会抛出 IOException 异常。

2) 接收数据

public InputStream getInputStream() throws IOException 方法返回一个输入流，可以从套接口读取数据。通常的用法如下：

```
try {
    //创建套接口对象
    Socket s = new Socket(“news.sina.com.cn”, 80);
    //获得输入流对象
    InputStream is = s.getInputStream();
    //后续处理
    ...
} catch(UnknownHostException uhex) {
    //进行异常处理
    ...
} catch(IOException ioex) {
    //进行异常处理
    ...
}
}
```

3) 发送数据

`public OutputStream getOutputStream()` throws `IOException` 返回一个 `OutputStream` 对象, 把数据写入套接口。通常的用法如下:

```
try {
    //创建套接口对象
    Socket s = new Socket( "news.sina.com.cn", 80);
    //获得输入流对象
    OutputStream os = s.getOutputStream();
    //创建经过缓冲的输入流对象
    OutputStream bos = new BufferedOutputStream(os);
    //创建 Writer 对象
    Writer writer = new OutputStreamWriter(bos, "ASCII");
    //写入数据
    writer.write( "GET / HTTP 1.0\r\n\r\n" );
    //后续处理
    ...;
} catch(UnknownHostException uhex) {
    //进行异常处理
    ...
} catch(IOException ioex) {
    //进行异常处理
    ...
}
```

4) 关闭连接

套接口对象在创建、使用完毕以后要最好进行关闭。虽然套接口对象在程序结束的时候如果没有在程序中显式地得到关闭, 垃圾收集器会自动回收, 但是在套接口使用完毕的时候在程序中显示地关闭是一个良好的编程习惯。套接口类提供了一个方法用来关闭一个套接口对象: `public synchronized void close()` throws `IOException`。通常的用法如下:

```
Socket s = null;
try {
    //创建套接口对象
    s = new Socket( "news.sina.com.cn", 80);
    //进行后续处理
    ...;
} catch(UnknownHostException uhex) {
    //进行异常处理
    ...;
} catch(IOException ioex) {
    //进行异常处理
    ...;
}
finally {
    //关闭套接口对象
    if(s != null) s.close();
}
```

C.2 服务器套接口

通过客户套接口的介绍，了解了如何为正在侦听连接的服务器打开一个套接口的过程。但是仅仅有客户套接口是不够的，而客户套接口不能够编写服务器程序。Java 为编写服务器套接口提供了 `java.net.ServerSocket` 类，用于编写服务器程序，负责侦听接入的 TCP 连接。

使用服务器套接口进行编程的一般步骤：在某个端口构造服务器套接口对象；侦听接入该端口的连接，该过程一直阻塞，直到客户进行连接，连接成功会返回一个套接口对象；发送和接受数据；客户或者服务器端关闭连接；服务器继续侦听接入该端口的下一次连接。

1) 创建服务器套接口对象

服务器套接口类共提供了 4 个构造方法，用于创建一个服务器套接口对象。这里主要介绍经常使用的一个构造函数：`public ServerSocket(int port) throws IOException`。这个构造函数用于在指定的端口创建一个服务器套接口，如果不能在指定的端口创建套接口，会抛出 `IOException` 异常。抛出异常的原因可能是因为在指定的端口已经存在另外一个服务器套接口在运行，这样就可以利用这样的特性来进行端口的检测。

该构造函数的典型用法如下：

```
try {
    //创建服务器套接口对象
    ServerSocket ss = new ServerSocket(90);
    //进行后续处理
    ...;
} catch(IOException ioex) {
    //进行异常处理
    ...;
}
```

2) 侦听连接

服务器套接口通常在不断地接受连接，在每次循环中都会调用一个方法来侦听接入指定端口的连接，该方法一直阻塞，直到连接发生；连接成功以后会返回一个套接口对象。这个方法就是 `public Socket accept() throws IOException`。

该方法的通常的用法如下：

```
ServerSocket ss = null;
try {
    //创建服务器套接口对象
    ss = new ServerSocket(90);
    //进行后续处理
    ...;
} catch(IOException ioex) {
    //进行异常处理
    ...;
}
try {
    //侦听接入的连接
    while(true) {
```

```

        //侦听方法
        Socket s = ss.accept();
        //进行后续处理
        ...;
    }
    ...;
} catch(IOException ioex) {
    //进行异常处理
    ...;
}

```

3) 发送和接收数据

在服务器套接口接受进入的连接并返回一个套接口对象以后,就可以利用上一节介绍的套接口的方法来发送和接收数据了,这里不再详细介绍。

4) 关闭连接

在连接建立并完成数据的发送和接收以后,无论是客户还是服务器端都可以进行关闭操作,套接口的关闭前面已经介绍,这里不再详细介绍。

5) 重新侦听新的连接

在一个客户连接接入、完成数据的发送和接受并关闭以后,服务器套接口可以重新侦听新的连接,这主要是通过一个循环来实现的,如上面的代码所示。

参考文献

- [1] Andrew S. Tanenbaum, David J. Wetherall 著, 严伟, 潘爱民译. 计算机网络(第 5 版). 清华大学出版社, 2012
- [2] W. Richard Stevens 著, 范建华等译. TCP/IP 详解 卷 1: 协议. 机械工业出版社, 2000
- [3] Douglas E. Comer 著, 林瑶, 张娟, 王海译. 用 TCP/IP 进行网际互连 第一卷—原理、协议与结构(第 5 版). 电子工业出版社, 2007
- [4] W. Richard Stevens, Bill Fenner, Andrew M. Rudoff 著, 杨继张译. UNIX 网络编程 第 1 卷: 套接口 API(第 3 版). 清华大学出版社, 2006
- [5] Ulf Lamping, Richard Sharpe, Ed Warnicke. Wireshark User' s Guide For Wireshark 1.99. 2014, <https://www.wireshark.org/docs/>
- [6] 徐明伟, 崔勇, 徐恪. 计算机网络原理实验教程(第 2 版). 机械工业出版社, 2013
- [7] 吴功宜, 吴英. 计算机网络课程设计. 机械工业出版社, 2012
- [8] David C. Plummer. Ethernet Address Resolution Protocol: Or Converting Network Protocol Addresses to 48. bit Ethernet Address for Transmission on Ethernet Hardware. IETF RFC 826, 1982, <http://www.rfc-editor.org/rfc/rfc826.txt>
- [9] S. Cheshire. IPv4 Address Conflict Detection. IETF RFC 5227, 2008, <http://www.rfc-editor.org/rfc/rfc5227.txt>
- [10] K. McCloghrie, M. Rose. Management Information Base for Network Management of TCP/IP-based internets: MIB-II. IETF RFC 1213, 1991, <http://www.rfc-editor.org/rfc/rfc1213.txt>
- [11] R. Presuhn. Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP). IETF RFC 3416, 2002, <http://www.rfc-editor.org/rfc/rfc3416.txt>
- [12] J. Klensin. Simple Mail Transfer Protocol. IETF RFC 5321, 2008, <http://www.rfc-editor.org/rfc/rfc5321.txt>
- [13] J. Myers, M. Rose. Post Office Protocol - Version 3. IETF RFC 1939, 1996, <http://www.rfc-editor.org/rfc/rfc1939.txt>
- [14] R. Siemborski, A. Menon-Sen. The Post Office Protocol (POP3) Simple Authentication and Security Layer (SASL) Authentication Mechanism. IETF RFC 5034, 2007, <http://www.rfc-editor.org/rfc/rfc5034.txt>